Epic-FHIR — Guides and Tutorials (sections 1 – 1.31)

Source: Epic on FHIR (fhir.epic.com) documentation, mirrored 2026-11. Text-only edition for AI ingestion (NotebookLM); pages follow the source order (its table of contents when one exists).

1 Guides and Tutorials

File: site/docs/01-guides/index.html

* open.epic MCP
* Agent Factory Connectors
* API Change Log
* App Default FHIR Version
* App Creation & Request Process Overview
* App Developer Guidelines
* Collaborate with Epic Community Members
* CDS Hooks Tutorial
* FHIRcast Tutorial
* Designing an Efficient Application
* Design Strategies to Increase Patient Data Sharing
* Developer Testing Guide
* ID Types for FHIR APIs
* FHIR Bulk Data Access Tutorial
* Implementing Apps at Epic Customers
* Interfaces (HL7v2)
* Patient-Facing Apps Using FHIR
* Requesting a Listing on Showroom
* Sharing Your App's Web Resources
* Epic as a Backend OAuth 2.0 Client
* FHIR Tutorial
* OAuth 2.0 Specification
* OAuth 2.0 Tutorial
* Patient Searching and Matching
* Search Parameters
* Sex, Gender, & Names
* Sandbox Test Data
* Troubleshooting
* User Context
* open.epic Terms of Use
* Frequently Asked Questions (FAQ)

1.1 open.epic MCP

File: site/docs/01-guides/01-mcp.html

Source: https://fhir.epic.com/Documentation?docId=mcp

The open.epic MCP server lets AI clients that support the Model Context Protocol (MCP) connect to open.epic content. Your use of this MCP is subject to the open.epic Terms of Use. The MCP server only retrieves content based on the input from your AI client. It does not provide any guidance. The MCP server is not designated to handle sensitive information, such as protected health information (PHI) or confidential information. You are responsible for any data transmitted.

The following open.epic content (including Epic on FHIR) is returned by the MCP:

* API Specifications
* Interface Summaries
* Additional Documentation, such as Data Sharing Playbooks and Tutorials
* FAQs

Connection Details

Setting | Value

Server URL | https://mcp.open.epic.com
Transport | Streamable HTTP (remote server)

Get Started with the open.epic MCP Server

Most AI clients that support remote MCP servers use the same basic flow. Menu names vary by client and change over time, so use these steps together with your client's documentation.

* Open your client's settings and look for a section named MCP, Connectors, Tools, Plugins, or Integrations.
* Choose the option to add a custom or remote MCP server.
* Select the HTTP or Streamable HTTP transport. Do not select stdio or command, which are for local servers.
* Enter a name, such as open-epic-mcp, and the server URL https://mcp.open.epic.com.
* Verify that the server shows as connected and that its tools are listed.

Some clients can only connect to local (stdio) MCP servers and cannot connect to this server. Some organizations restrict custom MCP servers, so an administrator might need to enable them for you.

Client Documentation

The following documentation explains how to add a custom remote MCP server in popular AI clients. Follow your client's documentation if its steps differ from the general steps above.

* ChatGPT
* Codex
* Claude
* Claude Code
* Cursor
* Microsoft Copilot Studio
* Visual Studio Code (GitHub Copilot)
* Google Antigravity

Other Clients

For any other AI client, search its documentation for "remote MCP server," "custom connector," or "Streamable HTTP," and use the values in the connection details table.

1.2 Agent Factory Connectors

File: site/docs/01-guides/02-agent-factory-connectors.html
Source: https://fhir.epic.com/Documentation?docId=agent-factory-connectors

Agent Factory Connectors: Early Information for Developers

Agent Factory Connectors are under development. We're sharing pre-release details because of strong interest from the developer community, and all information in this document is subject to change.

Last updated: 10/30/2026

What is Epic Agent Factory?

Agent Factory is Epic's AI orchestration engine: our platform for creating, monitoring, and evaluating AI agents for healthcare. It automates repetitive tasks and improves experiences for staff and patients using AI, while remaining integrated with existing data and workflows.

Epic's released agents run on Agent Factory, including Emmie for patients, Art for clinicians, and Penny for operational efficiency, with over 140 workflows deployed through the platform today. Customers have opportunities to shape Epic-released agents with their own data, such as organizational policies and knowledge bases. Additionally, customers can create custom agents from scratch with the visual drag-and-drop Agent Factory Builder.

Agent Factory Connectors

Epic is creating Agent Factory Connectors so that agents can do more on behalf of users, such as finding information and completing tasks in the other apps healthcare organizations rely on. With Connectors, patients, clinicians, and staff no longer have to do "swivel-chair" work, switching between apps and portals to get what they need.

Here are a few examples of how developers could leverage Agent Factory Connectors to support healthcare organizations:

* When an ambulance is on the way, emergency department staff could see arrival times that Art pulls from the ambulance app.
* Before discharge, surgical teams could confirm which assistive devices are available for a patient's recovery, using details Art gathers from a medical equipment app.

* A patient who isn't sure where to park could ask Emmie, then follow directions from a wayfinding app all the way to their appointment and back to their car.

* During a visit, clinicians could get real-time guidance from a clinical calculator or knowledge base, surfaced by Art right in their workflow.

* Revenue cycle staff could ask Penny about coverage or prior authorization status, and Penny could give the answer from a payer network app.

* Analysts could use Penny to pull request details from a help desk app and update the ticket once the work is done.

Healthcare organizations using Epic will be able to use Agent Factory Connectors to hook up your apps starting in Q1 2027, for these and many other use cases.

Capabilities and Technical Details

To start, Agent Factory Connectors will be built on the Model Context Protocol (MCP). Agent Factory will serve as the MCP client, and agents will be able to interact with your MCP servers. Customers choose which MCP servers to connect, including systems they've built themselves and third-party products.

You'll be able to register your MCP server configuration in Vendor Services and list your connector on Showroom, which is shown in the Agent Factory Marketplace so agent builders at healthcare organizations can find it. When a customer chooses your Agent Factory Connector, we'll deliver that configuration to their Agent Factory environments to speed up implementation. Vendor Services and Showroom will also support recommended practices for Agent Factory Connectors.

MCP connectors are one tool in the data exchange toolbox. Epic supports over 640 FHIR APIs and hundreds of additional industry-standard and Epic public APIs through open.epic. App developers are encouraged to use the right tool for the use case, which may include using only RESTful API calls, creating an MCP server, or a combination of both tools.

For more information, check out the MCP documentation and specification maintained by the Agentic AI Foundation. Our development plans do not currently include support for all aspects of the specification, such as extensions and experimental features.

Next Steps

Early-adopter customers will start piloting Agent Factory Connectors by the end of 2026. All customers that use Agent Factory Builder, as well as all vendors that want to list their Agent Factory Connector in Showroom, will be able to connect by the end of Q1 2027.

Customers will adopt Connectors at different paces throughout 2027 as they turn on Agent Factory features and train their staff to build with it. If you're working with an Epic customer and want to create a Connector for their use case, work directly with that customer to understand their timeline.

We will continue to update our documentation as design and development progress in the coming months. If you have additional questions, vendors enrolled in Vendor Services can contact our technical services team by creating a Sherlock.

1.3 API Change Log

File: site/docs/01-guides/03-changelog.html

Source: https://fhir.epic.com/Documentation?docId=changelog

Last updated: 10/28/26

This page explains non-backwards-compatible changes to certified APIs made in the past year. This will not include enhancements such as new APIs.

2026

November

Q- 8529745 Binary.Read Responses Now Include Browser-Hardening Headers

This change will be introduced in the November 2026 version of Epic. It applies to apps that meet all of the following conditions:

* Use any of Binary.Read APIs

* Display Binary document content inline in a browser

Background

The Binary.Read FHIR APIs can return either the Binary document or a FHIR resource representation of the document, depending on the value of the "Accept" header.

What's Changed

When an application reads a FHIR Binary resource as a document, the following headers are returned on the response:

* content-disposition: attachment

* content-security-policy: default-src 'none';base-uri 'none';form-action 'none';frame-ancestors 'none';sandbox;

* x-content-type-options: nosniff

If an application attempts to display the document inline in a browser, these headers prevent the document from being displayed and instead tell the browser to download the content as an attachment.

Applications that only render the Binary documents and don't attempt to display them inline receive the new headers but continue to render the documents as before.

Applications that use the "Accept" header set to "application/fhir+xml," "application/fhir+json," "application/xml+fhir," or "application/json+fhir" are unaffected because they return the FHIR resource representation of the Binary document. The new headers are not returned on these requests.

Mitigating impact

Your use of the FHIR Binary.Read APIs should account for downloading documents rather than displaying them inline in a browser, or be updated to render the documents without treating them as a webpage.

Suggested next steps

* Evaluate the effect of this change for your applications.

Q-8427283 Observation (Flowsheets) (R4) Now Required to Retrieve Non-Vital Signs Flowsheet Data

This change will be introduced in the November 2026 version of Epic (August 2026 by special update). It applies to apps that use any of the following APIs to interact with flowsheet data that is not a vital sign:

* Observation.Read (Vital Signs) (R4)

* Observation.Search (Vital Signs) (R4)

* Observation.Create (Vital Signs) (R4)

Background

Previously, you could use the Observation (Vital Signs) (R4) sub-resource to interact with flowsheet data that wasn’t better represented by another sub-resource, even if the data wasn’t a vital sign. When you searched for a flowsheet row by its ID, Epic returned the row with a category code of “vital-signs”, even for non-vital-sign data.

What’s Changed

We created a new sub-resource for non-vital-sign flowsheet data to improve differentiation and to support future scope changes. The Observation (Flowsheets) (R4) sub-resource now handles any non-vital-sign flowsheet data that isn’t already handled by an existing sub-resource, such as Observation (Assessments) (R4), Observation (Lines, Drains, Airways) (R4), or Observation (Vital Signs) (R4). It can also return vital sign and assessment data, either by itself or together with the more specific sub-resources. Non-vital-sign flowsheet rows are returned by the Observation (Flowsheets) (R4) sub-resource without a category code.

Applications that relied on the previous behavior of Observation (Vital Signs) (R4), such as specific element inclusion or cardinality, might no longer work as expected. Additionally, Observation (Flowsheets) (R4) requests including both a category and code parameter now fail if the request is for non-vital-sign flowsheet data.

Mitigating impact

To preserve backwards compatibility, Epic added Observation (Flowsheets) (R4) to existing applications that were using Observation (Vital Signs) (R4). However, if your application currently sends both category and code parameters to Observation (Vital Signs) (R4) to interact with non-vital-sign flowsheet data, you need to make additional updates to send only the code parameter.

Suggested next steps

* If your integration interacts with non-vital-sign flowsheet rows through Observation (Vital Signs) (R4), evaluate the effect of this change for your integration.

Q-8275152 Encounter (Outside Record) Participants Can Reference PractitionerRole

This change will be introduced in the November 2026 version of Epic. It applies to apps that meet the following conditions:

* The app uses one or more of the following APIs:

* Encounter.Read (Outside Record) (R4)

* Encounter.Search (Outside Record) (R4)

* The app reads Encounter.participant.individual and expects it to always reference a Practitioner resource.

Background

The Encounter (Outside Record) APIs return encounters that a health plan received from outside payer and provider systems, such as data exchanged through the CMS Payer-to-Payer API. The participant.individual element identifies the providers involved in each encounter.

In FHIR R4, participant.individual can reference either a Practitioner or a PractitionerRole resource. Practitioner carries a provider's name and identifiers such as NPI. PractitionerRole carries the provider's specialty and type, along with a reference back to the Practitioner. See HL7 FHIR R4 Encounter and HL7 FHIR R4 PractitionerRole.

What's Changed

Previously, participant.individual always referenced a Practitioner resource when it was sent.

Starting in November 2026, participant.individual references a PractitionerRole resource when a provider specialty (PractitionerRole.specialty) or provider type (PractitionerRole.code) is available for that participant. When neither is available, a Practitioner reference is sent, as before. The PractitionerRole resource includes a reference to the corresponding Practitioner, so the provider's name and identifiers remain available.

Rationale: This change makes provider specialty and type available to API consumers while continuing to support Practitioner references as required by the US Core Encounter profile.

Mitigating impact

Apps should check the resource type in each participant.individual reference rather than assuming it is a Practitioner. If your app needs provider details, resolve PractitionerRole references and follow PractitionerRole.practitioner to retrieve the Practitioner. If your app does not need provider details, it can safely ignore PractitionerRole references.

Suggested next steps

* Review your app's handling of Encounter.participant.individual and update it to accept both Practitioner and PractitionerRole references before your customers upgrade to November 2026.

August

Q-8376163 Encounter Time Comparisons Now Use a Consistent Time Zone

This change will be introduced in the February 2026 version of Epic. It applies to apps that meet all of the following conditions:

* You query the (outside record) FHIR R4 Encounter API with date-range parameters.

* You have previously done development to correct for the described behavior.

Background

Previously, when filtering (Outside Record) Encounter searches by a date range, Epic could compare an encounter's start and end times to the search parameters using the encounter's time zone data rather than a single consistent time zone. This could cause encounters to be incorrectly excluded from results — for example, an encounter matching a ge parameter but not a le parameter (a false non-overlap).

What’s Changed / Issue Description:

This change corrects a defect so that Encounter date-range comparisons are evaluated consistently regardless of encounter time zone data. Integrations that query the FHIR R4 Encounter (Outside Record) API with date-range filters (for example, date=ge or date=le) no longer have encounters incorrectly missing from results due to the encounter’s time zone data being in a different time zone than the search parameters.

We standardized Encounter date-range filtering to compare encounter start and end instants to the search parameters in UTC, preventing false non-overlap determinations (such as an encounter appearing for ge but not for le). Date search parameters don't need to be sent in UTC—they can continue to be provided in any time zone (including a time zone offset), and we convert both the search parameter values and the encounter instants to UTC before comparing them. Follow-up is required for downstream systems with custom configuration that were accounting for time zone mismatch to apply updates that enable this corrected UTC-based comparison.

API: Encounter.Read (Outside Record) (R4), Encounter.Search (Outside Record) (R4)

Tutorial: Epic on FHIR - FHIR Tutorial

Mitigating impact

If you built logic to compensate for the previous inconsistent behavior (for example, adjusting time zones yourself or expecting encounters to be filtered in the encounter's time zone), you should remove or update that logic, so you don't introduce new incorrect filtering.

Suggested next steps

* Evaluate the effect of this change for your applications

Q-8294158 DiagnosticReport (Results) Code System Changed

This change will be introduced in the August 2026 version of Epic. It applies to apps that interact with any of the following FHIR resources:

* DiagnosticReport.Search (Results) (R4)

* DiagnosticReport.Search (Results) (STU3)

* DiagnosticReport.Read (Results) (R4)

* DiagnosticReport.Read (Results) (STU3)

What’s Changed

When an order that is linked to a CPT code is included in one of the affected resources, the URL for the CPT code system (http://www.ama-assn.org/go/cpt) is now returned instead of the OID for the CPT code system (2.16.840.1.113883.6.12).

Rationale: Compliance with USCDI specifications.

Mitigating impact

Your use of the R4 and STU3 versions of the DiagnosticReport.Search (Results) and DiagnosticReport.Read (Results) FHIR resources should account for both the URL and the OID being returned for the CPT code system as Epic organizations upgrade through the August 2026 version on varying timelines.

Suggested next steps

* Evaluate the effect of this change for your applications.

Q-8252284 Updates to Subspace Resource content

This change will be introduced in the August 2026 version of Epic. It applies to apps that integrate with Subspace and use any of the following resources:

* Patient (R4)

* Encounter (R4)

* ImagingStudy (R4)

* DiagnosticReport (R4)

Background

Subspace is Epic's implementation of FHIRcast, which uses FHIR resources to synchronize clinical context between Epic's Hyperspace and external applications.

What's Changed

Previously, Subspace FHIRcast context-sync events (Patient, Encounter, ImagingStudy, & DiagnosticReport) used full resources every time they were generated. Starting in August 2026, they will use more concise data. These smaller resources only include elements that are defined as key elements by FHIRcast event definitions.

Below is an example of a Patient-open event generated with full resources. Text that is struck through will no longer be used when generating smaller resources starting in August 2026.

```
--- 2026-05-01 10:11:11.011 ---
WebSocket Message Received
{
  "timestamp": "2026-05-01T10:11:11.01",
  "id": "eNvVKcu2A-KUEKfXtp0YxQQ3",
  "event": {
    "hub.topic": "frd9kZPHPEWGGSlllt0zZw",
    "hub.event": "Patient-open",
    "context": [
      {
        "key": "patient",
        "resource": {
          "resourceType": "Patient",
          "id": "e5P83KaqpGP93ThmKaWsgeop",
          "extension": [ ... ],
          "identifier": [ ... ],
          "active": true,
          "name": [
            {
              "use": "official",
              "text": "Jane Doe",
              "family": "Doe",
              "given": [
                "Jane"
              ]
            },
            {
              "use": "usual",
              "text": "Jane Doe",
              "family": "Doe",
              "given": [
                "Jane"
              ]
            }
          ],
          "gender": "female",
          "birthDate": "1999-01-01",
          "deceasedBoolean": false,
          "managingOrganization": { ... }
        }
      }
    ]
  }
}
```

Visit https://fhircast.hl7.org/3_Events.html to dig deeper into the FHIRcast specifications that Epic FHIRcast events now match.

Exception: The Epic FHIRcast Patient resource will always include name, gender, and birthDate.

Affected specifications include:

* fhircast/Patient-open.read

* fhircast/Patient-open.write
* fhircast/Patient-close.read
* fhircast/Patient-close.write
* fhircast/Encounter-open.read
* fhircast/Encounter-open.write
* fhircast/Encounter-close.read
* fhircast/Encounter-close.write
* fhircast/ImagingStudy-open.read
* fhircast/ImagingStudy-open.write
* fhircast/ImagingStudy-close.read
* fhircast/ImagingStudy-close.write
* fhircast/DiagnosticReport-open.read
* fhircast/DiagnosticReport-open.write
* fhircast/DiagnosticReport-close.read
* fhircast/DiagnosticReport-close.write

Check out the open.epic :: Hyperdrive - Subspace Communication Framework topic for context on how Subspace uses FHIRcast resources.

This change better aligns with FHIR specifications and reduces processing time and CPU usage when generating resources for FHIRcast events.

Mitigating impact

Your use of FHIRcast Patient, Encounter, ImagingStudy, and DiagnosticReport resources should account for both the new limited resource behavior and existing full resource behavior as Epic organizations upgrade through the August 2026 version on varying timelines. Customers will be notified of this change in Release Note 927897.

Resolution - Action Needed by You

* Evaluate the effect of this change for your applications.
* Transition your apps to accept smaller FHIRcast resources.

May

Q-8325680 - Refresh Tokens Issued Before FHIR Sub-Resources Were Created Fail Authorization After Upgrade

This change was introduced in the November 2025 version of Epic. It applies to apps that meet all of the following conditions:

* Uses one or more of the following R4 FHIR resources prior to the November 2025 Upgrade:
* AllergyIntolerance
* Coverage
* Immunization
* Location
* Organization
* Practitioner
* PractitionerRole
* Specimen
* Your integration relies on refresh-token-based offline access or refresh token grants

Background

With the November 2025 release, Epic introduced new sub-resource designations for existing FHIR resources without changing how those resources behave. Authorization scopes for these sub-resources are evaluated at the time of the initial SMART-on-FHIR authorization. Refresh tokens issued before a customer upgraded to November 2025 or later were granted scopes under the pre-sub resources authorization model and do not include the new sub-resource scopes.

Issue Description:

When using a pre-upgrade refresh token to mint a new access token at a post-upgrade customer, the resulting access token does not carry scopes for the newly-created sub-resources, and FHIR requests against those sub-resources return authorization errors (INVALID-OAUTH2-TOKEN: The access token provided is valid, but is not authorized for this service). This continues until every affected token chain naturally expires and the vendor re-authenticates the end user.

Example of a response returned using a pre-upgrade refresh token for Allergy (Patient Chart):


```
{"resourceType":"OperationOutcome","issue":[{"severity":"warning","code":"suppressed","details":{"coding":[{"system":"ur
    n:oid:1.2.840.114350.1.13.324.2.7.2.657369","code":"59204","display":"The authenticated client's search request
    applies to a sub-resource that the client is not authorized for. Results of this sub-type will not be
```

returned."}],"text":"The authenticated client's search request applies to a sub-resource that the client is not authorized for. Results of this sub-type will not be returned."},"diagnostics":"Client not authorized for AllergyIntolerance - Patient Chart. Search results of this type have not been included."},{"severity":"fatal","code":"invalid","details":{"coding":[{"system":"urn:oid:1.2.840.114350.1.13.324.2.7.2.657369","code":"59022","display":"Combination of parameters is not valid for any authorized sub-resource. No search was performed."}],"text":"Combination of parameters is not valid for any authorized sub-resource. No search was performed."}}]}

Mitigating impact

You can re-prompt affected patients/users at your organizations for SMART-on-FHIR authorization.

This problem self-resolves as refresh tokens expire, however the duration depends on your app's refresh-token lifetime and the persistent access duration selected by a user during authorization.

Suggested next steps

* Evaluate the effect of this change for your applications

Q-8272574 - DocumentReference.Search (DSTU2) -Results Not Returned Unless Class Parameter Set

This issue was introduced in November 2025. It applies to organizations that meet all the conditions below:

* Your application uses the DocumentReference.Search (Generated CCDA) (DSTU2) FHIR API

* Your application sends your DocumentReference.Search (DSTU2) requests without the class parameter

Background

The DocumentReference.Search (Generated CCDA) (DSTU2) FHIR API allows applications to retrieve information about the patient as a generated CCDA document. The class parameter is an optional search parameter that filters results by document type. When the class parameter is not included in the request, the API is expected to return all available documents for the patient.

Issue Description:

The DocumentReference.Search (Generated CCDA) (DSTU2) FHIR API returns no results when the class parameter is not included in the search request, even when documents exist for the patient. The response contains only an OperationOutcome, indicating no results were found. This makes the API unusable for any integration that does not explicitly specify the class parameter, and integrations that previously worked correctly stop working upon a customer upgrading to an affected Epic version with no indication of what changed.

This change has been communicated to customers in Release Note 925159.

Mitigating impact

You can work around this by providing the class parameter. This API supports two classes with LOINC codes of 34133-9 (Summarization of Episode Note) and 11506-3 (Subsequent Evaluation Note).

To continue returning all documents through this API without specifying the class parameter, use the DocumentReference.Search (Generated CDA) (R4) FHIR API.

Resolution - Action Needed by You:

* Evaluate the effect of this change on your applications

Q-8205863 Observation Sub-Resource Security Overrides Encoded Flowsheet ID

This change was introduced in the August 2024 version of Epic. It applies to apps that meet all of the following conditions:

* Your app uses the Observation.Search (Vital Signs) (R4) or Observation.Read (Vital Signs) (R4) FHIR API.

* Your application uses customer-supplied encoded flowsheet (FLO) record IDs to identify and access non-vitals data.

* Your customers have mapped one of your encoded flowsheet IDs to one of the following US Core assessment codes:

* 31000005510-US CORE CATEGORY CODE - SDOH

* 31000005511-US CORE CATEGORY CODE - FUNCTIONAL-STATUS

* 31000005512-US CORE CATEGORY CODE - DISABILITY-STATUS

* 31000005513-US CORE CATEGORY CODE - COGNITIVE-STATUS

Background:

The Observation (Vital Signs) (R4) FHIR APIs allow a third-party application to access data about patient vitals. The APIs access relevant information in flowsheet rows. They can also access non-vitals flowsheet data through encoded flowsheet (FLO) record IDs.

What's Changed:

Some customers have now mapped additional flowsheet rows to assessment codes. If a non-vitals flowsheet row is mapped to an impacted assessment code, it can only be retrieved through Observation (Assessments) (R4). The data documented in the flowsheet row can't be retrieved through the Observation (Vital Signs) (R4) FHIR APIs. If your customer has not completed this mapping, your application is unaffected.

Mitigating impact

Customers can install a special update to restore backwards compatibility. They have been notified of this change in Release Note 921681. You can also begin using Observation (Assessments) R4 to retrieve the encoded flowsheet data now mapped to the assessments sub-resource. Work with your customers to determine the best course of action.

Suggested next steps

* Evaluate the effect of this change for your applications.

Q-7868260 Update to Medication.ingredient.itemCodeableConcept.coding

This change was introduced in the May 2025 version of Epic. It applies to apps that meet all the following conditions:

* Use Medication.Read or Medication.Search (STU3 or R4)

* Rely on RXNORM or other coding information in the ingredient.itemCodeableConcept.coding element

Background

Previously, the Medication FHIR resource returned RXNORM and other codes for multi-ingredient medications in both ingredient.itemCodeableConcept.coding and the code.coding element of its component Medication resources. Single-ingredient medications have always returned these codes in the code.coding element. To better support enhanced specificity of response elements on multi-ingredient medications, codes were removed from the ingredient.itemCodeableConcept.coding element of multi-ingredient medications and are now only available by referencing the code.coding element of the component Medication resources. Multi-ingredient medications are most commonly found in Inpatient and not Ambulatory (Outpatient) workflows.

What's Changed:

Starting with the May 2025 version Epic, the Medication STU3 and R4 FHIR resources will only make codes available for multi-ingredient medications through the referenced component Medication resources of a multi-ingredient Medication. Codes are no longer available directly within a multi-ingredient Medication.

Mitigating impact

If your app uses Medication.Read/Search (STU3 or R4) to pull medication data, review your application to determine if you are currently pulling medication codes through ingredient.itemCodeableConcept.coding. If you are, update your application code to call into the component Medication resources of that multi-ingredient medication and pull codes from code.coding.

Suggested next steps

* Evaluate the effect of this change, if any, for your applications

* If affected, update your application code to pull medication codes from code.coding.

February

Q-8157458 New Requirements for Valid Areas of Service

These new requirements are effective with the Epic February 2026 versions for Chronicles Imports (existing format), and is effective immediately for directories delivered via Nebula.

This change will be introduced in the February 2026 version of Epic. It applies to apps that meet all of the following conditions:

* Load Community Resource Directories into Epic by either

* Delivering flat files to customers for import (current method) or

* Delivering content via Nebula

* Use fields 170001-170003 in the HCS,1001 import specification

What's Changed:

We have new requirements in the CRD Vendor Import in Chronicles and Nebula. A valid "area of service" contains one of the following:

* A list of zip codes (field 170003)

* A list of counties (field 170001) with a single state (field 170002)

* A list of States (field 170002)

* Nothing populated in any of the area of service fields

In the case where the area of service passed by the vendor doesn't fall into any of the above categories, we will reject the area of service.

Previously, Areas of Service for Community Resource eligibility could be built in ways that cause provider finder or patient choice to display unexpected results, or exclude places that should be included.

Mitigating impact

We recommend adding validation logic to your exports that account for the updated requirements described above and avoid import errors at customers.

Suggested next steps

* Evaluate the effect of this change for your applications

* If you would like additional support for this change, open a new Sherlock ticket.

Q-8128386 FHIR U.S. State Alphabetical Code Affecting CRD

To ensure that Epic integrations are aligned with the industry's latest authorization standards, we've updated our FHIR Location.address.state property to send only the 2-letter abbreviation codes for U.S. states and territories.

This change will be introduced in the February 2026 version of Epic. It applies to apps that meet the following condition:

* Has Location.Search/Read (R4) FHIR API scoped

Background

Previously, FHIR through Epic sent a state's entire name. In U.S. Core 6.1.0, the Location.address.state property is bound (extensibly) to the 2-letter abbreviation codes for U.S. states.

Mitigating impact

Your use of your FHIR Resource, including in CRD, should account for the Location.address.state property to hold both the full state title and 2-letter abbreviated state title as Epic organizations upgrade through the February 2026 version on varying timelines.

Suggested next steps

* Evaluate the effect of this change for your applications.

Q-8140181 requiresAccessToken Parameter Update

This change will be introduced in the February 2026 version of Epic. It applies to apps that meet any of the following conditions:

* Uses Bulk Data Status Request and the requiresAccessToken parameter

Background

requiresAccessToken is a response element for Bulk Data Status Request. The requiresAccessToken parameter indicates if downloading the generated files requires a bearer access token.

What's Changed:

The requiresAccessToken parameter in Bulk Data Status Request has been updated to a Boolean value from a string value.

/Specifications?api=10170

This change better aligns with FHIR specifications.

Mitigating impact

Your use of Bulk Data Status Request should account for the requiresAccessToken being present as both a string value and a Boolean value as Epic organizations upgrade through the February 2026 version on varying timelines.

Suggested next steps

* Evaluate the effect of this change for your applications

Q-8130672 End Support for POST Parameters in Query URL

This change will be introduced in the February 2026 version of Epic. It applies to apps that meet all of the following conditions:

* Use the POST HTTP method for FHIR searches

* Include search parameters in the request URL

Background

The FHIR search interaction supports both the GET and POST HTTP methods. Both formats support the same functionality, but certain client or server considerations might make one or the other preferable in specific situations.

What's Changed:

FHIR POST searches no longer support URL query parameters. This applies to all FHIR resources and versions. Note that you can continue to use the _format parameter in the URL of POST requests.

This change removes ambiguity in the expected behavior of FHIR searches using the POST HTTP method.

Mitigating impact

Review your usage of FHIR search APIs. If you are using POST for any searches, verify that all search parameters are in the POST request body.

Suggested next steps

* Evaluate the effect of this change for your applications

2025

November

Q-8017955 Subspace Now Uses "information" as a sync error severity

This change will be introduced in Hyperdrive major version 100.2510. It applies to apps that meet all of the following conditions:

* Your organization manages apps that communicate with the Hyperdrive client using Epic's Subspace Communication Framework, which uses the FHIRcast specification.

* Your organization's apps have support for sync errors that use the "informational" severity.

Background

The FHIRcast specification recognizes four sync error severities:

* fatal

* error

* warning

* information

Epic's Subspace Communication Framework previously used "informational" as the fourth FHIRcast sync error severity, and vendors might have adopted that usage for testing or as a workaround.

What's Changed:

To conform to the FHIRcast specification, Epic's Subspace Communication Framework now uses "information" instead of "informational" as a sync error severity.

Mitigating impact

To ease the transition to FHIRcast-conformant severity, Subspace continues to recognize incoming sync errors using the "informational" severity. If Subspace receives this severity, it responds with an "informational" sync error of its own. However, Subspace only echoes the "informational" severity when it originates in the external system; any sync errors that originate in Hyperspace use "information" severity. Support for echoing the "informational" severity will be removed in Hyperdrive major version 100.2610.

Suggested next steps

* Evaluate the effect of this change for your applications. Ensure your app uses the "information" severity for sync errors instead of "informational".

Q-8017795 DocumentReference and Binary External CCDA resource IDs will now respect configured 64-character limits

This change will be introduced in the November 2025 version of Epic (August 2025 and May 2025 by special update). It applies only to clients that are configured to conform to this requirement, as noted in the "Constraining ID Length to 64 Characters in the R4 Version of FHIR" section of the FHIR Tutorial. For clients with that configuration the following resources are affected:

* DocumentReference.Read (STU3)

* DocumentReference.Search (STU3)

* DocumentReference.Read (External CCDAs) (R4)

* DocumentReference.Search (External CCDAs) (R4)

* Binary.Read (External CCDAs) (R4)

* Binary.Search (External CCDAs) (R4)

* Binary.Read (STU3)

Background

The HL7 FHIR standard requires IDs to be fewer than or equal to 64 characters. Previously even if clients were configured to observe the 64-character ID limit, the above DocumentReference and Binary resources could have received IDs from Epic that were longer than 64 characters.

What's Changed

Clients configured to observe this limit will no longer receive DocumentReference and Binary resources with IDs longer than 64 characters through the affected APIs listed above. To preserve existing integrations or for new integrations that rely on historical data, clients can still be configured to ignore this 64-character ID limit. Clients not observing this limit will continue to use the previous ID format that allows for more than 64 characters.

See Constraining ID Length to 64 Characters in the R4 Version of FHIR

Mitigating impact

Your use of the above DocumentReference and Binary resources (R4 and STU3) should account for IDs being present in both the previous format that could be greater than 64 characters and the new format that follows the 64-character limit as described in the HL7 FHIR specification as Epic organizations upgrade through the May 2025 version on varying timelines.

If your application has saved off FHIR IDs previously returned by the impacted resources, you may consider addressing this change through one of the following strategies or through another strategy determined by your team.

* Purge the saved FHIR IDs and re-query those resources

* Match old and new FHIR IDs on metadata such as author and date

Requests can still be made with previously received IDs that are longer than 64 characters, but resources will be returned with a new ID that respects the 64-character length limit.

Suggested next steps

* Evaluate the effect of this change for your applications

Q-7893893 FHIR / Observation (Labs) / FHIR ID generation for Observation (Labs) longer than 64 characters

This change will be introduced in the August 2025 version of Epic. It applies to apps that meet any of the following conditions:

* If your organization has any integrations that use the FHIR Observation (Labs) (STU3, R4) resource, you should inform the owners of those integrations that FHIR IDs for existing Observation (Labs) resources have changed and ask them to perform the build tasks.

Background

To align with the HL7 FHIR standard, applications can configure their application to respect the 64-character ID-length limit for all STU3 and R4 FHIR resources.

What's Changed:

Additional FHIR IDs for the Observation (Labs) (STU3, R4) resource are now constrained to 64 characters or fewer in order to conform to the FHIR specification. As such, historical FHIR IDs might no longer be valid after this change.

* Observation.Search (Labs) (R4)

* Observation.Read (Labs) (R4)

* Observation.Search (Labs) (STU3)

* Observation.Read (Labs) (STU3)

Mitigating impact

Your use of the R4 or STU3 Observation (Labs) FHIR resource should account for the FHIR ID length constraint of 64 characters as Epic organizations upgrade through the August 2025 version on varying timelines. It is recommended that you should use the Search interaction to query the relevant FHIR data instead of using the Read interaction. If you have cached FHIR IDs for this resource, you may need to re-query the customer's Epic system to retrieve FHIR IDs.

Update

In the November 2025 version of Epic (August 2025 by special update), this change will impact apps differently depending on whether they are configured to observe the 64-character ID limit as noted in the "Constraining ID Length to 64 Characters in the R4 Version of FHIR" section of the FHIR Tutorial.

Apps configured to observe the 64-character ID limit will see FHIR IDs for Observation (Labs) (STU3) and (R4) constrained to 64 characters or fewer, as described in the initial notification.

Additionally, the original FHIR ID, which may be greater than 64 characters in length, will be returned as an extension with the FHIR resource to allow apps to determine if the FHIR ID has changed. The extension URL will be http://open.epic.com/FHIR/StructureDefinition/extension/historical-fhir-id as shown in the example below:


```
"resource": {
    "resourceType": "Observation",
    "id": "f1jicnuG0x.ep.fR2BQ937qiv2ZvxrGcwN5PQ5Li1iKo4",
    "extension": [
        {
            "valueString": "94RRwn9jk44bL3wU5mF3aMMOV0dWt26NyFheBf1D7ApXOsr-OKhvn3n5ZUa.TGvr",
            "url": "http://open.epic.com/FHIR/StructureDefinition/extension/historical-fhir-id"
        }
    ]
}
```


Apps configured to use unconstrained FHIR IDs will not see the change described in the original notification if the customer has completed one of the following:

* Installed the special update in the August 2025 version of Epic OR

* Upgraded to November 2025 or a later version of Epic.

In either case, these apps will continue to receive FHIR IDs greater than 64 characters for Observation (Labs) (STU3) and (R4).

If a customer is using the August 2025 version of Epic and has not installed the special update, Observation (Labs) (STU3) and (R4) FHIR IDs will still be constrained to 64 characters or less.

Resolution - Action Needed by You

* Evaluate the effect of this change for your applications

1.4 App Default FHIR Version

File: site/docs/01-guides/04-appfhirversion.html

Source: https://fhir.epic.com/Documentation?docId=appfhirversion

An app’s FHIR version is selected during the app creation process and can be changed any time prior to activating an app. The FHIR version controls the behavior of different interoperability technologies, including:

* SMART on FHIR: Multiple pieces of the SMART on FHIR authentication workflow

* Authentication: OpenID Connect or Introspect for token inspection and user authentication

* CDS Hooks: Supported hooks and other features

* FHIR ID Generation Scheme: The control of FHIR ID length used in the app

Generally, an app FHIR version of STU3+ is recommended as those versions support functionality that the DSTU2 app FHIR version does not. Regardless of the version you choose though, you should test all relevant behaviors to ensure app compatibility with the selected FHIR version. Note that selecting an app FHIR version does not prevent you from using web services of other FHIR versions.

SMART on FHIR

The app’s FHIR version impacts multiple different behaviors during a SMART on FHIR launch, including:

* iss element

* JSON payload

iss Element

The value of the iss element sent during Step 1 of a SMART on FHIR launch is determined by the app's FHIR version, where the iss endpoint returned corresponds to the specific endpoint for that FHIR version on the server.

For example, if your app has a FHIR version of STU3 selected, then the server's STU3 FHIR endpoint will be returned as the iss value. In the sandbox, this would correspond to /interconnect-aocurprd-oauth/api/fhir/STU3.

JSON Payload

After successfully exchanging an authorization code for an access token, a JSON payload containing context about the app launch is returned. The JSON payload is impacted by the app FHIR version in the following ways:

* Encounter context is supported only in STU3+ app FHIR versions.

* Patient context returns a Patient FHIR ID of the version that matches the app's FHIR version.

Authentication

Epic offers multiple options for authenticating the user of a SMART app, each of which utilize the app's selected FHIR version to control how information about the authenticated user is returned. These affected methods include:

* OpenID Connect (OIDC)

* Introspect

OpenID Connect (OIDC)

In an OpenID Connect token, the app’s FHIR version determines whether or not the fhirUser claim included. The app’s FHIR version is respected for the sub claim beginning in the STU3 version.

fhirUser Element

When the app version is R4+ a fhirUser element can be returned when the fhirUser scope is included. This element returns an absolute reference to the authenticated user's FHIR ID.

Sub Element

The sub element will respect an app's FHIR version in STU3+ such that the FHIR reference returned in the sub element will match that of the app's FHIR version starting at an app FHIR version of STU3. While apps with a FHIR version of DSTU2 may still return a FHIR reference in the sub element, that reference will utilize an STU3 FHIR ID.

Introspect

Some integrations may utilize the Introspect web service to obtain information about the access token. When no epic_user_id_type parameter is specified in the Introspect request, the sub response element includes an absolute FHIR reference. That FHIR ID’s version will match the app’s specific FHIR version.

CDS Hooks

For a CDS Hooks integration, the app's FHIR version controls the following behaviors:

* Prefetch support

* Resource ID version

* Supported hooks

Prefetch Support

Prefetch is supported only in apps with a FHIR version of STU3 and above. Prefetch is not supported for DSTU2.

Resource ID Version

Some FHIR resources are returned in the CDS Hooks context. The app's FHIR version will determine if the resource IDs returned are DSTU2 or STU3+ FHIR IDs.

Supported Hooks

Some hooks are supported only in specific FHIR versions. For example, an app with a FHIR version of DSTU2 can use only the patient-view hook.

FHIR ID Generation Scheme

A new app’s FHIR version determines whether the FHIR ID Generation Scheme can be set. An app must have a FHIR version of STU3 or greater and use OAuth 2.0 or a FHIR API with a version of STU3 or greater to select one of the following:

* Use Unconstrained FHIR IDs

* Use 64-Character-Limited FHIR IDs for USCDI FHIR Resources

These settings do not have an effect for DSTU2 versions.

For historical reasons, some of Epic’s FHIR resources have identifiers that are longer than 64 characters. For example, Medication resource IDs sometimes exceed this length. Starting in the February 2022 version of Epic, new clients can be configured to respect the 64-character ID-length limit described in the HL7 FHIR standard for all STU3 and R4 version FHIR resources that are part of the USCDI data classes. To avoid disrupting existing integrations or new integrations that rely on historical data, this configuration is not enabled by default. You can set your client to observe the 64-character identifier length limit for USCDI resources by changing the FHIR ID Generation Scheme setting on the Build Apps page. To change this setting on an active app you can submit an email request.

Unconstrained FHIR IDs

Apps default to the unconstrained FHIR ID generation scheme. This means apps may use and may receive FHIR IDs longer than 64 characters.

64-Character-Limited FHIR IDs

The 64-Character-Limited FHIR ID option can be selected to ensure all FHIR IDs will be 64 characters or less for USCDI resources. This length limit will apply to both the production and non-production client IDs for your application. For cases where an ID was generated to be longer than 64 characters most versions of Epic have an alternate ID that represents the same resource but doesn’t exceed the 64-character limit.

1.5 App Creation & Request Process Overview

File: site/docs/01-guides/05-epiconfhirrequestprocess.html
Source: https://fhir.epic.com/Documentation?docId=epiconfhirrequestprocess

Table of Contents

* Overview

* Creating, Activating, and Licensing an App

* Registering an App

* Filling out the App Questionnaire

* Marking an App as Ready for Production

* Facilitating Epic Community Member Downloads

* Managing Customer Specific Configuration

* Requesting an App

* Before Starting the Request Process

* Obtaining the open.epic API Subscription Agreement

* Downloading Records

* Confirm the Client ID was Downloaded

* Deactivate an App

Overview

Epic on FHIR enables Epic community members to download client records for applications registered on the Epic on FHIR website. If the app is in Showroom customers can find the app by doing a keyword search for the app or vendor name. In order to download client records, each Epic community member must have someone with the "Purchase Apps" security point to download an app's client record on behalf of the organization.

The steps below describe the process of:

* The app registration, creation, and activation process.

* An Epic community member signing the open.epic API Subscription Agreement.

* The community member downloading an Epic on FHIR app.

Creating, Activating, and Licensing an App

Registering an App

When a developer registers an app, the website creates an app record in the Epic database and assigns the app production and non-production client IDs. To register an app:

* Navigate to the Build Apps page

* Select "Create My First App" or if you already have apps created, select "Create"

* Complete the Create an App form by filling out:

* The app name

* The app consumer type

* Applicable APIs

* The default FHIR version

* Applicable redirect URIs

* Whether you'll use the confidential client profile (required for refresh tokens)

* Save

After the app has been Saved and Marked Ready for Sandbox, it will be in a Draft status. When a new draft app is made or changes are made to the app on the website, the changes may take up to 1 hour to sync with the Sandbox. To get started on testing, we recommend reviewing the Developer Testing Guide. For any issues that arise in testing, refer to our Troubleshooting tutorial

Recommendations for Scoping APIs to your App

When you design your app’s API scope, less can be more. Scoping too broadly raises red flags for healthcare organizations reviewing your app and for patients authorizing data access. Because your app is authorized to use all the APIs on your client ID, that list directly defines the types of patient data a customer could expose to your app.

Customers will closely review that list to understand the breadth of data your app could access. Because of their legal and regulatory obligations to protect patient privacy, security officers and privacy reviewers are obligated to protect that data, and may delay or reject apps that request more access than they clearly need. Patients, too, may hesitate to authorize apps that appear to overreach. If your app is authorized for APIs that it doesn’t use or doesn’t need for its use cases, organizations may ask you to create a new client ID that only includes the necessary APIs. Keeping your scope tightly aligned to your app’s actual use cases builds trust and helps your app move through customer review and adoption more smoothly.

Because Epic’s industry-standard and Public APIs are licensed directly to healthcare organizations (not to app developers), broad API scopes can also make your app appear more expensive to enable than it actually is. While fees are based on production usage, not on your client ID scope, the scope determines which APIs are available to your app and which licenses your customers may need. Even small amounts of over-scoping can create confusion about required licenses and perceived costs, slowing down customer decisions.

Epic recommends that healthcare organizations proactively request a licensing estimate and complete a security review based on your client ID’s scope. This ensures they consider both privacy and cost implications of the data your app could request.

If the scope of your integration expands in the future, you can always create a new client ID to enable additional APIs. Starting narrow helps customers onboard faster and simplifies their review.

Best Practices for API Scoping

* Scope narrowly and intentionally. Only include the APIs that are essential for your app’s functionality.

* Avoid “just in case” APIs. Extra APIs can create confusion during customer review and inflate the perceived cost of your app.

* Map each API to a clear use case. Be ready to explain why each API is required.

* Plan for growth. You can always expand your scope later if your app’s functionality grows.

Examples

* Some Medication FHIR APIs are part of the USCDI group, which most healthcare organizations already license. If you add additional Medication APIs outside of USCDI, your customer may need to license more APIs, increasing their costs.

* Epic supports many DocumentReference APIs, each with specific use cases. Scoping many or all of them is almost never necessary. If your app uses DocumentReference.Search, make sure it targets only the APIs you truly need.

Recommendations for Creating your App Listing

When you first create your app, customers can search for the app by client ID on the Epic on FHIR Download page and view your application name, summary, and thumbnail. If you choose to list your app on Connection Hub once you have your first

live customer, there will be additional marketing fields that you may choose to fill out, which are described below.

Application Name

This is the name of your product. Keep your app name consistent throughout your app listing. Don't include "Epic" in your app name. Don't include any trademarks from other organizations without permission.

Public Documentation URL

This should be a public web page about your product. If you mention Epic in your public documentation, be sure to follow our Trademark Usage Guidelines.

Summary

The best summaries are 2-3 sentences and provide a high-level explanation of the product. The summary is meant to catch the reader's interest, so avoid including too much detail here. You can go into more detail in the description.

The summary can be no longer than 500 characters, including spaces.

Description

The description should give the reader a clear understanding of what the product does and how it integrates with Epic. The best descriptions follow a Why, What, How format – Why a customer should want to use this product, What the product does, and How the product integrates with Epic. Consider splitting your description into multiple short paragraphs or using bullet points to improve readability.

The description can be no longer than 1,998 characters (including spaces).

Data Use Questionnaire

We highly encourage developers to fill out the Data Use Questionnaire if the app is Patient-facing to benefit mutual Customers of Developers and Epic. The answers you provide to the Data Use Questionnaire are displayed to a patient during the app authorization flow, The questionnaire responses can help the patient make an informed decision about whether they want to grant your app access to their data based on the data use practices of the app. Failure to answer the DUQ will result in a warning being shown to the patient when they are prompted to authorize your app.

Thumbnail

This should be the logo for your organization or product. The recommended image dimensions are 500 x 300.

Screenshots

This is your opportunity to show viewers what your app looks like and how it works. Consider adding captions to your images to explain the main purpose of the image. If your product does not have a user interface, consider including a data flow diagram describing your integration.

The screenshot height is 1080 pixels, and the recommended dimensions are 1920 x 1080. You can include up to 10 screenshots.

Filling out the App Questionnaire

We highly encourage developers to fill out the App Questionnaire document and provide it to their prospective and installing customers. This questionnaire covers common questions customers ask such as how your app connects to Epic, security practices, patient safety, scalability, and more. Answering these questions helps instill Customers with confidence that Apps they choose are created in a safe and sustainable manner. Epic Customers use these to inform procurement and security evaluations prior to installing the app.

Marking an App as Ready for Production

After completing development and testing, you can mark your app Ready for Production use. The app cannot be used in any community member environments, either production or non-production, until the app has been marked Ready for Production. To mark your application Ready for Production:

* Navigate to the Build Apps page
* Select the app that will be activated
* Finalize details about the app
* Check the box to confirm compliance with the Terms and Conditions
* Click Save and Ready for Production

IMPORTANT! Neither you nor Epic can make updates to an app once it has been marked Ready for Production. With the exception of adding redirect URIs and modifying your JWK Set URLs, all technical changes will require you to build a new app.

Whether or not an app will auto-sync depends on whether it meets all auto-sync criteria. Apps that need to register client secrets or public keys that otherwise meet the auto-sync criteria will sync after a client secret or public key is provisioned to that organization via the "Review & Manage Downloads" button on the Build Apps tab. For all other apps, refer to the Facilitating Epic Community Member Downloads topic below.

Facilitating Epic Community Member Downloads

To allow an Epic community member to download an app, simply provide them with the Production or Non-Production Client ID from the app's detail page. Follow the steps in Managing Customer Specific Configuration to add any customer specific configurations when a community member requests the app.

Managing Customer Specific Configuration

When licensing your application to customers and depending on the configuration of your app, you will have options to specify customer configurations (e.g., redirect URLs and client credentials). To manage these settings:

* Navigate to the Build Apps page.

* Select Review and Manage Downloads for the relevant application.

* Select the relevant environment type. Note that you need to activate for Non-Production before you can activate for Production.

* Specify the necessary configurations following the recommendations below.

* Click Activate.

Customer Specific Endpoint URIs

If your app uses clinician- or patient-facing OAuth 2.0, you also have the option of specifying customer specific endpoint/redirect URIs when managing a download request. If you don't want to use the default URIs listed on your app, simply uncheck "Use app-level endpoint URIs" and enter the customer-specific URIs your app needs.

Provisioning Client Credentials

For backend apps or frontend confidential client apps, Epic recommends using a JWK Set URL (JKU) but you may upload a public key or client secret (frontend confidential client apps only). For apps that qualify for auto-synchronization functionality, a client credential must be set before the app can sync to the Epic community member's environment. For apps that do not meet auto-sync criteria, a client credential can be added once the community member has requested the app.

If using client secrets, you can use a client secret generated by the website or you can provide your own client secret. If using a public key, it should be exported to a base64 encoded X.509 certificate before being uploaded. Frontend confidential client apps must upload the key as a JSON Web Key Set (JWKS).

Epic requires JKUs for backend OAuth 2.0 apps following the timelines outlined on the OAuth 2.0 Tutorial document.

Epic recommends that you use a unique client credential for each community member and for production versus non-production.

Requesting an App

Before Starting the Request Process

Epic community members should do the following prior to the request process:

* Work closely with their Epic technical coordinator (TC) to research available integrations.

* Ensure the right Epic products and interfaces are in place to install an app.

* Evaluate potential costs of the app up front. In addition to developer's fees, consider other third-party software, hardware, and content costs, new interfaces, and additional license or subscription volume that may be triggered by the application.

When an Epic community member has committed to moving forward with an app registered on the Epic on FHIR website, they or the developer should organize a kickoff call to align key stakeholders on the goals, scope, processes, milestones, and timeline of the project. Key stakeholders include representatives from all three organizations: the developer, the Epic community member (e.g., operational sponsor, project manager, analyst, ECSA, network engineer), and Epic (e.g., the technical coordinator, TS who support affected applications, and, if applicable, the EDI representative). If you are not sure who from Epic to include, ask your Epic representative for assistance.

Including the right stakeholders from each organization at the start of the project enhances communication, sets the right expectations upfront, and helps identify and avoid potential issues before they impact the success of the project.

Epic does not endorse, certify, or verify the integrity, safety, performance, or practices of the developers who use Epic on FHIR or their software.

Obtaining the open.epic API Subscription Agreement

Community members who wish to use FHIR APIs with a third-party application registered on the Epic on FHIR website must sign the open.epic API Subscription Agreement. This agreement applies to the organization, not to individual apps, so this step is only required for organizations who have not previously integrated with an application that uses technology licensed under this agreement.

Downloading Client Records

To be able to download a client record, the following steps must have been completed:

* The developer has created an account on the Epic on FHIR website,

* The developer has registered their product on the Epic on FHIR website,

* The developer has marked their product Ready for Production use ("Production Ready" status), and

* The community member has signed the open.epic API Subscription Agreement.

From the Downloads page, community members can see apps which their organization has previously downloaded from the Epic on FHIR website. To download client records for additional apps, the community member's staff with the "Purchase Apps" security point can search using the app’s client ID, which must be obtained from the developer. Either the Production or Non-Production Client ID can be used.

The website will provide details about the app associated with the client ID and give the option to proceed with the download. If the app uses refresh tokens, the duration of the refresh token can also be specified at this time.

Any apps using backend OAuth 2.0 or the Confidential Client profile will need a JWT public key and/or a client secret uploaded to the website after you request a client ID download. This information is provided and uploaded by the app developer.

Confirm the Client ID was Downloaded

To confirm the client record was successfully downloaded, return to the Downloads page. The application should now appear on this page along with a request status. Apps using the confidential client profile or backend OAuth 2.0 will require app developer action. All other applications should have their download approved automatically. This download syncs the app's client record to the community member's Epic environments, allowing APIs listed on that client to be authorized by their server.

Epic community members can see the "Notes for Epic Customers" section at the top of the page for details on how to verify that a client record now exists in their Epic environments.

Deactivate an App

Apps that have not yet been deployed to any Production environment can be deactivated by the app developer by clicking the “Deactivate” button on the app's build page. Apps that have been activated for only a customer's Non-Production environment can be deactivated by the app developer in for that customer on the page where customer licenses are managed.

Once an app has been deployed to any Production environment, it cannot be deactivated. App developers must coordinate with customers live on the app to ensure it is safely uninstalled from workflows and then the customer can deactivate the app's client ID directly within their environments. Epic's recommendation is that app developers create their software to not exclusively rely on the client ID being disabled within customer environments to prevent unintentional or unexpected usage. This is particularly relevant for patient-facing applications that qualify for automatic client record distribution where coordination with individual customers is unrealistic.

1.6 App Developer Guidelines

File: site/docs/01-guides/06-developerguidelines.html

Source: https://fhir.epic.com/Documentation?docId=developerguidelines

Overview

As an app developer, it is important to be familiar with and practice principles for responsible healthcare app development. If your app connects to Epic, you and your apps are strongly encouraged to practice the App Developer Guidelines, detailed below. If you have any questions about these guidelines, please contact us at open.epic.com/Home/Contact.

App Developer Guidelines

Epic recommends apps adhere to the following principles:

* Safety. The app does not put patients or others at risk of harm.

* Security. The app does not introduce security vulnerabilities, cause security breaches, or increase the risk in either end users’ or Epic Community Members’ systems.

* Privacy & Data Use. The app respects the privacy of patients and their families, clinicians, other end users, and everyone else.

* Reliability. The app behaves as expected, delivering the right data at the right time. It is consistent and predictable.

* Scalability. The app is stable and does not negatively impact operations for users or Epic Community Members. The app performs as expected at scale and does not generate excessive or unexpected load on a user’s or Epic Community Member’s system.

* Data Integrity. The app does not corrupt or otherwise cause material inconsistencies in end users’ or Epic Community Members’ data(d)(2).

* System Integrity. The app does not cause the end users’ or Epic Community Members’ other systems to behave inaccurately or unexpectedly.

* Transparency & Honesty. The app and developer do not misrepresent products, product capabilities, business relationships, timelines, or anything else related to Epic or open.epic in any way.

* Intellectual Property. The app and developer protect the intellectual property of the developer, Epic, and the Epic Community Member.

1. Safety

* The app reasonably adheres to usability standards that impact patient safety, specifically safety-enhanced design and accessibility-centered design. Examples of usability standards and tools to consider include ISO 9241, NISTIR 7804-1, and those available on digital.gov.

Usability.

(g)(3)

(g)(5)

* The app incorporates Epic Community Member feedback into product design decisions to fit the app into the user's workflow as smoothly as possible.

* If the app allows an end user to search for a patient record, the app collects enough information from the end user to uniquely identify the correct patient record or displays enough information to the end user to enable the end user to uniquely identify the correct patient record.

Patient Search.

(g)(7)

* Epic strongly recommends apps require :

all of the following 3+1 demographics criteria

* Patient First and Last Name,

* Legal Sex,

* Date of Birth, and

* At least one additional identifier such as MRN, address, phone number, or email.

* Refer to the Patient Searching and Matching tutorial for guidance on how best to implement patient searching or matching for a given app type.

* Patient Identity. If the app is launched within an Epic patient workspace for patient-specific workflows, the app maintains the patient identity within the session. If the app is launched separately for patient-specific workflows, the app prominently displays the patient's identifying information to avoid any confusion by the end user about which patient's record they are interacting with.

* Accurate Display. If the app displays patient data (e.g., medications, observations, conditions), the app does so accurately.

* The app presents data in such a way that the end user is reasonably able to complete workflows without being required to remember information while jumping between screens/displays. Such a design may contribute to errors when making clinical decisions.

Data Separation.

* The app ensures the user can visually confirm they are accessing the correct patient record.

* Updated Data. The app provides users with a way to see when data was last updated so they may recognize if there may be new information they are not yet seeing.

* TALLman. If the app assists clinicians with patient-care decisions, the app displays any medications from ISMP's list of commonly confused drug names using TALLman lettering.

* If the app permits an end user to select options from a list, then the app makes obvious to end users that they are selecting from a list. Additionally:

Selection Lists.

* If the app displays only a portion of the list from which the end user may select, then the app makes obvious to the end user that additional options are available. This helps prevent end users from selecting a less desirable option from a shorter list than should have been considered or making decisions with incomplete information.

* Lists do not include irrelevant options that, if selected, could have negative outcomes.

* Consistency. The app uses a consistent user interface so end users know what to expect when moving between screens.

* Transparent Calculations. The app makes obvious to the end user whenever it stores or displays data that does not match what the end user entered (e.g., automatically converting weight from pounds to kilograms) or the factors that influence a calculation and how it should be interpreted.

* Reviewing and Correcting Data Entry. The app allows end users to review data that they enter into the app and make corrections as needed.

* Abbreviations. The app uses abbreviations appropriately and avoids those that could be misinterpreted or easily mistyped. (E.g., the app does not use "l" or "k" instead of "lb" or "kg" for weight entry. The "l" and "k" keys are next to each other on the keyboard.)

* The app expects that end users or systems will act in error and gracefully manages errors, guiding end users to appropriate error resolution. Developers should review the list of possible errors and faults included in the documentation for each API and verify that the app is coded to appropriately handle those error conditions, including scenarios that should be theoretically impossible. In addition to the error conditions listed in the API documentation, ensure the app can handle any HTTP errors it may encounter as well. To the extent necessary to avoid unsafe circumstances:

Manage Errors Gracefully.

* The app notifies end users of errors or potential errors that impact the proper or safe use of the app (e.g., if the data entered by an end user is unrealistic regarding magnitude, format, data type, etc., then the app notifies the end user of the potential error).

* The app notifies the right end user of the error or potential error at the right time in the right workflow.

* The app explains errors or potential errors to the end user in a clearly understandable manner (e.g., not an overly technical explanation).

* The app displays errors or potential errors in an internally consistent manner. For example, the app uses colors and symbols in error or potential error messages consistently.

* The app explains actionable steps that the end user may take to address the error or potential error.

* The app does not permit the end user to remain in a problematic error state.

* The app is designed with the expectation that end users or systems will experience interruptions and gracefully manages such interruptions. To the extent necessary to avoid unsafe circumstances:

Manage Interruptions Gracefully.

* The app manages workflow and connection interruptions, handling partially completed workflows or data entry in a manner that minimizes end user or system errors or potential errors.

* If the app may be accessed or viewed during a loss of connection and could display information that relies on the connection for updates, accuracy, or completeness, the app notifies the end user that the data displayed may be out of date.

* Critical Issues. The developer maintains processes with appropriately urgent timelines for addressing and notifying Epic Community Members of critical issues, such as potential patient safety concerns.

2. Security

* The app encrypts data stored in non-volatile storage using industry-standard encryption algorithms, such as AES-128 or higher. If encryption keys must be written to disk, then:

Protect Data at Rest.

* The app makes appropriate use of key storage functionality provided by the host platform/operating system.

* The app requires the administrative user to authenticate to gain access to the encryption keys.

* Protect Data in Transit. The app supports securing and sending/receiving data secured with the TLS 1.2 or higher encryption protocol. The app verifies that the server’s TLS certificate is valid and signed by a certificate authorized, recognized, and trusted by the host operating system. Data exchange between your app and Epic’s APIs and between your app and any other system should be secured with industry standard encryption while in transit(d)(9) and use authentication and authorization protocols(d)(1).

* The app does not request or store more protected health information than needed.

Minimize Use of Protected Health Information.

* The app is scoped for only the APIs required for the integration. Apps should not be scoped for unused APIs as this unnecessarily increases the amount of data that may be exposed to the app.

* The app supports single sign-on when appropriate. Best practice for authentication and authorizing access is to use OAuth 2.0. Use of other authentication mechanisms such as SAML, HTTP basic authentication, mutual TLS with client credentials or encrypted query strings with shared keys may be appropriate in limited scenarios but should be considered on a case-by-case basis. Other authentication mechanisms typically do not support fine-grained resource authorization, so that should be considered. The app complies with best practices related to the invalidation/rotation of security tokens and inactivity timeouts. For more guidance regarding OAuth 2.0 and Epic's recommended practices for implementing OAuth 2.0, review the OAuth 2.0 resources below:

Single Sign-On and Authentication.

* OAuth 2.0 Specification - This document is a technical specification for using OAuth 2.0 with Epic.

* OAuth 2.0 Tutorial - This tutorial provides guidance for defining your OAuth app architecture.

* The app properly incorporates its assigned client ID into all API calls and uses of other technologies that support client IDs.

Client IDs.

* The app keeps client identifiers confidential and does not disclose them to any other party or use them for any other purpose.

* Each app has its own unique client identifier.

* Apps that are confidential clients and using refresh tokens keep their client secret safe.

* Handling Authentication Credentials. The app protects authentication credentials provided by Epic or Epic Community Members, such as usernames, passwords, access tokens, or refresh tokens. Credentials used to access Community Members’ Epic software should be unique to each Epic Community Member and should only be used in conjunction with Epic APIs. If the app needs to persist this data to non-volatile storage, it uses approved secure storage functionality provided by the host platform/operating system.

* Handling User Credentials and Other Sensitive Data. If the app prompts the user for credentials, biometric data, or other sensitive information such as personally identifiable information, credit card numbers, or financial account information, the

app protects this data. If the data must be written to non-volatile storage, it should never be stored in clear text. The app makes use of one-way hashing algorithms such as SHA-256 or secure storage functionality provided by the host platform/operating system to persist this data. The app should secure all data on an end-user's device(d)(7)(d)(8) and enforce inactivity time-outs (d)(5).

* Software Implementation of Cryptography and Authentication. For performing low-level cryptography and authentication operations, the app leverages well-known, established software packages that are actively maintained and have published processes for reporting and responding to security vulnerabilities. Developers are strongly discouraged from implementing custom cryptography or authentication functionality.

* The app accounts for commonly known security vulnerabilities, such as those described in and addresses any applicable vulnerabilities. For example:

Addresses Security Risks.

OWASP Top 10

* All data entered by an end user should be strictly validated or escaped prior to rendering, passing to a database, or sending to an external API. Prior to invoking an API, the App validates that all user-supplied input values conform to the expected format described in the API documentation. The app takes reasonable steps to prevent a user from accidentally or purposefully entering obviously invalid data to be passed to an API.

* The app employs standard protective measures against cross-site scripting attacks.

* The app uses PHI-safe, production-like protections in all customer environments, even in non-production testing environments. Any connection to a customer's Epic environment, even non-production environments like POC and TST, has the potential to access PHI.

PHI-safe Non-Production Customer Connections.

* HIPAA: Non-production customer connections to the app should meet data handling requirements for HIPAA.

* Access: Control and provisioning of access to customer non-production connections should be handled the same as production connections.

* Code and testing: The developer should have the same requirements for development and testing of code before it is deployed to both non-production and production customer connections.

* Change control: Change control, deployment, and release should be handled similarly between non-production and production, including access to introduce changes and deploy changes.

3. Privacy and Data Use

* Data Use Questionnaire. For a patient app, the developer has clearly, accurately, and completely responded to the Data Use Questionnaire. If the responses to the Data Use Questionnaire become out of date or inaccurate, then the Developer promptly submits updated and accurate responses by creating and distributing a new app.

* The developer provides patient users and Epic Community Members with a clear and understandable data use or privacy policy that includes, at minimum, an accurate and truthful explanation of the following:

Data Use Policy.

* Any data elements that the developer or app obtains from or writes to a Community Member's Epic environment

* How and where the developer or app uses data

* Whether the developer or app retains data

* The length of time that the developer or app retains data

* Any secondary uses of data, including transfer of data to a third party or third-party product, or any access to data provided by the developer or app to any third party or third-party product

Obtain Consent. The developer obtains consent from patient users and Epic Community Members to obtain, write, or retain data and for any secondary use of or access to data, including transfer to or use by a third party or third-party product. The Developer obtains updated consent to obtain, write, or retain additional data and for additional secondary uses. The developer does not circumvent the display of any authentication or authorization mechanisms from Epic or any Epic Community Member.

A patient app provides patient users with options to give such consent, decline such consent and, if such consent has been given, to revoke such consent. If a patient app establishes a persistent connection to a Community Member's Epic Environment on behalf of a patient user (e.g., by use of a refresh token), then the developer has a HIPAA-compliant Business Associate Agreement with the Epic Community Member, or the app informs the patient user of the period of time for which the app is seeking authorization and obtains the patient user's consent to access, obtain, and use data for that period.

* The developer accesses and uses data appropriately, as described below.

Appropriate Use.

* The developer and app do not provide data or access to data to any third party or third-party product without consent from patient users and Epic Community Members.

* The developer and app do not use data for direct-to-consumer marketing or advertising without consent from patient users and Epic Community Members.

* The developer and app do not sell data to third parties.

* The developer and app may access only the data described in the developer's data use or privacy policy, discussed above, and agreed to by the patient user and Epic Community Members.

* The developer and app use data only for the purposes described in the developer's data use or privacy policy, discussed above, and agreed to by the patient user and Epic Community Members.

* Record Use. When the developer or app uses, retains, or distributes data, the developer or app documents who used, retained, or distributed what data, when they did so, and for what purpose they did so. The developer or app provides the patient user and Epic Community Member access to that documentation upon request. The developer does not prohibit Epic Community Members from sharing that documentation with Epic.

* Cease Use, Delete or Correct Data. The developer or app provides a method for patient users and Epic Community Members to stop the app from reading, writing, or retaining data and to delete or correct retained data. The developer or app provides clear and prominent notice to patient users and Epic Community Members of how to use these methods.

Note: Obligations related to patient users only apply if the app is a patient app.

4. Reliability

* Stability. Software must work. The app must not crash, hang, or lead to exhaustion of compute resources under typical use.

* Predictable behavior. Given the same inputs, the app yields consistent results, with no unexplained variability.

* No surprises. The app avoids "silent failures". Every significant action leaves an observable trail (logs, audit records, or UI confirmation).

* Version consistency. Each release behaves as documented, with backward compatibility for APIs or interfaces unless deprecation is clearly communicated in advance.

* Monitoring & alerting. Apps include health checks, heartbeat monitoring, and robust logging. Developers work with Epic Community Members to monitor impact on the Community Member's system.

* Business Continuity. Developers have redundancies in place such that users experience less than 1% downtime. Developers offer resources to mitigate the impact of disruptions to their service.

5. Scalability

* The app consumes no more than 1% of overall Epic operational database (ODB) system resources during peak hours and consumes no more than 5% of overall ODB resources during non-peak hours. Peak hours are typically 7:00 am to 7:00 pm local time, though such times may vary by Community Member.

System Impact.

* If the app makes a significant number of API calls, the developer or app should provide programmatic guardrails and be able to monitor and throttle API call volumes if necessary to mitigate system load impact.

* If the app is embedded within Epic user workflows, then the app has an average initial load response time (i.e., the amount of time from when the user first launches the app until it is ready for user interaction) of 1.5 seconds or less. If the app loads incrementally, then the app can be considered ready for interaction before completely loading, if the user can reasonably interact with the app at that time. If the app does not meet this threshold, then the app:

Reasonable Initial Load Response Time.

* Shows a loading indicator with its specific branding while loading.

* Has an average initial load response time of 3 seconds or less.

* The app's load response times are not consistently greater than 5 seconds.

* The documentation for the app clearly and prominently discloses the app's expected performance impact to the Epic Community Member prior to planning a production go-live date.

Document Expected Performance Impact.

* If the developer expects an increase in API traffic, they work with the Epic Community Member to review the potential impact and ensure the Epic Community Member system is prepared for the change.

For guidance on creating a performant app please see Designing an Efficient Application.

6. Data Integrity

* Recommended Configuration. In order to reduce the likelihood of adverse data integrity issues, the app conforms to all interface design and configuration recommendations provided by Epic.

* The app uses only appropriate, as determined by Epic, industry-standard or Epic-published methods to obtain data from or write data to a Community Member's Epic Environment.

Appropriate Methods of Data Exchange.

* Writing data to a Community Member's Epic Environment must be done in a manner that does not adversely affect the Community Member's Epic Environment, and in some cases, there may be no such appropriate method.

* Software that reads data from or writes data to a user interface (i.e., risky robotic process automation) is highly susceptible to error, including because of normal software upgrades or configuration updates.

* Backward Compatibility. The app has backward compatibility when features are updated that could still reference older data structures.

7. System Integrity

* The app appropriately manages users' assumptions and expectations of healthcare software to avoid negative outcomes. For example:

Manage User Assumptions and Expectations.

* If the app permits users to input data that is typically input and stored in Epic software, or if the app uses an API to look up data from Epic software and permits users to edit that data, then users are likely to assume or expect that the app will file the new or updated data to Epic.

* If the app facilitates part of a workflow, then users are likely to assume or expect the app to trigger downstream steps in the workflow (e.g., trigger a message to another end user, trigger an interface message, etc.).

* Imitation. The app does not imitate, bypass, or interfere with third-party products or Epic software in a way that may confuse users or lead users to believe that the app is another product or has the capabilities of another product when it does not.

* Testing in Non-Production. The app is thoroughly tested in a non-production environment before going live in an Epic Community Member's production environment. Both expected workflows and edge cases are tested.

* Advertising. The app does not display advertisements of any nature within any Epic software user interface.

8. Transparency & Honesty

* Relationship with Epic. The developer accurately represents the app and developer's relationship with Epic or Epic Community Members.

* App Status. The developer accurately represents the app and developer's status in any Epic process including the app or developer's status in the sales cycle or implementation with any Epic Community Member.

* The developer accurately represents the app's features and functionality, including:

App Functionality.

* The developer's progress designing, developing, testing, or enhancing the app.

* The app's cleared or approved use under any applicable regulations (e.g., FDA clearance).

* The data that the app accesses from Epic systems, how the app or developer uses that data, any secondary uses that the app or developer make of the data, and any access to that data that the app or developer provide to other parties or products.

* The app and developer accurately represent Epic products and their features and functionality including:

Relation to Epic Products.

* Similar functionality that Epic's products have to the app.

* The app's and Epic products' abilities to interface with each other.

* Cost. The app and developer accurately represent the apps' total cost to the Epic Community Member or end user, including all license/sales, maintenance, subscription, implementation, training, hardware, and other fees.

* Accuracy. The app and developer do not otherwise misrepresent products, product capabilities, business relationships, timelines, or anything else related to Epic not specified above.

9. Intellectual Property

* Replication. The app does not copy source code, data structures, or user interfaces from another app or Epic product without the owner's prior written permission.

* Third-Party Use. The app does not provide third parties or third-party products direct or indirect access to or use of the Epic Materials, Epic services, or Epic software.

* Third-Party Content Flag. The Epic Community shares clinical and operational content by periodically uploading content configuration to Epic's Community Library. If the app includes content to be imported into an Epic environment, then the content import file appropriately marks proprietary content for exclusion from the Community Library uploads.

Footnotes

45 CFR 170.315 (d)(1) (Authentication, Access Control, Authorization): "Verify against a unique identifier(s) (e.g., username or number) that a user seeking access to electronic health information is the one claimed; and [...] establish the type of access to electronic health information a user is permitted based on the unique identifier(s) provided".

45 CFR 170.315 (d)(2) (Auditable Events and Tamper-resistance): "The health IT records actions pertaining to electronic health information [...] when health IT is in use; changes to user privileges when health IT is in use; and records the date and time [each action occurs]. [...] The health IT records the audit log status [...] when the audit log status is changed and records the date and time each action occurs. [...] The health IT records the information [...] when the encryption status of locally stored electronic health information on end-user devices is changed and records the date and time each action occurs."

45 CFR 170.315 (d)(5) (Automatic Access Time-out): "Automatically stop user access to health information after a predetermined period of inactivity. [...] Require user authentication in order to resume or regain the access that was stopped."

45 CFR 170.315 (d)(7) (End-user Device Encryption): “Technology that is designed to locally store electronic health information on end-user devices must encrypt the electronic health information stored on such devices after use of the technology on those devices stops [or] technology is designed to prevent electronic health information from being locally stored on end-user devices after use of the technology on those devices stops.”

45 CFR 170.315 (d)(8) (Integrity): “Verify [...] upon receipt of electronically exchanged health information that such information has not been altered.”

45 CFR 170.315 (d)(9) (Trusted Connection): “Health IT needs to provide a level of trusted connection using either 1) encrypted and integrity message protection or 2) a trusted connection for transport.”

45 CFR 170.315 (g)(3) (Safety-enhanced Design): “User-centered design processes must be applied to each capability technology.”

45 CFR 170.315 (g)(5) (Accessibility-centered Design): “The use of a health IT accessibility-centered design standard or law in the development, testing, implementation and maintenance of that capability must be identified.”

45 CFR 170.315 (g)(7) (Application Access - Patient Selection): “The technology must be able to receive a request with sufficient information to uniquely identify a patient and return an ID or other token that can be used by an application to subsequently execute requests for that patient’s data.”

1.7 Collaborate with Epic Community Members

File: site/docs/01-guides/07-eofcollaborate.html

Source: https://fhir.epic.com/Documentation?docId=eofcollaborate

Strategies to Effectively Collaborate with Epic Community Members

Healthcare software implementations are large, complex projects. Healthcare organizations demand robust software solutions that can integrate seamlessly with their existing operations and IT infrastructure.

Even across organizations on the same version of Epic, the exact requirements demanded by operations across the two sites may differ significantly, so project scoping is an integral part of any new implementation.

In addition to demanding usability requirements, you also must keep data protection and security at the forefront of your product development due to working with sensitive patient health information.

This document provides guidance on how to prepare for and collaborate with all stakeholders at your prospective Epic Community Member. There are three main groups you’ll work with during these projects:

* Operations: The stakeholders who directly represent end users. Often department leads, super users, and clinical representatives like physician/nurse champions.

* IT: Primary technical support for Epic end users. This is commonly Epic certified analysts who work directly for the organization.

* Epic Support: Epic Technical and Implementation Services staff from Epic who support the IT staff from the organization.

Understanding Epic Community Members

Our organizations have a deep partnership between operations, IT, and their Epic support. They take ownership of their Epic software deployment, and IT collaborates with operations on project intake, prioritization, and implementation timelines to configure the organization’s Epic instance to meet users’ needs. No project can be successfully implemented without buy-in and approvals from both groups.

It’s important to understand that Epic customers are not all cloud-hosted on one single database or version. Even customers who are hosted by Epic are each on their own independent instance. This means organizations do not all upgrade at the same time. When you see an Epic version labeled by a Month + Year (e.g., August 2025), that indicates Epic’s release date for that version, not when every customer will be running it. In practice, customers set their own production upgrade timelines, often 3 to 12 months (sometimes more) after Epic's release date.

The organization’s IT team has dedicated staff to support each Epic application and all infrastructure that goes into the deployment. Each of those staff has dedicated Epic Technical Services (TS) Support who can provide advanced assistance if needed.

Any new healthcare software product that involves Epic will need to involve representatives from potentially multiple areas across IT and operations. Their involvement is critical to successfully implementing changes to end user workflows, especially those involving integrations between Epic and your software.

From Concept to First Customer

Operations will know how they use Epic today and will be a key voice in many workflow design decisions. Effective engagement with operations is predicated on understanding the day-to-day functions and workflows of staff. You should focus on how your software will streamline these workflows, reduce administrative burden, or enhance the delivery of care. For example, where do they want to display your information in Epic? Or when will the information they already document in Epic flow over to you? It’s important to keep in mind that some of these decisions may be specific to this first organization. Design your integration with enough flexibility to tackle future integrations as well.

IT knows their Epic system, their project priorities, and how your integration may fit in. Epic has many integration points and a wide array of technologies that could be used in a new software integration, so integration design must be a collaborative

discussion. Experts from IT's integration team can help you pick the right integration technologies and overall design to meet your project goals, like FHIR or HL7v2 interfaces. If needed, they'll also loop in their Epic support for additional help to scope out workflow and technical integration options.

After technology and workflow decisions are complete, your development and implementation teams should familiarize themselves with the technologies and integration points to prepare for proof of concepting. Be aware of resources like specifications and sandbox environments on open.epic at your team's disposal. When you're ready, work with IT and operational stakeholders to set realistic proof of concepting, validation, and implementation timelines given the scope of the project, required resources, and resource availability.

Implementation, Ongoing Maintenance, and Version Upgrades

When you're ready to kick off the install, you'll engage with IT to begin configuring Epic to integrate with your product. Epic provides materials and support to help IT set up integration technologies. Continue to pay attention to difficult areas of the install that may serve as good lessons learned to note down in documentation for your future projects.

Be sure users are prepared to go live with your software integration. This may include user validation sessions to ensure your design still resonates with users or training sessions, so users are prepared to use your software.

Ongoing support and open lines of communication post-implementation are key to resolving any issues swiftly. Be prepared to re-engage IT and operations in preparation for any new version upgrades that you may roll out and be aware that they may also re-engage your team for new Epic version testing or troubleshooting assistance.

Scoping out Subsequent Installs

Be agile and flexible when you approach each new integration. Even if you have an existing integration design that has worked for other organizations, lean on operations for integration decisions related to how their staff wants to use your application in their existing workflows. Organizations can configure their Epic instance to best meet the needs of their staff, so you will need to scope out whether your pre-existing integration design assumptions still hold for this new group of users.

If changes are needed to your integration design to best-support operations at this organization, IT can help you expand upon or modify your initial integration design to suit the needs of their users.

In Summary

Designing and deploying healthcare software that integrates with an organization's Epic software is a complex but manageable process. It's critical that you engage with both operations and IT at each organization to ensure you're meeting the needs of their users while using the most appropriate technologies. By maintaining clear communication with all stakeholders, adapting to feedback, and ensuring continuous support, you can ensure your software integration is prioritized, implemented efficiently, and well-received by users. When you are pitching your product to an organization, invite leaders from both operations and IT to discussions to begin this collaboration from the beginning of the project.

1.8 CDS Hooks Tutorial

File: site/docs/01-guides/08-cds-hooks.html

Source: https://fhir.epic.com/Documentation?docId=cds-hooks

The patient-view hook is triggered in two scenarios:

CDS Hooks Tutorial

Introduction

Clinical Decision Support (CDS) Hooks is an HL7 standard for in-workflow decision support integrations between electronic health record systems and remote, real-time, provider-facing decision support services. This tutorial describes Epic's support for this standard.

Epic's support for CDS Hooks is built on a workflow engine and CDS rule engine. Organizations using Epic can configure native CDS alerts by defining the workflow point at which they are triggered, known as the hook, and the criteria to evaluate when they are triggered. These criteria determine whether a CDS alert appears to the clinician and, if it does, what information it contains. Epic's implementation of CDS Hooks enables native criteria to call a remote CDS Hooks API when evaluating whether to show a CDS alert. In Epic, the native CDS alert is referred to as an OurPractice Advisory (OPA).

Guidance to CDS Developers

CDS Hooks is a powerful integration technology because it directly interacts with clinician workflows. Treat the ability to interact with clinicians carefully. As a CDS developer, you now share responsibility for a positive user experience. Your CDS service must be fast, must avoid alert fatigue, and should improve over time.

Hooks

As of Epic version May 2021, Epic primarily supports three standard hooks built on pre-existing CDS alert workflow triggers. Note that Epic supports many more native workflow triggers than CDS Hooks has standardly defined.

patient-view

* When a patient's chart is opened. The native Epic trigger is named Open Patient Chart. Epic recommends against firing a CDS Hooks request every time a patient chart is opened because it results in a poor user experience. Instead, work with the

health system to use this trigger in combination with more specific, native business criteria in Epic to limit the number of CDS Hooks requests.

* When a native Epic trigger calls a CDS Hooks service that is not standardized by CDS Hooks. For example, Enter Allergy is a native Epic workflow trigger that an organization can configure to show an OPA, including evaluating criteria that calls a CDS Hooks service. Because there is no industry standard hook for the Enter Allergy workflow step, we reuse the patient-view hook. The Epic-specific "com.epic.cdshooks.request.bpa-trigger-action" extension (see below) can be used to differentiate between native Epic triggers.

order-select

Epic supports the order-select hook in workflows when a clinician enters orders for a patient. In addition to the new selection of one or more orders (which triggers the order-select event), the clinician might have already selected other orders that have not yet been signed. The draftOrders JSON object contains a bundle of both the newly selected orders and the previously selected, unsigned orders. The "selections" array of FHIR identifiers identifies which of the unsigned orders from the draftOrders bundle are newly selected. Decision support using this hook should focus on the newly selected orders.

The draftOrders bundle can contain the MedicationRequest, ServiceRequest, or ProcedureRequest resource.

order-sign

The order-sign hook is triggered in Epic as the final step in the ordering process. It occurs after a clinician has clicked the Sign Orders button, but before the system has finalized the order. Decision support from this hook is the final chance for the clinician to revise the order. This hook allows for collection of all unsigned order details to send in the CDS Hooks call as MedicationRequest, ServiceRequest, or ProcedureRequest resources.

CDS Hooks Request

Prefetch

Prefetch can be a valuable method for optimizing the performance of your CDS service. Evaluate whether you can collect the data needed to inform your CDS service using the CDS Hooks prefetch model.

Prefetch within Epic can be configured in your CDS Hooks service's OurPractice Advisory (LGL) record. Any resource that you'd like to prefetch must be appropriately scoped to your client and match the client's primary FHIR version. Epic does not support automatically retrieving the contents of a CDS Hooks service's discovery endpoint.

Here is an example of how prefetch might be configured for a CDS service:

Prefetch Property Name | FHIR Query
patient | Patient/{{context.patientId}}
medications | MedicationRequest?patient={{context.patientId}}&status=active
encounterDx | Condition?patient={{context.patientId}}&encounter={{context.encounterId}}&category=encounter-diagnosis
user | {{context.userId}}

Note that the {{context.patientId}}, {{context.encounterId}}, and {{context.userId}} prefetch tokens are all used here. These IDs are returned by default in the CDS Hooks request context field and are also available to use in constructing contextual read/search FHIR requests that are included in the initial request to the CDS Hooks service. In contrast to the FHIR IDs, which are always sent in the context, the patient prefetch property defined above returns the entire patient resource within the prefetch. For searches, the medications and encounterDx prefetch properties above show how to use prefetch tokens as query parameters when implementing your CDS Hooks service.

The {{context.userId}} prefetch token is unique in that it returns both a FHIR resource name and FHIR resource ID. If the clinician triggering the CDS Hooks request has a provider record in Epic, the request returns "PractitionerRole/". If the clinician does not have a provider record, the request returns "Practitioner/". Additionally, the {{userPractitionerId}} and {{userPractitionerRoleId}} prefetch tokens can be used in prefetch to return the FHIR resource ID for their respective resources.

Input Fields for JSON POST Body

Field | Description
hook | Epic supports the following hooks: order-select order-sign patient-view
hookInstance | A universally unique identifier (UUID) for each hook call.
fhirServer | The base FHIR URL for the health system. Epic will always provide this. Reach out to the Client Systems Web and Server Systems team or EDI TS for the organization to obtain this.
fhirAuthorization | A structure holding an OAuth 2.0 bearer access token, which can be used to authenticate to Epic APIs for a short period of time.
context | The CDS Hooks specification (for example, patient-view) defines the context elements associated with each hook. Epic always sends patientId and userId. If the hook is initiated from an encounter context, encounterId is sent. For the order-select and order-sign hooks, draftOrders is always sent. Specific to order-select, the selections array identifies which orders are newly selected.
prefetch | Resources sent in the prefetch are configured by the health system and can include FHIR resources that are already supported by Epic.

Epic Extensions

Extension | Description
com.epic.cdshooks.request.bpa-trigger-action | The specific trigger action in Epic that is mapped to the hook. Common values include, but are not limited to: 5 (General OPA section) 6 (Enter problem) 7 (Enter diagnosis) 18 (Enter order) 23 (Sign orders) 26 (IP Admission OPA section) 27 (IP Discharge OPA section) 29 (IP Transfer OPA section) 60 (Open patient chart)
com.epic.cdshooks.request.cds-hooks-specification-version | The CDS hooks specification version used by Epic.
com.epic.cdshooks.request.fhir-version | The primary FHIR version of the CDS service as specified during OAuth registration.

com.epic.cdshooks.request.criteria-id | The ID of the OPA criteria record in Epic. This value can be helpful during troubleshooting.
com.epic.cdshooks.request.epic-version | The version of Epic that the health system is currently using.
com.epic.cdshooks.request.cds-hooks-implementation-version | The internal version Epic assigns to CDS Hooks implementations. Can be used to determine what features are supported.

Example HTTP Request


```
{
  "hookInstance": "f399c67c-c703-11ea-af16-460231621f93",
  "fhirServer": "https://example.com/interconnect-instance-oauth/api/FHIR/R4",
  "hook": "patient-view",
  "fhirAuthorization": {
    "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJhdWQiOiJ1cm46ZXBpYzpjZWMuY2RlIiwiY2xpZW50X2lkIjoiYWVhYzNkYT
    ctNjQ4MS00NWFmLWFkNDEtNDJlNWU3MDExNDFkIiwiZXBpYy5lY2kiOiJ1cm46ZXBpYzpDdXJyZW50LURldmVsb3BtZW50LUVudmlyb25tZW50LUVudEl1Nk
    BpYy5tZXRhZGF0YSI6Ik4yTDdoSFE3clNxLXdQbkstcVMyZlBTSjY1Q1EzS1ZOVHhhTUlBZGRVTllwemQyOEE0Q2d4ZHA0SVg0NUI5YmE4TVVJdEl1Nk
    EzYjBkYmU1c1ZlY093V1ZZN0tTd2VZb09KZDR5R2hmblcxdk9yOENoX195N2NrT1VtV005cnVhIiwiZXBpYy50b2tlbnR5cGUiOiJhY2Nlc3MiLCJleH
    AiOjE1OTQ4NjMzMzcsImlhdCI6MTU5NDg2MzAzNywiaXNzIjoidXJuOmVwaWM6Y2VjLmNkZSIsImp0aSI6ImU4MjFhMDA5LTgzNjAtNDRkYy1iZmMwLT
    EyZmU4ODA3MjQyYSIsIm5iZiI6MTU5NDg2MzAzNywic3ViIjoiZUJxZ0Foc2pVcHF3a3lJWm9LZU1jVUEzIn0.hF2Ir9717QxJ0TgiFXKN1bhRdicgUH
    Ef_0tjZ7qiy2Enzsn7_dW4HaueJqg22vcnrEayLxlW4h6Q6J5He3-U7VXmGBFd5AKXxVLbTQZupN6x4owt5n_N1OJKDu5UVJdlWoq-
    fBVn2sEyDfvITQ-BK21MWXxGaEcnttJNxS0Mk847JnxBby2oLzAd5NnIbdT733S2nL36ViK_mVnkucLu4vxwVRcaec2zCpvqF8Pn2Crl3yKaPKQbRqc6
    y12hprdUeB2VxOm3H5KxJQKYnEGTKNCmCy5w3Yaz86KtP9OlX1e3n6Km3TSCtZbj0Gm7QIVyDARAlREPA_XzbgQ3ItJang",
    "token_type": "Bearer",
    "expires_in": 300,
    "scope": "ALLERGYINTOLERANCE.READ ALLERGYINTOLERANCE.SEARCH Observation.Read (Smoking History) Observation.Read
    (Labs) (R4) Observation.Read (Vitals) Observation.Read (Vitals) (R4) Observation.Search (Labs) (R4)
    Observation.Search (Smoking History) Observation.Search (Vitals) Observation.Search (Vitals) (R4) Practitioner.Read
    (R4) Practitioner.Search (DSTU2) PROCEDURE.READ PROCEDURE.SEARCH MedicationRequest.Read (R4)
    MedicationRequest.Search (R4) CONDITION.READ CONDITION.SEARCH Condition.Read (R4) Condition.Search (R4) PATIENT.READ
     PATIENT.SEARCH PRACTITIONER.READ PRACTITIONER.SEARCH OBSERVATION.READ OBSERVATION.SEARCH urn:Epic-
    com:Informatics.2014.Services.Order.AnnotateOrders AllergyIntolerance.Search (R4) AllergyIntolerance.Read (R4)
    Patient.Read (R4) Patient.Search (R4) Location.Read (R4) Encounter.Read (R4) Encounter.Search (R4) urn:Epic-
    com:Core.2016.Services.DataUtility.GetImportDataLog Condition.Create (R4) Medication.Read (R4) MEDICATION.READ
    (DSTU2) MEDICATION.READ MEDICATIONORDER.READ MEDICATIONORDER.READ (DSTU2) MEDICATIONORDER.SEARCH MEDICATION.SEARCH
    MedicationStatement.Read (R4) MEDICATIONSTATEMENT.READ (DSTU2) MEDICATIONSTATEMENT.READ MEDICATIONSTATEMENT.SEARCH
    MedicationStatement.Search (R4) Condition.Search (R4) Patient.Read (R4) Practitioner.Read (R4) Patient.Search (R4)
    Practitioner.Search (R4) Condition.Read (R4) Condition.Create (R4) AllergyIntolerance.Read (R4)
    AllergyIntolerance.Search (R4) Observation.Read (R4) Observation.Search (R4) Practitioner.Search (R4)
    PractitionerRole.Search (R4) PractitionerRole.Search (R4) medication medication.create Encounter.Read (R4)
    Encounter.Search (R4) ProcedureRequest.Read (R4) ProcedureRequest.Search (R4)
    EPIC.FHIR.R4.SERVICES.MEDICATIONREQUEST.CREATE EPIC.FHIR.R4.SERVICES.MEDICATIONREQUEST.CREATE Condition.Create
    (Encounter Diagnosis) (R4) Condition.Create (Problems) (R4) EPIC.FHIR.R4.SERVICES.PROCEDUREREQUEST.CREATE
    EPIC.FHIR.R4.SERVICES.SERVICEREQUEST.CREATE Condition.Read (Encounter Diagnosis) (R4) Condition.Read (Health
    Concern) (R4) Condition.Read (Problems) (R4) Condition.Search (Encounter Diagnosis) (R4) Condition.Search (Health
    Concern) (R4) Condition.Search (Problems) (R4) Observation.Read (Core Characteristics) (R4) Observation.Read (Labs)
    (R4) Observation.Read (LDA-W) (R4) Observation.Read (Obstetric Details) (R4) Observation.Read (Smoking History) (R4)
     Observation.Read (Vitals) (R4) Observation.Search (Core Characteristics) (R4) Observation.Search (Labs) (R4)
    Observation.Search (LDA-W) (R4) Observation.Search (Obstetric Details) (R4) Observation.Search (Smoking History)
    (R4) Observation.Search (Vitals) (R4) MedicationRequest.Read (R4) MedicationRequest.Search (R4)
    condition.create(diagn Condition.Create Problem (R4) PractitionerRole.Read (R4) PractitionerRole.Read (R4)",
    "subject": "aeac3da7-6481-45af-ad41-42e5e701141d"
  },
  "context": {
    "patientId": "eXoGxqgBaJuNkuahMYmiDhg3",
    "encounterId": "eFyoeOuWgXtlQmOQzPdkQWwy3s8a49yrUc-LtjwhWT6g3",
    "userId": "PractitionerRole/e-QokEGUJIzyynNdkCFrs9w3"
  },
  "prefetch": {
    "-user": {
      "resourceType": "PractitionerRole",
      "id": "e-QokEGUJIzyynNdkCFrs9w3",
      "active": true,
      "practitioner": {
        "reference": "https://example.com/interconnect-instance-
    oauth/api/FHIR/R4/Practitioner/eqZb1t82mP4YeBjiLldSFPQ3",
        "display": "Family Medicine Physician"
      },
      "code": [
        {
          "coding": [
            {
              "system": "urn:oid:1.2.840.114350.1.13.861.1.7.10.836982.1040",
              "code": "1",
              "display": "Physician"
            }
          ],
          "text": "Physician"
        }
      ],
      "specialty": [
        {
          "coding": [
            {
              "system": "urn:oid:1.2.840.114350.1.72.1.7.7.10.688867.4160",
              "code": "10",
              "display": "Cardiology"
            }
          ],
          "text": "Cardiology"
```

```
    },
    {
      "coding": [
        {
          "system": "urn:oid:1.2.840.114350.1.72.1.7.7.10.688867.4160",
          "code": "5",
          "display": "Anesthesiology"
        }
      ],
      "text": "Anesthesiology"
    },
    {
      "coding": [
        {
          "system": "urn:oid:1.2.840.114350.1.72.1.7.7.10.688867.4160",
          "code": "15",
          "display": "Dermatology"
        }
      ],
      "text": "Dermatology"
    },
    {
      "coding": [
        {
          "system": "urn:oid:1.2.840.114350.1.72.1.7.7.10.688867.4160",
          "code": "18",
          "display": "Endocrinology"
        }
      ],
      "text": "Endocrinology"
    }
  ]
},
"patient": {
  "resourceType": "Patient",
  "id": "eXoGxqgBaJuNkuahMYmiDhg3",
  "extension": [
    {
      "extension": [
        {
          "valueCoding": {
            "system": "http://hl7.org/fhir/us/core/ValueSet/omb-race-category",
            "code": "UNK",
            "display": "Unknown"
          },
          "url": "ombCategory"
        },
        {
          "valueString": "Unknown",
          "url": "text"
        }
      ],
      "url": "http://hl7.org/fhir/us/core/StructureDefinition/us-core-race"
    },
    {
      "extension": [
        {
          "valueCoding": {
            "system": "http://hl7.org/fhir/us/core/ValueSet/omb-ethnicity-category",
            "code": "UNK",
            "display": "Unknown"
          },
          "url": "ombCategory"
        },
        {
          "valueString": "Unknown",
          "url": "text"
        }
      ],
      "url": "http://hl7.org/fhir/us/core/StructureDefinition/us-core-ethnicity"
    },
    {
      "valueCode": "M",
      "url": "http://hl7.org/fhir/us/core/StructureDefinition/us-core-birthsex"
    }
  ],
  "identifier": [
    {
      "use": "usual",
      "type": {
        "text": "EPI"
      },
      "system": "urn:oid:1.2.840.114350.1.1",
      "value": "113822"
    },
    {
      "use": "usual",
      "type": {
        "text": "EXTERNAL"
```

```
      },
      "value": "Z16282"
    },
    {
      "use": "usual",
      "type": {
        "text": "FHIR"
      },
      "value": "TDjw8.slp7UtBKr3jNvYPsSGUb0qoZMC5OEhD5iu5ticB"
    },
    {
      "use": "usual",
      "type": {
        "text": "FHIR R4"
      },
      "value": "eXoGxqgBaJuNkuahMYmiDhg3"
    },
    {
      "use": "usual",
      "type": {
        "text": "INTERNAL"
      },
      "value": "    Z16282"
    },
    {
      "use": "usual",
      "system": "urn:oid:2.16.840.1.113883.4.1"
    }
  ],
  "active": true,
  "name": [
    {
      "use": "official",
      "text": "Family Medicine Physician",
      "family": "Physician",
      "given": [
        "Family Medicine"
      ]
    },
    {
      "use": "usual",
      "text": "Family Medicine Physician",
      "family": "Physician",
      "given": [
        "Family Medicine"
      ]
    }
  ],
  "gender": "male",
  "birthDate": "1981-06-24",
  "deceasedBoolean": false,
  "managingOrganization": {
    "reference": "https://example.com/interconnect-instance-
oauth/api/FHIR/R4/Organization/e1aU.Joccsz0IPnyexqMjXw3",
    "display": "EXAMPLE ORGANIZATION"
  }
},
"conditions": {
  "resourceType": "Bundle",
  "type": "searchset",
  "total": 0,
  "link": [
    {
      "relation": "self",
      "url": "https://example.com/interconnect-instance-oauth/api/FHIR/R4/Condition?patient=eXoGxqgBaJuNkuahMYmiDhg3
&_include=Condition:patient&_include=Condition:patient:organization"
    }
  ],
  "entry": [
    {
      "resource": {
        "resourceType": "OperationOutcome",
        "issue": [
          {
            "severity": "warning",
            "code": "processing",
            "details": {
              "coding": [
                {
                  "system": "urn:oid:1.2.840.114350.1.13.861.1.7.2.657369",
                  "code": "4101",
                  "display": "Resource request returns no results."
                }
              ],
              "text": "Resource request returns no results."
            }
          }
        ]
      },
```

```
      "search": {
        "mode": "outcome"
      }
    }
  ]
},
"medications": {
  "resourceType": "Bundle",
  "type": "searchset",
  "total": 2,
  "link": [
    {
      "relation": "self",
      "url": "https://example.com/interconnect-instance-
oauth/api/FHIR/R4/MedicationRequest?patient=eXoGxqgBaJuNkuahMYmiDhg3"
    }
  ],
  "entry": [
    {
      "link": [
        {
          "relation": "self",
          "url": "https://example.com/interconnect-instance-oauth/api/FHIR/R4/MedicationRequest/e4jAeTRRFdgN-
QzpjNRbwQkDnp1BQlqrukNdC6BvYzU43"
        }
      ],
      "fullUrl": "https://example.com/interconnect-instance-oauth/api/FHIR/R4/MedicationRequest/e4jAeTRRFdgN-
QzpjNRbwQkDnp1BQlqrukNdC6BvYzU43",
      "resource": {
        "resourceType": "MedicationRequest",
        "id": "e4jAeTRRFdgN-QzpjNRbwQkDnp1BQlqrukNdC6BvYzU43",
        "identifier": [
          {
            "use": "usual",
            "system": "urn:oid:1.2.840.114350.1.13.861.1.7.2.798268",
            "value": "1000182255"
          }
        ],
        "status": "active",
        "intent": "order",
        "category": {
          "coding": [
            {
              "system": "http://hl7.org/fhir/medication-request-category",
              "code": "inpatient",
              "display": "Inpatient"
            }
          ],
          "text": "Inpatient"
        },
        "medicationReference": {
          "reference": "https://example.com/interconnect-instance-oauth/api/FHIR/R4/Medication/ehjM3OtjgdW8Bea3jIQgM
FVhzPMcMELCBS5v775xzbLetNaOP5vZRG-7BUwFy1hYFzJ7rXwELBf63P6zbB4tI56DXbUTYtEZEWyFIzxmyONE3",
          "display": "IBUPROFEN 200 MG PO TABS"
        },
        "subject": {
          "reference": "https://example.com/interconnect-instance-
oauth/api/FHIR/R4/Patient/eXoGxqgBaJuNkuahMYmiDhg3",
          "display": "Physician, Family Medicine"
        },
        "authoredOn": "2020-07-06T20:25:56Z",
        "recorder": {
          "reference": "https://example.com/interconnect-instance-
oauth/api/FHIR/R4/Practitioner/eqZb1t82mP4YeBjiLldSFPQ3",
          "display": "Family Medicine Physician"
        },
        "dosageInstruction": [
          {
            "extension": [
              {
                "valueQuantity": {
                  "value": 2,
                  "unit": "tablet",
                  "system": "http://unitsofmeasure.org",
                  "code": "{tbl}"
                },
                "url": "https://open.epic.com/fhir/extensions/admin-amount"
              },
              {
                "valueQuantity": {
                  "value": 400,
                  "unit": "mg",
                  "system": "http://unitsofmeasure.org",
                  "code": "mg"
                },
                "url": "https://open.epic.com/fhir/extensions/ordered-dose"
              }
            ],
            "timing": {
```

```
              "repeat": {
                "boundsPeriod": {
                  "start": "2020-07-06T20:25:32Z"
                },
                "frequency": 1,
                "period": 6,
                "periodUnit": "h"
              },
              "code": {
                "text": "Q6H PRN"
              }
            },
            "asNeededBoolean": true,
            "route": {
              "coding": [
                {
                  "system": "urn:oid:1.2.840.114350.1.13.861.1.7.4.798268.7025",
                  "code": "15",
                  "display": "Oral"
                },
                {
                  "system": "http://snomed.info/sct",
                  "code": "260548002",
                  "display": "Oral"
                }
              ],
              "text": "Oral"
            },
            "doseQuantity": {
              "value": 400,
              "unit": "mg",
              "system": "http://unitsofmeasure.org",
              "code": "mg"
            }
          }
        ]
      },
      "search": {
        "mode": "match"
      }
    },
    {
      "link": [
        {
          "relation": "self",
          "url": "https://example.com/interconnect-instance-
oauth/api/FHIR/R4/MedicationRequest/es5s.w.VqTbI-3sTtgDY0f.oJSZ8leYTXykPvcBCZrWQ3"
        }
      ],
      "fullUrl": "https://example.com/interconnect-instance-
oauth/api/FHIR/R4/MedicationRequest/es5s.w.VqTbI-3sTtgDY0f.oJSZ8leYTXykPvcBCZrWQ3",
      "resource": {
        "resourceType": "MedicationRequest",
        "id": "es5s.w.VqTbI-3sTtgDY0f.oJSZ8leYTXykPvcBCZrWQ3",
        "identifier": [
          {
            "use": "usual",
            "system": "urn:oid:1.2.840.114350.1.13.861.1.7.2.798268",
            "value": "1000182291"
          }
        ],
        "status": "active",
        "intent": "order",
        "category": {
          "coding": [
            {
              "system": "http://hl7.org/fhir/medication-request-category",
              "code": "inpatient",
              "display": "Inpatient"
            }
          ],
          "text": "Inpatient"
        },
        "medicationReference": {
          "reference": "https://example.com/interconnect-instance-oauth/api/FHIR/R4/Medication/eIBy3xFZsuXmpUpNLdojk
XC2jhRNQyq0NyjvMBlrASIS4dl.7I3naftOtzE1KVToZwnuJ5ty7Zonwkhzz9kHIeEYjUV102TvDEv7zaGJFkKQ3",
          "display": "MITOMYCIN 40 MG IV SOLR"
        },
        "subject": {
          "reference": "https://example.com/interconnect-instance-
oauth/api/FHIR/R4/Patient/eXoGxqgBaJuNkuahMYmiDhg3",
          "display": "Physician, Family Medicine"
        },
        "authoredOn": "2020-07-06T20:40:24Z",
        "recorder": {
          "reference": "https://example.com/interconnect-instance-
oauth/api/FHIR/R4/Practitioner/eqZb1t82mP4YeBjiLldSFPQ3",
          "display": "Family Medicine Physician"
        },
```

```
            "dosageInstruction": [
              {
                "extension": [
                  {
                    "valueQuantity": {
                      "value": 10,
                      "unit": "mg",
                      "system": "http://unitsofmeasure.org",
                      "code": "mg"
                    },
                    "url": "https://open.epic.com/fhir/extensions/ordered-dose"
                  }
                ],
                "timing": {
                  "repeat": {
                    "boundsPeriod": {
                      "start": "2020-07-06T23:00:00Z"
                    },
                    "frequency": 2,
                    "period": 1,
                    "periodUnit": "d"
                  },
                  "code": {
                    "text": "0900 & 1800"
                  }
                },
                "asNeededBoolean": false,
                "route": {
                  "coding": [
                    {
                      "system": "urn:oid:1.2.840.114350.1.13.861.1.7.4.798268.7025",
                      "code": "11",
                      "display": "Intravenous"
                    }
                  ],
                  "text": "Intravenous"
                }
              }
            ]
          },
          "search": {
            "mode": "match"
          }
        }
      ]
    }
  },
  "extension": {
    "com.epic.cdshooks.request.bpa-trigger-action": "18",
    "com.epic.cdshooks.request.cds-hooks-specification-version": "1.0",
    "com.epic.cdshooks.request.fhir-version": "R4",
    "com.epic.cdshooks.request.criteria-id": "2852",
    "com.epic.cdshooks.request.epic-version": "9.4",
    "com.epic.cdshooks.request.cds-hooks-implementation-version": "1.0",
    "com.epic.cdshooks.request.cds-hooks-su-version": "0",
    "internal.epic.cdshooks.request.epic-emp-id": "17805"
  }
}
```


CDS Hooks Response

Upon receiving a CDS Hooks request from Epic, a CDS Service should quickly and synchronously respond with a CDS Hooks response. Optionally, the CDS Service may query Epic's FHIR server to learn additional information before responding using the OAuth 2.0 access token provided in the fhirAuthorization attribute in the request.

Upon evaluating the CDS Hooks request (including the relevant information contained in the request, and any data retrieved) according to your app’s business logic, your application responds. This response determines whether content is displayed to end users and what that content should be. Possible types of content include one or more of:

* textual information

* suggested actions to be taken, such as adding diagnoses

* links to your SMART app or other web pages

Suggestion or app launch?

Generally, launching and interacting with an app is more time-consuming and therefore more disruptive to a user's workflow than returning a suggestion. If your CDS recommendations can be fully determined with only the information provided in the CDS Hooks request and other APIs, you should return these suggestions in the CDS Hooks response.

If you are unable to determine recommendations with only the information from the CDS Hooks request and other APIs, a link to launch an app enables additional user interaction. The app launch link is returned in the CDS Hooks response. A card response is a one-time event. CDS Hooks suggestions can only be returned in a CDS Hooks response. An app launched from a CDS Hooks response can't use CDS Hooks APIs.

Generally, the user needs to manually click a link to launch your app. There's nothing forcing the user to do this, and therefore they may not.

Beginning in Aug 23, Epic supports an "auto-launch" feature, such that a CDS Service may inform the EHR in its CDS Hooks response that it's appropriate to auto-launch an app because, for example, there's no other CDS guidance in the card. In the case where the only content in a CDS Hooks' response is an app url, and there's no other cards or native CDS, Epic may auto-launch an app.

Actions

In the CDS Hooks response, a CDS Service can suggest suggestions, which include one or more actions. An action creates or deletes a FHIR resource. For example, a CDS Service can recommend that a problem be added to the current patient's problem list by returning a FHIR Condition resource and a card.suggestion.action.type of "create". The list of suggestions supported by Epic are listed in the API library (search for CDS Hooks).

If using the MedicationRequest, ServiceRequest, or ProcedureRequest resources to create an unsigned order in Epic, order overrides are NOT set with information from the service response, even if additional order details are specified. All order details come from the default values defined on the medication record, procedure record, or preference list. To use these CREATE resources to specify a preference list item, specify the system as "urn:com.epic.cdshooks.action.code.system.preference-list-item". The code, assigned while building the preference list, will be the corresponding key used to identifty a particular order on the preference list.

HTTP Response Fields

Field | Description

cards | An array of cards that provide any of the following: Information Text to be shown to the user, optionally formatted as GitHub-flavored markdown, or with html markup if also providing an Epic-specific extension defining the content type. Suggestions Work with the health system to map the problem list items, encounter diagnoses, medications, procedures and/or multi-order sets. Containing CREATE actions for Condition resources, specifically Problems, or Encounter Diagnosis. Medication single order follow-up suggestions with CREATE actions using MedicationRequest resources Medication single order follow-up suggestions with DELETE actions using MedicationRequest resources Procedure single order follow-up suggestions with CREATE actions using ProcedureRequest (STU3) or ServiceRequest (R4) resources Procedure single order follow-up suggestions with DELETE actions using ProcedureRequest (STU3) or ServiceRequest (R4) resources Order Set, SmartSet, Pathway or Express Lane multiple order follow-up suggestions with CREATE actions using ProcedureRequest (STU3) or ServiceRequest (R4) resources Links Reference links do not require any additional configuration SMART App links need to be added to the allowlist in Epic

systemActions | Beginning in Feb 24, Epic supports system actions to annotate orders via the ServiceRequest.Update (Unsigned Order) (R4) API.

Card Attributes

Epic-specific nuances regarding Card attributes:

Field | Description

indicator | The info, warning, and critical values can be mapped to Epic-specific values by the Epic application team for display.

uuid | Unique identifier, used for auditing and logging suggestions. This field is optional. However, it is required if you intend to receive feedback.

selectionBehavior | Only the value "any" is currently supported.

detail | A CDS Service may return content as mere plain text, as GitHub flavored markdown, or, with an Epic-specific extension, as html. (See "com.epic.cdshooks.card.detail.content-type" extension, below).

source.topic.code | Epic only supports alpha-numeric strings as the code of source.topic, for example: "Card123" or "869e7c5587e04d0da96a60a84b5b8eac". The value returned in source.topic.code is used for logging and auditing, and is returned to the CDS Service in the feedback request. Maximum Length: 100 Characters Source.topic.code should be a static identifier representing the particular topic of a card. When an end user overrides a given card, their acknowledgment is associated with this identifier. If you want the end user's override to be respected on subsequent requests to your CDS service, the topic identifier should remain static if sending the same card content.

links | Allows your service to suggest a link to a user for additional information or a SMART app. Allowed links are allow-listed by the health system.

Suggestion Attributes

Epic-specific nuances regarding Suggestions and Action attributes:

Field | Description

label | This field is not used, but is required.

uuid | Unique identifier, used for auditing and logging suggestions. This field is optional. However, it is required if you intend to receive feedback.

isRecommended | Boolean. When there are multiple suggestions, allows a service to indicate that a specific suggestion is recommended from all the available suggestions on the card. In Epic, this is used to control whether the suggested action is pre-selected or not.

actions | Epic only supports a single action per suggestion.

actions.type | For a given resource, confirm the action types supported in the API Library.

actions.description | This value is the primary content of the OPA. A CDS Service may return content as plain text.

actions.resource | The FHIR resource provided by the CDS Service representing the action to be suggested to the user. Check the API Library to see what resources (and action types) are supported.

Codesets for Create APIs

The following table calls out codesets that may be used when specifying a coding for a given Create action.

System | Description | Resource(s)

urn:com.epic.cdshooks.action.code.system.preference-list-item | Used to order a specific preference list item. | MedicationRequest.Create ServiceRequest.Create ProcedureRequest.Create

urn:com.epic.cdshooks.action.code.system.orderset-item | Used to order a specific SmartSet, OrderSet, or Pathway. | ServiceRequest.Create ProcedureRequest.Create

urn:com.epic.cdshooks.action.code.system.cms-hcc | Used to suggest a Visit Diagnosis through a CMS-HCC model. | Condition.Create

urn:com.epic.cdshooks.action.code.system.hhs-hcc | Used to suggest a Visit Diagnosis through a HHS-HCC model. | Condition.Create

urn:oid:2.16.840.1.113883.6.90 | Used to suggest ICD-10 codes. | Condition.Create
urn:oid:2.16.840.1.113883.6.69 | Used to suggest NDC codes. | MedicationRequest.Create
urn:oid:2.16.840.1.113883.6.88 | Used to suggest by RxNorm code. | MedicationRequest.Create

Link Attributes

Attribute | Description
label | This field is not used, but is required.
url | The URL to load.
type | This field is required, and may either be absolute or smart.
appContext | This field is optional, and will share information from the CDS card with the subsequently launched SMART app.
autolaunchable | Supported by Epic beginning in Aug 23, this will autolaunch the SMART app if only one OPA requests an autolaunch at a given time.

Supported Extensions

Extension | Description
com.epic.cdshooks.card.detail.content-type | If this is not set, or set to "text/markdown" or "text/markdown; variant=GitHub", Epic will treat it as GitHub Flavored Markdown Other options are "text/plain", "text/html", and "text/markdown; variant=CommonMark".

Basic Example HTTP Response with Card

```
{
  "cards": [
    {
      "summary": "Testing Informational Card",
      "detail": "* Suspected colorectal cancer. \n\n* Please add to patient's diagnoses.\n\nSuspected Bladder cancer.
    Please add to patient's diagnoses.",
    "indicator": "critical",
      "source": {
        "topic": {
          "code": "examplecode1"
        }
      },
    },]
}
```

Example HTTP Response for no decision support needed

```
{
  "cards": []
}
```

Example HTTP Response with Links, Override Reasons, and Suggestions

```
{
   "cards":[
    {
         "summary":"Example",
         "indicator":"info",
         "extension":{
            "com.epic.cdshooks.card.detail.content-type":"text/html"
         },
         "detail":"Another card to test suggestions",
     "source": {
      "label": "Clinical Inferences",
      "url": "https://www.example.com/",
      "icon": "file://example/CDSHooks/images/example.png",
      "topic": {
        "code": "BCSCard2",
        "system": "card-system",
        "display": "BCS Card 2"
      }
      },
      "links": [
      {
        "label": "Github",
        "url": "https://github.com",
        "type": "absolute"
      },
      {
        "label": "R4 SMART Example App",
        "url": "https://example.com/EpicSMARTApp/Default.aspx?appFhirVersion=R4",
        "type": "smart",
        "appContext": "%FNAME%-%EXTENSION;74901%-420fe522-193c-11eb-9552-460231621f93~!@#$%^&*()-+{}[]|\\"
      },
      ],
      "overrideReasons":[
      {
        "code":"patrefused",
        "system":"http://example.org/cds-services/fhir/CodeSystem/override-reasons",
        "display":"Patient refused"
      },
      {
        "code":"seecomment",
        "system":"http://example123.org/cds-services/fhir/CodeSystem/override-reasons",
      }
     ],
```

```
    "suggestions":[
{
        "label":"Arthritis",
  "uuid": "cf72fe83-1eb9-410c-94aa-04ec98736388",
        "actions":[
           {
              "type":"create",
    "description":"Arthritis",
              "resource": {
     "resourceType": "Condition",
     "category": [
       {
       "coding": [
         {
         "system": "http://terminology.hl7.org/CodeSystem/condition-category",
         "code": "encounter-diagnosis",
         "display": "Encounter diagnosis"
         }
       ],
       "text": "Encounter diagnosis"
       }
     ],
     "code": {
       "coding": [
       {
         "system": "urn:com.epic.cdshooks.action.code.system.cms-hcc",
         "code": "40",
         "display": "Arthritis"
       }
       ],
       "text": "Stomach ache"
     }
     },
           }
        ]
     },
{
        "label":"Stroke",
  "uuid": "12035ae1-5d60-4f58-b922-882140b98283",
        "actions":[
           {
              "type":"create",
    "description":"Stroke",
              "resource": {
     "resourceType": "Condition",
     "category": [
       {
       "coding": [
         {
         "system": "http://terminology.hl7.org/CodeSystem/condition-category",
         "code": "encounter-diagnosis",
         "display": "Encounter diagnosis"
         }
       ],
       "text": "Encounter diagnosis"
       }
     ],
     "code": {
       "coding": [
       {
         "system": "urn:com.epic.cdshooks.action.code.system.cms-hcc",
         "code": "100",
         "display": "Stroke"
       }
       ],
       "text": "Stroke"
     }
     },
           }
        ]
     },
{
        "label":"Stroke",
  "uuid": "1a53ba14-a06b-4e82-bbb0-09ae01a75515",
        "actions":[
           {
              "type":"create",
              "description":"Stroke prognosis",
              "resource": {
     "resourceType": "Condition",
     "category": [
       {
       "coding": [
         {
         "system": "http://terminology.hl7.org/CodeSystem/condition-category",
         "code": "problem-list-item",
         "display": "Problem List"
         }
       ],
```

```
        "text": "Problem list"
        }
      ],
      "code": {
        "coding": [
        {
          "system": "urn:oid:2.16.840.1.113883.6.90",
          "code": "R10.9"

        },
        {
          "system": "urn:oid:2.16.840.1.113883.6.96",
          "code": "271681002"

        }
        ],
        "text": "Stomach ache"
      }
      },
            }
         ]
      },
{
"label":"Diabetes",
"uuid": "85126ce5-b0a7-4a54-86f4-d7b52426cc58",
      "actions":[
         {
            "type":"create",
            "description":"Diabetes Order Set from CDS Hooks",
            "resource": {
    "resourceType": "ServiceRequest",
    "status": "draft",
    "intent": "proposal",
    "category": [
      {
      "coding": [
      {
        "system": "http://terminology.hl7.org/CodeSystem/medicationrequest-category",
        "code": "outpatient",
        "display": "Outpatient"
      }]
    }],
    "code": {
      "coding": [
        {
        "system": "urn:com.epic.cdshooks.action.code.system.orderset-item",
        "code": "DIABETES"
      }
      ]
    }
    },
         },
      ]
   },
 {
  "label":"Test IP Medication Order",
  "uuid": "b1e0575b-2381-4625-b687-2def804d7c79",
  "actions":[
     {
      "type":"create",
      "description":"Test Medication IP Order",
      "resource": {
      "resourceType": "MedicationRequest",
      "status": "draft",
      "intent": "proposal",
      "category": [
      {
        "coding": [
        {
          "system": "http://terminology.hl7.org/CodeSystem/medicationrequest-category",
          "code": "inpatient",
          "display": "Inpatient"
        }
        ],
        "text": "Inpatient"
      }
      ],
      "medicationCodeableConcept": {
        "coding": [
        {
          "system": "urn:oid:2.16.840.1.113883.6.69",
          "code": "55111-682-01"
        }
        ],
        "text": "Test Med Display name"
      }
      },
     },
  ]
```

```
      },
      {
       "label":"Test Medication Order from Pref",
       "uuid": "5e0ce5e2-d33a-467c-bfc6-cde8d304e73d",
       "actions":[
          {
           "type":"create",
           "description":"Test Medication Order from Pref",
           "resource": {
           "resourceType": "MedicationRequest",
           "status": "draft",
           "intent": "proposal",
           "category": [
           {
             "coding": [
             {
               "system": "http://terminology.hl7.org/CodeSystem/medicationrequest-category",
               "code": "inpatient",
               "display": "Inpatient"
             }
             ],
             "text": "Inpatient"
           }
           ],
           "medicationCodeableConcept": {
             "coding": [
             {
               "system": "urn:com.epic.cdshooks.action.code.system.preference-list-item",
               "code": "BENAD25"
             }
             ],
             "text": "Test Proc Display name"
           }
           },
          },
       ]
      },
    {
           "label":"CBC",
     "uuid": "613b0192-4243-4384-8294-4316dfb726bb",
           "actions":[
              {
                 "type":"create",
                 "description":"CBC from CDS Hooks",
                 "resource": {
        "resourceType": "ServiceRequest",
        "status": "draft",
        "intent": "proposal",
        "category": [
          {
          "coding": [
          {
            "system": "http://terminology.hl7.org/CodeSystem/medicationrequest-category",
            "code": "outpatient",
            "display": "Outpatient"
          }]
        }],
        "code": {
          "coding": [
            {
            "system": "urn:com.epic.cdshooks.action.code.system.preference-list-item",
            "code": "CBC_IP"
          }
          ],
          "text": "Test Proc Display name"
        }
        },
             },
          ]}
   ]
     },
]
}
```

When sending override reasons in your response, configuration must be completed within the CDS Hooks criteria record to map overrideReasons.code to released acknowledgment reasons. overrideReasons.system does not have to be a specific string; it can be any non-empty string.

1.9 FHIRcast Tutorial

File: site/docs/01-guides/09-fhircast-developer-tutorial.html

Source: https://fhir.epic.com/Documentation?docId=fhircast-developer-tutorial

FHIRcast Context Synchronization Developer Guide

This guide is intended to help app developers connect to Epic Hyperspace application using FHIRcast for context synchronization. While examples in this guide reference PACS applications, the guidance applies to any application type that wants to use FHIRcast for context synchronization with Epic's Hyperdrive desktop browser.

Intro to FHIRcast

HL7 FHIRcast is designed to enable simple, modern application context synchronization using FHIR and WebSockets.

This technology framework establishes a tighter connection than SMART on FHIR EHR launches as the applications using FHIRcast remain in sync as the user navigates across contexts such as different patients. Context synchronization has commonly been used by providers such as radiologists and cardiologists who simultaneously work in Epic and PACS/CPACS applications, and who would like to have one system follow when the user navigates in the other. The same framework can be used for additional user workflows and use cases.

Epic's support for FHIRcast is built into an API layer of Epic's Hyperdrive desktop browser, referred to as Subspace. Subspace supports the FHIRcast standard as a localhost Hub for context synchronization (presenting the same record to the user in each system) starting with the release of Epic Hyperdrive in 2022 and content sharing (exchanging measurements in real time) starting in the Epic November 2024 release.

Technology Background and Roadmap

Epic has long supported a context synchronization specification based on our Epic public XML spec. Over the past several years, we have worked with various industry leaders and HL7 International to create the FHIRcast industry standard. Our future development plans are focused on this industry standard, enabling faster connections for customers. Additionally, the FHIRcast standard is the baseline for future enhancements, such as the sharing of discrete data between imaging products, which will not be retrofitted to the legacy XML specification.

Epic recommends using the FHIRcast standard for all new context synchronization integrations with Epic. For the time being, existing live connections such as PACS integrations can continue to use the XML specification, though we recommend that developers start moving to the FHIRcast standard.

Technical Framework and Supported Events

Get started by reviewing the Hyperdrive documentation available on open.epic.com, including the Subspace Communication Framework document, which describes overall capabilities of Subspace and how to connect.

The HL7 FHIRcast event library defines industry-standard FHIRcast events and Epic supports many of them. FHIRcast events are authorized as OAuth 2.0 scopes, per the HL7 FHIRcast spec. You must identify the FHIRcast events your application requires when registering your client with Epic.

Epic has additionally defined a few implementer-specific events as permitted by the HL7 FHIRcast specification, including events and APIs related to Subspace User Session Management, Scan Acquisition workflows, Speech Recognition, Front End Coding, and Other Subspace APIs.

FHIRcast Application Roles

There are three FHIRcast roles to know:

* The Hub (aka coordinating application) maintains the shared context, such as the patient or the study, for all other applications to coordinate the session. As the EHR, when using FHIRcast to integrate Epic with other systems, Epic is always the coordinating application.

* A Subscribing Application is any user-facing app that wants to stay in sync. Every Subscriber connects to that one Hub instead of connecting to each other directly. Instead of building a custom connection between every pair of apps, everyone just talks to the Hub, and the Hub keeps them all in sync. For example, a PACS, a speech recognition product, or an AI tool could subscribe to the Hub.

* The Driving Application is whichever app is directing context at a given moment. Context can be driven from any side; it's not locked to one app. FHIRcast is a two-way street and should always be considered bidirectional; to keep context in sync, each application must listen to events and not assume it is always driving. For example, a generally PACS-driven workflow can be interrupted by the radiologist choosing to open another patient and study in Epic and the PACS needs to respond to that ad-hoc change in context (and vice versa for RIS-driven workflows) to meet the user's synchronization expectations.

Getting Started with App Contexts and Client IDs

Client ID Context Considerations

Starting with Epic version August 2026, Epic recommends launching your application from Epic using a SMART on FHIR EHR launch for all FHIRcast context synchronization integrations, including both web-based and Windows-native (desktop) applications. This approach is recommended for several reasons:

* SMART App Launch provides user-specific security; the token is scoped to the authenticated end user, which enables proper audit trails.

* Apps configured for SMART app launches can include both Subspace (FHIRcast) APIs and incoming RESTful FHIR web services on a single client registration, eliminating the need for multiple client IDs and managing their related tokens, thus simplifying your integration. For more information on constraints with backend context apps, refer to Subspace Security Considerations.

To enable SMART on FHIR launch, create a provider context app, with an Application Audience set to Clinicians or Administrative Users.

This strategy for initial application launch does not dictate whether the workflow is Epic-driven or driven from worklists in your application. Once Hyperdrive and your application are launched and subscribed to each other via FHIRcast, either application can drive the workflow, for example, using the Patient Chart button in Epic or using a worklist in PACS to launch patients.

Note: Prior to Epic version August 2026, we do not recommend FHIRcast for web-based applications due to limitations with duration of API access. Although FHIRcast can be used with web-based apps, the complexity of using a SMART on FHIR launch followed by the dynamic registration of a client with the client_credentials OAuth 2.0 grant type is not recommended. The rest of this guide highlights our recommendations for Epic version August 2026 and later.

Client Record Registration

To consume FHIRcast APIs, products need to pre-register with Epic to receive a client record (client ID). This client ID includes the app's requested API scopes. An app can only connect using APIs registered to its client ID.

Complete the following steps:

* Check out our step-by-step , where you can find the steps needed for and more. For every app, select:

Create a client ID for your application.

Developer Guide

Creating a Client ID

* Application Audience: Clinicians or Administrative Users

* Use Case: General

* Is Confidential Client: Selected

* Requires Persistent Access: Selected (note Uses Online Access is then automatically selected)

* Select any non-FHIRcast APIs this client will use, for example APIs to do patient and study matching:

* Patient.$match (Patient Match) (R4)

* DiagnosticReport.Search (Results) (R4)

* Under Endpoint URIs, enter one or more SMART on FHIR redirect URIs.

* Enter your applicable Production and Non-Production JWK Set URLs.

* Next, to add FHIRcast scopes to that client. Provide your client ID and the list of Subspace APIs for your use case. See Epic's and for more information on the APIs.

send an email to open.epic

FHIRcast events specification

Subspace Communication Framework Overview

* Always include:

* fhircast/Patient-open.read

* fhircast/Patient-open.write

* fhircast/Patient-close.read

* fhircast/Patient-close.write

* fhircast/Home-open.read

* fhircast/Home-open.write

* fhircast/syncerror.read

* fhircast/syncerror.write

* fhircast/userlogout.read

* fhircast/userlogout.write

* fhircast/userhibernate.read

* fhircast/userhibernate.write

* For synchronization of , also include:

context

imaging workflows

* fhircast/ImagingStudy-open.read

* fhircast/ImagingStudy-open.write

* fhircast/ImagingStudy-close.read

* fhircast/ImagingStudy-close.write

* fhircast/DiagnosticReport-open.read

* fhircast/DiagnosticReport-open.write

* fhircast/DiagnosticReport-close.read

* fhircast/DiagnosticReport-close.write

* For synchronization of , also include:

context

encounters

* fhircast/Encounter-open.read

* fhircast/Encounter-open.write

* fhircast/Encounter-close.read

* fhircast/Encounter-close.write

* Include the following if your app will also do synchronization:

content

* fhircast/DiagnosticReport-update.read

* fhircast/DiagnosticReport-update.write

FHIRcast Workflows

FHIRcast Context Synchronization Workflow

This is a typical example of a RIS-driven (Epic-driven) PACS integration:

* The user logs in to Epic.

* Epic launches the PACS (automatically at login or from a button) using a SMART on FHIR launch.

* The PACS uses the SMART on FHIR handshake to complete SSO for the user.

* The PACS subscribes to the Epic FHIRcast Hub, with specific events for the user’s session.

* The user selects and opens a study to read in Epic.

* Epic broadcasts Patient-open, DiagnosticReport-open, and ImagingStudy-open FHIRcast events to subscribed applications.

* The PACS reads and interprets these events, validates the content and patient/study matching, and then opens the study in the PACS.

* The user closes the study in Epic.

* Epic sends ImagingStudy-close, DiagnosticReport-close, and Patient-close FHIRcast events.

* The PACS interprets the DiagnosticReport-close event and closes the study in the PACS.

* Optionally, if the study status in the DiagnosticReport-close FHIRcast event indicates that the exam has been read, the PACS marks the study as read.

* The user logs out of Epic.

* Epic sends a userlogout FHIRcast event.

* The PACS interprets the userlogout event and logs the user out of the PACS.

In other (non-imaging) workflows, it’s more common for the user to open the patient without imaging study and diagnostic report contexts. In many cases, when the user opens the patient, Epic triggers Patient-open and Encounter-open events.

Application Initialization, Authorization, and Authentication

Launching Your Application from Epic

Epic's "User Toolbar" launch approach provides the ability for the user to launch your application before opening any patient and provides your application with authorization to view multiple patients' charts without re-launching. This requires coordination with the Epic community member (the healthcare organization using Epic) to show the launch button to end users. Starting in the August 2026 release, Epic also supports auto-start at login for SMART on FHIR integrations, which launches your application automatically when a user logs into Hyperspace, eliminating the need for end users to manually initiate the workflow.

Either approach triggers a SMART on FHIR EHR Launch of your app with a launch token that it may redeem in an Authorization Code flow to receive an access token for subsequent API calls. If your app is a desktop-based (Windows-native) app, review the Considerations for Native Apps such as using App URI Handlers.

Duration of Application Access

By default, Subspace applications receive 1 hour of API authorization per launch. FHIRcast applications typically require longer access, such as when radiologists are using both Epic and PACS for hours at a time.

Starting in the Epic August 2026 release, use SMART on FHIR EHR launch with refresh tokens and the online_access scope. The app receives a refresh token that stays valid for the duration of the user’s Hyperspace session. When the user logs out or switches users, tokens are automatically revoked. This eliminates the need for Dynamic Client Registration (DCR) to maintain longer-lived access. Web-based PACS integrations prior to August 2026 are not well-supported for sustained FHIRcast sessions.

FHIRcast Subscription Sequence

App receives hub.url and hub.topic

Following a successful OAuth 2.0 exchange, Epic's authorization server returns the standard access token response. For FHIRcast, that response additionally includes the base URL of the hub and the session identifier. For example:

```
{
  "access_token": "i8hweunweunweofiwweoijewiwe",
   ...
  "hub.url": "https://127.0.0.1:54797/RZiJaxe1zE_i75Tsx6KZSQ",
  "hub.topic": "fdb2f928-5546-4f52-87a0-0648e9ded065"
}
```

See our OAuth 2.0 Specification and Tutorial for additional details on OAuth2.0 exchanges

Application subscribes to events

The application completes a Subscription Handshake with Hyperspace's Subspace server, using the access token, hub.url, and hub.topic obtained above.

At a high level:

* POST to the hub.url, with the access token in the Authorization header and a form-encoded body specifying hub.topic, hub.mode=subscribe, the events to subscribe to, and hub.channel.type=websocket.

* Epic responds 202 Accepted with a hub.channel.endpoint (a wss:// URL).

* The application connects to that WebSocket.

* Epic sends the final result over the WebSocket: hub.mode: "subscribe" with a hub.lease-seconds value if approved, or hub.mode: "denied" with a hub.reason if not.

Sample subscription request for the Patient-open and Patient-close events:

```
POST https://127.0.0.1:54797/RZiJaxe1zE_i75Tsx6KZSQ
Authorization: Bearer i8hweunweunweofiwweoijewiwe
Content-Type: application/x-www-form-urlencoded

hub.topic=fdb2f928-5546-4f52-87a0-0648e9ded065&
hub.mode=subscribe&
hub.events=patient-open,patient-close&
hub.channel.type=websocket
```

See the Subspace Communication Framework document for additional information on this workflow, including sample accepted and denial responses, resubscriptions, unsubscribing, and subscription/token expiration.

Context Synchronization Events

What’s in a Study?

An ImagingStudy FHIR resource is defined by HL7 to include extensive DICOM study metadata, data that do not exist in Epic. On the other hand, users create DiagnosticReport data in Epic or otherwise connect systems such that Epic becomes the repository for report data. Therefore, in Epic’s implementation, you can think of both resources roughly as representations of an accession number, with a DiagnosticReport adding the details of the result such as narrative and measurements.

In a typical workflow, when a reading provider opens a study in Epic, the Patient-open, DiagnosticReport-open, and ImagingStudy-open events will all be raised. Both ImagingStudy and DiagnosticReport events contain references to the Patient. DiagnosticReport further includes a reference to the ImagingStudy.

In uncommon workflows, such as when a provider opens a study before any technologist documentation has been started, a report context has not yet been created. This can lead to situations where an ImagingStudy event is raised without a DiagnosticReport, so your FHIRcast app should account for this situation in order to keep context in sync. For example, only reload your context if the most recent ImagingStudy processed had a different id. Similar logic is needed to correctly handle scenarios where subscribers send an ImagingStudy-open message, which results in the hub echoing out the ImagingStudy-open message while also sending a related Patient-open message.

Timing of Open and Close Events

Epic’s FHIRcast events are sent in a specific sequence. When opening a study in Epic, the following are sent in order:

* Patient-open

* DiagnosticReport-open (called if the report context exists)

* ImagingStudy-open

Then, when closing, the events are called in the reverse order:

* ImagingStudy-close

* DiagnosticReport-close (called if the report context exists)

* Patient-close

Multi-Tab Considerations

Epic can have multiple contexts open at the same time. For example, a study for patient A can be open on one tab while a study for patient B is open on another.

Additionally, Epic can have workspaces open that do not have a patient, study, or encounter context, such as a reading worklist.

Review the HL7 FHIRcast implementer guidance for additional details around situations such as receiving multiple open events in succession on different patients without close messages, and using the Home-open event. If these situations are not handled well (for example, automatically closing workspaces upon receiving another open message), they can lead to performance issues that occur with repeatedly re-loading context in your app as a user navigates between contexts.

Third-Party Driving Applications

In FHIRcast integrations, Epic is always the coordinating application. However, while the coordinating and driving applications are often the same, there are scenarios where providers may want to use another product as the driving application. For example, in a PACS-driven workflow, the radiologist reads from a worklist of studies in the PACS. When Epic is not the driving application, the driving application can send events to Epic, using its .write scopes, to change the context in Hyperspace.

Patient and Study Matching for Third-Party Driving Applications

When your product is the driving app and a study is opened in your system, you'll raise a DiagnosticReport-open to Epic. To do so, you need the Patient and DiagnosticReport FHIR IDs to identify the context.

If you don't already have the patient's FHIR ID, start by finding it using Patient.$match. Then, using that patient and the accession number or other known metadata, perform a DiagnosticReport.Search to match the specific study. Finally, raise the FHIRcast DiagnosticReport-open event:

```
{
    "timestamp": "2026-10-20T14:21:48.37",
    "id": "k3x8mwq2tbn5r7y",
    "event": {
        "hub.topic": "Qx7mT2pLd9RwKcYe4HbNaA",
        "hub.event": "DiagnosticReport-open",
        "context": [{
                "key": "patient",
                "resource": {
                    "resourceType": "Patient",
                    "id": "eR4mWq2ZtLk8vNx1-bPcJyA3",
                    //...other properties...
                }
            }, {
                "key": "report",
                "resource": {
                    "resourceType": "DiagnosticReport",
                    "id": "eHm6VcQd8ZrTnY2xLw4BsKj9PfA3gUoE7NiXtRyMzWb43",
                    //...other properties...
                }
            }
        ]
    }
}
```

Content Sharing

Content sharing allows your FHIRcast app to file discrete measurements to a study in Epic, such as the ejection fraction from an echocardiography study. This is achieved via a DiagnosticReport-update event as shown below.

```
{
    "timestamp": "2026-10-20T16:38:27.69",
    "id": "8f3a6b92-4c17-4e5d-a0b8-71d2c9e46f35",
    "event": {
        "hub.topic": "Qx7mT2pLd9RwKcYe4HbNaA",
        "hub.event": "diagnosticreport-update",
        "context.versionId": "c5e91d47-82fa-4b36-9d0e-3a7b648f12c0",
        "context.priorVersionId": "6b0d2f85-e3a1-47c9-b5f4-d98a1c7e3052",
        "context": [{
                "key": "patient",
                "resource": {
                    "resourceType": "Patient",
                    "id": "eR4mWq2ZtLk8vNx1-bPcJyA3",
                    //...other properties...
                }
            }, {
                "key": "study",
                "resource": {
                    "resourceType": "ImagingStudy",
                    "id": "eTq9ZkLw3xYpR7mVd2NcHs5JfB0uAoE8G-KiXtWyPz143",
                    //...other properties...
                }
            }, {
                "key": "report",
                "resource": {
                    "resourceType": "DiagnosticReport",
                    "id": "eHm6VcQd8ZrTnY2xLw4BsKj9PfA3gUoE7NiXtRyMzWb43",
                    //...other properties...
                }
            }, {
                "key": "updates",
                "resource": {
                    "resourceType": "Bundle",
                    "type": "transaction",
                    "entry": [{
```

```
                        "resource": {
                            "resourceType": "Observation",
                            "id": "a47e30c8-1b5d-4f92-8e6a-d2c03b9f7714",
                            "status": "preliminary",
                            "code": {
                                "coding": [{
                                        "system": "http://loinc.org",
                                        "code": "79991-6",
                                        "display": "LVEF biplane"
                                    }
                                ]
                            },
                            "issued": "2026-10-20T09:52:46-05:00",
                            "valueQuantity": {
                                "value": 54,
                                "unit": "%",
                                "system": "http://unitsofmeasure.org",
                                "code": "%"
                            }
                        },
                        "request": {
                            "method": "POST"
                        }
                    }
                ]
            }
        }
    ]
  }
}
```

To withdraw a measurement you previously sent, send another DiagnosticReport-update message that repeats the same Observation id and sets request.method to DELETE. Epic clears the value only if that Observation is the one most recently filed for the measurement.

Review the FHIRcast Content Sharing in Subspace specification for additional details on version context, request sequence, and technical limitations.

Connection Testing

Getting Access to the Epic Hyperdrive Client Test Harness

Epic provides the Hyperdrive Test Harness for third parties to test connections with Hyperspace, such as FHIRcast. The FHIRcast test harness is built on the released, production Hyperspace codebase to help ensure that developers experience the same behavior while testing as when deploying their application in customer environments, including production. See full documentation of the process to gain access to the test harness. You'll need two different accounts to access the test harness:

* An account, provisioned by your organization's administrator

Epic on FHIR

* If your organization is not yet enrolled in Epic on FHIR, create a free account.

* If your organization is enrolled, but you do not yet have an account, you must identify the administrator at your organization. If you are unsure who your current admin is, contact us with information about your organization and we can help you identify your organization's admin.

* A Sandbox account for Hyperspace

* Send an email to open@epic.com to request a Hyperspace account for each user.

Once open@epic.com has provisioned and communicated your Hyperdrive account, you must configure and log in to the Test Harness within 48 hours. The "INSTALL INSTRUCTIONS.txt" file within the zipped Test Harness download contains configuration instructions.

1.10 Designing an Efficient Application

File: site/docs/01-guides/10-designing-an-efficient-application.html

Source: https://fhir.epic.com/Documentation?docId=designing_an_efficient_application

App developers have a responsibility to create apps that perform well at scale and do not create an excessive or unexpected load on a customer's system. Be mindful that high API volumes from your app could have a negative outcome for a customer's users.

Performance issues in a customer's production environment are often caused by web service applications generating either (1) extremely high query volumes or (2) moderate volumes of resource-intensive queries.

To avoid these issues, follow our recommended practices below.

Recommended Practices

Your app should be stable and predictable and should not negatively impact operations for users or healthcare organizations. Follow the best practices below to develop an application that runs efficiently.

For apps launched from Hyperspace, aim for an average initial load response time of under 1.5 seconds.

This is a best practice Epic applications also follow. Delays beyond 1.5 seconds will be noticeable to end users and may result in a poor user experience. If your app reads a large amount of data from Epic upon launching the app, this could result in a slower response time. Where appropriate, consider either requesting data ahead of the app launch or queuing data to be loaded after the app is made responsive to the end user so the user is not waiting for the app to load.

Use the right tool for your use case.

Epic offers many different technologies that can be used to exchange data and integrate products. Depending on your use case, certain types of technologies may be more efficient than others. Consider the examples below and keep in mind when you need the data your app pulls from Epic and how frequently you need to pull data.

Event-driven Interfaces

One of the best tools available to you for keeping systems in sync in real time is event-driven interfaces, most commonly HL7v2 interfaces. HL7v2 interfaces are open industry-standard technology that cover a range of data types and triggers that are built for system-to-system integrations where two systems need to stay in-sync about particular types of data over time. Compared to continuously polling APIs, interfaces significantly reduce load on the operational database as the work is only being done at the time of the event.

Example: Your app needs to maintain a list of currently admitted patients. You may use the Outgoing Patient Administration HL7v2 interface to receive updates when a patient is admitted or discharged.

CDS Hooks

For clinical decision support apps, consider CDS Hooks as a way to get a notification of a user workflow and the ability to respond with CDS content in real time. Similar to event-driven interfaces, CDS Hooks allows your app to only pull data at the time of a particular event (ex. placing a specific order) when your app is able to make an actionable suggestion to the end user.

Note that CDS Hooks should only be used to display an actionable alert to the end user. It is not intended to be used exclusively as a notification framework. If your app only needs a notification, see Event-driven Interfaces above.

Example: Your app evaluates medication orders and recommends more appropriate options and dosages. You may use CDS Hooks to trigger an alert when certain medication orders are placed.

RESTful APIs

RESTful APIs, such as FHIR APIs, are very useful for pulling specific data points about individual patients in real time. Apps that launch from Epic with patient context commonly use FHIR APIs to retrieve various types of information about the current patient, including demographics, diagnoses, current medications, and more.

Example: Your app launches from Epic and needs to retrieve information about the current patient's wounds. You might get the patient's FHIR ID through a SMART on FHIR launch and then use Observation.Search (LDA-W) to query for that patient's active wounds in real time when the app is launched.

Only pull the data you need

You should avoid pulling extraneous data wherever possible, as this can put unnecessary strain on your customer's system. Use search parameters to filter your search to only the necessary data. For example, if you are using FHIR Search APIs, you can use the date and time parameters to avoid pulling extra data. Use discrete APIs rather than CDA documents since they provide targeted data sets that are more performant for the consuming and producing systems.

Native vs. Post-Filter Search Parameters

R4 FHIR APIs may support both native and post-filter search parameters. Native search parameters narrow down the search performed by Epic, therefore reducing the performance impact on a customer system. Post-filter parameters, however, are applied to search results after the initial search has been performed using the native search parameters. Post-filter parameters reduce the work required by your application, but do not reduce the load on the customer system.

You should generally avoid making Search queries without any native filtering parameters. Many patient charts contain a large amount of data spanning over a long period of time. Consider whether your app will make use of all of the data returned, and if not, filter your search results to a more specific data set. For example, if you only need clinical notes from the patient's most recent encounter, don't request all clinical notes for that patient. Instead, filter on the encounter identifier.

Avoid API polling

API polling is a practice in which an app constantly or repeatedly queries an API, looking for new data. When apps are designed to poll for information updates, this requires significant resources and degrades performance of the Epic system. There is also the chance that data might be stale by the time it is needed for a workflow and would require additional queries anyway. While use cases for API polling do exist in limited forms, the practice is inherently wasteful. This is because an app does not know when changes have been made to the requested data.

Event-driven technologies can be a way to avoid API polling. Rather than continually calling FHIR APIs to pull the same data points—which may or may not have changed—you can receive updates as they are made to the patient chart.

Cache relatively static data

Your app might require some data that doesn't change often. For this data, consider taking a snapshot of the data and update or refresh as needed. Many FHIR resources, such as Location and Practitioner, do not change frequently, which means your app can store that data and avoid pulling those resources every time they are referenced. For resources that support date filtering, you may be able to cache older clinical data that is not likely to change, such as encounters that have been closed for an extended period of time. Consider how frequently you want to update each resource in your app's database to minimize unnecessary API calls.

Set appropriate expectations with customers

Each customer should understand the data you are requesting from their system and at what frequency. Both you and your customer should understand the expected volume of data being exchanged. For example, if your app makes 100 API calls per patient per day, the total number of API calls per day could vary greatly depending on the size of the customer organization. Make sure your customers understand the impact that your app will have on their system so they can evaluate whether any infrastructure changes are required. Customers can work with their Epic Server Systems TS to make this evaluation.

Include programmatic guardrails to limit the number of API calls made

Add programmatic caps on the volume of API calls made in a given time range that would be outside the bounds of expected usage. Alternatively, implement monitoring or alerting mechanisms to proactively identify issues that could impact the customer's system. Your customers will have different system resources and levels of comfort with different levels of system impact that APIs can cause. Make sure your app has the ability to decrease the frequency of API calls if the customer's system can't handle updates as frequently as you've initially designed.

Stagger API calls

There may be situations where you cannot avoid pulling a very large amount of data. If your app requires a lot of data to be pulled at regular intervals, consider staggering your API calls to minimize your impact on the customer system. For example, rather than making 1,000 API calls at once to pull data for 1,000 different patients, you might split those API calls into groups of 50 or 100 and spread them out over a longer period of time. Coordinate with each customer to schedule your API calls outside of their peak hours.

Test in each customer's non-production environment

Regardless of the technologies you are using or the amount of data you are requesting, it is important to conduct realistic non-production testing. While test data in a customer's non-production environments will likely be more limited than production data, you should aim to replicate a production workflow as closely as possible in terms of volume and frequency. This will allow you and your customer to address any issues prior to going live and avoid disrupting end user workflows.

1.11 Design Strategies to Increase Patient Data Sharing

File: site/docs/01-guides/11-increasingoauthsuccess.html

Source: https://fhir.epic.com/Documentation?docId=increasingoauthsuccess

Application Design Strategies to Increase Patient Data Sharing

This resource outlines our recommended practices to increase the success rate of patient data sharing for patient mediated applications that connect with Epic. We analyzed data from a large set of applications using patient-facing external (i.e., standalone) launches of OAuth 2.0. We found that patients are more likely to share their health data when their user experience is simple, guided, and the use of their data is transparent. While that may sound intuitive and individual application results vary, consistent trends emerged among the applications with the highest patient data-sharing success rates. One application that followed each of our recommended best practices boasted a 90.9% success rate with over 11,000 OAuth flow attempts.

Definitions

Let's get on the same page. Some terms used throughout this resource are defined below.

Success Rate (%): The percentage of OAuth flow attempts that complete end-to-end.

Formula: Success Rate = OAuth Flow Successes ÷ OAuth Flow Attempts × 100

OAuth Flow Attempt (Interactions): When an authorization code is requested. This occurs when a user launches a connection request to a hospital system from the application and is directed to the MyChart login.

OAuth Flow Success (Grants): When the user completes the entire OAuth flow and an access token is granted. This includes entering MyChart login credentials and approving access to health data.

Best Practices

Explain why data access is needed before MyChart login

Applications that included a pre-login disclosure were associated with a 1.85x higher median success rate compared to applications that did not. Effective disclosures explained how the patient's data would be used and what data would be accessed. Implement this into your application by creating a pop-up or notice that a user must view before being redirected to the MyChart login.

What should you tell your users?

* Data security

* Duration of data use

* Deletion of data

* Expected workflow

* Features granted by sharing data

* Privacy protections (e.g., HIPAA, GDPR)

Example pre-login disclosure:

Complete the Data Use Questionnaire

Applications that completed the Data Use Questionnaire had a 1.40x higher success rate compared to those that did not, regardless of their answers. The Data Use Questionnaire can be accessed at the bottom of an app's build page and can only be filled out once per application. Completing the Data Use Questionnaire increases transparency and builds user trust in your application.

Allow users to opt-in or out of data sharing and delete stored health data

Applications that requested permission to share the user's health data and gave users the option to delete stored data were associated with a 1.36x higher success rate compared to applications that did not. Giving users more control over their health data strengthens their confidence in your application.

Other Trends Observed

API Usage

While we recommend only using APIs required for the core functionality of your application, the number of APIs an application uses was shown to have little to no impact on success rate. While using more APIs generally means accessing more categories of patient data, patients can select which information they want to share from the requested list of information.

Application Modality

Success rate varied greatly by application modality (website, mobile app, web-and-mobile). Web-only applications had the highest success rate: 1.70x higher than mobile-only applications and 2.20x higher than web-and-mobile applications. We recommend maintaining consistency in content across both modalities, while designing the user experience to suit the platform.

1.12 Developer Testing Guide

File: site/docs/01-guides/12-testingguide.html

Source: https://fhir.epic.com/Documentation?docId=testingguide

Developer Testing Guide

Our assortment of test harnesses and simulation tools are available to help you achieve a satisfactory proof-of-concept app for many types of data exchange prior to starting an implementation with an Epic customer. Depending on the technologies used to connect to Epic software, you'll find different utility and applicability for each tool. This guide progresses through different stages of testing beginning with no-code Try It (returning results for individual APIs), progress to testing API calls in an API Testing Platform such as Bruno/Postman and then launching from Hyperdrive. All of these tools connect to our FHIR Developer Sandbox ("the Sandbox").

Epic Instances and Environments

Customers

Each of Epic's customers has their own instance, which is a single deployment of Epic's software for an entire health system. Within each instance, there are multiple environments that each contain their own data and configuration for training, testing, reporting, production, and other purposes. Here is a general overview of a typical health system's environment strategy:

* Proof of Concept (POC): Allows for access to initial build and validation of workflows. While this is usually where application build starts and later migrates up, the connectivity for API calls is not always set up in POC.

* Test (TST): Likely where most testing of your app will take place. Better data is available for testing as well as features and network architecture needed for connectivity that aren't always in POC, such as APIs, Interconnect, and interfaces.

* Production (PRD): Where all users' work and patient data is stored.

* Support (SUP): A nightly or, less commonly, weekly copy of PRD. Typically, only Hyperspace is available. It is meant to troubleshoot issues with production data without potentially affecting Production.

* Reporting (RPT): A near real-time copy of PRD and a data source for analytical reporting.

Depending on the customer, POC and TST environments might be refreshed at some point. Make sure to know these schedules before starting to test.

FHIR Developer Sandbox

Epic provides a single Sandbox environment for testing. We ship major version releases of our software each quarter (February, May, August, November) and the Sandbox will be upgraded to the current version within a few weeks of each quarter. During this upgrade, which typically lasts for 1 day, the Sandbox will be unavailable.

The Sandbox is refreshed every Sunday at approximately 8:00 PM Central Time from a static “Source” environment only available to Epic. If you attempt to connect around this time, you may see intermittent errors. Additionally, any data you wrote to the Sandbox from the prior week will be erased. Some developers take the strategy of running scripts on Monday mornings to create the data they need throughout the week for testing.

Each Epic environment has a unique endpoint for your app to use for authorization and API calls. A list of customer-provided endpoints for OAuth2 are published by Epic. Your app should generally be using the most normalized FHIR version applicable to the resource called.

Sandbox Base URL Endpoint: https://fhir.epic.com/interconnect-fhir-oauth/

Sandbox Type | FHIR Base URL
fhir.epic.com Sandbox - R4 | https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/
fhir.epic.com Sandbox - STU3 | https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/STU3/
fhir.epic.com Sandbox - DSTU2 | https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/DSTU2/

API Testing

Use Try It for quick exploration, then switch to an HTTP client to test a complete call.

FHIR API Specifications and Test Data

Explore the full catalog of APIs on open.epic. If you’re using FHIR APIs, you can dive right in to the API specifications through our FHIR developer toolbox and select an individual API to view its spec and Try It examples.

Anatomy of a FHIR API

Lets take an example such as Patient.Search (Demographics) (R4):

* Patient: First, you have the Resource. When we say FHIR allows “modular healthcare data exchange”, the resource is what we’re alluding to. It specifies what kind of data we’re exchanging. You can visit the HL7 FHIR website for additional information about published resources.

* .Search: Second, you have the action. This dictates how we’re exchanging that data. There are a handful of actions you can take with FHIR, but the main ones supported in Epic's implementation are Read—which gives you data about a resource, Search—which helps you find a resource you want to read, Create—which lets you write new data to Epic, Update—which lets you modify an existing resource, and Delete—which let’s you remove data in the context of a clinical decision support workflow. Most resources have both Read and Search, a handful of resources have Create, while Update and Delete are a bit more unique, so check that the action you want is available before planning around it.

* (Demographics): For some FHIR APIs, there is also a subresource which helps us narrow down the scope of some of the more broadly scoped FHIR resources. Subresources are important since it allows your application to more narrowly scope to a specific type of data housed in a give resource. This implementation also affords patients more flexibility in choosing which aspects of their data they want to share with your application. Not all FHIR resources have subresources.

* (R4): And lastly, the FHIR version. The number at the end can help you compare how recent two different standards are. The reason we create new standards is it allows us to update and expand resources without breaking previously established connections. When possible, we recommend working with the most recent or normative FHIR version available. The different versions won’t look fundamentally different from one another, though you’ll often get more data and functionality by going with the newest FHIR version.

At the top of a spec, you’ll see a General Information section that represents the information you’ll need to call the API including the URL template and HTTP method which you’ll use to structure the API call. You’ll want to grab these and paste them into your HTTP client. You’ll also see a Description of the API which explains how the API interacts with Epic’s data structure and any important considerations (e.g., You can't use the Encounter (Patient Chart) resource to find upcoming appointments. Instead, use the Appointment resource to retrieve information on upcoming appointments.)

As you continue to scroll, you’ll first see the native request elements so you know what data is expected to construct a successful call. Pay mind to the “Is Optional” column so you can determine what data is required initially to then later put into the “optional” or “conditional” fields. On certain specs, you’ll see a separate list of parameters that use a post-filtering mechanism. When responding to a request, the Epic FHIR server first retrieves all results that match your search (using any native search parameters you’ve provided), then filters down those results based on the additional post-filtered parameters you’ve specified.

For more information about post-filter parameters and related considerations, refer to the General Considerations topic of the FHIR Search Parameters resource.

When you’re ready to give it a go, use Try It on individual API specification pages to execute preconfigured requests and view responses without writing code. The Try It forms are configurable so you can adjust the parameters to query different results. A list of test patients is available to reference the availability of different resources on each record.

Find an API -> click “Try It” in the top right of the spec page.

The information returned in any one API call can be used as inputs to other APIs. For example, a Patient.Search response may include a generalPractitioner element containing a FHIR ID that can be used with Practitioner.Read.

While Try It cases are convenient and a great way to have reliable test data to reference, they use a hardcoded test app. We ultimately recommend that you test your API calls with an HTTP client such as Bruno or Postman. If you're not quite sure that your URL is formatted correctly, you can expand the Raw Request from a Try It and copy this into your HTTP client.

Responsible Sandbox Testing

The sandbox has many concurrent app connections, and we strive to create high availability through our network architecture. Over time, we have noticed a few potential testing pitfalls that put strain on our Sandbox and may affect not just your testing, but others' testing as well.

* Schedule API calls for a limited window. If you schedule recurring calls, set an endpoint on that schedule and keep it to about a week. Disable the schedule once validated.

* Avoid infinite looping through records. Do not loop across all records in the Sandbox. Instead, select a specific subset/array of records to exercise.

When you've checked all the boxes, you're go for launch! The topics below represent common connection methods to the Sandbox.

Launching & Authorizing Your App with Epic

Prerequisites to Testing

* You have a registered application as described in our App Creation & Request Process Overview and know your Non-PRD client ID.

* If you are building an app that implements OAuth 2.0 as a form of SSO, you have reviewed the Securing your Application topic of the OAuth 2.0 Tutorial and chosen an appropriate SMART App Launch or OAuth 2.0 Library.

* You have reviewed the steps in the topic of the OAuth 2.0 Tutorial, including selection of the relevant APIs for your data exchange use case.

Build Your Application

* Note: There is a rolling sync between the Build Apps page and the Sandbox. You may need to wait up to 30 minutes for newly added APIs to be recognized in the Sandbox environment. This is a common source of 403 errors.

* You have reviewed the FHIR Tutorial and FHIR Search Parameters resource to gain further understanding of how to use FHIR APIs in a performant manner.

* You have reviewed the App Architecture Considerations topic of the OAuth 2.0 Tutorial and have implemented the relevant recommendations.

Standalone OAuth 2.0

SMART on FHIR is a healthcare IT specific standard implementing OAuth 2.0, designed to enable applications to be EHR agnostic. The sections below explain how to instantiate an EHR Launch, however, you can also connect to the Sandbox by implementing a standalone launch. You can use this workflow when you'd like to use OAuth 2.0 for SSO, but when you do not need to be launched from Epic. Your app initiates a login on its own, and Epic presents a login screen where the user can enter their credentials.

LaunchPad

* SMART on FHIR LaunchPad: SMART on FHIR (OAuth 2.0)-> Try It

* HTTP Get LaunchPad: HTTP GET-> Try It

LaunchPad is a lightweight way to validate your app launch from Epic without Hyperdrive. It’s easier to set up and ideal for quick tests of launch, authentication, and context passing. You can also use this tool to receive an access token for use with your HTTP client.

Hyperdrive

Hyperdrive, Epic’s new Windows client application, is a secure client application that includes an embedded browser based on Chromium. Hyperdrive is Epic’s native browser, created specifically for users to access Hyperspace activities and complete workflows while meeting the unique needs of a healthcare organization not found in typical browsers, such as connectivity to peripheral devices and heightened security measures for PHI.

The Hyperdrive client testing harness is a development tool that can help app developers simulate a variety of the common connection types with the Epic Hyperdrive client. The Hyperdrive test harness is particularly useful when your app requires interactions with the Hyperspace client through APIs or when embedding your app in the Hyperdrive client/browser (e.g., through a SMART on FHIR EHR launch). You can review the full list of supported use cases—including E-Signature, Login, Voice Recognition, Scan Acquisition, Scan Viewing, Scan Signature Deficiencies, FHIRcast, XML, Encoder, Web PACS, and SMART on FHIR—and their associated specifications on the Hyperdrive resource page.

When creating your app’s test plan, keep in mind that this is a testing tool to support common client integration use cases. This tool is not intended to provide access to full end user workflows, including Epic's UI. Refer to the steps published here to request credentials, download, and install the client test harness.

MyChart and Patient-Facing Apps

A list of MyChart account credentials is available in the test patients resource to facilitate patient-facing standalone OAuth2.

Backend Systems Apps

Backend apps (i.e., apps without direct end user or patient interaction) can also use OAuth2 through the client_credentials grant type. Epic's OAuth2 implementation for backend services follows the SMART Backend Services: Authorization Guide, though it currently differs from that profile in some respects.

Review our SMART Backend Services (Backend OAuth 2.0) Tutorial for the steps to connect to the Sandbox.

Testing with Epic Customers

For as much testing as you can achieve using our simulators and test harness, collaborating directly with an Epic customer to test and build out your app in one of their Non-PRD environments will be required. Beyond accounting for environment-specific variations like endpoints and the dynamic data points within APIs, the following necessitate customer involvement since they are not available on open.epic:

* Non-OAuth2 launches: Our Sandbox is exclusively configured with an OAuth2 security policy. If you must use Basic Authentication, Client Certificates, or SAML tokens (not recommended), work with a customer to establish the needed connectivity.

* Interface testing: The customer's Interface team will set up the required interfaces and can contact their Epic EDI support for assistance. For general guidance on implementing interfaces, refer to the Getting Started with HL7 Interfaces tutorial.

* MyChart EHR Launch: If your patient-facing app is designed to be embedded or launched from MyChart after a patient has logged in, you can use the SMART on FHIR LaunchPad to validate successful completion of the OAuth2 flow. When you implement with a customer, their IT team will build out a custom MyChart menu item to be available to patients.

* Mobile App Launches and Native Apps: Epic offers several iOS and Android mobile apps for patients and providers, including MyChart, Haiku, Canto, and Rover. All these apps have the same support for SMART on FHIR as their desktop platform counterparts. When you implement with a customer, their IT team will need to complete a nominal amount of application build to make your app available to users of these apps. Starting in the August 2019 version of Epic, the Epic implementation of OAuth 2.0 supports PKCE and Epic recommends using PKCE for native mobile apps. See the Considerations for Native Apps topic in our OAuth 2.0 Specification, if applicable.

Troubleshooting & Assistance

If you run into technical issues with your app's data exchange, first reference our Web App and Web Service Troubleshooting guide for possible solutions. If you're already working in collaboration with an Epic customer, you may also find a resolution by requesting support from their IT team.

If you encounter functional issues with any of the tools or notice something erroneous (e.g., a broken Try It), contact our open.epic team.

For more information about support—including options to receive individualized guidance and testing with workflows that would otherwise require collaboration with an Epic customer—visit our Developer Support page.

1.13 ID Types for FHIR APIs

File: site/docs/01-guides/13-epicidtypes.html

Source: https://fhir.epic.com/Documentation?docId=epicidtypes

ID Types for APIs

Table of Contents

FHIR API IDs and HL7 OIDs

* Identifier Type

* Examples

* Identifier System

* Obtaining OIDs from Customers

* Organization-Specific IDs and Value Sets

FHIR API IDs and HL7 OIDs

Epic's FHIR APIs often use object identifiers (OIDs) to uniquely identify and exchange information about particular entities. An OID is a global, unambiguous identifier for an object. It is formatted as a series of numbers and periods that represent a hierarchy of information. Organizations such as HL7, Epic, and individual healthcare organizations have registered their own unique node in the OID namespace with the International Standards Organization (ISO). Some examples include:

* Epic: 1.2.840.114350

* NPI: 2.16.840.1.113883.4.6

* HL7: 2.16.840.1.113883

OIDs are widely used by a variety of applications. FHIR APIs are only one place where you might see them. For example, HL7v2 messages, HL7v3 messages, and CDA exchanges all involve the use of OID identifiers. HL7 provides a useful site for

reviewing existing registered OIDs: https://www.hl7.org/oid/.

Each organization is responsible for managing additional sub-nodes within their registered space. For example, Epic has registered OID 1.2.840.114350, and has assigned the following child OIDs (among many others):

* 1.2.840.114350.1.13.0.1.7.2.657380 – Represents the Interface Specification (AIP) master file in the open.epic sandbox environment

* 1.2.840.114350.1.13.0.1.7.5.737384.0 – Represents the Epic MRN ID type in the open.epic sandbox environment

* 1.2.840.114350.1.72.3.10 – Represents an observation about a patient FYI in Care Everywhere

Epic also issues child OIDs to each unique organization that uses Epic software, so that identifiers can be uniquely distinguished across organizations. When Epic issues a child OID to an organization, we have a specific structure we use. A common pattern is:

1.2.840.114350.1.13.<CID>.<EnvType>.7.<Value>

The pieces of this example OID represent the concepts outlined in the following table:

OID Segment	Meaning
1.2.840.114350	Epic's root OID
.1	Indicates that this is an application-type ID
.13	The application ID. Some common application IDs include: 13: EDI 72: Care Everywhere
.<CID>	The unique Epic organization ID (FHIR Sandbox Value = 529)
.<EnvType>	The environment type. 1-Epic refers to Epic environments, such as the open.epic sandboxes. 2-Production and 3-Non-production refer to instances of Epic deployed at healthcare organizations. 1: Epic 2: Production 3: Non-production
.7	Indicates that this OID is intend for use in an HL7-type message
.<Value>	The value. This varies greatly depending on the concept we are generating an OID for. It might include multiple additional sub-nodes.

Note that the structure shown above is just one of several that Epic uses. It is included as an illustrative example. OIDs should be treated as opaque identifiers by external applications and systems. However, knowing this common structure can help analysts easily identify issues when troubleshooting.

Identifier Type

Epic uses this term differently than HL7. The distinction is important.

* Within HL7 standards, an identifier type is a categorization of the identifier. For example, "MR" for medical record numbers or "DL" for driver's license numbers. A given patient might have multiple identifiers with the same type (e.g., multiple medical record numbers).

* Within Epic, identifier types are represented by Identity ID Type (IIT) records. IIT records store metadata about a specific set of identifiers. That metadata includes both the HL7 identifier type, (e.g., "MR"), which is stored in the "HL7 ID Type" field of the IIT record, as well as the HL7 identifier system, which is stored in the "HL7 Root" field of the IIT record. The identifier system is the globally unique namespace identifier for the set of identifiers. See the definition of identifier system below.

Examples

The following example in the HL7v2 message format demonstrates how HL7 identifiers are used:

* Format: <ID>^^^<HL7 Assigning Authority>^<HL7 Identifier Type>

* Example: 1234^^^Example Health System^MR

This example indicates that the identifier 1234 was assigned by Example Health System and is a Medical Record (MR) identifier. This example uses a human-readable "Example Health System" value for the assigning authority, but for integrations internal to an organization, often a short textual mnemonic like "EHSMRN" is used to communicate the assigning authority. However, for cross-organizational exchanges, an OID might be necessary because a short textual mnemonic might not be unique across organizations.

Here is a similar example in a FHIR format:

```
<identifier>
    <use value="usual"/>
    <type>
        <coding>
            <system value="http://terminology.hl7.org/CodeSystem/v2-0203"/>
            <code value="MR"/>
        </coding>
    </type>
    <system value="urn:oid:1.2.36.146.595.217.0.1"/>
    <value value="1234"/>
    <assigner>
        <display value="Example Health System"/>
    </assigner>
</identifier>
```

Identifier System

From an HL7 perspective, you might have multiple identifiers with the same type (MR) but with different systems. If two organizations merge, their shared patient population might have two medical record numbers. Both of those identifiers are MR-type identifiers, but they are from different identifier systems. Identifier systems are important because, without qualifying an identifier with an identifier system, you cannot uniquely identify a patient. Ignoring identifier systems can lead to matching to the wrong patient, which can have patient safety implications.

In Epic, the HL7 identifier system is stored in the “HL7 Root” field of the ID Type record. In some cases, if this value is not specified in the IIT record, Epic falls back to using an auto-generated OID that is specific to the current environment. While this value may often follow the structure under Epic's root OID as noted in the table above, healthcare organizations may also register for their own nodes and populate this field on a given ID type with that value.

The meaning of the “HL7 Root” field

HL7 version 3 defined two methods of ensuring identifiers were globally unique: Universally Unique Identifiers (UUIDs), and Object Identifiers (OIDs). Epic software uses the OID method. To communicate unique identifiers, HL7v3 defined an “instance identifier” data type, which had two attributes: a root and an extension. The root attribute contained the root OID of the identifier system, and the extension attribute contained the identifier.

When Epic first developed the ability to store an OID for the identifier system, we created the “HL7 Root” field in Epic IIT records. This was initially created to support HL7 version 3, and the item was named according to the HL7v3 terms, but the use of that item has since evolved to be used by Care Everywhere, HL7v2 and FHIR.

In many integrations, Epic will look first to the “HL7 Root” field for the OID, but if this field is blank, we fall back to an auto-generated OID that is specific to the environment. The auto-generated OID includes a sub-node that indicates the environment type (PRD vs. non-PRD). It also includes a sub-node that identifies the specific healthcare organization. This often works well, as most IIT records are used to represent identifiers generated by that organization. However, this approach won’t work if the IIT record is used to store identifiers you receive from an external organization.

Obtaining OIDs from Customers

Given that specific pieces of an OID are customer / environment specific, you'll need your customers to provide relevant OID values to you. Ask your customer contacts to reference the following Galaxy topic, which will provide them instructions on running our OID Generator utility to pull the needed values. Only customers and Epic have access to the documentation and utility. Simply send them the link.

(Galaxy) Customer-facing OID Generator Utility

Here are examples of common OIDs vendors ask for:

* OID for an Epic Master File

* E.g If you need to know if an ID from Location.Read is a department, you'll need the OID for the "DEP" master file.

* OID for a Category List

* E.g The list of possible note types in a customer environment. (For DocumentReference.Search)

* OID for an Epic Item

* E.g The OID for an Encounter CSN. (For Encounter.Search)

* OID for an Identity Type record

* E.g Identifying a patient's MRN or other customer-specific ID

Organization-Specific IDs and Value Sets

FHIR APIs can sometimes require organization-specific identifiers or value sets. In these cases, developers and Epic organizations need to work together to coordinate usage of those identifiers or value sets.

* FHIR APIs that use Epic-specific values such as Observation.Create (Vitals) use organization-specific OIDs. FHIR APIs that use standard value sets such as AllergyIntolerance.Search use either an OID or an industry URL as the coding system. For FHIR Create operations, you will need to use the OID or URL format, and the IDs will differ from the identifiers used by non-FHIR Epic APIs such as AddAllergy.

When an API returns data, it often returns multiple IDs that reference the same entity, such as a diagnosis or procedure. Each returned ID uses a different coding system. Therefore, you might be able to retrieve the ICD-10 or industry standard codes when an API returns a particular entity. For example, calling the Condition.Search FHIR API for a patient with hypertension returns both the ICD-10 code and an Epic ID for hypertension.

FHIR APIs typically use shared value sets that are linked in the “system” element, and represent well-defined HL7 standards.

The ValueSet.$expand API can in some cases help determine systems and codes supported by a resource. Epic currently supports ValueSet.$expand for the DocumentReference resource. Additional resources might be supported in future Epic versions.

1.14 FHIR Bulk Data Access Tutorial

File: site/docs/01-guides/14-fhir-bulk-data.html

Source: https://fhir.epic.com/Documentation?docId=fhir_bulk_data

FHIR Bulk Data Access Tutorial

Contents

* Background

Important Considerations for Using Bulk Data

* Technical Constraints

When to Use Bulk Data
* Good Use Cases for Bulk Data
* Poor Use Cases for Bulk Data
* Best Practices
* Epic's Implementation of the Bulk Data Standard
Using the Bulk Data APIs
* Client Configuration
Kicking Off the Bulk Data Request
* _type Parameter
* Include Associated Data
* _typeFilter Parameter
* Requirements and Restrictions
* Checking the Status of the Request
* Viewing Resource Files
* Viewing Request Errors
* Deleting Requests

Background

The FHIR Bulk Data Access specification, also known as Flat FHIR, gives healthcare organizations the ability to provide external clients with large amounts of data for a population of patients formatted as FHIR resources.

Important Considerations for Using Bulk Data

The bulk data API is powerful, and because it uses the FHIR standard, you might find it easier to export and process data from bulk data if you are already familiar with that standard. However, there are important things to take into consideration before deciding to use bulk data. As always, carefully consider your use case to determine whether bulk data is the best solution.

Technical Constraints

Bulk data runs on an organization's operational database, so it is important to consider performance. Bulk data exports large amounts of data for large groups of patients, which takes more time to complete the larger the data set or patient population. Responses are not instantaneous, so use cases of bulk data should not rely on immediate responses. Later in this tutorial, we will cover how to set up requests to help minimize this wait time.

Additionally, bulk data requests are not incremental. The API collects all data for the requested patients and resources before it starts returning any data. You cannot retrieve any results until all results are ready.

Once a request has kicked off, you have a window of fourteen days for the search to complete and to download the result files. After two weeks the data is deleted. Once the data is deleted, it is no longer accessible using the bulk data access APIs.

When to Use Bulk Data

As with any data exchange, form should follow function. While having all of this data formatted as FHIR resources might be exciting or sound easier to work with, the data won't be useful if bulk data's technical capabilities don't align with your use case. Consider your exchange paradigm and workflow needs first, then see if bulk data meets those requirements. If it does - great! If it doesn't, explore other options to interoperate with Epic on open.epic. Below are examples of use cases that fit bulk data and some that don't.

Good Use Cases for Bulk Data

* A one-time load of data in preparation for continuous data exchange using other methods
* Monthly loads of a targeted set of data (for example, patient demographics and allergies)
* Weekly export of a dynamic group of patients (for example, all patients discharged in the last week with a certain diagnosis)
* Weekly loads of small patient populations (less than one hundred), such as for registry submissions

Poor Use Cases for Bulk Data

* Data synchronization with data warehouses or other databases
* Periodic loads of large amounts of clinical data
* Incremental data loads

Best Practices

Epic has several recommendations that can help you have the best experience with bulk data APIs.

* Use the _type and _typeFilter parameters whenever possible to improve response time and minimize storage requirements.

* For groups of under one hundred patients, check the request status every ten minutes, or use exponential backoff. For groups of over one hundred patients, check the status every thirty minutes, or use exponential backoff. More information on

exponential backoff can be found in the bulk data export specification.

* After retrieving all of the resource file content, use the FHIR Bulk Data Delete Request API to allow the server to clean up the data from your request.

Epic's Implementation of the Bulk Data Standard

The implementation guide for Bulk Data Access is on HL7's website. Epic supports the 1.0.1 version of the specification for R4 FHIR resources. We have also incorporated some features from the 1.1.0 version to provide additional functionality. Epic supports only the Group Export operation. We do not support _since or other bulk data operations at this time.

Using the Bulk Data APIs

Now that you have determined that bulk data is an appropriate solution for your use case, let’s go through how to use the APIs. This diagram shows the typical flow of the bulk data APIs.

Client Configuration and Authorization

External clients must be authorized to use the bulk data APIs: Bulk Data Kick-off, Bulk Data Status Request, Bulk Data File Request, and Bulk Data Delete Request. Additionally, they must be authorized for the R4 search API for each resource they want to request, for example, AllergyIntolerance.Search (R4). The healthcare organization you're integrating with also needs to authorize your client to access the specific groups of patients. Work with that organization to enable a group that is appropriate for your use case.

This tutorial assumes you are passing a form of authorization covered in one of our authentication guides. It is generally assumed that in production environments FHIR APIs will use OAuth 2.0; however, it's important to note FHIR APIs do support HTTP Basic Authentication.

Kicking Off the Bulk Data Request

To begin, you need the base URL of the organization you want to integrate with, as previously described in the FHIR Tutorial. Let's say this is our base URL:

```
https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4
```

Bulk data uses the Group resource for the export operation. Contact the organization you are integrating with to discuss what group of patients to use for your integration and to get the FHIR ID for that group.

Use the Group FHIR ID to call the group export operation:

```
https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/Group/eIscQb2HmqkT.aPxBKDR1mIj3721CpVk1suC7rlu3yX83/$export
```

The request above returns results for a default set of R4 resources in the ndjson format. The default set includes the resources in the U.S. Core Data for Interoperability (USCDI) data classes, resources from the patient compartment of the bulk data access specification, and additional supporting resources outside of the patient compartment.

Epic also supports the following parameters for the group export operation:

* _type

* includeAssociatedData

* _typeFilter

_type Parameter

The _type parameter accepts a comma-delimited list of FHIR resource types. When used, bulk data returns only the resource types specified in the parameter. Epic recommends using this parameter whenever possible because limiting the scope of the request to only the resources you need decreases both response times and the amount of data stored.

The _type parameter is also the only way to retrieve Binary resources. Binary files can become very large in this workflow, so Binary resources are not returned by default. Certain very large Binary files cannot be retrieved by bulk data operations at all. If a Binary file is not returned, you can still get the ID from the resource that it's associated with, for example, from the DocumentReference.content.attachment.url element, and perform a separate Binary.Read request for the content. If the resource cannot be returned, an OperationOutcome with the resource type and FHIR ID is included in the OperationOutcome response file.

For resources where Epic doesn't support a search by Patient ID (for example, Medication), a resource that has search by Patient ID and references the resource of interest (for example, MedicationRequest) should be included. For example, you could use the _type parameter to limit the request to include only Patient, MedicationRequest, and Medication resources as follows:

```
https://fhir.epic.com/interconnect-fhir-
    oauth/api/FHIR/R4/Group/eIscQb2HmqkT.aPxBKDR1mIj3721CpVk1suC7rlu3yX83/$export?_type=patient,medicationrequest,medica
    tion
```

includeAssociatedData Parameter

The includeAssociatedData parameter can be set to "LatestProvenanceResources" to include the Provenance resource associated with each resource instance included in the bulk data files.

```
https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/Group/eIscQb2HmqkT.aPxBKDR1mIj3721CpVk1suC7rlu3yX83/$export?in
    cludeAssociatedData=LatestProvenanceResources
```

_typeFilter Parameter

The _typeFilter parameter accepts a comma-delimited list of FHIR resource search queries. This is used to filter the results of the bulk data export so you can retrieve only the data you need. To use _typeFilter for a specific resource type, that resource must be included in the _type parameter. You cannot use _typeFilter without also using _type. Query strings for multiple resources can be included in _typeFilter.

Query strings in the _typeFilter parameter use the same search parameters as the R4 search APIs, and are formatted in much the same way. The general format is “_typeFilter=Resource%3Fparameter%3Dvalue\,value%26parameter%3Dvalue.” The standard query string reserved characters (question mark, equals sign, and ampersand) must be URL encoded within _typeFilter and commas must be backslash escaped.

Here is an example of a simple query string in _typeFilter. This would limit the MedicationRequest resources returned in the bulk data export to those with a category of “inpatient.”

```
https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/Group/eIscQb2HmqkT.aPxBKDR1mIj3721CpVk1suC7rlu3yX83/$export?_t
    ype=MedicationRequest&_typeFilter=MedicationRequest%3Fcategory%3Dinpatient
```

This is an example of a more complex use of _typeFilter. This would limit the Observation resources returned to vital signs from 2023 and laboratory results from 2022 onwards, and the Condition resources returned to active problems on the problem list.

```
https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/Group/eIscQb2HmqkT.aPxBKDR1mIj3721CpVk1suC7rlu3yX83/$export?_t
    ype=Observation,Condition&_typeFilter=Observation%3Fcategory%3Dvital-
    signs%26date%3D2023,Observation%3Fcategory%3Dlaboratory%26date%3Dge2022,Condition%3Fcategory%3Dproblem-list-
    item%26clinical-status%3Factive
```

When you use _typeFilter, we recommend following these best practices:

* Avoid using multiple query strings that would have overlapping results. For example, including “Observation%3Fcategory%3Dlaboratory” and “Observation%3Fcategory%3Dlaboratory\,vital-signs” in the same request.

* When possible, simplify _typeFilter values into a single query string. For example, rather than “_typeFilter=MedicationRequest%3Fstatus%3Dactive,MedicationRequest%3Fstatus%3Don-hold” use “_typeFilter=MedicationRequest%3Fstatus%3Dactive\,on-hold”

* Test your query strings with the respective resources in a search request to ensure they return the results you expect before using the query strings in _typeFilter.

Note that the following are not supported by _typeFilter:

* Searching by patient, subject, or _id

* Query strings with search result parameters, such as _count, _include, or _revInclude

* Query strings for the Patient resource

* Query strings for resources that don't contain patient information

* Applying the search query strings to resources included by reference

* Payer-to-Payer API Clients

Requirements and Restrictions

When you have compiled all components of the request URL, you can submit the GET request to kick-off the bulk data workflow. Note that by default, a client can request a specific group of patients only once in a twenty-four-hour period. If you need to request bulk data more frequently, work with the Epic organization you're integrating with to configure an appropriate request window.

Your request must include the following headers:

* Accept: application/fhir+json

* Prefer: respond-async

Checking the Status of the Request

After kicking off the bulk data request, use the status API to track the progress of the request. Note that the same client and user that made the kickoff request must make the status request. In the response headers of the kick-off request, the value of the “Content-Location” header is the status URL for this bulk data request. Each bulk data request has a unique identifier used to request the status of a specific bulk data request.

```
https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/BulkRequest/B0F84FB8D37411EB92726C04221B350C
```

If the bulk data request has not finished processing, the response body is empty. An approximate measure of progress is available in the "X-Progress" response header. Starting in the November 2026 version of Epic, this is set to a value made up of the stage of the request, if processing is actively happening, and the approximate percentage of work complete for the current stage. For example, “Processing - patient data, 15% complete.” See the Bulk Data Status Request API for more details on the possible values of this header. In earlier Epic versions, the value of this header is "Searched X of Y patients" where "Y" is the number of patients in the group and "X" is the number of patients processed so far. This is only an approximate measure of progress because there is additional processing required after all patients have been searched, so it's possible for the header to be set to "Searched 100 of 100 patients" with no response body returned.

Epic recommends pinging for the request status every ten minutes for groups with a hundred or fewer patients, every thirty minutes for groups over a hundred, or using exponential backoff as described in the bulk data export specification.

After the request is completed, the status API returns the URLs for the resource files. For more information on the structure of the response, reference the API details.

```
{
    "transactionTime": "2021-06-23T16:39:52Z",
    "request": "https://fhir.epic.com/interconnect-fhir-
    oauth/api/FHIR/R4/Group/eIscQb2HmqkT.aPxBKDR1mIj3721CpVk1suC7rlu3yX83/$export?_type=patient,encounter,condition",
    "requiresAccessToken": true,
    "output": [
        {
            "type": "Patient",
            "url": "https://fhir.epic.com/interconnect-fhir-
    oauth/api/FHIR/BulkRequest/9ED3042CD44111EB84F2D2068206269D/e19upATM-PTKGZuHsy04IUQ3"
        },
        {
            "type": "Condition",
            "url": "https://fhir.epic.com/interconnect-fhir-
    oauth/api/FHIR/BulkRequest/9ED3042CD44111EB84F2D2068206269D/e8D8N1JwB3qCiHJkmeV.98w3"
        }
    ],
    "error": [
        {
            "type": "OperationOutcome",
            "url": "https://fhir.epic.com/interconnect-fhir-
    oauth/api/FHIR/BulkRequest/9ED3042CD44111EB84F2D2068206269D/eKbGgVw9pUn6xIiB8kZ756w3"
        }
    ]
}
```


Viewing Resource Files

Bulk data generates a separate file for each resource type. The "output" elements in the status API response lists each resource type and the corresponding file request URL. If a single resource has a very large number of results, the data is split into multiple files, all linked in the response. There is a maximum of three thousand resource instances per file, but the actual number of resource instances in a given file varies based on the size of the resource instances included in the file. To view a file, you'll use the file request API. The file URLs for this API are found in the file manifest from the status API. Note that the same client and user that made the kickoff request must make the file request.

```
https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/BulkRequest/9ED3042CD44111EB84F2D2068206269D/e19upATM-
    PTKGZuHsy04IUQ3
```

The format of the bulk data files is ndjson. The ndjson format is similar to JSON, but is newline-sensitive. Resource instances are included in the bulk data file through a search by Patient ID or through a reference in a previously gathered resource instance. The files do not differentiate by patient, so results for all patients are included in each resource file.

Viewing Request Errors

In addition to the resource files, if there are any request-level errors, the "error" element in the status API response includes the URL for the OperationOutcome file. This file includes only request-level errors. Bulk data does not return resource-level errors.

Error Code | Error Text | Notes

59130 | "Some information was not or may not have been returned due to business rules, consent or privacy rules, or access permission constraints." | This error is logged when a resource is requested that the client is not authorized for. OperationOutcome.issue.diagnostics lists the resource.

59100 | "Content invalid against the specification or a profile." | This error is logged when a parameter is included in the request that Epic doesn't support in bulk data. OperationOutcome.issue.diagnostics lists the parameter.

59136 | "The resource or profile is not supported." | This error is logged when a resource is requested that Epic doesn't support in bulk data. OperationOutcome.issue.diagnostics lists the resource.

59176 | "Error Converting FHIR Resource to ndjson. Resource: <resource type> FHIR ID: <resource FHIR ID&gt" | This error is logged when a resource instance cannot be included in the resource file because of its size. The resource instance can be requested separately using the resource's read interaction. This error is logged starting in the November 2021 version of Epic and in August 2021 by special update.

Deleting Requests

If you no longer want to run a request after starting it, or no longer need the data stored, the bulk data delete API is available. This API uses the same URL as the status API, but uses the DELETE HTTP method rather than GET.

```
DELETE https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/BulkRequest/B0F84FB8D37411EB92726C04221B350C
```

If you delete a request before it is completed (the status API hasn’t returned any content), the request does not finish processing and is marked as deleted in the receiving system. The deleted request does not count towards your request per group per client per time period limit.

If you delete a request after it is completed (the status API returns file URLs), the request remains marked as completed in the system, but the data generated by the request is deleted. Deleting the generated data after you've retrieved it can help prevent excessive storage requirements for the server. In this case, the request does count towards the client's request limit. Epic recommends running the delete API after you finish retrieving the data files.

1.15 Implementing Apps at Epic Customers

File: site/docs/01-guides/15-implementing.html

Source: https://fhir.epic.com/Documentation?docId=implementing

Project Planning for a New App Implementation

The table below outlines some basic tasks your team may need to complete while installing an integration with an organization's Epic software. We encourage you to use this as a template for your own project planning materials.

The table does not include early steps like clinical governance approvals or site-specific user acceptance and product design, which are described in Collaborate with Epic Community Members.

The Who Should Be Involved column includes some broader groups to help keep the table easier to read, but here are more specifics on which stakeholders are included in each of those groups:

* Vendor Stakeholders: Project leadership, implementation team, technical team

* Customer Operational Stakeholders: Operational leadership, Super users, Subject matter experts

* Customer IT Stakeholders: Point person for client ID downloads, Epic Application analyst(s) and TS (for the application(s) that owns the workflow that integrates with your app), Interface administrator and Epic EDI (if the app requires interfaces) , ECSA (Web & Service Servers administrator) and Epic Client Systems TS (if the app requires Incoming or Outgoing API traffic or is embedded within Epic) , ECSA (Citrix & Desktop administrator) and Epic Client Systems TS (if the app is embedded in Epic) , Users & security team (if the app requires backend user build or security evaluation for API usage) , Training team (for the application that owns the workflow that integrates with the app being installed, if the app requires changes to user workflows) , Epic Technical Coordinator and BFF (optional)

Task | Who Should Be Involved | Description

Project Prioritization & Contracting | Operational leadership, IT leadership, Vendor leadership | The organization has communicated interest in your app to their operational & IT leadership and the project is ready to begin.

Kickoff Call | Operational stakeholders, IT stakeholders, Vendor stakeholders | Identify who should be involved, designate responsibilities, and set initial timelines for the project.

App Request Process | Point person for client ID downloads, Vendor administrator | To initiate the app implementation process, someone from the organization will request to download your app via the App Request Process.

User & Security Setup | Users & security team | Configure users and security as required by your integration. This requirement is integration and technology-specific. It will be fully owned by the organization.

Interface Setup | Interface administrator and Epic EDI | Configure the required interfaces as needed by your app. You will need to communicate your Interface Type, triggering, and field-level message requirements to the organization. See Interface Setup for Apps for more information about how to approach this communication.

Interconnect Setup | ECSA (Web & Service Servers administrator) and Epic Client Systems TS | Configure the web services server as needed by your app. They can look at your app's client ID to determine your API requirements.

Allow Content | ECSA (Citrix & Desktop administrator) and Epic Client Systems TS | Configuration that allows your app to communicate with their Epic software from outside the organization's firewall and function appropriately. This requirement is integration and technology-specific.

Application Build | Application analyst and TS | Configuration that allows your app to work well with relevant Epic application user workflows. This requirement is integration and technology-specific. See some of the additional sections below for more information about how to approach this communication.

End-to-End Testing | Application analyst and TS, Vendor implementation team | Test standard and edge case workflows to ensure your app is working as expected.

Training | Application analyst and training team, Vendor implementation team | Train users how to use your app and on changes to their Epic workflow.

Cutover to Production | All involved Stakeholders | Prepare to go-live! Move all relevant build in Epic and outside system configuration from test environments to production.

Implementation Kick-off Call

Kick off your app implementation by putting together a meeting with the relevant project stakeholders from your team and the implementing organization. Use this time to talk through the scope of the integration, required setup steps, plan timelines, and inform the organization's project owners of their tasks during this project.

Include representatives from every area that might have a stake in the project. Err on the side of too much representation at the start. Having everyone necessary present during the kickoff is better than trying to bring teams up to speed several weeks into your implementation. The organization may also include their Epic support to help discuss the configuration needed to integrate with your app.

We encourage you to have a list of repeatable, specific project scoping questions for your kick-off meetings to ensure all stakeholders are on the same page to begin any new project. Some initial ideas for project scoping questions that may be relevant to your product are:

* Has the organization used the technologies required by this app before? For example, have they used FHIR? It can be helpful to identify team members who have experience configuring your product's integration technologies.

* Does your app have other hardware or software dependencies? If your app install relies on other hardware or software dependencies, consider how tasks like licensing, ordering, and installing those dependencies could impact project timelines and the teams involved in this project at your organization.

* Which types of users will use this app? Work with the relevant operational stakeholders to help determine the scope of the install and workflow impact of this app on those types of users.

* How much time should we allow for training? Work with your trainers to understand the complexity of workflow changes that this app could introduce to your users.

* What are upcoming timelines for Epic upgrades in different environments? What about build freezes in different environments? Consider how your project timelines might need to account for these events.

Common Implementation Documents

Epic customers will need to review many different parts of your application as part of their governance and scoping reviews. Below are a few examples of resources that Epic customers will likely find useful during the implementation process. Providing these documents at the start of an implementation can help reduce the back-and-forth emails or meetings required to get the information needed for their internal reviews.

API Usage Scope

Epic customers will review your application and all the APIs it will use to access their data. For each API, they will want to know whether it is part of your core functionality, used only in specific use cases, or tied to an optional feature that not all customers use. They will also want a description of the data you plan to retrieve with it. Here is an example of an API Usage Scope document that customers have requested as part of their implementation process.

API Query Parameters

After Epic customers review the total scope of APIs and the data you intend to retrieve via those APIs, they want to know how you are going to retrieve that data. For example, are you planning to retrieve all clinical notes for a patient, only a patient's notes from a specific provider, or only a patient's notes from a specific provider written in the last 6 months or since the beginning of the year? These parameters change the breadth of data you will retrieve and place different performance demands on their system. Here is an example of an API Query Parameters document that customers have requested as part of their implementation process.

Data Dictionary

After customers review the total scope of the APIs and how much data you are planning to retrieve via those APIs, they want to know whether any of the data points you are retrieving are tied to specific code systems (LOINC, SNOMED, etc.). This lets them confirm that the codes you plan to use actually map to those data points in their system. Each organization owns the data in its system and configures it differently, so one organization could use one code for a diagnosis while another uses a different code for the same diagnosis. Providing this resource allows organizations to confirm whether their system uses the codes you are expecting, and Epic customers can then provide a list of codes that will need to be updated in your system to retrieve specific data. Here is an example of a Data Dictionary document that customers have requested as part of their implementation process.

Common Build Resources

Epic customers reference our Galaxy portal for instructions on Epic build. Here is a list of common Galaxy resources that may come up during an implementation. Only Epic customers have access to Galaxy, but you can provide these links in your own implementation materials. Many of the URLs below are included in our generalized Third-Party App Setup and Support Guide or included in support documentation that is linked from that guide.

Galaxy Topic (Customer resources) | Relevant Workflows (App developer resources) | What it covers

User & Security Setup for Apps | SMART on FHIR Backend Apps CDS Hooks Bulk FHIR | Enables users to access your application and/or authorize API calls on their behalf. For frontend apps, this security is applied to real users (clinicians or patients logging into Epic). For backend apps, security is configured for a background user.

Interface Strategy Handbook | Interfaces | Outlines the tools Epic uses for configuring and maintaining interfaces.. Describes Epic's recommendations for the major steps in a successful interface project using those tools.

Interconnect Setup for Apps | SMART on FHIR Backend Apps CDS Hooks Bulk FHIR Interfaces | Interconnect is the secure data transport framework that supports third-party data exchange through Epic. Interconnect needs to be configured to allow the types of exchange your app requires. Customers with many integrations under their belt may have little additional Interconnect setup needed.

Allow Content for Apps | SMART on FHIR Sharing Your App's Web Resources | Deploying web apps at Epic organizations often involves navigating network security. It's important to precisely document the domains that your web app needs to access so customers can add those domains to their web security allowlists.

Launch an App from Hyperspace Launch an App from MyChart Launching an App from Other Locations (Haiku, Canto, & Rover) | SMART on FHIR | Setting up the launch locations and technical configuration needed to launch a web app from Epic.

Launch an App from an OurPractice Advisory Web Service | CDS Hooks | Setting up External Server and OurPractice Advisory (fka. BestPractice Advisory (BPA)) records for use with CDS Hooks apps. Additional SMART on FHIR launch setup if launching a web app from a CDS Hooks card.

Initial Setup for Bulk Data Access Per Application Setup for Bulk Data Access | Bulk FHIR | One-time initial setup for a customer’s first Bulk FHIR app. Per-app setup required for each new Bulk FHIR app.

FHIR Resources API Specifications | Any app with APIs | Some APIs require specific build to function such as code mapping or security points. Customers should refer to the FHIR Resources section of the FHIR setup and support guide for build considerations for FHIR APIs.

Troubleshooting | Apps using OAuth 2.0 Apps launching from Epic | Working through common error messages presented during an app launch from Epic, OAuth 2.0 errors, and web service error codes.

Training for Apps Cutover to Production for Your App Wrap Up Your App Implementation Ongoing Maintenance for Your App | All apps | Miscellaneous recommended practices and reminders as customers seek to go live with an application.

Implementing a SMART on FHIR EHR or Standalone Launch

Many apps launch from a user workflow in Hyperspace to an external web application and use a single-sign-on workflow through SMART on FHIR to log the user in to the external page. SMART on FHIR is the recommended practice for app integrations that launch from Epic. For more information on building with this technology, see our SMART on FHIR launch simulator, and our Hyperdrive Test Harness for testing out your integration using self-service tools.

SMART on FHIR is unique in its support of standalone launch. During a standalone launch, an app can redirect the Epic user to an Epic login page. By authenticating, the user authorizes the app to access information from Epic. You will need to verify that your app has the correct user type set and understand which login credentials can be used in each case. This is the related to the "Who will primarily be using this app?" question. If you have selected:

* Patients: Users must use their MyChart login credentials to authenticate.

* Clinicians or Administrative Users: Users must use their EMP login credentials to authenticate.

As you develop your SMART on FHIR integration, consider how to make your app's integration as performant as possible by limiting FHIR API calls or performing them asynchronously from the web page load. Users want to interact with your app right away, not wait for it to load.

Information to send to the Customer

When you've tested it out and are ready to implement your SMART on FHIR app with a customer, the customer will need just a couple of pieces of technical information to configure the SMART on FHIR launch in Epic. This is the same information you would have used yourself in the SMART on FHIR simulator:

* Client IDs – the organization will follow the App Request process to download your client ID to prepare for your install. For an EHR launch, they will use either the non-production or production client ID in their SMART on FHIR launch configuration, depending on the environment.

* Launch URL – For an EHR launch, the organization's application build team will need the launch URL for the initial landing page that kicks off your SMART on FHIR launch's OAuth 2.0 handshake.

* – for an EHR launch, the organization's application build team will need the list of context tokens that your app needs at the point of launch. These will be in the form of Key=Value pairs. Refer to for a list of possible tokens. It can be easiest to send your customers a table with the values you need. Here is an example:

Launch Context

Token Library

Key | Value
csn | %CSN%

* Test Type (i.e., Launch Type) – for an EHR launch, the organization's application build team will need to understand how the app will launch. The simulator only has a couple options for ease of testing. However, when working with a customer, this gets a bit trickier. We'll dig in below.

* Domains and page actions used by your app – if your app is embedded in Hyperspace, the organization will need to know the launch URL's domain and all subsequent navigation domains as well as the actions and functionality required by your app (i.e., use of JavaScript or cookies). This information is used in the Epic Browser Allowlist and Trusted Sites configuration so your embedded app is allowed to perform actions necessary to your user experience. Review the Sharing Your App's Web Resources tutorial for tips on breaking down the domains your app uses.

Information to get from the Customer

When you're ready to implement, be sure you know what organization-specific information your SMART on FHIR app needs and request that information from their IT team with as much notice as possible. This could include information like:

* Technical Connection information – i.e., the Interconnect instance base URL for FHIR API traffic or IP addresses for their Hyperspace servers that you need to allow.

* Data Mapping Requirements – i.e., - hospital location identifiers that you need to map, clinical note types that you need to map, or authorized users that you need to allow if your app uses security controls based on specific users.

Organization-Specific Considerations

There are numerous launch points and display options that a customer can choose to use when setting up a SMART on FHIR EHR launch. It is key that you work with your users, operational leadership, and IT stakeholders to understand how your app will best-integrate your customers. You will make joint decisions with your users that lead to how the organization's IT team will configure the SMART on FHIR launch to your app. After you've communicated the technical information described above and decided on the workflow design, the organization's IT team can take it from there and engage their Epic Technical Services support if necessary.

For more information about how we recommend engaging all the relevant stakeholders for a new implementation, see Strategies to Effectively Collaborate with Epic Community Members.

Launching Your App

Users will need a point to access your SMART on FHIR EHR launch app and building the workflow overall will help inform which launch point may be best for your users. Even once you've settled on how to embed your app, users may have preferences on how they access your app across organizations, so it's important to have an in-depth scoping discussion ahead of each install.

Certain information about the context of the launch comes from launch button placement, so the information required for the launch, like the required web tokens, might influence how a launch can be built. For example, if the launch requires a patient MRN, the launch must come from a patient chart. Some common launch locations are described below:

* Launch from a Menu Button

* Launch from a Hyperlink in a user's navigators

* Launch from a Hyperlink in a report display

* Launch from a User Toolbar – as opposed to the options above, this launch type is typically reserved for launches that do not need patient or encounter-level information at the point of launch.

* Launch Automatically when a patient chart opens

Displaying Your App

A key piece of your app's user experience is how it's displayed in or alongside Epic. The Hyperdrive Test Harness is a great way to begin to explore options to launch embedded in Epic or as a floating window alongside Epic. Even once you've settled on how to embed your app, users may have preferences on page size or launch type across organizations, so it's important to have an in-depth scoping discussion ahead of each install. Some common launch types are described below:

* Embedded in Hyperspace in the patient chart

* Embedded in Hyperspace as a sidebar

* Embedded in Hyperspace as a report display

* Launched in an external browser running on Citrix alongside Hyperspace

* Launched in an external browser running on a user's local desktop alongside Hyperspace

Be mindful that certain ways to embed an app may require you to have developed for a specific page size, like embedding in the sidebar or report display.

Helpful References for Implementation Documents

We recommend compiling the information above into a document that you can share with customers. While documentation you create should primarily focus on your application's requirements and support resources, you may also choose to include references to Epic's documentation to help customer IT teams easily find the relevant steps from our Third-Party App Setup & Support Guide on Galaxy. You can reference our full list of Common Build Resources and the phrases below can be copied into your materials:

* User Security: Provider-facing apps that use OAuth 2.0 authentication require certain security points on the users launching the app. Ensure that users who will be accessing the app have the security points for the APIs used by this app and have access to launch the button for the app. To determine which security points might be needed for an app install, review the list of APIs on the app and identify this app's FHIR Resources in the FHIR Setup & Support Guide or refer to the API Specifications for specific security points. Refer to the User & Security Setup for Apps topic for more information.

* Interconnect Setup: Refer to the Interconnect Setup for Apps topic for information on how to configure Interconnect as appropriate for the technologies that this app uses. If you need assistance enabling web services or determining the appropriate rewrite patterns for this application, contact your Epic Client Systems - Web and Service Servers TS, and reference SLG 7948970.

* If your application exclusively uses OAuth 2.0 for authentication, you may also include the following statement: If you are currently using the Chronicles-Configured OAuth2 Instance and the OAuth2 ARR Use Case, you do not need to complete any additional OAuth2 Interconnect build for this application. For more information on these topics, see the Interconnect Setup and Support Guide or reach out to your Epic Client Systems - Web and Service Servers TS.

* Allow Content: Refer to the Allow Content for Apps topic for information on how to add this app's web resources to the appropriate allowlists in Hyperspace and in locations throughout your network. Contact your ECSA (Web & Service Servers administrator) (if self-hosted) or Epic Security Administrators (if Epic-hosted) and Epic Client Systems – Hyperspace & Desktop TS if you need assistance.

* Application Build: Refer to the Launch an App from Hyperspace topic for information on and setup steps for the possible user workflow launch points that can be used to launch the app. Work with your operational stakeholders and Epic application representative, as needed, to determine how users should launch and interact with the app.

Implementing a Backend FHIR App

Many apps just need to pull data via FHIR APIs through a secure backend connection. OAuth 2.0 Backend Services is the best-practice for app integrations that need to retrieve patient data via FHIR APIs. Use our TryIt cases in the API Specification library as a starting point.

As you develop your Backend FHIR integration, consider how to make your app's integration as performant as possible by limiting FHIR API calls to just what's needed. For more information on how to think about building a performant app, see the Designing an Efficient Application tutorial.

Information to send to the Customer

When you've tested out your API calls against the sandbox and are ready to implement your Backend FHIR app with a customer, the customer will need to do configuration to allow your app to pull data from the FHIR APIs that are listed as Incoming APIs – this may include User & Security Setup or other application configuration, but you won't need to be involved in this, just like how it's all set up out-of-the-box in the sandbox. They will be able to identify the required configuration based on your Client ID, so the only piece of information you may need to give them is:

* Client IDs – the organization will follow the App Request process to download your client ID to prepare for your install. They will use either the Non-Production or Production client ID in their Backend OAuth 2.0 configuration, depending on the environment. The customer's IT team will map the client ID to an Epic user account for the purposes of auditing and providing

security to invoke web service calls made by your backend application. Epic community members can reference the Backend System Integrations section in the FHIR Setup and Support Guide for the steps needed to complete this build.

* JKUs: Starting in the May 2026 version of Epic, all backend OAuth 2.0 apps must use a JWK Set URL (JKU) instead of a static public key. In addition to adding the JKUs to your app listing and/or customer licenses, you should also explicitly provide your JKU to your customers as they will need to map it to your app as part of their setup, or add it to an allowlist for outgoing network traffic.

Information to get from the Customer

When you're ready to implement, be sure you know what organization-specific information your Backend Systems integration needs and request that information from their IT team with as much notice as possible. This could include information like:

* Technical Connection information – i.e., the Interconnect instance base URL for FHIR API traffic or IP addresses for their Interconnect instances that you need to allow.

* Data Mapping Requirements – i.e., - hospital location identifiers that you need to map, clinical note types that you need to map, or authorized users that you need to allow if your app uses security controls based on specific users.

Helpful References for Implementation Documents

We recommend compiling the information above into a document that you can share with customers. While documentation you create should primarily focus on your application's requirements and support resources, you may also choose to include references to Epic's documentation to help customer IT teams easily find the relevant steps from our Third-Party App Setup & Support Guide on Galaxy. You can reference our full list of Common Build Resources and the phrases below can be copied into your materials:

* User Security: Backend Systems apps that use OAuth 2.0 authentication require a background user for authentication of the application to your database and to enforce security points. To determine which security points might be needed for an app install, review the list of APIs on the app and identify this app's FHIR Resources in the FHIR Setup & Support Guide or refer to the API Specifications for specific security points. Refer to the Backend Authentication User Setup Considerations topic in the User & Security Setup for Apps guide for more information.

* Interconnect Setup: Refer to the Backend OAuth2 (Client Credentials) topic for information on how to set the background user and map the JKU to the app. If you need assistance enabling web services or determining the appropriate rewrite patterns for this application, contact your Epic Client Systems - Web and Service Servers TS, and reference SLG 7948970.

* If your application exclusively uses OAuth 2.0 for authentication, you may also include the following statement: If you are currently using the Chronicles-Configured OAuth2 Instance and the OAuth2 ARR Use Case, you do not need to complete any additional OAuth2 Interconnect build for this application. For more information on these topics, see the Interconnect Setup and Support Guide or reach out to your Epic Client Systems - Web and Service Servers TS.

Interface Setup for Apps

Some apps use one or more interfaces to communicate with Epic. The customer's Interface team will set up the required interfaces and can contact their Epic EDI support for assistance. For general guidance on implementing interfaces, refer to the Getting Started with HL7 Interfaces tutorial

Information to send to the Customer

* A list of interfaces your app requires – Refer to the interface specifications on open.epic

* The communication method you want to use – HL7v2 interfaces support HTTPS or TCP/IP

* For interfaces:

outgoing

* Your HTTPS endpoint or TCP/IP hostname and port

* For HTTPS: Any additional authentication details you require to process interface messages from Epic.

* A high-level description of interface filter criteria:

* For example, you may only be interested in receiving messages for specific events or patients.

Information to get from the Customer

* For incoming interfaces:

* Interface endpoints (HTTPs) or host/port names (TCP/IP)

* For interfaces using communication:

outgoing

TCP/IP

* The customer's sending IP address (in case you need to add it to your allowlist)

* ID types to use in interface messages – Patients, Providers, Encounters, and more can have many different types of IDs in a customer's system. Work with the customer to determine which ID types you'll send on your incoming messages or receive on your outgoing messages.

* HL7v2 versions set on the interface - Interfaces in Epic can be configured to have a set HL7 version they expect to receive (incoming interfaces) or send out (outgoing interfaces) in the MSH-12 - Version ID field.

Helpful References for Implementation Documents

We recommend compiling the information above into a document that you can share with customers. While documentation you create should primarily focus on your application's requirements and support resources, you may also choose to include references to Epic's documentation to help customer IT teams easily find the relevant steps from our Third-Party App Setup & Support Guide on Galaxy. You can reference our full list of Common Build Resources and the phrases below can be copied into your materials:

* Interface Setup for Apps: Work with your Interface team to set up the required interfaces. Refer to the Interface Strategy Handbook and contact your Epic EDI representative for assistance.

Implementing a CDS Hooks App

CDS Hooks apps use web services associated with OurPractice Advisories (OPAs – formerly BPAs) for real-time decision support workflows. Certain Epic workflows can trigger an outbound message to your CDS service which can send back a card that displays as an OPA alert to the Epic user. Depending on your app's functionality, these cards can display text recommendations, enable a specific action like placing an order, or serve as a notification to launch a SMART on FHIR app.

Designing your Workflow

CDS Hooks apps provide decision support at key points during a provider's workflow in interacting with a patient chart in Epic. As such, it's important to have discussions at the beginning of implementation about how your decision support fits into providers' daily work. Here are some key questions to ask of your users:

When should you see our Decision Support?

Your users' desired timing and location of your decision support will inform the IT team's Epic configuration. In the May 2021 version and above, Epic supports the patient-view, order-select, and order-sign hooks. (Refer to the Hooks section of the CDS Hooks Tutorial.) Under the hood, these are broken down into several distinct workflow points when an OPA could be triggered and where it could display.

Discuss with your end users whether your decision support needs to show at a specific point in the workflow, or after a user takes a specific action. This will determine whether your alert will be interruptive or passive. Interruptive alerts show as a popup to the user, and they must act on it to continue their workflow. Passive alerts are displayed alongside their current workflow, and there may be color coding, a score, or an icon to indicate if a user may want to review, but they are not required to act on the CDS to continue their workflow. Here are some common examples of when CDS Hooks cards can show.

Alerts should show: | Interruptive / Passive? | Notes
Once when opening a patient's chart | Interruptive | This is not recommended in most scenarios. Consider the other options below first.
As its own section in-line with the user's workflows | Passive | Customer IT would configure this to show as a "General Navigator section" or "General BPA section" depending on their Epic version.
It should always be visible. | Passive | Customer IT can configure the alert to display in the Storyboard: a sidebar patient summary that is always visible when in the patient's chart.
After selecting an order | Interruptive | This is before the order is signed and active on the patient.
After signing an order | Interruptive
After entering a lab result | Interruptive
After filing other patient data | Interruptive | Example: vitals readings in a flowsheet

What if our Clinical Decision Support is complex and can take a long time to process?

If your CDS Hooks service takes more than a few seconds to process, you should work on improving the overall performance and response time of your service..

If slow response times are unavoidable, and it is okay for users to continue their workflows while waiting for your OPA to load, customer IT teams can enable Deferred Evaluation for your app's OPA record. When enabled, this allows users to begin/continue their workflows while the OPA is processing. If not enabled, Hyperspace will pause until the OPA has finished evaluation.

Should we make our linked SMART on FHIR app launch automatically?

Starting in the August 2023 version of Epic, it's possible for CDS Hooks cards to automatically launch a SMART on FHIR app by including the autolaunchable link attribute. This may be appropriate in situations when your CDS Hooks card has no other decision support besides the app link. Customer IT teams will need to enable this behavior on their side as well, so be sure to discuss with them and your users whether this behavior is appropriate.

Information to send to Customer IT teams

The customer will need just a few pieces of technical information to configure the CDS Hooks app in Epic.

* Client IDs – the organization will follow the App Request process to download your client ID to prepare for your install. They will use either the non-production or production client ID in their CDS Hooks configuration, depending on the environment.

* CDS/Application Endpoint – the organization's application build team will need the endpoint for your CDS service so they can send decision support opportunities your way.

* Prefetch Template – FHIR queries that will provide your CDS service with additional patient data when it's invoked. Customers will enter your prefetch as Key:Value pairs, so it will be easier for them if you provide it in the form of a table instead of in JSON format. Here is an example:

Key | Value
demographics | Patient/{{context.patientId}}

* SMART on FHIR launch details – If your CDS Hooks card can launch a SMART on FHIR app, customers will need information launch your app. Refer to the Implementing a Smart on FHIR App tutorial.

* CDS Hooks API-specific details – If using any of the CDS Hooks ordering .Create APIs (CDS Hooks MedicationRequest.Create, ServiceRequest.Create, or ProcedureRequest.Create). Provide the customer with a list of procedures and/or medications your app can place orders for.

* Workflow Decisions – Let them know the general direction of the workflow decisions made like when users should see your decision support and whether it should be interruptive or passive.

Information to get from Customer IT teams

When you're ready to implement, be sure you know what organization-specific information your CDS Hooks app needs and request that information from their IT team with as much notice as possible. This could include information like:

Technical Connection Information

* The Interconnect instance base URL for FHIR API traffic

* Their JWT Public Keys URL (JKU) for JWT-Based Authentication.

* This is usually in the format of https://<Interconnect-base-URL>api/epic/2019/Security/Open/PublicKeys/530013/530013, however, it can be overridden by the customer on an app-by-app basis.

* Other information needed for Trusting CDS Clients such as the expected values of the JWT claims.

Data Mapping Requirements

* For apps using any of the CDS Hooks ordering APIs, you'll need the mapping of procedures/medications to external IDs your app will use to place orders.

* To receive CDS Hooks Feedback, you'll need the customer's list of possible OPA Acknowledgement Reasons, which you can send in your CDS Hooks card as overrideReasons.

Implementing a Bulk FHIR App

The FHIR Bulk Data Access specification gives you the ability to retrieve large amounts of data for a population of patients formatted as FHIR resources. Work with the customer to determine criteria and estimated size of the groups of patients you will query with your application.

Information to send to the Customer

* Client IDs – the organization will follow the App Request process to download your client ID to prepare for your install. They will use either the non-production or production client ID in their Bulk FHIR configuration, depending on the environment.

* – Work with your customer to determine the population of patients you will query with bulk FHIR. These should be tightly defined groups of patients for specific purposes. Some examples could be "Patients with <A, B, C> conditions" or "Patients currently admitted to <X, Y, Z> departments."

High-level patient group criteria

* Bulk FHIR performance varies based on the size of the patient groups and the number/type of resources you query. Epic conservatively recommends groups of 1000 patients or fewer.

Information to get from the Customer

* Group FHIR IDs – Customers run a utility to produce FHIR IDs for each group of patients your app can query with Bulk FHIR.

1.16 Interfaces (HL7v2)

File: site/docs/01-guides/16-interfaces.html
Source: https://fhir.epic.com/Documentation?docId=interfaces

What is an Interface?

An interface is a mechanism to exchange information from a sending system to a receiving system that contains discrete information about an event that occurred in the sending system. Epic supports a variety of standards-based interfaces including HL7v2, HL7v3, X12, NCPDP, DICOM, and more. By far the most popular standard used by Epic customers and the healthcare industry in general is HL7v2.

Why use an Interface?

Interfaces have been used to exchange healthcare information for decades. Almost all organizations that use Epic have an existing interface team dedicated to implementing and supporting this type of integration. Due to the event-based nature of

interfaces, they typically excel at notifying a receiving system of an event that occurred in the sending system. For example, if you want to be notified when something occurs in Epic (such as a provider placing and order), you can implement an outgoing interface. Conversely, incoming interfaces are used to notify Epic of events that occur in your system.

What are some considerations for using an Interface?

Implementing a new interface connection can sometimes involve more work than exposing and calling an API/web service endpoint. The implementation typically involves setting up a connection between the sending and receiving systems, as well as any required application build and mapping.

Which Interfaces Does Epic support and How Do I See the Specs?

open.epic contains a list of standards-based interfaces supported by Epic along with public specifications https://open.epic.com/Interface

Connecting Interfaces with Organizations that use Epic

Interfaces can be connected directly to Epic, or, more commonly, they are routed through an interface engine. Historically, the vast majority of HL7v2 interfaces used MLLP direct socket TCP/IP connections. If your application is connecting from outside of the organization's network, this type of connection might require additional security considerations like VPNs and adding IPs to an allowlist.

Epic natively supports HL7v2 over HTTP as defined by the HAPI Standard for HL7 over HTTP. Most interface engines support HTTP traffic, but there might be engine-specific considerations like potential licensing fees. When using HL7 over HTTP for Incoming Interface messages (Incoming into Epic), we recommend authenticating your messages with Backend OAuth 2.0. See the Incoming Interfaces section below for more information.

So, when connecting with an organization, consider whether the connection needs to go through an interface engine due to a security requirement by the organization or because using an engine makes the most technical sense for the type of interface.

For example, high volume interfaces that send messages to many receiving systems are almost always sent to the interface engine first and then distributed to each receiving system. An example of this is the Outgoing ADT from Epic interface. Conversely, it's common for incoming interfaces into Epic to be a unique feed from a sending system and thus have more flexibility regarding whether it needs to go through an interface engine.

Organization-specific considerations

There are many organization-specific configuration options that affect how interfaces behave. When implementing at a specific organization, consider the following:

Identity ID Mapping

Each organization has its own configurable set of ID types depending on the entity in question. Examples can include patient IDs, provider IDs, department IDs, hospital IDs, order IDs, result IDs, and more. How these ID types are configured affects how the ID types appear in an interface message. For example, the ID type for a patient's MRN in the PID-3 segment is defined in the fourth piece (PID-3.4). At one organization, it might look like "PID|||1234567^^^MRN|". But at another organization, it might look completely different, such as: "PID|||1234567^^^MEDID|"

Refer to the ID Types for APIs tutorial for additional background on ID types in Epic.

Interface Configurations

Each type of interface is highly configurable to tailor it to certain use cases. There are often dozens or hundreds of configuration options. For example, ORC-2 on an orders interface can contain one of many different order IDs depending on the interface configuration (The internal Epic order ID, accession number, or the external system's order ID).

Some interfaces are shared between multiple vendors, meaning you may need to conform your implementation to what the organization is already using.

HL7v2 Versions

Interfaces in Epic can be configured to have a set HL7 version they expect to receive (incoming interfaces) or send out (outgoing interfaces) in the MSH-12 - Version ID field. Because many Epic organizations copy prebuilt interfaces from Epic's Foundation System, most of their interfaces will have this value set to a specific version (Ex: 2.3). Epic's interface implementation is version-agnostic; we do not change anything about how we process/interpret interface messages based on what's set in MSH-12 besides checking that the value you provide in incoming messages matches what's set on the interface.

When implementing an interface with an Epic organization, ask their interface analyst what's set in the "Version ID (MSH-12)" field on the interface. For Incoming interfaces, you'll need to provide that value in your MSH-12 field. For outgoing interfaces, your system will need to accept that version value when receiving messages from Epic

Time Zones

The organization's time zone is often different from your own. The organization might also have multiple physical locations that cross time zones. This time zone difference affects how you use timestamps in your interface messages.

Encounters In Epic

Epic uses encounters as containers for almost all clinical and financial data tied to a patient. Encounters may sometimes be referred to as visits or contacts. Within Epic, encounters are uniquely identified by the Contact Serial Number, or CSN for short. When you file information using an interface, you must always consider how you are resolving this encounter - either by matching or creation on the fly.

Picking the right encounter can impact clinical workflows for Epic end users when reviewing information about the patient and making clinical decisions. Additionally, it can also impact how a patient is billed for care that is provided. Careful consideration of your workflow is needed to make sure you're implementing the safest, most robust solution.

Encounter Matching

The most robust way to match to an encounter in Epic is through the CSN. For some interfaces, other discrete identifiers may be used as an indirect linkage back to the CSN. We strongly recommend that you leverage CSN matching whenever available in your workflow.

If the CSN or other discrete identifiers are not available in your workflow, Epic interfaces often support fuzzy encounter matching. Fuzzy matching leverages data like date range, departments, providers, procedures, and more to identify the most appropriate encounter. What type of data can be leveraged varies from interface to interface.

Encounter Creation

Many Epic interfaces that write data can also support encounter creation in the event that an appropriate encounter does not exist. It is almost always required to first search for an existing encounter prior to creating a new encounter. For example, if your app has already created an encounter today, it should try to re-use that existing encounter for new data instead of creating another one. As mentioned in the Encounter Matching section, encounter fuzzy matching can generally accomplish this task in the absence of a discrete identifier like CSN.

Appointments

Appointments in Epic broadly cover any encounter that has yet to happen. Because appointments can be cancelled, filing clinical data to an appointment can be risky. Each interface may handle filing to appointments slightly differently, so it's important to think through your app's workflow and how appointment cancelation should be handled.

Outgoing Interfaces

When using Outgoing interfaces, provide the customer with the following information:

* A detailed description of your use cases.

* Endpoint information:

* If using HTTPS:

* What is your HTTPS endpoint?

* Do you require any additional authentication to receive our messages?

* Refer to the Outgoing Interface authentication options (HTTP(s) only) section below.

* If using TCP/IP:

* Ask the customer what their sending IP is (in case you need to add it to your allowlist).

* What is your hostname and port to which we should route messages?

Outgoing Interface authentication options (HTTP(s) only)

If using HTTP(s) with outgoing interfaces, and your system requires some additional authentication to receive outgoing interface messages from Epic, we support the following authentication options. For all of these, you'll need to provide authentication information to the customer.

* Custom HTTP Headers

* We can send out any custom HTTP header with a hardcoded value you provide.

* Example: “api-key: 12345”

* Username & Password

* We can support either Username or Password on their own, or both at the same time.

* X .509 Client Certificates

* Refer to Security Profile Level 3 from the HAPI spec for more information.

* OAuth 2.0 (Starting in the)

November 2023 version of Epic

* We support authenticating to your servers with OAuth 2.0 using the client_credentials grant type. We support JWTs or client_secrets as credentials. If you wish to use OAuth 2.0 with your outgoing interface messages, provide your customer the following information:

* The Client ID you want us to use to authenticate to you.

* The Token URL where Epic should initially retrieve access tokens.

* The to be used in the token request.

client authentication type

* Client Secret

* You'll want to provide the customer your client secret in a more secure method than email.

* JWT

* Provide the customer your desired signing algorithm. (E.g. SHA-384)

* Any additional scopes we should send with our OAuth 2.0 request.

* For your primary endpoint for receiving messages, provide us with your desired HTTP Method (Eg. GET or POST)

Incoming Interfaces

HTTPS endpoints for interfaces will follow the format: {{Customer base URL}}/api/epic/2015/EDI/HTTP/HL7v2/<Interface ID>. The interface ID will be provided by the customer.

To send interface messages via HTTPs, you'll need to create an app with the Backend Systems user type and add HL7v2 as an incoming API. You should authenticate your interface messages with Backend OAuth 2.0. Follow the steps in the Backend OAuth 2.0 specification to create a JWT and obtain an access token. With your HL7 over HTTPS interface message, include an Authorization header set to "Bearer theAccessTokenYouJustObtained"

When testing in customer non-production environments, you should always use an MSH-12 (Processing ID) value of "T" for test. Using "P" for production will fail unless you are in a customer production environment.

When you send an incoming interface message, you might get a basic acknowledgement message that Epic received it. This message does not necessarily indicate that the information in your message filed correctly. Work with your customers to understand their interface error monitoring procedures.

1.17 Patient-Facing Apps Using FHIR

File: site/docs/01-guides/17-patientfacingfhirapps.html

Source: https://fhir.epic.com/Documentation?docId=patientfacingfhirapps

FHIR API Response Variations in Patient-Facing Applications

Some FHIR APIs may return different data when the application is used in a patient-facing context. This typically occurs because data that should not be visible to a patient is filtered out of FHIR API responses in patient-facing applications. Some filtering criteria is configurable by the community member such that two community members could filter the same data in different ways. Below are some examples of possible differences in the results returned by a FHIR API in a patient context:

* FHIR APIs will only return results relevant to the authenticated patient. If one patient, Susan, is logged in to the application, the Observation (Labs) API won't return lab results for a patient other than Susan.

* FHIR APIs may not return patient-entered data that has not yet been reviewed and reconciled by a clinician.

* FHIR APIs may return patient friendly names for resources, such as medications. For example, a patient-facing app may return the medication name pseudoephedrine instead of PSEUDOPHEDRINE HCL 30 MG PO TABS.

* FHIR APIs may be configured to exclude specific lab results to comply with state and local regulations

Community members may choose to disable this filtering functionality for their Epic instance, resulting in near identical behavior between patient-facing FHIR API responses and the response of apps with other audiences.

Epic recommends that you thoroughly test each API at each Community Member's site prior to a go-live in production to ensure that you understand the behavior of each API for that community member's Epic deployment.

Automatic Client Record Distribution

Enabling Your App to Customer Production Environments

Epic customers receive licenses of applications automatically on a rolling 12-hour cycle. Each customer maintains their own instance of Epic, so each environment they maintain is likely to have a unique client record sync cycle. When enabling your app, keep in mind that it may take up to 12 hours before a successful authentication can occur. Refer to our Troubleshooting Tutorial for additional information on common OAuth 2.0 errors.

Automatic Client ID Distribution: USCDI Apps

Client IDs for USCDI apps will automatically download to a community member's Epic instances when all of the following conditions are met:

* The application:

* In Epic August 2024 and later:

* Uses only USCDI v3 FHIR APIs, which are documented in the appendix below †

* In Epic May 2024 and earlier:

* Uses only USCDI v1 FHIR APIs, which are documented in the appendix below †

* Only reads data from Epic

* Is patient-facing

* Does not use refresh tokens OR uses refresh tokens and has a client credential uploaded by the vendor for that community member

* Selects "Enable Auto-download" and "USCDI v1" or "USCDI v3" on the app creation page

* Is marked "Ready for Production" and was marked ready after Sept. 3rd, 2020

* Apps can be saved for sandbox use to test with our Epic on FHIR environment prior to marking the app "Ready for Production"

* The Community Member:

* Is marked to use USCDI APIs in their open.epic licensing agreement

* Has not disabled auto-download functionality

Automatic Client ID Distribution: CMS Payer Apps

Client IDs for apps meant to meet CMS payer requirements will automatically download to a community member's Epic instances when all of the following conditions are met:

* The application:

* Uses only ExplanationOfBenefit (of any supported version) and/or QuestionnaireResponse.Read (Prior Auth) and/or any FHIR APIs (of any supported version) from the list in the appendix below †

* Only reads data from Epic

* Selects "Enable Auto-download" and "CMS Patient Access API" on the app creation page

* Is patient-facing

* Uses OAuth 2.0

* Is marked "Ready for Production"

* Apps can be saved for sandbox use to test with our Epic on FHIR environment prior to marking the app "Ready for Production"

* The Community Member:

* Is marked to use CMS Payer APIs in their open.epic licensing agreement

* Has not disabled auto-download functionality

Automatic Client ID Distribution: TEFCA™ IAS via FHIR Apps

Client IDs for TEFCA IAS via FHIR apps will automatically download to a community member's Epic instances when all of the following conditions are met:

* The application:

* Is patient-facing

* Has completed registration with Epic Nexus

* Has completed testing with Epic Nexus

* Has an active entry in the Production RCE (TEFCA) Directory

* The Community Member:

* Is live on TEFCA and IAS via FHIR

* Has not disabled auto-download functionality

For more information see Epic Nexus's Approach to Individual Access Services in TEFCA via FHIR.

Manual Client ID Distribution

Any applications that do not meet all of the above criteria will need to be manually downloaded to the community member's Epic instances. Steps to do so can be found in the App Creation and Request Process guide.

Appendix

† The following FHIR APIs qualify for USCDI v1 auto-distribution.

* AllergyIntolerance.Read (DSTU2)

* AllergyIntolerance.Read (R4)

* AllergyIntolerance.Read (STU3)

* AllergyIntolerance.Search (DSTU2)

* AllergyIntolerance.Search (R4)

* AllergyIntolerance.Search (STU3)

* Binary.Read (Clinical Notes) (R4)

* Binary.Read (Clinical Notes) (STU3)

* Binary.Read (Generated CCDA) (DSTU2)

* Binary.Read (Generated CCDA) (R4)

* Binary.Read (Labs) (R4)

* Binary.Search (Clinical Notes) (R4)

* Binary.Search (Generated CCDA) (R4)
* Binary.Search (Labs) (R4)
* CarePlan.Read (Encounter-Level) (DSTU2)
* CarePlan.Read (Encounter-Level) (R4)
* CarePlan.Read (Longitudinal) (DSTU2)
* CarePlan.Read (Longitudinal) (R4)
* CarePlan.Search (Encounter-Level) (DSTU2)
* CarePlan.Search (Encounter-Level) (R4)
* CarePlan.Search (Longitudinal) (DSTU2)
* CarePlan.Search (Longitudinal) (R4)
* CareTeam.Read (Longitudinal) (R4)
* CareTeam.Search (Longitudinal) (R4)
* Condition.Read (Care Plan Problem) (R4)
* Condition.Read (Encounter Diagnosis, Problems) (STU3)
* Condition.Read (Health Concern) (R4)
* Condition.Read (Problems) (DSTU2)
* Condition.Read (Problems) (R4)
* Condition.Search (Care Plan Problem) (R4)
* Condition.Search (Encounter Diagnosis, Problems) (STU3)
* Condition.Search (Health Concern) (R4)
* Condition.Search (Problems) (DSTU2)
* Condition.Search (Problems) (R4)
* Device.Read (Implants and External Devices) (STU3)
* Device.Read (Implants) (DSTU2)
* Device.Read (Implants) (R4)
* Device.Search (Implants and External Devices) (STU3)
* Device.Search (Implants) (DSTU2)
* Device.Search (Implants) (R4)
* DiagnosticReport.Read (Results) (DSTU2)
* DiagnosticReport.Read (Results) (R4)
* DiagnosticReport.Read (Results) (STU3)
* DiagnosticReport.Search (Results) (DSTU2)
* DiagnosticReport.Search (Results) (R4)
* DiagnosticReport.Search (Results) (STU3)
* DocumentReference.Read (Clinical Notes) (R4)
* DocumentReference.Read (Clinical Notes) (STU3)
* DocumentReference.Read (Generated CCDA) (DSTU2)
* DocumentReference.Read (Generated CCDA) (R4)
* DocumentReference.Read (Labs) (R4)
* DocumentReference.Search (Clinical Notes) (R4)
* DocumentReference.Search (Clinical Notes) (STU3)
* DocumentReference.Search (Generated CCDA) (DSTU2)
* DocumentReference.Search (Generated CCDA) (R4)
* DocumentReference.Search (Labs) (R4)
* Encounter.Read (Patient Chart) (R4)
* Encounter.Read (STU3)
* Encounter.Search (Patient Chart) (R4)
* Encounter.Search (STU3)
* Goal.Read (Care Plan) (R4)
* Goal.Read (Care Plan) (STU3)
* Goal.Read (Patient) (DSTU2)
* Goal.Read (Patient) (R4)

* Goal.Read (Patient) (STU3)
* Goal.Search (Care Plan) (R4)
* Goal.Search (Care Plan) (STU3)
* Goal.Search (Patient) (DSTU2)
* Goal.Search (Patient) (R4)
* Goal.Search (Patient) (STU3)
* Immunization.Read (DSTU2)
* Immunization.Read (R4)
* Immunization.Read (STU3)
* Immunization.Search (DSTU2)
* Immunization.Search (R4)
* Immunization.Search (STU3)
* Introspect
* Location.Read (R4)
* Location.Read (STU3)
* Location.Search (R4)
* Medication.Read (DSTU2)
* Medication.Read (R4)
* Medication.Read (STU3)
* Medication.Search (DSTU2)
* Medication.Search (R4)
* MedicationOrder.Read (DSTU2)
* MedicationOrder.Search (DSTU2)
* MedicationRequest.Read (Orders) (R4)
* MedicationRequest.Read (Orders) (STU3)
* MedicationRequest.Search (Orders) (R4)
* MedicationRequest.Search (Orders) (STU3)
* MedicationStatement.Read (DSTU2)
* MedicationStatement.Read (STU3)
* MedicationStatement.Search (DSTU2)
* MedicationStatement.Search (STU3)
* Observation.Read (Labs) (DSTU2)
* Observation.Read (Labs) (R4)
* Observation.Read (Labs) (STU3)
* Observation.Read (Social History) (DSTU2)
* Observation.Read (Social History) (R4)
* Observation.Read (Social History) (STU3)
* Observation.Read (Vitals) (DSTU2)
* Observation.Read (Vitals) (R4)
* Observation.Read (Vitals) (STU3)
* Observation.Search (Labs) (DSTU2)
* Observation.Search (Labs) (R4)
* Observation.Search (Labs) (STU3)
* Observation.Search (Social History) (DSTU2)
* Observation.Search (Social History) (R4)
* Observation.Search (Social History) (STU3)
* Observation.Search (Vitals) (DSTU2)
* Observation.Search (Vitals) (R4)
* Observation.Search (Vitals) (STU3)
* Organization.Read (R4)
* Organization.Read (STU3)
* Organization.Search (R4)

* Patient.Read (DSTU2)
* Patient.Read (R4)
* Patient.Read (STU3)
* Patient.Search (DSTU2)
* Patient.Search (R4)
* Patient.Search (STU3)
* Practitioner.Read (DSTU2)
* Practitioner.Read (R4)
* Practitioner.Read (STU3)
* Practitioner.Search
* Practitioner.Search (DSTU2)
* Practitioner.Search (R4)
* Practitioner.Search (STU3)
* PractitionerRole.Read (R4)
* PractitionerRole.Read (STU3)
* PractitionerRole.Search (R4)
* PractitionerRole.Search (STU3)
* Procedure.Read (Orders) (DSTU2)
* Procedure.Read (Orders) (R4)
* Procedure.Read (Orders, Surgeries) (STU3)
* Procedure.Read (Surgeries) (R4)
* Procedure.Search (Orders) (DSTU2)
* Procedure.Search (Orders) (R4)
* Procedure.Search (Orders, Surgeries) (STU3)
* Procedure.Search (Surgeries) (R4)
* Provenance.Read (R4)
* RelatedPerson.Read (Proxy) (R4)
* RelatedPerson.Search (Proxy) (R4)

In addition to the USCDI v1 FHIR APIs above, the following FHIR APIs qualify for USCDI v3 auto-distribution.

* Binary.Read (Study) (R4)
* Binary.Search (Study) (R4)
* Condition.Read (Encounter Diagnosis) (R4)
* Condition.Search (Encounter Diagnosis) (R4)
* Coverage.Read (R4)
* Coverage.Read (STU3)
* Coverage.Search (R4)
* Coverage.Search (STU3)
* Media.Read (Study) (R4)
* Media.Search (Study) (R4)
* MedicationDispense.Read (Fill Status) (R4)
* MedicationDispense.Search (Fill Status) (R4)
* Observation.Read (Assessments) (R4)
* Observation.Read (SDOH Assessments) (R4)
* Observation.Read (SmartData Elements) (R4)
* Observation.Read (Study Finding) (R4)
* Observation.Search (Assessments) (R4)
* Observation.Search (SDOH Assessments) (R4)
* Observation.Search (SmartData Elements) (R4)
* Observation.Search (Study Finding) (R4)
* Procedure.Read (SDOH Intervention) (R4)
* Procedure.Search (SDOH Intervention) (R4)
* RelatedPerson.Read (Friends and Family) (R4)

* RelatedPerson.Search (Friends and Family) (R4)
* ServiceRequest.Read (Community Resource) (R4)
* ServiceRequest.Read (Order Procedure) (R4)
* ServiceRequest.Search (Community Resource) (R4)
* ServiceRequest.Search (Order Procedure) (R4)
* Specimen.Read (R4)
* Specimen.Read (STU3)
* Specimen.Search (R4)
* Specimen.Search (STU3)
* AllergyIntolerance.Read (Outside Record) (R4)
* AllergyIntolerance.Search (Outside Record) (R4)
* Binary.Read (Outside Record Clinical Notes) (R4)
* Binary.Search (Outside Record Clinical Notes) (R4)
* CarePlan.Read (Outside Record) (R4)
* CarePlan.Search (Outside Record) (R4)
* CareTeam.Read (Outside Record) (R4)
* CareTeam.Search (Outside Record) (R4)
* Condition.Read (Outside Record Encounter Diagnosis) (R4)
* Condition.Search (Outside Record Encounter Diagnosis) (R4)
* Condition.Read (Outside Record Health Concern) (R4)
* Condition.Search (Outside Record Health Concern) (R4)
* Condition.Read (Outside Record Problems) (R4)
* Condition.Search (Outside Record Problems) (R4)
* Coverage.Read (Outside Record) (R4)
* Coverage.Search (Outside Record) (R4)
* Device.Read (Outside Record) (R4)
* Device.Search (Outside Record) (R4)
* DiagnosticReport.Read (Outside Record Results) (R4)
* DiagnosticReport.Search (Outside Record Results) (R4)
* DocumentReference.Read (Outside Record Clinical Notes) (R4)
* DocumentReference.Search (Outside Record Clinical Notes) (R4)
* Encounter.Read (Outside Record) (R4)
* Encounter.Search (Outside Record) (R4)
* Goal.Read (Outside Record) (R4)
* Goal.Search (Outside Record) (R4)
* Immunization.Read (Outside Record) (R4)
* Immunization.Search (Outside Record) (R4)
* Location.Read (Outside Record) (R4)
* Location.Search (Outside Record) (R4)
* Medication.Read (Outside Record) (R4)
* Medication.Search (Outside Record) (R4)
* MedicationDispense.Read (Outside Record) (R4)
* MedicationDispense.Search (Outside Record) (R4)
* MedicationRequest.Read (Outside Record) (R4)
* MedicationRequest.Search (Outside Record) (R4)
* Observation.Read (Outside Record Activities of Daily Living) (R4)
* Observation.Search (Outside Record Activities of Daily Living) (R4)
* Observation.Read (Outside Record Occupation) (R4)
* Observation.Search (Outside Record Occupation) (R4)
* Observation.Read (Outside Record Pregnancy Status) (R4)
* Observation.Search (Outside Record Pregnancy Status) (R4)
* Observation.Read (Outside Record Results) (R4)

* Observation.Search (Outside Record Results) (R4)
* Observation.Read (Outside Record Screening Assessment) (R4)
* Observation.Search (Outside Record Screening Assessment) (R4)
* Observation.Read (Outside Record SDOH Assessment) (R4)
* Observation.Search (Outside Record SDOH Assessment) (R4)
* Observation.Read (Outside Record Sexual Orientation) (R4)
* Observation.Search (Outside Record Sexual Orientation) (R4)
* Observation.Read (Outside Record Smoking Status) (R4)
* Observation.Search (Outside Record Smoking Status) (R4)
* Observation.Read (Outside Record Vital Sign) (R4)
* Observation.Search (Outside Record Vital Sign) (R4)
* PractitionerRole.Read (Outside Record) (R4)
* PractitionerRole.Search (Outside Record) (R4)
* Procedure.Read (Outside Record) (R4)
* Procedure.Search (Outside Record) (R4)
* Specimen.Read (Outside Record) (R4)
* Specimen.Search (Outside Record) (R4)

1.18 Requesting a Listing on Showroom

File: site/docs/01-guides/18-chlistingrequest.html
Source: https://fhir.epic.com/Documentation?docId=chlistingrequest

Connection Hub

A listing in Connection Hub on Showroom is available to any application or software product in Production use with at least one Epic customer using mature, repeatable technologies to exchange data. While the listing offers the added benefit of publicly sharing information about your product, a listing is not required for you to connect your software to organizations using Epic. You can facilitate downloads of your application at no cost by providing customers with the associated client IDs.

Eligibility

Review the Connection Hub Listing Terms. By proceeding to request a listing, among other terms, you are attesting that all information you submit is accurate and that your product has a live, active connection to an Epic system using safe, repeatable technologies (e.g., industry standard APIs or interfaces). You also agree to pay the annual, non-refundable Connection Hub listing fee of $500 per product per year.

Listing Request Steps

The steps below assume that you have already logged in to your user account. If you do not yet have an Epic on FHIR account, complete registration by clicking "Sign Up" on the top right corner of the website.

* An app record is required to represent your connection to Epic. If your organization already has an app record on the website in a Production Ready status, proceed to step 2. If you need more information on creating your app, see our App Creation and Request Process document. We recommend completing all available fields.

* HL7 or Interfaces-only apps: Your application may already by live with Epic customers and using technologies to exchange data (e.g., HL7 interfaces) without ever having created an app record. In this case, we recommend only selecting the "Backend Systems" user type. Additionally, in the Description field, briefly explain the technologies your app uses to exchange data.

* Select the app you want to list from your Build Apps page and scroll to the bottom to click the "Request Connection Hub Listing" button.

* Review and accept the Connection Hub Terms of Use.

* Within 15 days, your organization's point of contact should hear back about payment for the listing fee before finalizing your listing.

Toolbox

Within Showroom, Toolbox is a designation for products that use established integration technology in select categories to meet Epic defined recommended connection practices for that area. It's an inclusive program where many vendors who make a product in an eligible category can participate. The selected categories, the design requirements, and products in the categories will be evaluated annually to maintain focus on the most impactful and innovative areas for our customers.

Request a Blueprint and Evaluation

You may request the Blueprint for a Toolbox category by submitting an Technology Guidance Form on open.epic.com. Select "I need to request something not listed here" and describe your product in as much detail as possible.

Participation in Toolbox is at Epic's discretion and requesting Toolbox services requires enrollment in Vendor Services. If you decide to proceed, you will share information about your existing or planned product with Epic to determine the next steps. Apps in a Toolbox-eligible category developed to meet the corresponding Blueprint must complete Toolbox Readiness to receive Toolbox designation on Showroom.

Toolbox Categories

The selection of Toolbox categories is based on a number of criteria including but not limited to: interest from customers; technical feasibility; potential of additional services creating positive outcomes for customers and their patients; and availability of Epic resources to provide services, among others. Epic regularly evaluates categories for Toolbox eligibility.

The list below represents the current Toolbox categories.

Category Name | Category Definition

Alert Manager | Alert Communicator is included in EpicCare Inpatient and requires a third-party Alert Manager. Alert Managers ingest alerts from a variety of devices, such as inpatient vitals monitors, and determine who should get notified for each alert. They then send Epic's Alert Communicator a standardized PCD-06 message indicating the alert recipient, and the relevant alert details. We then display it to the appropriate user(s) as a toast notification in Hyperspace or a push notification in Haiku and Rover. They also handle escalation in scenarios where the original recipient does not respond in an appropriate amount of time.

Ambient Voice Recognition | Ambient Voice Recognition (AVR) apps integrate with Epic's Ambient Voice Recognition Module. Clinicians record their clinical encounters from beginning to end in Haiku or Rover, verbalizing their evaluative process throughout. When they finish recording, the audio is processed by a third-party ambient listening vendor. The vendor processes the conversation and converts it into clinical content. The documentation is then sent to Hyperspace for the clinician to review and finalize. With integration in this area, clinicians can expect to spend less time documenting, provide more attention to patients during visits, and have reduced after-hours time.

Anesthesia Infusion Injectable Hardware | Anesthesia infusion injectable hardware integrations support a hardware device that manages manual IV bolus injections by identifying medication and concentration, performing allergy checks, measuring doses, and automatically documenting the process in real-time in Epic's Anesthesia application. This integration enhances workflow efficiency and accuracy, increasing clinician focus on patient care and reducing manual documentation.

Automated Dispensing System | Automated Dispensing Systems (ADS) manage and include computerized devices that store, dispense, and track medications at the place of service. ADS commonly receive patient and order information to determine what medications are available for dispensing and send dispense information back to Epic when medications are removed from the ADS.

Bayesian Medication Dosing Decision Support | A third-party decision support system used to provide personalized medication dosing recommendations. These products launch from Hyperspace using SMART on FHIR, pull patient and medication information, and provide the end user with optimized medication dosing recommendations. Bayesian analysis is commonly used for medications with narrow therapeutic windows or medications requiring precise dosing based on patient factors. This Toolbox category will no longer be offered in 2027. If you have questions or are interested in this category, reach out to vendorservices@epic.com.

Bedside Dining | Bedside Dining applications present patients dining menus and options to place dining orders within MyChart Bedside.

Bedside TV Hardware | To provide a TV-centric experience in hospitals, vendors need to support hardware-level features such as deploying and app/OS updating, making Bedside TV the main screen, and facilitating seamless video calls using in-room cameras.

Clinical Documentation Improvement | This category consists of two subcategories: NoteReader CDI: NoteReader CDI is a tool used to help clinicians write complete, accurate, and specific clinical documentation. It works with a third-party Computer-Assisted Physician Documentation (CAPD) engine to suggest documentation improvements, called queries or nudges, to clinicians when they are writing a note. These nudges prompt clinicians to address potential issues in their documentation, which can help reduce denied claims and ensure accurate reimbursement for the care provided. Today, NoteReader CDI only supports the inpatient context. Outpatient Risk Adjustment CDI: Outpatient Risk Adjustment CDI is a system that supports clinical documentation improvement (CDI) processes for outpatient encounters by automating chart review, evidence gathering, and clarification management. This functionality integrates risk adjustment recommendations from third-party systems directly into Epic's pre-visit and point-of-care workflows, facilitating comprehensive HCC reviews in a singular decision support tool. CDI staff can review suggestions during pre-visit workflows and tee them up for providers to review during point-of-care workflows, without needing to navigate to other systems or disparate decision support advisories. Supplemental data explaining the insights and evidence will be displayed to end users, and providers' actions will be sent back to the third-party systems, creating a feedback loop with vendors that was not previously available.

Community Resource Network (CRN) | Community Resource Network integrations support closed loop referrals from Epic community members to other organizations on the health grid, like post-acute facilities, community resources, DME suppliers, and more. Case Managers can find availability for their patients nearing discharge, and send information via vendors to post-acute facilities who can accept or decline placement based on the patients' needs and history, as well as capacity and provider availability. Similarly, clinicians who identify Social Drivers of Health impacting patients can refer them to community organizations who can help address patients' needs, all without leaving their workflows within Epic.

Contact Center Productivity Tools | Contact Center Productivity apps enable Computer Telephony Integration (CTI), allowing Epic users to make and receive telephone calls in Hyperspace. Incoming calls bring the user to the activity in Epic that matches the purpose of the call and with the caller identified. Outgoing calls allow the user to initiate a call in the phone system to a selected phone number with a single click.

Collaborative Care Plan Content | Collaborative Care Plans (CCPs) are used by care managers to track and evaluate progress toward resolving problems related to a patient's long- or short-term condition. Vendors provide importable care plans that can replace or augment care plans that customers may already have built. CCP is our recommended tool for almost all outpatient disciplines including Behavioral Health, Rehab, and Care Coordination.

Digital Pathology | Digital Pathology is an integration between Beaker (Epic's laboratory application) and a 3rd party Image Management System (IMS) to allow pathologists to gain access via a hyperlink to high-resolution, digitized images of case slides for examination and interpretation. The hyperlink can be set up as one link for the overall case with all the associated slide images (Case-level) and/or as multiple links to the individual slide images (Slide-level).

Discharge Instructions | The Discharge Instructions category supports integrating third-party educational content directly into Epic workflows, allowing organizations to deliver tailored, current, clinically relevant materials to patients through MyChart and other Epic modules at key points in care. This streamlines the delivery and management of educational materials, reduces paper waste, and ensures patients have easy access to validated content within their existing Epic experience. When content is loaded as SmartText records, it integrates directly into Epic functionality such as SmartSets, the After Visit Summary, and Patient Education workflows, and surfaces in downstream experiences like MyChart Bedside and Care Companion, helping organizations deliver consistent education while reducing workflow fragmentation for clinicians and patients.

Embedded Payment Processor | The embedded payment processor integration (often referred to as the external payment page or EPP) allows MyChart websites, Welcome kiosks and tablets, and/or Hyperspace users to collect credit card and bank account payments in real time, while helping comply with standards to safely handle payment information. This is done by embedding a gateway's URL as a payment page in Epic. The gateway's website handles the collection of the payment directly and posts non-sensitive transaction information back to Epic.

Encoder | Encoder integrations involve using a third-party system to determine and file diagnosis, procedure, and DRG codes based on clinical documentation. Coders or CDS users launch from Epic into the encoder, which interfaces with Epic to ensure accurate coding and data storage.

Encoders can integrate for facility coding workflows and/or for Clinical Documentation Improvement (CDI) workflows.

Endoscope Data Capture | Endoscope data capture integrations allow vendors to gather data from a scope, format it, and then send it downstream. Currently, Epic is focused on image data, though that will expand to video, event, and other data in the future. This data is used throughout Lumens workflows, e.g., documenting findings or writing a procedure note.

End-User Authentication | End-User Authentication includes two subcategories, Reauthentication and Login and Session Management. Reauthentication applications verify the identity of end-users before performing certain actions within Epic's applications such as Hyperspace, Haiku, Canto, or Rover. Login and Session Management applications provide secure and efficient mechanisms for users to access Epic's applications such as Hyperspace, Haiku, Canto, or Rover.

Fully Autonomous Coding | Fully Autonomous Coding applications utilize AI and machine learning to assign billing codes to patient charges or accounts. Fully Autonomous Coding apps receive patient information, clinical documentation, result and charge details from various outgoing interfaces and APIs. Without user intervention, the Fully Autonomous Coding app processes this information, updated charges are sent back to the EHR via an incoming financial transactions (DFT) interface and filed to Professional Billing or Hospital Billing. If the flow cannot be completed autonomously, feedback should be sent to Epic allowing the coder to complete the review in Epic.

Hemodynamics System | Hemodynamic Monitoring Systems are used in Cardiac Cath Labs, Cardiac EP Labs, and some Interventional Radiology Labs to monitor patients during invasive procedures. These systems monitor the patient's vitals and pressures associated with specific cardiac structures. Epic's procedure log is used in conjunction with the existing hemodynamic monitoring system, and the hemodynamic system writes relevant data into Epic's procedure log to keep all the data integrated with the patient's chart in Epic.

Histocompatibility Laboratory Information System | These integrations support transplant programs by performing critical tests that ensure compatibility between organ or tissue donors and recipients and sharing test results with health systems. A health system will place an order for specimen, accension, or laboratory test which will then be performed by the lab. The lab then sends these results via HL7 interface to the health system. Any health system performing Solid Organ or Blood Marrow transplants will use Human Leukocyte Antigen (HLA) crossmatch labs tests.

Home Dialysis | Home dialysis machines are used by patients to administer hemodialysis and peritoneal dialysis at home. These machines also collect clinical data such as vitals that are written to device-entered or patient-entered flowsheets in Epic, maintaining continuity of the patient's chart throughout treatment.

Hospital Utilization Review Creation | Hospital Utilization Review Creation apps provide guidelines for treatment of various medical conditions and the associated reimbursement. Apps in this category integrate with the Utilization Review activity (Clinical Case Management). Utilization managers launch the application and document a patient's symptoms, events, and treatments to see if the patient meets sufficient criteria for admission or discharge and what level of care is needed in the hospital. Afterwards, the app will send the criteria and notes added by the case manager into the Utilization Review activity in Epic.

Identity Verification | The Identity Verification parent category includes Identity Verification for EpicCare Link and MyChart. It involves connecting reusable identity platforms to streamline account creation and recovery for both applications. For EpicCare Link, identity platforms let users assert their identity during New Account Request submission, replacing manual applicant review. For MyChart, the same specification integrates with identity platforms during self-signup, letting users assert their identity regardless of account status or existing patient record, and assists with account recovery when traditional 2FA methods fail.

Imaging Appropriate Use Criteria | Imaging Appropriate Use Criteria (AUC) apps provide decision support for appropriateness of imaging procedures given clinical indications. When imaging orders are placed in Epic, AUC apps are notified and given some clinical context to determine the appropriateness of the order. The decision support outcome may file directly in an Epic OurPractice (OPA, formerly BPA) response, while other times, the app's UI is shown to request additional information.

Infusion Pumps | Bi-directional infusion pump integration allows clinicians to send order details from Epic to auto-program the pump, and then receive documentation information from the pump, such as rate changes and volumes, to file into the patient's chart.

Inpatient Care Plan Content | Inpatient Care Plans can be imported to create the templates, problems, goals, and interventions that will be used to manage and coordinate care for patients.

Inpatient Virtual Care | Inpatient Virtual Care products are in-room cameras used to monitor admitted patients virtually, enabling and supporting virtual nursing, continuous remote monitoring, and teleconsult workflows for admitted patients. The clinician-facing video feed from these cameras can be viewed via an external web application or embedded within Epic Monitor. The patient-facing video feed can be viewed via an external web application or embedded within MyChart Bedside TV. These products may use computer vision-based detection on the in-room video feed to monitor patients for fall risk or other adverse events.

Medication Ordering Decision Support | A third-party decision support system used to suggest medication changes. These products launch from Hyperspace using CDS Hooks to pull medication order information and provide the end user with medication ordering recommendations. The ordering decision support provided should augment the customer's existing medication warning configuration by evaluating patient-specific information to provide a more tailored warning.

MyChart Care Companion Care Plan Content | Apps that create signed XML documents used to import MyChart Care Companion care plans into customer environments. These care plans deliver notifications, analyze data provided by patients and connected devices, and orchestrate changes to the plan and escalations as needed to help patients, their caregivers, and care managers stay on top of a patient's care. A care plan comprises modules of patient-assigned tasks, which are delivered to patients as periodic reminders. These tasks could be reading education materials, responding to questionnaires, performing periodic symptom check-ins, or measuring weight, height, or blood pressure for example.

Nursing Quality Metrics | The Quality Program reporting infrastructure is intended to facilitate the flow of data used to calculate Nursing Sensitive Indicators (NSIs). Vendors use Kit APIs to pull data from Epic and use the data to calculate NSIs for their customers. This functionality is available to customers using Epic's Bugsy application.

Oncology Clinical Pathways Decision Support | A third-party decision support system used to suggest oncology treatment protocols. These products launch from Beacon's Treatment Plan navigator, embedded within Hyperspace, and post the user's selected oncology protocol back to Epic.

Outbound Emergency Medical Services Transportation Ordering | Emergency Medical Services (EMS) Transportation Ordering applications enable hospital staff to order EMS-based transportation services for patients requiring additional medical care during transportation, commonly between medical facilities, by integrating Epic with Computer Aided Dispatch (CAD) systems. These integrations center around outbound transportation from the ordering location. Companies offering these applications may provide transportation services themselves or facilitate integration with transportation services employed directly by the hospital system.

Outpatient Pharmacy Patient Locker | This Toolbox category will no longer be offered in 2027. If you have questions or are interested in this category, reach out to vendorservices@epic.com. The Patient Locker integration for Outpatient Pharmacy is for third-party systems that store pre-filled prescriptions in a locker for patient pick-up. Pharmacists/technicians complete their usual workflow to fill prescribed medications, but rather than waiting for patients to come to the pharmacy and pick up the prescription in person the medication is stored in a patient locker. Once patients know their prescription is ready to be picked up they travel to the locker and the system is able to collect necessary patient identification, dispense the correct medication, and offer counseling services if required.

Outpatient Pharmacy Will-Call | This Toolbox category will no longer be offered in 2027. If you have questions or are interested in this category, reach out to vendorservices@epic.com. The Will-Call integration for Outpatient Pharmacy is for third-party systems that improve the organization of prescribed fills waiting to be picked up. When the prescriptions are filled, they are stored in bags, bins, etc. such that an indicator (e.g., a light) can call attention to the prescription location. This system allows pharmacists, technicians, or other users to easily find the prescription a patient at the counter has come to pick up.

Patient Education Content | The Patient Education Content category allows third parties to streamline the delivery and management of patient-facing educational materials, reduce paper waste, and ensure patients have easy access to relevant, validated content within their existing Epic experience.

Patient Feedback & Insights | Patient Feedback & Insight applications help organizations use Epic to manage and improve patient experience. These applications analyze patient satisfaction surveys administered through MyChart to generate patient insights, give positive recognition to providers, generate customer relationship management tasks for patients whose survey responses require follow-up, and import provider ratings and reviews into MyChart Provider Finder.

Payer Medical Necessity Guideline Creation | Payer Medical Necessity Guideline Creation apps provide care guidelines for treatment of various medical conditions for authorization review. Apps in this category integrate with the Guideline Review activity (Tapestry Utilization Management).

Payment Device Integrations | The Payment Device Integration allows payments to be collected in real time. This is handled by outgoing requests sent to your endpoint during a payment workflow. The first request checks whether the device is initialized while the second request is needed for a payment transaction response. The payment is processed outside of Epic and through the payment processor.

Point-of-care Middleware Integrations | Point-of-care Middleware Integrations are middleware that connect lab hardware to Epic to facilitate receiving results from point-of-care devices into Beaker.

Printing Output Management Integrations | Output Management applications integrate with Epic Print Services (EPS) to collect the jobs exiting from the service that are bound for physical printer devices, which would normally be done through Windows print queues

Procedural Consent Content Import | A clinical consents content vendor provides consent content such as procedure-specific risks, benefits and alternatives that is imported into customer environments. The content can then be pulled into consents created in Epic by clinical staff, spanning many clinical workflows and specialties.

Radiation Therapy Treatment Management System | A radiation therapy Treatment Management System (TMS) is a central hub for patients' radiotherapy treatment plans and manages sending those plans to treatment modalities (e.g., linear accelerators and brachytherapy systems) and receiving and storing records of those treatments. A TMS can focus on plan management, safety, and tracking, leaving other tasks like scheduling, note-taking, and charging to the EHR. This Blueprint describes the recommended integration between a TMS and Epic for smooth workflows between the two systems.

Real Time Location Systems | A Real-Time Location System (RTLS) is used within the healthcare industry to provide real-time visibility into where patients, staff, and assets are located within their facilities. RTLS uses a tag that gets assigned to the entity (patient/staff/asset) they want to locate and a sensory network that's deployed within the facility to determine location of the tag. Epic integrates with RTLS leveraging an HL7v3 interface to communicate incoming patient, staff, and medical device locations, and tag associations. In Epic, the RTLS data is used to provide visibility and automate workflows.

Real Time Prescription Benefit | An integrated real time prescription benefit (RTPB) application uses the native RTPB interface to provide accurate patient-level cost estimates and alternatives to providers at the point of ordering. Using information provided by the interface, RTPB applications offer: more accurate prior authorization information and automatic prior authorization status adjustments, out of pocket costs for the patient before signing orders, potential medication alternatives and their costs, and up front coverage alerts such as step therapy requirements.

Remote Patient Monitoring with Devices | RPM with devices allows providers to monitor and manage their patients' chronic conditions. Devices that are distributed to a patient gather patient-generated health data such as blood pressure, heart rate, or weight. Data is sent back to Epic from the device. These products require patients to be explicitly enrolled in an RPM episode in Epic. Clinicians review the data in Hyperspace and take appropriate follow-up actions.

Remote Interpretation over Telehealth Invitations | Clinicians or staff members can send an invitation to a remote audio or video interpreter for a specified language from within their standard workspace for a telehealth visit. The remote interpreter will join on-demand via the health system's video visit platform to conduct audio or video remote interpretation between the patient and the treatment team. A detailed log of events will be audited in the customer's environment.

Shipping Management System | This Toolbox category definition is Under Construction with expected future renovations. A Shipping Management System vendor provides software that integrates with Epic to manage package shipments. The system receives data such as prescription or supply shipment details from Epic, and returns information including tracking numbers, expected delivery dates, and shipping labels. This integration streamlines the shipping process by embedding package management directly within Epic workflows.

Staff Duress | This integration will enhance staff safety by enabling third-party panic button devices to send location-aware staff duress alerts into Epic using FHIR-based APIs. When triggered, the integration with Epic will identify the user and their location which will then be used within Epic to notify designated security response and informational groups through Epic's Push Notification framework. Critical and informational push notifications will support timely response and coordination, with responders able to acknowledge or decline alerts from within Epic. Other users stay in the loop with informational notifications.

Standard Lab Middleware Integrations | Standard Lab Middleware Integrations are middleware that connect lab hardware to Epic to facilitate receiving results from lab instruments into Beaker.

Translation of Patient Communications | This Toolbox category definition is Under Construction with expected future renovations. Language-to-language translation of MyChart messages, discharge instructions, and other text communication with patients to support quick turnarounds of translation and communication in a patient's preferred language.

Total Parenteral Nutrition Compounding | Total Parenteral Nutrition (TPN) Compounding is the process of preparing sterile solutions for patients to receive intravenously. This process is conducted via TPN compounding machines (compounders), which are computerized medical devices that receive ingredient level information and compound precise amounts of each component. TPNs can be compounded in-house or offsite by a third-party compounding service.

Vitals Gateway and Middleware Device Integration | Vendors in this category receive vitals data from devices like bedside monitors or anesthesia carts. Vendors support either a Spot Check Vitals integration, a Continuous Vitals integration, or both. Spot Check Vitals devices receive discrete data that is validated on the device and then sent to EpicCare Inpatient Clinical Documentation. Continuous Vitals devices receive a constant stream of data which is then sent to EpicCare Inpatient Clinical Documentation or Epic's Anesthesia application.

Waveform Visualization | Waveform visualization apps display waveform data in near real time via a web and/or mobile app that launches from Epic.

Wayfinding | Wayfinding applications using a patient's appointment information, such as encounter department, to help users navigate healthcare organizations.

Rev Cyclers Program

The Rev Cyclers Program is intended to help customers and vendors optimize their use of Epic software to improve their revenue cycle outcomes. Vendors receive customer-level access to Epic tools, materials, and other benefits.

The vendors we choose to invite to the Rev Cyclers Program are driven by various criteria including customer feedback, scope of revenue cycle outsourcing work, and more. This is a new program. As such, we are iterating with a couple early adopters and plan to expand our group of vendors at a later time. If you have any questions about the program, please reach out to us at ConsultantInquiries@epic.com.

Please note that vendors can work with our customers regardless of if they participate in the program. For additional information on ways to interoperate with Epic customers, please visit open.epic where you can find specifications for hundreds of FHIR APIs and other resources to exchange data with members of the Epic community. Additionally, you can reach out to our Consultant Relations team if you have any questions on acquiring system access to support our customers.

1.19 Sharing Your App's Web Resources

File: site/docs/01-guides/19-sharing-web-resources.html

Source: https://fhir.epic.com/Documentation?docId=sharing_web_resources

Deploying web apps at Epic organizations often involves navigating network security. To make your install successful and secure, it's important to precisely document the domains that your web app needs to access. This allows Epic organizations you work with to add those domains to their web security allowlist.

Categorizing Your Resources

When documenting the resources and domains that your application accesses, you should classify that content into the following three categories:

Top-Level Navigations:

Any domain that is navigated to directly or entered into the browser's URL search bar. This includes:

* Link navigation targets

* Domains specified in an iframe src attribute

* HTTP Location header targets

* Targets of a window.location.href assignment

JavaScript or Cookie Accessors:

Any domain that will need to run JavaScript or set cookies.

This includes the domains of any pages containing script tags, as well as the src domains of those scripts. If a domain needs to set cookies in an HTTP Response header (or JavaScript) it should be included as well.

Static Resources:

All other resource domains that your site accesses that do not fall into the above categories belong here.

Be Specific:

Be as specific as possible when listing out the domains and subdomains that your app will access. For example, you should note that your app accesses content from smart.app.com, rather than simply noting that the app accesses the app.com domain.

Epic customers vary widely in their network setup and the strictness of their network security protocols, so you need to be prepared to present the precise subdomains used by your app whenever possible. Let your Epic customer's network administrators identify candidates for wildcarding or broader security allowances.

Wildcarding

While we prefer specificity whenever possible, there are some specific scenarios where wildcarding a subdomain is warranted:

* Your app dynamically generates subdomains per user session. Something like <random_string>.app.com can only be used by wildcarding *.app.com at the appropriate levels.

* You access multiple domains that only differ by the leading subdomain, AND you own the domain. Calls to sub1.app.com, sub2.app.com and etc. could be bundled into *.app.com if there are more than a handful of subdomains to allow, and if you own the app.com domain.

Example Resource Breakdown

```
GET https://smart.app.com/oauth2/redirect HTTP/1.1
Host: smart.app.com

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <body>
        <img src="./fhir.png">
        <script src="./index.js"></script>
        <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
    </body>
</body>
</html>
```

In the site above, there are four HTTP resources that need to be addressed.

* https://smart.app.com/oauth2/redirect

* https://smart.app.com/oauth2/redirect/fhir.png

* https://smart.app.com/oauth2/redirect/index.js

* https://code.jquery.com/jquery-3.5.1.slim.min.js

Top Level Navigations

The following domains need to access "Top Level Navigations"

* smart.app.com - The HTML page returned at /oauth2/redirect is intended for the search bar of a browser

JavaScript or Cookie Accessors

The following domains need to access JavaScript/cookies

* smart.app.com - The script tag set to ./index.js tells us two things about this domain:

* It needs to be able to run JavaScript (Script tags are present in HTML from this domain)

* It should be allowed to return JavaScript over the network (The src of a script tag is for the smart.app.com domain)

* code.jquery.com - To run this jQuery code, the CDN domain needs to run JavaScript as well. While there is no HTML "from" this domain with a script tag, we are targeting this domain to fetch JavaScript over the network.

Other Resources

The following domains need to have generic network access with no special permissions

* smart.app.com - The image returned at /oauth2/redirect/fhir.png is a static resource we know doesn't need to access cookies. In theory the server returning this image could set cookies in it's HTTP Response headers, but for this example assume this is a simple static image fetch.

1.20 Epic as a Backend OAuth 2.0 Client

File: site/docs/01-guides/20-epic-as-client-oauth.html

Source: https://fhir.epic.com/Documentation?docId=epic_as_client_oauth

Epic as Backend Client Using OAuth 2.0 Client Credentials Grant

Updated: February 23, 2026

High Level Technical Summary

Epic's web API client supports the client_credentials OAuth 2.0 grant type, per RFC 6749. A JSON Web Key Set Url (JKU), specific to a given health system, is pre-registered with the external authorization server in exchange for a client id. When calling a web API, the Epic client signs a JSON Web Token (JWT) with a private key to obtain an access token with a client_assertion_type of urn:ietf:params:oauth:client-assertion-type:jwt-bearer per RFC 7515 and RFC 7523.

The authorization server retrieves a list of public keys from the pre-registered JSON Web Key Set Url (JKU). The returned key set contains one to many public keys, each with a key identifier (kid). The JWT used to authenticate to the authorization server identifies the kid of the public key corresponding to the private key used to sign the JWT.

Key Rotation

JWK Set URLs streamline implementation and make key rotation feasible. Epic's web API client's keys can be automatically or manually rotated. Once a new key-pair is generated, the previous key-pair is no longer used to sign credentials. The previous key-pair remains accessible from the JKU for a short window of time.

Registration

Each health system using Epic software should manually register for their own client id with the authorization server and will provide their own JKU.

Additionally, the resource server hosting the APIs accessed by Epic's web API client registers a client id with Epic, which is used to audit outgoing API calls.

Signing Algorithms

Epic's web API client supports the following signing algorithms:

* RSASSA-PKCS1-v1_5 using SHA-256 [RS256]

* RSASSA-PKCS1-v1_5 using SHA-384 [RS384]

* RSASSA-PKCS1-v1_5 using SHA-512 [RS512]

* ECDSA using P-256 and SHA-256 [ES256]

* ECDSA using P-384 and SHA-384 [ES384]

* ECDSA using P-512 and SHA-512 [ES512]

Starting in August 2025, if you use an RSA algorithm, you have the following options for the RSA key's bit length:

* 2048 bit
* 3072 bit
* 4096 bit
* 8192 bit

Examples

Example Token Request

```
POST https://as.example.com/oauth2/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&client_id=<URL encoded client id from authorization
    server>&client_assertion_type=urn%3Aietf%3Aparams%3Aoauth%3Aclient-assertion-type%3Ajwt-bearer&client_assertion=eyJh
    bGciOiJFUzM4NCIsImprdSI6Imh0dHBzOi8vdmVuZG9yc2VydmljZXMuZXBpYy5jb20vaW50ZXJjb25uZWN0LWFtY3VycHJkLW9hdXRoL29hdXRoMi9r
    ZXlzLzIvRTNEOUE5MkNGNTE2QkNEMTVGMjNCMjU0RkNCNTVBNjMiLCJraWQiOiJwLzVwWUo0SjNQVU9MeGhRb0NqZk5KRkNtOE01aTJicVN0bjcvODRG
    T1B3PSIsInR5cCI6IkpXVCJ9.eyJhdWQiOiJodHRwczovL2VuOWYzMWU1d2kzeG1mcS5tLnBpcGVkcmVhbS5uZXQiLCJleHAiOjE2OTIwMjY2NTAsIml
    hdCI6MTY5MjAyNjUzMCwiaXNzIjoiYjcwNjUyOTYtY2M0OC00YWE0LWI2NWMtNzI4YzYzODNmZWMwIiwianRpIjoiZGYzZTkwNGYtNzY4YS00ODhlLTl
    lY2EtNWRkNzA2N2NhNWRjIiwibmJmIjoxNjkyMDI2NDcwLCJzdWIiOiJiNzA2NTI5Ni1jYzQ4LTRhYTQtYjY1Yy03MjhjNjM4M2ZlYzAifQ.yJdbzVcD
    IZotiudVvlzgoJF3wWJ5ZSCh4es9Ro7OtnIV8FgeXCnF1QiVOU9Aj2Uth9SYLyXgc09VgHJ9pgzG6CZVWJRDs0uZJAHPq7TBhvLmpr9bYvqHFKC1qpAv
    w22a
```

Example Token Response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
    "access_token": ******,
    "token_type": "Bearer",
    "expires_in": 3600,
    "scope": "..."
}
```

Example JWT

(Also see in jwt.io).

```
{
    "alg": "ES384",
    "jku": "https://vendorservices.epic.com/interconnect-amcurprd-oauth/oauth2/keys/2/E3D9A92CF516BCD15F23B254FCB55A63",
    "kid": "p/5pYJ4J3PUOLxhQoCjfNJFCm8M5i2bqStn7/84FOPw=",
    "typ": "JWT"
}
.
{
    "aud": "https://en9f31e5wi3xmfq.m.pipedream.net",
    "exp": 1692026650,
    "iat": 1692026530,
    "iss": "b7065296-cc48-4aa4-b65c-728c6383fec0",
    "jti": "df3e904f-768a-488e-9eca-5dd7067ca5dc",
    "nbf": 1692026470,
    "sub": "b7065296-cc48-4aa4-b65c-728c6383fec0"
}
```

Example JKU Request

```
GET https://app.example.com/oauth2/keys/2/480786374A752C71B56B83471AAFEDFF
```

Example JKU Response

```
HTTP/1.1 200 OK
Cache-Control: public, must-revalidate, max-age=3300,no-store
Content-Length: 258
Content-Type: application/json; charset=utf-8
Access-Control-Allow-Credentials: true
Access-Control-Allow-Headers: origin, authorization, accept, content-type, x-requested-with, prefer, Epic-User-ID, Epic-
    User-IDType, Epic-Client-ID, soapaction, Epic-MyChartUser-ID, Epic-MyChartUser-IDType
Access-Control-Allow-Methods: GET, HEAD, POST, PUT, DELETE, TRACE, OPTIONS
Access-Control-Allow-Origin: *

{
    "keys": [{
        "kty": "EC",
        "kid": "iTqXXI0zbAnJCKDaobfhkM1f-6rMSpTfyZMRp_2tKI8",
        "crv": "P-521",
        "x": "C1uWSXj2czCDwMTLWV5BFmwxdM6PX9p-Pk9Yf9rIf374m5XP1U8q79dBhLSIuaoj",
        "y": "svOT39UUcPJROSD1FqYLued0rXiooIii1D3jaW6pmGVJFhodzC31cy5sfOYotrzF",
        "alg": "ES512",
        "use": "sig"
    },
    {
        "kty": "EC",
        "crv": "P-521",
        "x": "AUbOrsnOvzbJUmtX75Y4gAK9R5xwlRYn_u1SD1FiN6qkR82Nn0fUB_Am07fTQDCY9tXGIC3U-zu0OaWEHIuXRU1s",
        "y": "ANBHM0k9mOY_Rpq8N7rr-UE7NQfZ4OhIMvB1vhbELskVHzqo-sdoP39kppS105uKRopnGc7oHyZY8xjkO-REMw5L",
        "use": "sig",
        "alg": "ES512",
        "kid": "EwWsxS6/Fedwvbx2qTE3qx++7h9N6bA4Q0RbRNg2bkA="
```

```
    }]
}
```

Reference Documentation

Standards Documentation

* RFC5246, The Transport Layer Security Protocol, V1.2

* RFC6749, The OAuth 2.0 Authorization Framework

* RFC7515, JSON Web Signature

* RFC7517, JSON Web Key

* RFC7518, JSON Web Algorithms

* RFC7519, JSON Web Token (JWT)

* RFC7521, Assertion Framework for OAuth 2.0 Client Authentication and Authorization Grants

* RFC7523, JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants

1.21 FHIR Tutorial

File: site/docs/01-guides/21-fhir.html

Source: https://fhir.epic.com/Documentation?docId=fhir

Can be executed using a POST using this format: If both query string parameters and a POST body are provided to the _search endpoint, the POST body is ignored. Starting in the February 2026 version of Epic, query string parameters, except for _format, are ignored by the _search endpoint. There are a few unique things to note about Epic's implementation of date search prefixes: Currently in FHIR, most searching is done by Date only, while the response can return a DateTime value which will be in the same format as would be used in a search request.

FHIR makes it easy to quickly get patient data from a clinical system, and to connect together queries to obtain a wide variety of related healthcare information. The tutorial below includes a walkthrough introduction to FHIR data and sample patient and clinical information - everything you need to turn up the heat on your first FHIR app!

Let's begin with a few examples using jQuery — in a larger application, you might build your own framework, or use an existing one, to interact with FHIR, but FHIR is easy enough to use without any special libraries. This tutorial assumes you are passing a form of authorization covered in one of our authentication guides. It is generally assumed that in production environments FHIR APIs will use OAuth 2.0; however, it's important to note FHIR APIs do support HTTP Basic Authentication.

Interacting with the API

To get started, let's define the FHIR server base URL. This is a constant address for a server where all FHIR API endpoints for a given FHIR version live. In a SMART on FHIR launch, this is specified with the iss parameter. We'll store this variable as a separate entity, so we can reference it across our queries:

```
var baseUrl = "https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4"
```

You can expect the bold part of the above URL to be different for each unique instance of Epic you integrate with.

Finding Patients via Demographic Data

With the base URL defined, we can start building queries. Because we're interacting with patient data, there's an obvious place to get started: finding the patient whose data we are interested in. Epic's FHIR support provides two methods for applications to find and interact with patients:

* The Epic application can provide a patient to you as part of the application launch context, determined from the Epic workflow or the patient selection activity. This method can apply to both clinician and patient-facing workflows.

* Our Patient API provides a way to match on a single high-confidence patient using demographic information such as patient name, birthdate, address, and other identifying information. We use this method when your application is independent from Epic.

```
var patientMatchString = "/Patient/$match"
```

We can use jQuery's $.ajax() function to send patient demographic data to the server using an HTTP POST request. It takes a parameter describing the URL, the data to send to the server, and a callback for doing something with the data once the API responds. In our case, let's just log the data to the console so we can inspect it before developing code that works with it:

```
$.ajax({
  url: baseUrl + patientMatchString,
  type: "POST",
  contentType:"application/json; charset=utf-8",
  dataType: "json",
  data: dataToSend,
  success: function(data) { console.log(data); }
});
```

The complete request URL at this point will read:

```
"https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/Patient/$match"
```

The dataToSend will be a FHIR Parameters resource. In this case, there are two parameters:

* The “resource” parameter, which is a partial Patient resource with demographic information about the patient you want to find.

* The “onlyCertainMatches” parameter, which is set to true, indicating that the server should only return single high-confidence matches based on your demographic information.

```
{
  "resourceType": "Parameters",
  "id": "example",
  "parameter": [
    {
      "name": "resource",
      "resource": {
        "resourceType": "Patient",
        "identifier": [
            {
                "use": "usual",
                "type": {
                    "text": "EPI"
                },
                "system": "urn:oid:1.2.840.114350.1.1",
                "value": "27475"
            }
        ],
        "name": [
          {
            "family": "Smith",
            "given": [
              "John"
            ]
          }
        ],
        "birthDate": "1906-03-04"
      }
    },
    {
      "name": "count",
      "valueInteger": "1"
    },
    {
      "name": "onlyCertainMatches",
      "valueBoolean": "true"
    }
  ]
}
```

If you execute the above request, you'll see the following full Patient resource response in your console:

```
{
    "resourceType": "Bundle",
    "type": "searchset",
    "total": 1,
    "link": [
        {
            "relation": "self",
            "url": "https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/Patient/$match"
        }
    ],
    "entry": [
        {
            "link": [
                {
                    "relation": "self",
                    "url": "https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/Patient/ecBjL8PUhljoHWMtwx63UhA3"
                }
            ],
            "fullUrl": "https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/Patient/ecBjL8PUhljoHWMtwx63UhA3",
            "resource": {
                "resourceType": "Patient",
                "id": "ecBjL8PUhljoHWMtwx63UhA3",
                "extension": [
                    {
                        "valueCodeableConcept": {
                            "coding": [
                                {
                                    "system": "urn:oid:1.2.840.114350.1.13.5325.1.7.10.698084.130.768080.39128",
                                    "code": "male",
                                    "display": "male"
                                }
                            ]
                        },
                        "url": "http://open.epic.com/FHIR/StructureDefinition/extension/legal-sex"
                    },
                    {
                        "valueCodeableConcept": {
                            "coding": [
                                {
                                    "system": "urn:oid:1.2.840.114350.1.13.5325.1.7.10.698084.130.768080.35144",
                                    "code": "male",
                                    "display": "male"
```

```
                        }
                    ]
                },
                "url": "http://open.epic.com/FHIR/StructureDefinition/extension/sex-for-clinical-use"
            },
            {
                "extension": [
                    {
                        "valueCoding": {
                            "system": "http://terminology.hl7.org/CodeSystem/v3-NullFlavor",
                            "code": "UNK",
                            "display": "Unknown"
                        },
                        "url": "ombCategory"
                    },
                    {
                        "valueString": "Unknown",
                        "url": "text"
                    }
                ],
                "url": "http://hl7.org/fhir/us/core/StructureDefinition/us-core-race"
            },
            {
                "extension": [
                    {
                        "valueString": "Unknown",
                        "url": "text"
                    }
                ],
                "url": "http://hl7.org/fhir/us/core/StructureDefinition/us-core-ethnicity"
            },
            {
                "valueCode": "248153007",
                "url": "http://hl7.org/fhir/us/core/StructureDefinition/us-core-sex"
            },
            {
                "valueCodeableConcept": {
                    "coding": [
                        {
                            "system": "http://loinc.org",
                            "code": "LA29518-0",
                            "display": "he/him/his/his/himself"
                        }
                    ]
                },
                "url": "http://open.epic.com/FHIR/StructureDefinition/extension/calculated-pronouns-to-use-for-
text"
            }
        ],
        "identifier": [
            {
                "use": "usual",
                "type": {
                    "text": "CEID"
                },
                "system": "urn:oid:1.2.840.114350.1.13.5325.1.7.3.688884.100",
                "value": "YA5WT9X7ZMB69TB"
            },
            {
                "use": "usual",
                "type": {
                    "text": "EPI"
                },
                "system": "urn:oid:1.2.840.114350.1.1",
                "value": "27475"
            },
            {
                "use": "usual",
                "system": "urn:oid:2.16.840.1.113883.4.1",
                "_value": {
                    "extension": [
                        {
                            "valueString": "xxx-xx-3745",
                            "url": "http://hl7.org/fhir/StructureDefinition/rendered-value"
                        }
                    ]
                }
            }
        ],
        "active": true,
        "name": [
            {
                "use": "official",
                "text": "John Smith",
                "family": "Smith",
                "given": [
                    "John"
                ]
            },
```

```
                    {
                        "use": "usual",
                        "text": "John Smith",
                        "family": "Smith",
                        "given": [
                            "John"
                        ]
                    }
                ],
                "gender": "male",
                "birthDate": "1906-03-04",
                "deceasedBoolean": false,
                "managingOrganization": {
                    "reference": "Organization/ePUQySFaW.ofXm2GhaXRMDA3",
                    "display": "Epic Facility"
                }
            },
            "search": {
                "extension": [
                    {
                        "valueCode": "certain",
                        "url": "http://hl7.org/fhir/StructureDefinition/match-grade"
                    }
                ],
                "mode": "match",
                "score": 1
            }
        }
    ]
}
```


Let's pick out a couple of key elements to learn more about:

* The entry[n].fullUrl element points to the permanent location of a unique patient. If we make a request to this absolute URL, we can directly access the Patient resource for that patient.

* The entry[n].resource.id contains the FHIR ID of the resource. This ID is assigned by the server responsible for storing the resource. Storing this FHIR ID can be useful if you want to save off a list of recently used patients, for example.

* The entry[0].resource element contains all of the actual data about John Smith. Within this object, you'll find information about John such as his address, phone number, and birth date.

Ideally, you provide enough demographic information in your initial request for the API to return the correct patient. If we were building a more complex app, we might need to handle other scenarios gracefully, such as when no patients are found.

The following response is an example of what to expect if a single high-confidence patient match is not found. Note the text contained within the OperationOutcome resource:


```
{
    "resourceType": "OperationOutcome",
    "issue": [
        {
            "severity": "fatal",
            "code": "processing",
            "details": {
                "coding": [
                    {
                        "system": "urn:oid:1.2.840.114350.1.13.5325.1.7.2.657369",
                        "code": "59013",
                        "display": "The matching operation found one or more possible matches, but did not find a
    certain match."
                    }
                ],
                "text": "The matching operation found one or more possible matches, but did not find a certain match."
            },
            "location": [
                "/f:patient"
            ],
            "expression": [
                "patient"
            ]
        }
    ]
}
```


That's a quick overview of the Patient $match endpoint, but if you're interested in learning more about our Patient.$match API, you can read more here. Let's continue by saving off John's FHIR ID so we can refer to it later:

```
var patientId = data.entry[0].resource.id;
```

Interlude: Relative Resource URLs (R4 and Later)

There are several versions of the FHIR standard. We generally recommend using Epic's latest R4 FHIR resources, as we did in the example above, but you might also need to use DSTU2 resources or STU3 resources in some cases.

For R4 and later versions of FHIR, Epic uses relative URLs for all referenced FHIR resources included in a query response. STU3 and earlier versions use absolute URLs. For example, if we look back at the sample response from our Patient query above, if that query was an STU3 version Patient query, instead of returning a relative reference URL for the managingOrganization:

```
"managingOrganization": {
    "reference": "Organization/ePUQySFaW.ofXm2GhaXRMDA3",
    "display": "Epic Facility"
}
```

An STU3 resource would return an absolute URL instead:

```
"managingOrganization": {
    "reference": "https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/STU3/Organization/ePUQySFaW.ofXm2GhaXRMDA3",
    "display": "Epic Facility"
}
```

This distinction is important to keep in mind, particularly if you're working with some FHIR resources that use the DSTU2 or STU3 standard, and other resources that use the R4 standard. Make sure to append the baseURL to the front of the relative URL when querying referenced FHIR resources.

Searching for Clinical Data by Patient ID

We've found our patient, and we have some demographics that we could parse and display to an end user. But how do we get access to clinical information about that patient? To retrieve this data, we need to use another endpoint within our API, along with the patient ID we saved off above.

Let's start with a simple query — retrieving a patient's allergies. The AllergyIntolerance endpoint exposes exactly that, and provides a search endpoint to find allergies by Patient ID. We can construct a query that provides John's FHIR ID to grab all the active allergies recorded in our system:

```
var allergySearchString = "/AllergyIntolerance?patient=" + patientId;
```

We can use jQuery's $.getJSON() function to call into the URL and retrieve our data in JSON format. We will also log the response bode to the console as a simple means to view the data.

```
$.getJSON(baseUrl + allergySearchString, function(data,error) { console.log(data); });
```

Executing the above request returns the following:

```
{
    "resourceType": "Bundle",
    "type": "searchset",
    "total": 1,
    "link": [
        {
            "relation": "self",
            "url": "https://fhir.epic.com/interconnect-fhir-
    oauth/api/FHIR/R4/AllergyIntolerance?patient=ecBjL8PUhljoHWMtwx63UhA3"
        }
    ],
    "entry": [
        {
            "link": [
                {
                    "relation": "self",
                    "url": "https://fhir.epic.com/interconnect-fhir-
    oauth/api/FHIR/R4/AllergyIntolerance/efnBJhT6lf969qC4prH5jVqor3vnmeJyvRfMzJX9-yZ43"
                }
            ],
            "fullUrl": "https://fhir.epic.com/interconnect-fhir-
    oauth/api/FHIR/R4/AllergyIntolerance/efnBJhT6lf969qC4prH5jVqor3vnmeJyvRfMzJX9-yZ43",
            "resource": {
                "resourceType": "AllergyIntolerance",
                "id": "efnBJhT6lf969qC4prH5jVqor3vnmeJyvRfMzJX9-yZ43",
                "clinicalStatus": {
                    "coding": [
                        {
                            "system": "http://terminology.hl7.org/CodeSystem/allergyintolerance-clinical",
                            "version": "4.0.0",
                            "code": "active",
                            "display": "Active"
                        }
                    ],
                    "text": "Active"
                },
                "verificationStatus": {
                    "coding": [
                        {
                            "system": "http://terminology.hl7.org/CodeSystem/allergyintolerance-verification",
                            "version": "4.0.0",
                            "code": "confirmed",
                            "display": "Confirmed"
                        }
                    ],
                    "text": "Confirmed"
                },
                "category": [
                    "medication"
                ],
                "criticality": "high",
                "code": {
                    "coding": [
                        {
                            "system": "urn:oid:2.16.840.1.113883.3.26.1.5",
```

```
                        "code": "N0000005840",
                        "display": "PENICILLINS"
                    }
                ],
                "text": "PENICILLINS"
            },
            "patient": {
                "reference": "Patient/ecBjL8PUhljoHWMtwx63UhA3",
                "display": "Smith, John"
            },
            "onsetDateTime": "2024-08-27",
            "recordedDate": "2024-08-27T20:34:34Z",
            "reaction": [
                {
                    "manifestation": [
                        {
                            "coding": [
                                {
                                    "system": "http://snomed.info/sct",
                                    "code": "201272008",
                                    "display": "(URTICARIA NOS) OR (HIVES)"
                                }
                            ],
                            "text": "Hives"
                        }
                    ],
                    "description": "Hives"
                }
            ]
        },
        "search": {
            "mode": "match"
        }
    }
  ]
}
```


Looking at the response, there are a couple of key elements to highlight to help us understand the allergy information:

* The total element lets you know the number of resources returned for a query. In this case, it lets us know how many allergies our system has documented for John Smith.

* The entry[n].fullUrl element points to the permanent location of a unique AllergyIntolerance resource. If we make a request to this URL, we will always get back data for this specific AllergyIntolerance for the patient. The FHIR ID for this resource can also be retrieved directly from entry[n].resource.id. This might be useful for future reference to check whether the patient has developed additional reactions to this allergy.

* The entry[n].resource.patient element describes the patient the allergy relates to. This allows us to confirm the association between allergy and patient and shows how FHIR works by linking these smaller resources to paint a larger picture. In this case, the element in each allergy entry points to our patient, John.

* The entry[n].resource.code element describes the allergen that causes the allergy. This element has a text element, to make it easy to read, as well as coded values, to associate and develop clinical rules against. In this example response, we can see that John is allergic to Penicillin.

* The entry[n].resource.reaction element describes the allergic reactions of the patient documented in the system.

* The entry[n].resource.onsetDateTime element tells you when the allergy first started happening.

* The entry[n].resource.reaction[n].manifestation element tells you what kind of reaction occurred. For example, John's allergy to Penicillin gives him hives.

That's a lot of information with a simple query, and it demonstrates how quickly we can build up and associate the different endpoints available on our API.

Wrap Up

Nice work! You made it to the end of our tutorial, and you understand how to find patients, and start working with their data. There's even more to discover in our documentation, like additional data types, and more complex searches.

Advanced Topics

In this section we'll cover a few advanced topics that may come up in your implementation of FHIR.

Searching from Patient Context in Coordination with OAuth 2.0

For patient-facing applications using OAuth 2.0 authorization, including the patient or subject search parameter(s) is optional when making requests for the patient on whose behalf authorization to the system was granted. This is true for STU3 and R4 versions of FHIR resources.

Including the patient or subject ID in the request is still recommended and encouraged. Requests can be made with these parameters so long as the FHIR ID matches the patient whose data is being requested with the access token.

Content negotiation in FHIR

Epic’s FHIR server defaults to using XML for FHIR resources, but many apps may prefer sending and receiving JSON instead. FHIR allows both XML and JSON MIME-types and specifies how a client can request one or the other. Per the FHIR

spec and in Epic's FHIR server, a client can specify XML or JSON through either the _format query parameter or by specifying the MIME-type in an HTTP header. A client should use the Accept HTTP header to specify the MIME-type of the content that it wishes to receive from the server and the Content-Type HTTP header to specify the MIME-type of the content that it is sending in the body of its request, for example, as part of a create or update; the Content-Type HTTP header will also be considered for requests in which another method (_format query parameter, or Accept HTTP header) is not provided.

Epic's FHIR server processes the available options in the following hierarchy:

* _format query parameter
* Accept HTTP Header
* Content-Type HTTP Header
* Server Default: XML

Epic supports all of the following MIME-types for FHIR interactions:

* application/fhir+json
* application/json+fhir
* application/xml+fhir
* application/fhir+xml
* application/json
* application/xml
* text/xml

In addition to the MIME-types listed above, the _format query parameter supports:

* xml
* json

Retrieving Additional ID Types with Patient and Practitioner Resources

You can also use FHIR search APIs such as Patient.Search and Practitioner.Search to get FHIR IDs for other FHIR versions and a list of other identifiers given one type of identifier. For Patient.Search, use the following format:

```
<base url>/R4/Patient?identifier=<ID Type>|<ID>
```

Where ID Type can be one of the following: CID, CSN, External, ExternalKey, FHIR, FHIR STU3, Internal, Name, NationalID, CEID, MyChartLogin, <any IIT identifier>

The resulting identifiers are identical to those returned by GetPatientIdentifiers.

For practitioners, you can use the Practitioner.Search endpoint to retrieve a list of practitioner identifiers. Use the following format:

```
<base url>/R4/Practitioner?identifier=<ID Type>|<ID>
```

Where ID Type can be one of the following: CID, External, FHIR, Internal, NPI, CSN.

Constraining ID Length to 64 Characters in the R4 Version of FHIR

For historical reasons, some of Epic's FHIR resources have identifiers that are longer than 64 characters. For example, Medication resource IDs sometimes exceed this length. New clients can be configured to respect the 64-character ID-length limit described in the HL7 FHIR standard for all STU3 and R4 version FHIR resources that are part of the USCDI data classes. To avoid disrupting existing integrations or new integrations that rely on historical data, this configuration is not enabled by default. You can set your client to observe the 64-character identifier length limit for USCDI resources by changing the FHIR ID Generation Scheme setting on the Build Apps page. To change this setting on an active app, you will need to contact your Epic representative. See the App Default FHIR Version Tutorial for more information.

This document explains the ways in which the date and datetime parameters can be configured during a search with a FHIR resource. The general format for dates and times is yyyy-mm-ddThh:mm:ss[Z|(+|-)hh:mm], an example being 2019-05-20T08:00:00Z which corresponds to May 20th 2019 at 8:00 AM UTC.

FHIR Search Using HTTP POST

In some cases, you might want to perform a search interaction using an HTTP POST body. In rare scenarios, the query string might be too long and a POST body could make more sense. This method is supported for all versions of FHIR. For example, a search query like this:

```
GET <base url>/api/FHIR/<version>/<resource>?param1=val1&ampparam2=val2...
```

```
POST <base url>/api/FHIR/<version>/<resource>/_search
Content-Type: application/x-www-form-urlencoded
param1=val1&ampparam2=val2...
```

Date/Time Information

Prefixes

Epic supports a subset of HL7-defined prefixes that can be added to a date or datetime parameter to modify the way in which searching occurs. If you choose to omit a prefix, the default behavior will be the same as that of an "equal" prefix, such that the search will include a results for which the specified value is within range.

Refer to the Prefix Considerations section below for additional caveats about Epic's date searching behavior.

Prefix | Prefix Meaning | Example Search | Example Comparisons
eq | equal | =eq2013-01-14 | 2013-01-14T00:00 matches
2013-01-14T10:00 matches
2013-01-15T00:00 does not match - it's not in the range
lt | less than | =lt2013-01-14T10:00 | 2013-01-14 matches, because it includes the part of 14-Jan 2013 before 10am
le | less than or equal to | =le2013-03-14 | "from 21-Jan 2013 onwards" is included because that period may include times before 14-Mar 2013
gt | greater than | =gt2013-01-14T10:00 | 2013-01-14 matches, because it includes the part of 14-Jan 2013 after 10am
ge | greater than or equal to | =ge2013-03-14 | "from 21-Jan 2013 onwards" is included because that period may include times after 14-Mar 2013
sa | starts after | =sa2013-03-14 | "from 15-Mar 2013 onwards" is included because that period starts after 14-Mar 2013
"from 21-Jan 2013 onwards" is not included because that period starts before 14-Mar 2013
"before and including 21-Jan 2013" is not included because that period starts (and ends) before 14-Mar 2013
eb | ends before | =eb2013-03-14 | "from 15-Mar 2013 onwards" is not included because that period starts after 14-Mar 2013
"from 21-Jan 2013 onwards" is not included because that period starts before 14-Mar 2013, but does not end before it
"before and including 21-Jan 2013" is included because that period ends before 14-Mar 2013

Prefix Considerations

* In some FHIR APIs, the date parameter supports fuzzy date searching, meaning that the search will also return values with dates close to but outside of the specified date or date range. If you need strict date searching, we suggest that you do additional validation on the date values returned in the response.

* The and prefixes behave differently than defined by HL7:

* sa behaves identically to gt in Epic. In other words, sa will match a period even if only part of that period occurs after the specified date or datetime.

* eb behaves identically to lt in Epic. In other words, eb will match a period even if only part of that period occurs before the specified date or datetime.

* Epic does not support the ne or ap date search prefixes.

Date Parameters

Level of Precision

Searching by a date or time will do a search based on the level of precision you enter.

Search by year:

date=yyyy

Ex: date=2019

This will search for values with a date that occurred in 2019. This encompasses a range from January 1st, 2019 00:00:00 to December 31st, 2019 23:59:59.

Search by year and month:

date=yyyy-mm

Ex: date=2019-05

This will search for values with a date that occurred in May 2019. This encompasses a range from May 1st, 2019 00:00:00 to May 31st, 2019 23:59:59.

Search by year, month, and day:

date=yyyy-mm-dd

Ex: date=2019-05-15

This will search for values with a date of May 15th, 2019.

Date Ranges

Search on or near a specific date:

date=yyyy-mm-dd

Ex: date=2019-05-15

This will search for values with a date of May 15th, 2019.

Search starting at a specific date:

date=ge yyyy-mm-dd

Ex: date=ge2019-05-10

This will search for values with a date on or after May 10th, 2019.

Search ending at a specific date:

date=le yyyy-mm-dd

Ex: date=le2019-05-20

This will search for values with a date on or before May 20th, 2019.

Search starting and ending at a specific date:

date=ge yyyy-mm-dd&date=le yyyy-mm-dd

Ex: date=ge2019-05-10&date=le2019-05-20

This will search for values with a date on or after May 10th, 2019 and on or before May 20th, 2019.

DateTime (Instant)

Level of Precision

Search by Hour:

parameter=yyyy-mm-ddThh:mm

Ex: parameter=2019-05-15T12:00

This will search for values with a date of May 15th, 2019 and a time of 12:00 pm local time. If you include an hour value, you must include a minute value.

Search by Hour and Minute:

parameter=yyyy-mm-ddThh:mm

Ex: parameter=2019-05-15T12:30

This will search for values with a date of May 15th, 2019 and a time of 12:30 pm local time.

Search by Hour, Minute, and Second:

parameter=yyyy-mm-ddThh:mm:ss

Ex: parameter=2019-05-15T12:00:30

This will search for values with a date of May 15th, 2019 and a time of 12:00:30 pm local time.

Time Zones

When a time and time offset are specified, the specified timezone will be used. If only a date with no time is specified or a time is specified without a timezone, the timezone defaults to the local timezone.

Specify UTC Time Zone:

parameter=yyyy-mm-ddThh:mm:ssZ

Ex: parameter=2019-05-15T12:30:00Z

This will search for values with a date of May 15th, 2019 and a time of 12:30 pm UTC. This also works with an offset of +00:00 or -00:00.

Specify a non-UTC Time Zone:

parameter=yyyy-mm-ddThh:mm:ss(+/-)hh:mm

Ex: parameter=2019-05-15T12:30:00-05:00

This will search for values with a date of May 15th, 2019 and a time of 12:30 pm EST.

1.22 OAuth 2.0 Specification

File: site/docs/01-guides/22-oauth2.html

Source: https://fhir.epic.com/Documentation?docId=oauth2

can be used for some OAuth 2.0 workflows. JWTs are required to be used by the client credentials flow used by backend services, and can be used, as an alternative to client secrets, for apps that use the confidential client profile or refresh tokens. You will generate a one-time use JWT to authenticate your app to the authorization server and obtain an access token that can be used to authenticate your app's web service calls, and potentially a refresh token that can be used to get another access token without a user needing to authorize the request. There are several libraries for creating JWTs. See for some examples. Epic supports forms of authentication in addition to OAuth 2.0.

Refer to the following table for Epic versions when this requirement will take effect.

Implementation Consideration: Starting in the May 2026 version of Epic, customers must manually configure the JWK Set URL (JKU) you specify for your backend OAuth 2.0 clients. This customer-specific configuration is referred to as the local JKU.

Update: Customers will be able to configure a local JWK using a static public key in .pem format that you provide, instead of using a JKU. If your app relies on static keys and your JWKs change, you will need to communicate the new keys directly to each customer to maintain functionality. For ease of key rotation and ongoing maintenance, Epic recommends supporting a JKU.

Epic Version | Epic Sandbox JKU Support | Epic Customer JKU Support

February 2025 and prior | Optional | Optional - Each customer can choose to enforce JKU support for backend OAuth apps. This enforcement is off by default.

August 2025 | Vendor Services and Epic on FHIR no longer allow new static key uploads for use in the sandbox.

February 2026 | Static keys no longer supported in the sandbox for backend OAuth. Vendor Services and Epic on FHIR websites no longer allow static key uploads for new apps or new customer download requests. Both sites still support rotating keys for existing live customers. | Vendor Services and Epic on FHIR websites no longer allow static key uploads for new apps or new customer download requests. Both sites still support rotating keys for existing live customers.

May 2026 | | Static keys are no longer supported in customer environments for backend OAuth.

August 2026 | | Customers may configure local JWKs if provided .pem files.

Using OAuth 2.0

Applications must secure and protect the privacy of patients and their data. To help meet this objective, Epic supports using the OAuth 2.0 framework to authenticate and authorize applications.

Why OAuth 2.0?

OAuth 2.0 enables you to develop your application without having to build a credential management system. Instead of exposing login credentials to your application, your application and an EHR's authorization server exchange a series of authorization codes and access tokens. Your application can access protected patient data stored in an EHR's database after it obtains authorization from the EHR's authorization server.

Before You Get Started

To use OAuth 2.0 to authorize your application's access to patient information, some information needs to be shared between the authorization server and your application:

* client_id: The client_id identifies your application to authentication servers within the Epic community and allows you to connect to any organization.

* : The confirms your identity and is used to validate and redirect authentication requests that originate from your application. Epic's implementation allows for multiple s.

redirect_uri

redirect_uri

redirect_uri

* Note that a redirect_uri is not needed for backend services using the client_credentials grant type.

* An https protocol is required for use in a production environment, but http protocol-based redirect_uris are allowed for development and testing.

* Registered redirect_uris must not contain anything in the fragment (i.e. anything after #).

* Credentials: Some apps, sometimes referred to as confidential clients, can use credentials registered for a given EHR system to obtain authorization to access the system without a user or a patient implicitly or explicitly authorizing the app. Examples of this are apps that use refresh tokens to allow users to launch the app outside of an Epic client without needing to log in every time they use the app, and backend services that need to access resources without a specific person launching the app, for example, fetching data on a scheduled basis.

You can register your application for access to both the sandbox and Epic organizations here. You'll provide information to us, including one or more redirect_uris, and Epic will generate a client_id for you.

Apps can be launched from within an existing EHR or patient portal session, which is called an EHR launch. Alternatively, apps can be launched standalone from outside of an existing EHR session. Apps can also be backend services where no user is launching the app.

EHR launch (SMART on FHIR): The app is launched by the EHR calling a launch URL specified in the EHR's configuration. The EHR launches the launch URL and appends a launch token and the FHIR server's endpoint URL (ISS parameter) in the query string. The app exchanges the launch token, along with the client identification parameters to get an authorization code and eventually the access token.

* See the Embedded Launch section of this guide for more details.

Standalone launch: The app launches directly to the authorize endpoint outside of an EHR session and requests context from the EHR's authorization server.

* See the Standalone Launch section of this guide for more details.

Backend services: The app is not authorized by a specific person and likely does not have a user interface, and therefore calls EHR web services with system-level authorization.

* See the Backend Services section of this guide for more details.

Desktop integrations through Subspace: The app requests access to APIs directly available on the EHR's desktop application via a local HTTP server.

* See the Subspace Communication Framework specification for more details.

Note: For all requests to OAuth 2.0 endpoints, your application must not include the same parameter more than once.

Starting in the August 2019 version of Epic, the Epic implementation of OAuth 2.0 supports PKCE and Epic recommends using PKCE for native mobile app integrations. For earlier versions or in cases where PKCE cannot be used, Epic recommends the use of Universal Links/App Links in lieu of custom redirect URL protocol schemes.

The app you build will list both a production Client ID and a non-production Client ID. While testing in the Epic on FHIR sandbox, use the non-production Client ID.

The base URL for the Current Sandbox environment is:

```
https://fhir.epic.com/interconnect-fhir-oauth/
```

EHR Launch (SMART on FHIR)

Contents

* Step 1: Your Application is Launched from the Patient Portal or EHR

Step 2: Your Application Retrieves the Conformance Statement or SMART Configuration

* Additional Header Requirements

* Step 3: Your Application Requests an Authorization Code

* Step 4: EHR's Authorization Server Reviews The Request

Step 5: Your Application Exchanges the Authorization Code for an Access Token

* Non-confidential Clients

* Frontend Confidential Clients

* OpenID Connect id_tokens

* Validating the OpenID Connect JSON Web Token

* Step 6: Your Application Uses FHIR APIs to Access Patient Data

* Step 7: Use a Refresh Token to Obtain a New Access Token

How It Works

The app is launched by the EHR calling a launch URL specified in the EHR's configuration. The EHR launches the launch URL and appends a launch token and the FHIR server's endpoint URL (ISS parameter) in the query string. The app exchanges the launch token, along with the client identification parameters to get an authorization code and eventually the access token.

Step 1: Your Application is Launched from the Patient Portal or EHR

Your app is launched by the EHR calling the launch URL which is specified in the EHR's configuration. The EHR sends a launch token and the FHIR server's endpoint URL (ISS parameter).

Note: If you are writing a web-based application that you intend to embed in Epic, you may find that your application may require minor modifications. We offer a simple testing harness to help you validate that your web application is compatible with the embeddable web application viewer in Epic. As embedding your application in Hyperspace may not behave the same as presenting it in a browser, ensure that your app works with the testing harness and that you can run multiple instances of the app at the same time within the same session.

* : This parameter is an EHR generated token that signifies that an active EHR session already exists. This token is one-time use and will be exchanged for the authorization code.

launch

* See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling launch codes.

* : This parameter contains the EHR's FHIR endpoint URL, which an app can use to find the EHR's authorization server.

iss

* Your application should create an allowlist of iss values (AKA resource servers) to protect against phishing attacks. Rather than accepting any iss value in your application, only accept those you know belong to the organizations you integrate with.

Step 2: Your Application Retrieves the Conformance Statement or SMART Configuration

To determine which authorize and token endpoints to use in the EHR launch flow, you should make a GET request to the metadata endpoint which is constructed by taking the iss provided and appending /metadata. Alternatively, when communicating with Epic organizations using the August 2021 version or Epic or later, you can make a GET request to the SMART configuration endpoint by taking the iss and appending /.well-known/smart-configuration.

If no Accept header is sent, the metadata response is XML-formatted and the smart-configuration response is JSON-formatted. An Accept header can be sent with a value of application/json, application/xml, and the metadata endpoint additionally supports application/fhir+json and application/json+fhir to specify the format of the response.

Metadata Example

Here's an example of what a full metadata request might look like.

```
GET https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/DSTU2/metadata HTTP/1.1
Accept: application/fhir+json
```

Here is an example of what the authorize and token endpoints would look like in the metadata response.

```
"extension": [
{
"url": "http://fhir-registry.smarthealthit.org/StructureDefinition/oauth-uris",
"extension": [
  {
    "url": "authorize",
    "valueUri": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2/authorize"
  },
  {
    "url": "token",
    "valueUri": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token"
```

```
  }
]
}
]
```

Metadata Example with Dynamic Client Registration (starting in the November 2021 version of Epic)

Here's an example of what a full metadata request might look like. By including an Epic Client ID, clients authorized to perform dynamic client registration can then determine which endpoint to use when performing dynamic client registration. Note that this capability is only supported for STU3 and R4 requests.

```
GET https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/metadata HTTP/1.1
Accept: application/fhir+json
Epic-Client-ID: 0000-0000-0000-0000-0000
```

Here is an example of what the authorize, token, and dynamic registration endpoints would look like in the metadata response.

```
"extension": [
{
"url": "http://fhir-registry.smarthealthit.org/StructureDefinition/oauth-uris",
"extension": [
  {
    "url": "authorize",
    "valueUri": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2/authorize"
  },
  {
    "url": "token",
    "valueUri": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token"
  },
  {
    "url": "register",
    "valueUri": https://fhir.epic.com/interconnect-fhir-oauth/oauth2/register
  }
]
}
]
```

Here's an example of what a smart-configuration request might look like.

```
GET https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/.well-known/smart-configuration HTTP/1.1
Accept: application/json
```

Here is an example of what the authorize and token endpoints would look like in the smart-configuration response.

```
"authorization_endpoint": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2/authorize",
"token_endpoint": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token",
"token_endpoint_auth_methods_supported": [
    "client_secret_post",
    "client_secret_basic",
    "private_key_jwt"
]
```

Retrieving Conformance Statement: Additional Header Requirements

In some cases when making your request to the metadata endpoint, you'll also need to include additional headers to ensure the correct token and authorize endpoints are returned. This is because in Epic, it is possible to configure the system such that the FHIR server's endpoint URL, provided in the iss parameter, is overridden on a per client ID basis. If you do not send the Epic-Client-ID HTTP header with the appropriate production or non-production client ID associated with your application in your call to the metadata endpoint, you can receive the wrong token and authorize endpoints. Additional CORS setup might be necessary on the Epic community member's Interconnect server to allow for this header.

Starting in the November 2021 version of Epic, by including an Epic Client ID, clients authorized to perform dynamic client registration can then determine which endpoint to use when performing dynamic client registration. Note that this capability is supported for only STU3 and R4 requests.

Here's an example of what a full metadata request might look like with the additional Epic-Client-ID header.

```
GET https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/metadata HTTP/1.1
Accept: application/fhir+json
Epic-Client-ID: 0000-0000-0000-0000-0000
```

Step 3: Your Application Requests an Authorization Code

Your application now has a launch token obtained from the initial EHR launch as well as the authorize endpoint obtained from the metadata query. To exchange the launch token for an authorization code, your app needs to either make an HTTP GET or POST request to the authorize endpoint that contains the parameters below. POST requests are supported for EHR launches in Epic version November 2020 and later, and are not currently supported for standalone launches.

For POST requests, the parameters should be in the body of the request and should be sent as Content-Type "application/x-www-form-urlencoded". For GET requests, the parameters should be appended in the querystring of the request. It is Epic's recommendation to use HTTP POST for EHR launches to overcome any size limits on the request URL.

In either case, the application must redirect the User-Agent (the user's browser) to the authorization server by either submitting an HTML Form to the authorization server (for HTTP POST) or by redirecting to a specified URL (for HTTP GET). The request should contain the following parameters.

* response_type: This parameter must contain the value "code".

* client_id: This parameter contains your web application's client ID issued by Epic.

* redirect_uri: This parameter contains your application's redirect URI. After the request completes on the Epic server, this URI will be called as a callback. The value of this parameter needs to be URL encoded. This URI must also be registered with the EHR's authorization server by adding it to your app listing.

* scope: This parameter describes the information for which the web application is requesting access. Starting with the Epic 2018 version, a scope of "launch" is required for EHR launch workflows. Starting with the November 2019 version of Epic, OpenID Connect scopes are supported, specifically the "openid" scope is supported for all apps and the "fhirUser" scope is supported by apps with the R4 (or greater) SMART on FHIR version selected.

* launch: This parameter is required for EHR launch workflows. The value to use will be passed from the EHR.

* aud: Starting in the August 2021 version of Epic, health care organizations can optionally configure their system to require the aud parameter for EHR launch workflows if a launch context is included in the scope parameter. Starting in the May 2023 version of Epic, this parameter will be required. The value to use is the FHIR base URL of the resource server the application intends to access, which is typically the FHIR server returned by the iss.

* : This optional parameter is generated by your app and is opaque to the EHR. The EHR's authorization server will append it to each subsequent exchange in the workflow for you to validate session integrity. While not required, this parameter is recommended to be included and validated with each exchange in order to increase security. For more information see .

state

RFC 6819 Section 3.6

* Large values in combination with launch tokens that are JWTs (see above) may cause the query string for the HTTP GET request to the authorization endpoint to exceed the Epic community member web server's max query string length and cause the request to fail. You can mitigate this risk by:

Note:

state

* Following the SMART App Launch Framework's recommendation that the state parameter be an unpredictable value ... with at least 122 bits of entropy (e.g., a properly configured random uuid is suitable), and not store any actual application state in the state parameter.

* Using a POST request. If you use this approach, note that you might still need to account for a large state value in the HTTP GET redirect back to your own server.

Additional parameters for native mobile apps (available starting in the August 2019 version of Epic):

* code_challenge: This optional parameter is generated by your app and used for PKCE. This is the S256 hashed version of the code_verifier parameter, which will be used in the token request.

* code_challenge_method: This optional parameter indicates the method used for the code_challenge parameter and is required if using that parameter. Currently, only the S256 method is supported.

Here's an example of an authorization request using HTTP GET. You will replace the [redirect_uri], [client_id], [launch_token], [state], [code_challenge], and [audience] placeholders with your own values.

```
https://fhir.epic.com/interconnect-fhir-oauth/oauth2/authorize?scope=launch&response_type=code&redirect_uri=[redirect_uri]&client_id=[client_id]&launch=[launch_token]&state=[state]&code_challenge=[code_challenge]&code_challenge_method=S256&aud=[audience]
```

This is an example HTTP GET from the SMART on FHIR launchpad:

```
https://fhir.epic.com/interconnect-fhir-oauth/oauth2/authorize?scope=launch&response_type=code&redirect_uri=https%3A%2F%2Ffhir.epic.com%2Ftest%2Fsmart&client_id=d45049c3-3441-40ef-ab4d-b9cd86a17225&launch=GwWCqm4CCTxJaqJiROISsEsOqN7xrdixqTl2uWZhYxMAS7QbFEbtKgi7AN96fKc2kDYfaFrLi8LQivMkD0BY-942hYgGO0_6DfewP2iwH_pe6tR_-fRfiJ2WB_-1uwG0&state=abc123&aud=https%3A%2F%2Ffhir.epic.com%2Finterconnect-fhir-oauth%2Fapi%2Ffhir%2Fdstu2
```

And here's an example of what the same request would look like as an HTTP Post:

```
POST https://fhir.epic.com/interconnect-fhir-oauth/oauth2/authorize HTTP/1.1
Content-Type: application/x-www-form-urlencoded

scope=launch&response_type=code&redirect_uri=https%3A%2F%2Ffhir.epic.com%2Ftest%2Fsmart&client_id=d45049c3-3441-40ef-ab4d-b9cd86a17225&launch=GwWCqm4CCTxJaqJiROISsEsOqN7xrdixqTl2uWZhYxMAS7QbFEbtKgi7AN96fKc2kDYfaFrLi8LQivMkD0BY-942hYgGO0_6DfewP2iwH_pe6tR_-fRfiJ2WB_-1uwG0&state=abc123&aud=https%3A%2F%2Ffhir.epic.com%2Finterconnect-fhir-oauth%2Fapi%2Ffhir%2Fdstu2
```

Step 4: EHR's Authorization Server Reviews the Request

The EHR's authorization server reviews the request from your application. If approved, the authorization server redirects the browser to the redirect URL supplied in the initial request and appends the following querystring parameter.

* : This parameter contains the authorization code generated by Epic, which will be exchanged for the access token in the next step.

code

* See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling authorization codes.

* state: This parameter will have the same value as the earlier state parameter. For more information, refer to Step 3.

Here's an example of what the redirect will look like if Epic's authorization server accepts the request:

```
https://fhir.epic.com/test/smart?code=yfNg-rSc1t5O2p6jVAZLyY00uOOte5KM1y3YUxqsJQnBKEMNsYqOPTyVqcCH3YXaPkLztO9Rvf7bhLqQTw
```

```
    ALHcHN6raxpTbR1eVgV2QyLA_4K0HrJO92et3qRXiXPkj7&state=abc123
```

Step 5: Your Application Exchanges the Authorization Code for an Access Token

After receiving the authorization code, your application trades the code for a JSON object containing an access token and contextual information by sending an HTTP POST to the token endpoint using a Content-Type header with value of "application/x-www-form-urlencoded". For more information, see RFC 6749 section 4.1.3.

Access Token Request: Non-confidential Clients

The following parameters are required in the POST body:

* grant_type: For the EHR launch flow, this should contain the value "authorization_code".

* code: This parameter contains the authorization code sent from Epic's authorization server to your application as a querystring parameter on the redirect URI as described above.

* redirect_uri: This parameter must contain the same redirect URI that you provided in the initial access request. The value of this parameter needs to be URL encoded.

* client_id: This parameter must contain the application's client ID issued by Epic that you provided in the initial request.

* code_verifier: This optional parameter is used to verify against your code_challenge parameter when using PKCE. This parameter is passed as free text and must match the code_challenge parameter used in your authorization request once it is hashed on the server using the code_challenge_method. This parameter is available starting in the August 2019 version of Epic.

Here's an example of what an HTTP POST request for an access token might look like:

```
POST https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&code=yfNg-rSc1t5O2p6jVAZLyY00uOOte5KM1y3YUxqsJQnBKEMNsYqOPTyVqcCH3YXaPkLztO9Rvf7bhLqQTwALH
    cHN6raxpTbR1eVgV2QyLA_4K0HrJO92et3qRXiXPkj7&redirect_uri=https%3A%2F%2Ffhir.epic.com%2Ftest%2Fsmart&client_id=d45049
    c3-3441-40ef-ab4d-b9cd86a17225
```

The authorization server responds to the HTTP POST request with a JSON object that includes an access token. The response contains the following fields:

* : This parameter contains the access token issued by Epic to your application and is used in future requests.

access_token

* See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling access tokens.

* token_type: In Epic's OAuth 2.0 implementation, this parameter always includes the value bearer.

* expires_in: This parameter contains the number of seconds for which the access token is valid.

* scope: This parameter describes the access your application is authorized for.

* id_token: Returned only for applications that have requested an openid scope. See below for more info on OpenID Connect id_tokens. This parameter follows the guidelines of the OpenID Connect (OIDC) Core 1.0 specification. It is signed but not encrypted.

* patient: This parameter identifies provides the FHIR ID for the patient, if a patient is in context at time of launch.

* epic.dstu2.patient: This parameter identifies the DSTU2 FHIR ID for the patient, if a patient is in context at time of launch.

* encounter: This parameter identifies the FHIR ID for the patient's encounter, if in context at time of launch. The encounter token corresponds to the FHIR Encounter resource.

* location : This parameter identifies the FHIR ID for the encounter department, if in context at time of launch. The location token corresponds to the FHIR Location resource.

* appointment: This parameter identifies the FHIR ID for the patient's appointment, if appointment context is available at time of launch. The appointment token corresponds to the FHIR Appointment resource.

* loginDepartment: This parameter identifies the FHIR ID of the user's login department for launches from Hyperspace. The loginDepartment token corresponds to the FHIR Location resource.

* state: This parameter will have the same value as the earlier state parameter. For more information, refer to Step 3.

Note that you can include additional fields in the response if needed based on the integration configuration. For more information, refer to the Launching your app topic. Here's an example of what a JSON object including an access token might look like:

```
{
"access_token": "Nxfve4q3H9TKs5F5vf6kRYAZqzK7j9LHvrg1Bw7fU_07_FdV9aRzLCI1GxOn20LuO2Ahl5RkRnz-
    p8u1MeYWqA85T8s4Ce3LcgQqIwsTkI7wezBsMduPw_xkVtLzLU2O",
"token_type": "bearer",
"expires_in": 3240,
"scope": "openid Patient.read Patient.search ",
"id_token": "eyJhbGciOiJSUzI1NiIsImtpZCI6IktleUlEIiwidHlwIjoiSldUIn0.eyJhdWQiOiJDbGllbnRJRCIsImV4cCI6RXhwaXJlc0F0LCJpYXQ
    iOklzc3VlZEF0LCJpc3MiOiJJc3N1ZXJVUkwiLCJzdWIiOiJFbmRVc2VySWRlbnRpZmllciJ9",
"__epic.dstu2.patient": "T1wI5bk8n1YVgvWk9D05BmRV0Pi3ECImNSK8DKyKltsMB",
"patient": "T1wI5bk8n1YVgvWk9D05BmRV0Pi3ECImNSK8DKyKltsMB",
"appointment": "eVa-Ad1SCIenVq8CYQVxfKwGtP-DfG3nIy9-5oPwTg2g3",
"encounter": "eySnzekbpf6uGZz87ndPuRQ3",
"location": "e4W4rmGe9QzuGm2Dy4NBqVc0KDe6yGld6HW95UuN-Qd03",
"loginDepartment": "e4W4rmGe9QzuGm2Dy4NBqVc0KDe6yGld6HW95UuN-Qd03",
```

```
"state": "abc123",
}
```

At this point, authorization is complete and the web application can access the protected patient data it requests using FHIR APIs.

Access Token Request: Frontend Confidential Clients

Consult Standalone Launch: Access Token Request for Frontend Confidential Clients for details on how to obtain an access token if your app is a confidential client.

OpenID Connect id_tokens

Epic started supporting the OpenID Connect id_token in the access token response for apps that request the openid scope since the November 2019 version of Epic.

A decoded OpenID Connect id_token JWT will have these headers:

Header | Description

alg | The JWT authentication algorithm. Currently only RSA 256 is supported in id_token JWTs so this will always be RS256.

typ | This is always set to JWT.

kid | The base64 encoded SHA-256 hash of the public key.

A decoded OpenID Connect id_token JWT will have this payload:

Claim | Description | Remarks

iss | Issuer of the JWT. This is set to the token endpoint that should be used by the client. | Starting in the May 2020 version of Epic, the iss will be set to the OAuth 2.0 server endpoint, e.g. https://<Interconnect Server URL>/oauth2. For Epic versions prior to May 2020, it is set to the OAuth 2.0 token endpoint, e.g. https://<Interconnect Server URL>/oauth2/token.

sub | STU3+ FHIR ID for the resource representing the user launching the app. | For Epic integrations, the sub and fhirUser claims reference one of the following resources depending on the type of workflow: Provider/user workflows: Practitioner resource MyChart self access workflows: Patient resource MyChart proxy access workflows: RelatedPerson resource Note apps with a SMART on FHIR version of DSTU2 will still get a STU3+ FHIR ID for the user. The STU3+ Practitioner.Read or RelatedPerson.Read APIs would be required to read the Practitioner or RelatedPerson FHIR IDs. Patient FHIR IDs can be used across FHIR versions in Epic's implementation.

fhirUser | Absolute reference to the FHIR resource representing the user launching the app. See the HL7 documentation for more details. | The fhirUser claim will only be present if app has the R4 (or greater) SMART on FHIR version selected, and requests both the openid and fhirUser scopes in the request to the authorize endpoint. See the remark for the sub claim above for more information about what the resource returned in these claims represents.

aud | Audiences that the ID token is intended for. This will be set to the client ID for the application that was just authorized during the SMART on FHIR launch.

iat | Time integer for when the JWT was created, expressed in seconds since the "Epoch" (1970-01-01T00:00:00Z UTC).

exp | Expiration time integer for this authentication JWT, expressed in seconds since the "Epoch" (1970-01-01T00:00:00Z UTC). | This is set to the current time plus 5 minutes.

preferred_username | The user's LDAP/AD down-level logon name. | This claim is supported in Epic version February 2021 and later. The preferred_username claim will be present only if the app requests both the openid and profile scopes in the request to the authorize endpoint.

nonce | The nonce for this client session. | The nonce claim will be present only if it was specified in the authorization request. This is an identifier for the client session. For more information about the nonce, refer to the Authentication Request section of the OpenID specification.

Here is an example decoded id_token that could be returned if the app has the R4 (or greater) SMART on FHIR version selected, and requests both the openid and fhirUser scopes in the request to the authorize endpoint:

```
{
"alg": "RS256",
"kid": "liCulTIaitUzjfUh2AqNiMro47X9HcVcd9XPi8LDJKA=",
"typ": "JWT"
}
{
"aud": "de5dae1a-4317-4c25-86f1-ed558e85529b",
"exp": 1595956317,
"fhirUser": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2/api/FHIR/R4/Practitioner/exfo6E4EXjWsnhA1OGVElgw3",
"iat": 1595956017,
"iss": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2",
"sub": "exfo6E4EXjWsnhA1OGVElgw3"
}
```

Here is an example decoded id_token that could be returned if the app requests at least the openid scope in the request to the authorize endpoint, but either doesn't request the fhirUser claim or doesn't meet the criteria outlined above for receiving the fhirUser claim:

```
{
"alg": "RS256",
"kid": "liCulTIaitUzjfUh2AqNiMro47X9HcVcd9XPi8LDJKA=",
"typ": "JWT"
}
{
"aud": "de5dae1a-4317-4c25-86f1-ed558e85529b",
"exp": 1595956317,
"iat": 1595956017,
"iss": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2",
"sub": "exfo6E4EXjWsnhA1OGVElgw3"
}
```

Validating the OpenID Connect JSON Web Token

When completing the OAuth 2.0 authorization workflow with the openid scope, the JSON Web Token (JWT) returned in the id_token field will be cryptographically signed. The public key to verify the authenticity of the JWT can be found at

https://<Interconnect Server URL>/api/epic/2019/Security/Open/PublicKeys/530005/OIDC, for example https://fhir.epic.com/interconnect-fhir-oauth/api/epic/2019/Security/Open/PublicKeys/530005/OIDC.

Starting in the May 2020 version of Epic, metadata about the server's OpenID Connect configuration, including the jwks_uri OIDC public key endpoint, can be found at the OpenID configuration endpoint https://<Interconnect Server URL&gt/oauth2/.well-known/openid-configuration, for example https://fhir.epic.com/interconnect-fhir-oauth/oauth2/.well-known/openid-configuration.

Starting in the August 2021 version of Epic, metadata about the server's OpenID Connect configuration can also be found at the FHIR endpoint https://<Interconnect Server URL>/api/FHIR/R4/.well-known/openid-configuration, for example https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/.well-known/openid-configuration.

Step 6: Your Application Uses FHIR APIs to Access Patient Data

With a valid access token, your application can now access protected patient data from the EHR database using FHIR APIs. Queries must contain an Authorization header that includes the access token presented as a Bearer token.

Here's an example of what a valid query looks like:

```
GET https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/DSTU2/Patient/T1wI5bk8n1YVgvWk9D05BmRV0Pi3ECImNSK8DKyKltsMB
    HTTP/1.1
Authorization: Bearer Nxfve4q3H9TKs5F5vf6kRYAZqzK7j9LHvrg1Bw7fU_07_FdV9aRzLCI1GxOn20LuO2Ahl5RkRnz-
    p8u1MeYWqA85T8s4Ce3LcgQqIwsTkI7wezBsMduPw_xkVtLzLU2O
```

Step 7: Use a Refresh Token to Obtain a New Access Token

Refresh tokens are not typically needed for embedded (SMART on FHIR) launches because users are not required to log in to Epic during the SMART on FHIR launch process, and the access token obtained from the launch process is typically valid for longer than the user needs to use the app.

Consult the Standalone Launch: Use a Refresh Token to Obtain a New Access Token for details on how to use a refresh token to get a new an access token if your app uses refresh tokens.

Standalone Launch

Contents

* How It Works

* Step 1: Your Application Requests an Authorization Code

* Step 2: EHR's Authorization Server Authenticates the User and Authorizes Access

Step 3: Your Application Exchanges the Authorization Code for an Access Token

* Non-confidential Clients

* Frontend Confidential Clients

* Step 4: Your Application Uses FHIR APIs to Access Patient Data

* Step 5: Use a Refresh Token to Obtain a New Access Token

Offline Access for Native and Browser-Based Applications

* Step 1: Get the initial access token you'll use to register a dynamic client

* Step 2: Register the dynamic client

* Step 3: Get an access token you can use to retrieve FHIR resources

* Step 3a: Get an access token using the JWT bearer flow

* Step 3b: Alternate flow with refresh tokens

How It Works

The app launches directly to the authorize endpoint outside of an EHR session and requests context from the EHR's authorization server.

Step 1: Your Application Requests an Authorization Code

Your application would like to authenticate the user using the OAuth 2.0 workflow. To initiate this process, your app needs to link (using HTTP GET) to the authorize endpoint and append the following querystring parameters:

* response_type: This parameter must contain the value "code".

* client_id: This parameter contains your web application's client ID issued by Epic.

* redirect_uri: This parameter contains your application's redirect URI. After the request completes on the Epic server, this URI will be called as a callback. The value of this parameter needs to be URL encoded. This URI must also be registered with the EHR's authorization server by adding it to your app listing.

* state: This optional parameter is generated by your app and is opaque to the EHR. The EHR's authorization server will append it to each subsequent exchange in the workflow for you to validate session integrity. While not required, this parameter is recommended to be included and validated with each exchange in order to increase security. For more information see RFC 6819 Section 3.6.

* scope: This parameter describes the information for which the web application is requesting access. Starting with the November 2019 version of Epic, the "openid" and "fhirUser" OpenID Connect scopes are supported. While scope is optional in Epic's SMART on FHIR implementation, it's required for testing in the sandbox. Providing scope=openid is sufficient.

* aud: Starting in the August 2021 version of Epic, health care organizations can optionally configure their system to require the aud parameter for Standalone and EHR launch workflows if a launch context is included in the scope parameter. Starting in the May 2023 version of Epic, this parameter will be required. The value to use is the base URL of the resource server the application intends to access, which is typically the FHIR server.

Additional parameters for native mobile apps (available starting in the August 2019 version of Epic):

* code_challenge: This optional parameter is generated by your app and used for PKCE. This is the S256 hashed version of the code_verifier parameter, which will be used in the token request.

* code_challenge_method: This optional parameter indicates the method used for the code_challenge parameter and is required if using that parameter. Currently, only the S256 method is supported.

Here's an example of an authorization request using HTTP GET. You will replace the [redirect_uri], [client_id], [state], and [audience] placeholders with your own values.

```
https://fhir.epic.com/interconnect-fhir-oauth/oauth2/authorize?response_type=code&redirect_uri=[redirect_uri]&client_id=
    [client_id]&state=[state]&aud=[audience]&scope=[scope]
```

This is an example link:

```
https://fhir.epic.com/interconnect-fhir-oauth/oauth2/authorize?response_type=code&redirect_uri=https%3A%2F%2Ffhir.epic.c
    om%2Ftest%2Fsmart&client_id=d45049c3-3441-40ef-
    ab4d-b9cd86a17225&state=abc123&aud=https%3A%2F%2Ffhir.epic.com%2Finterconnect-fhir-
    oauth%2Fapi%2Ffhir%2Fdstu2&scope=openid
```

Step 2: EHR's Authorization Server Authenticates the User and Authorizes Access

The EHR's authorization server reviews the request from your application, authenticates the user (sample credentials found here), and authorizes access. If approved, the authorization server redirects the browser to the redirect URL supplied in the initial request and appends the following querystring parameter.

* code: This parameter contains the authorization code generated by Epic, which will be exchanged for the access token in the next step.

* See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling authorization codes.

* state: This parameter will have the same value as the earlier state parameter. For more information, refer to Step 1.

Here's an example of what the redirect will look like if Epic's authorization server accepts the request:

```
https://fhir.epic.com/test/smart?code=yfNg-rSc1t5O2p6jVAZLyY00uOOte5KM1y3YUxqsJQnBKEMNsYqOPTyVqcCH3YXaPkLztO9Rvf7bhLqQTw
    ALHcHN6raxpTbR1eVgV2QyLA_4K0HrJO92et3qRXiXPkj7&state=abc123
```

Step 3: Your Application Exchanges the Authorization Code for an Access Token

After receiving the authorization code, your application trades the code for a JSON object containing an access token and contextual information by sending an HTTP POST to the token endpoint using a Content-Type header with value of "application/x-www-form-urlencoded". For more information, see RFC 6749 section 4.1.3.

Access Token Request: Non-confidential Clients

The following parameters are required in the POST body:

* grant_type: For the Standalone launch flow, this should contain the value "authorization_code".

* code: This parameter contains the authorization code sent from Epic's authorization server to your application as a querystring parameter on the redirect URI as described above.

* redirect_uri: This parameter must contain the same redirect URI that you provided in the initial access request. The value of this parameter needs to be URL encoded.

* client_id: This parameter must contain the application's client ID issued by Epic that you provided in the initial request.

* code_verifier: This optional parameter is used to verify against your code_challenge parameter when using PKCE. This parameter is passed as free text and must match the code_challenge parameter used in your authorization request once it is hashed on the server using the code_challenge_method. This parameter is available starting in the August 2019 version of Epic.

Here's an example of what an HTTP POST request for an access token might look like:

```
POST https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&code=yfNg-rSc1t5O2p6jVAZLyY00uOOte5KM1y3YUxqsJQnBKEMNsYqOPTyVqcCH3YXaPkLztO9Rvf7bhLqQTwALH
    cHN6raxpTbR1eVgV2QyLA_4K0HrJO92et3qRXiXPkj7&redirect_uri=https%3A%2F%2Ffhir.epic.com%2Ftest%2Fsmart&client_id=d45049
    c3-3441-40ef-ab4d-b9cd86a17225
```

The authorization server responds to the HTTP POST request with a JSON object that includes an access token. The response contains the following fields:

* access_token: This parameter contains the access token issued by Epic to your application and is used in future requests.

* See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling access tokens.

* token_type: In Epic's OAuth 2.0 implementation, this parameter always includes the value bearer.

* expires_in: This parameter contains the number of seconds for which the access token is valid.

* scope: This parameter describes the access your application is authorized for.

* id_token: Returned only for applications that have requested an openid scope. See above for more info on OpenID Connect id_tokens. This parameter follows the guidelines described earlier in this document.

* : For patient-facing workflows, this parameter identifies the FHIR ID for the patient on whose behalf authorization to the system was granted.

patient

* The patient's FHIR ID is not returned for provider-facing standalone launch workflows.

* : For patient-facing workflows, this parameter identifies the DSTU2 FHIR ID for the patient on whose behalf authorization to the system was granted.

epic.dstu2.patient

* The patient's FHIR ID is not returned for provider-facing standalone launch workflows.

Note that you can pass additional parameters if needed based on the integration configuration. Here's an example of what a JSON object including an access token might look like:


```
{
"access_token": "Nxfve4q3H9TKs5F5vf6kRYAZqzK7j9LHvrg1Bw7fU_07_FdV9aRzLCI1GxOn20LuO2Ahl5RkRnz-
    p8u1MeYWqA85T8s4Ce3LcgQqIwsTkI7wezBsMduPw_xkVtLzLU2O",
"token_type": "bearer",
"expires_in": 3240,
"scope": "Patient.read Patient.search ",
"patient": "T1wI5bk8n1YVgvWk9D05BmRV0Pi3ECImNSK8DKyKltsMB"
}
```


At this point, authorization is complete and the web application can access the protected patient data it requested using FHIR APIs.

Access Token Request: Frontend Confidential Clients

There are two authentication options available for confidential apps that can keep authentication credentials secret:

* Client Secret Authentication

* JSON Web Token (JWT) Authentication

Access Token Request: Frontend Confidential Clients with Client Secret Authentication

After receiving the authorization code, your application trades the code for a JSON object containing an access token and contextual information by sending an HTTP POST to the token endpoint using a Content-Type header with value of "application/x-www-form-urlencoded". For more information, see RFC 6749 section 4.1.3.

The Epic on FHIR website can generate a client secret (effectively a password) for your app to use to obtain refresh tokens, and store the hashed secret for you, or Epic community members can upload a client secret hash that you provide them when they activate your app for their system. If you provide community members a client secret hash to upload, you should use a unique client secret per Epic community member and per environment type (non-production and production) for each Epic community member.

The following parameters are required in the POST body:

* grant_type: This should contain the value authorization_code.

* code: This parameter contains the authorization code sent from Epic's authorization server to your application as a querystring parameter on the redirect URI as described above.

* redirect_uri: This parameter must contain the same redirect URI that you provided in the initial access request. The value of this parameter needs to be URL encoded.

Note: The client_id parameter is not passed in the POST body if you use client secret authentication, which is different from the access token request for apps that do not use refresh tokens. You will instead pass an Authorization HTTP header with client_id and client_secret URL encoded and passed as a username and password. Conceptually the Authorization HTTP header will have this value: base64(client_id:client_secret).

For example, using the following client_id and client_secret:

client_id: d45049c3-3441-40ef-ab4d-b9cd86a17225

URL encoded client_id: d45049c3-3441-40ef-ab4d-b9cd86a17225 Note: base64 encoding Epic's client IDs will have no effect

client_secret: this-is-the-secret-2/7

URL encoded client_secret: this-is-the-secret-2%2F7

Would result in this Authorization header:

```
Authorization: Basic base64Encode{d45049c3-3441-40ef-ab4d-b9cd86a17225:this-is-the-secret-2%2F7}
```

or


```
Authorization: Basic ZDQ1MDQ5YzMtMzQ0MS00MGVmLWFiNGQtYjljZDg2YTE3MjI1OnRoaXMtaXMtdGhlLXNlY3JldC0yJTJGNw==
```


Here's an example of what a valid HTTP POST might look like:

```
POST https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded
```

```
Authorization: Basic ZDQ1MDQ5YzMtMzQ0MS00MGVmLWFiNGQtYjljZDg2YTE3MjI1OnRoaXMtaXMtdGhlLXNlY3JldC0yJTJGNw==

grant_type=authorization_code&code=yfNg-rSc1t5O2p6jVAZLyY00uOOte5KM1y3YUxqsJQnBKEMNsYqOPTyVqcCH3YXaPkLztO9Rvf7bhLqQTwALH
    cHN6raxpTbR1eVgV2QyLA_4K0HrJO92et3qRXiXPkj7&redirect_uri=https%3A%2F%2Ffhir.epic.com%2Ftest%2Fsmart
```

The authorization server responds to the HTTP POST request with a JSON object that includes an access token and a refresh token. The response contains the following fields:

* refresh_token: This parameter contains the refresh token issued by Epic to your application and can be used to obtain a new access token. For more information on how this works, see Step 5.

* See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling refresh tokens.

* access_token: This parameter contains the access token issued by Epic to your application and is used in future requests.

* See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling access tokens.

* token_type: In Epic's OAuth 2.0 implementation, this parameter always includes the value bearer.

* expires_in: This parameter contains the number of seconds for which the access token is valid.

* scope: This parameter describes the access your application is authorized for.

* id_token: Returned only for applications that have requested an openid scope. See above for more info on OpenID Connect id_tokens. This parameter follows the guidelines of the OpenID Connect (OIDC) Core 1.0 specification. It is signed but not encrypted.

* : For patient-facing workflows, this parameter identifies the FHIR ID for the patient on whose behalf authorization to the system was granted.

patient

* The patient's FHIR ID is not returned for provider-facing standalone launch workflows.

* : For patient-facing workflows, this parameter identifies the DSTU2 FHIR ID for the patient on whose behalf authorization to the system was granted.

epic.dstu2.patient

* The patient's FHIR ID is not returned for provider-facing standalone launch workflows.

* : This parameter identifies the FHIR ID for the patient's encounter, if in context at time of launch. The encounter token corresponds to the FHIR Encounter resource.

encounter

* The encounter FHIR ID is not returned for standalone launch workflows.

* : This parameter identifies the FHIR ID for the ecounter department, if in context at time of launch. The location token corresponds to the FHIR Location resource.

location

* The location FHIR ID is not returned for standalone launch workflows.

* : This parameter identifies the FHIR ID for the patient's appointment, if appointment context is available at time of launch. The appointment token corresponds to the FHIR Appointment resource.

appointment

* The appointment FHIR ID is not returned for standalone launch workflows.

* : This parameter identifies the FHIR ID of the user's login department for launches from Hyperspace. The loginDepartment token corresponds to the FHIR Location resource.

loginDepartment

* The loginDepartment FHIR ID is not returned for standalone launch workflows.

Note that you can pass additional parameters if needed based on the integration configuration. Here's an example of what a JSON object including an access token and refres token might look like:

```
{
"access_token": "Nxfve4q3H9TKs5F5vf6kRYAZqzK7j9LHvrg1Bw7fU_07_FdV9aRzLCI1GxOn20LuO2Ahl5RkRnz-
    p8u1MeYWqA85T8s4Ce3LcgQqIwsTkI7wezBsMduPw_xkVtLzLU2O",
"refresh_token": "H9TKs5F5vf6kRYAZqzK7j9L_07_FdV9aRzLCI1GxOn20LuO2Ahl5R1MeYWqA85T8s4sTkI7wezBsMduPw_xkLzLU2O",
"token_type": "bearer",
"expires_in": 3240,
"scope": "Patient.read Patient.search ",
"patient": "T1wI5bk8n1YVgvWk9D05BmRV0Pi3ECImNSK8DKyKltsMB"
}
```

At this point, authorization is complete and the web application can access the protected patient data it requested using FHIR APIs.

Access Token Request: Frontend Confidential Clients with JSON Web Token (JWT) Authentication

After receiving the authorization code, your application trades the code for a JSON object containing an access token and contextual information by sending an HTTP POST to the token endpoint using a Content-Type header with value of "application/x-www-form-urlencoded". For more information, see RFC 6749 section 4.1.3.

You can use a one-time use JSON Web Token (JWT) to authenticate your app to the authorization server and obtain an access token and/or refresh token. There are several libraries for creating JWTs. See jwt.io for some examples. You'll

pre-register a JSON Web Key Set or JWK Set URL for a given Epic community member environment on the Epic on FHIR website and then use the corresponding private key to create a signed JWT. Frontend apps using JWT authentication must use a JSON Web Key Set or JWK Set URL as static public keys are not supported.

See the Creating a Public Private Key Pair for JWT Signature and the Creating a JWT sections for details on how to create a signed JWT.

The following parameters are required in the POST body:

* grant_type: This should contain the value authorization_code.

* code: This parameter contains the authorization code sent from Epic's authorization server to your application as a querystring parameter on the redirect URI as described above.

* redirect_uri: This parameter must contain the same redirect URI that you provided in the initial access request. The value of this parameter needs to be URL encoded.

* client_assertion_type: This should be set to urn:ietf:params:oauth:client-assertion-type:jwt-bearer.

* client_assertion: This will be the one-time use JSON Web Token (JWT) your app generated using the steps mentioned above.

Here's an example of what a valid HTTP POST might look like:

```
POST https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&client_assertion_type=urn%3Aietf%3Aparams%3Aoauth%3Aclient-assertion-type%3Ajwt-
    bearer&code=yfNg-rSc1t5O2p6jVAZLyY00uOOte5KM1y3YUxqsJQnBKEMNsYqOPTyVqcCH3YXaPkLztO9Rvf7bhLqQTwALHcHN6raxpTbR1eVgV2Qy
    LA_4K0HrJO92et3qRXiXPkj7&redirect_uri=https%3A%2F%2Ffhir.epic.com%2Ftest%2Fsmart&client_assertion=eyJhbGciOiJSUzM4NC
    IsInR5cCI6IkpXVCJ9.eyJpc3MiOiJkNDUwNDljMy0zNDQxLTQwZWYtYWI0ZC1iOWNkODZhMTcyMjUiLCJzdWIiOiJkNDUwNDljMy0zNDQxLTQwZWYtY
    WI0ZC1iOWNkODZhMTcyMjUiLCJhdWQiOiJodHRwczovL2ZoaXIuZXBpYy5jb20vaW50ZXJjb25uZWN0LWZoaXItb2F1dGgvb2F1dGgyL3Rva2VuIiwia
    nRpIjoiZjllYWFmYmEtMmU0OS0xMWVhLTg4ODAtNWNlMGM1YWVlNjc5IiwiZXhwIjoxNTgzNTI0NDAyLCJuYmYiOjE1ODM1MjQxMDIsImlhdCI6MTU4M
    zUyNDEwMn0.dztrzHo9RRwNRaB32QxYLaa9CcIMoOePRCbpqsRKgyJmBOGb9acnEZARaCzRDGQrXccAQ9-
    syuxz5QRMHda0S3VbqM2KX0VRh9GfqG3WJBizp11Lzvc2qiUPr9i9CqjtqiwAbkME40tioIJMC6DKvxxjuS-St5pZbSHR-
    kjn3ex2iwUJgPbCfv8cJmt19dHctixFR6OG-YB6lFXXpNP8XnL7g85yLOYoQcwofN0k8qK8h4uh8axTPC21fv21mCt50gx59XgKsszysZnMDt8OG_G4g
    jk_8JnGHwOVkJhqj5oeg_GdmBhQ4UPuxt3YvCOTW9S2vMikNUnxrhdVvn2GVg
```

The authorization server responds to the HTTP POST request with a JSON object that includes an access token and a refresh token. The response contains the following fields:

* : If your app is configured to receive refresh tokens, this parameter contains the refresh token issued by Epic to your application and can be used to obtain a new access token. For more information on how this works, see Step 5.

refresh_token

* See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling refresh tokens.

* : This parameter contains the access token issued by Epic to your application and is used in future requests.

access_token

* See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling access tokens.

* token_type: In Epic's OAuth 2.0 implementation, this parameter always includes the value bearer.

* expires_in: This parameter contains the number of seconds for which the access token is valid.

* scope: This parameter describes the access your application is authorized for.

* id_token: Returned only for applications that have requested an openid scope. See above for more info on OpenID Connect id_tokens. This parameter follows the guidelines of the OpenID Connect (OIDC) Core 1.0 specification. It is signed but not encrypted.

* : For patient-facing workflows, this parameter identifies the FHIR ID for the patient on whose behalf authorization to the system was granted.

patient

* The patient's FHIR ID is not returned for provider-facing standalone launch workflows.

* : For patient-facing workflows, this parameter identifies the DSTU2 FHIR ID for the patient on whose behalf authorization to the system was granted.

epic.dstu2.patient

* The patient's FHIR ID is not returned for provider-facing standalone launch workflows.

* : This parameter identifies the FHIR ID for the patient's encounter, if in context at time of launch. The encounter token corresponds to the FHIR Encounter resource.

encounter

* The encounter FHIR ID is not returned for standalone launch workflows.

* : This parameter identifies the FHIR ID for the encounter department, if in context at time of launch. The location token corresponds to the FHIR Location resource.

location

* The location FHIR ID is not returned for standalone launch workflows.

* : This parameter identifies the FHIR ID for the patient's appointment, if appointment context is available at time of launch. The appointment token corresponds to the FHIR Appointment resource.

appointment

* The appointment FHIR ID is not returned for standalone launch workflows.

* : This parameter identifies the FHIR ID of the user's login department for launches from Hyperspace. The loginDepartment token corresponds to the FHIR Location resource.

loginDepartment

* The loginDepartment FHIR ID is not returned for standalone launch workflows.

Note that you can pass additional parameters if needed based on the integration configuration. Here's an example of what a JSON object including an access token might look like:

```
{
  "access_token": "Nxfve4q3H9TKs5F5vf6kRYAZqzK7j9LHvrg1Bw7fU_07_FdV9aRzLCI1GxOn20LuO2Ahl5RkRnz-
    p8u1MeYWqA85T8s4Ce3LcgQqIwsTkI7wezBsMduPw_xkVtLzLU2O",
  "refresh_token": "H9TKs5F5vf6kRYAZqzK7j9L_07_FdV9aRzLCI1GxOn20LuO2Ahl5R1MeYWqA85T8s4sTkI7wezBsMduPw_xkLzLU2O",
  "token_type": "bearer",
  "expires_in": 3240,
  "scope": "Patient.read Patient.search ",
  "patient": "T1wI5bk8n1YVgvWk9D05BmRV0Pi3ECImNSK8DKyKltsMB"
}
```

At this point, authorization is complete and the web application can access the protected patient data it requested using FHIR APIs.

Step 4: Your Application Uses FHIR APIs to Access Patient Data

With a valid access token, your application can now access protected patient data from the EHR database using FHIR APIs. Queries must contain an Authorization header that includes the access token presented as a Bearer token.

Here's an example of what a valid query looks like:

```
GET https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/DSTU2/Patient/T1wI5bk8n1YVgvWk9D05BmRV0Pi3ECImNSK8DKyKltsMB
    HTTP/1.1
Authorization: Bearer Nxfve4q3H9TKs5F5vf6kRYAZqzK7j9LHvrg1Bw7fU_07_FdV9aRzLCI1GxOn20LuO2Ahl5RkRnz-
    p8u1MeYWqA85T8s4Ce3LcgQqIwsTkI7wezBsMduPw_xkVtLzLU2O
```

Step 5: Use a Refresh Token to Obtain a New Access Token

If your app uses refresh tokens (i.e. it can securely store credentials), then you can use a refresh token to request a new access token when the current access token expires (determined by the expires_in field from the authorization response from step 3).

There are two authentication options available for confidential apps that can keep authentication credentials secret:

* Client Secret Authentication

* JSON Web Token (JWT) Authentication

Using Refresh Tokens: If You Are Using Client Secret Authentication

Your application trades the refresh_token for a JSON object containing a new access token and contextual information by sending an HTTP POST to the token endpoint using a Content-Type HTTP header with value of "application/x-www-form-urlencoded". For more information, see RFC 6749 section 4.1.3.

The Epic on FHIR website can generate a client secret (effectively a password) for your app when using refresh tokens, and store the hashed secret for you, or Epic community members can upload a client secret hash that you provide them when they activate your app for their system. If you provide community members a client secret hash to upload, you should use a unique client secret per Epic community member and per environment type (non-production and production) for each Epic community member.

The following parameters are required in the POST body:

* grant_type: This parameter always contains the value refresh_token.

* refresh_token: The refresh token received from a prior authorization request.

An Authorization header using HTTP Basic Authentication is required, where the username is the URL encoded client_id and the password is the URL encoded client_secret.

For example, using the following client_id and client_secret:

client_id: d45049c3-3441-40ef-ab4d-b9cd86a17225

URL encoded client_id: d45049c3-3441-40ef-ab4d-b9cd86a17225 Note: base64 encoding Epic's client IDs will have no effect

client_secret: this-is-the-secret-2/7

URL encoded client_secret: this-is-the-secret-2%2F7

Would result in this Authorization header:

```
Authorization: Basic base64Encode{d45049c3-3441-40ef-ab4d-b9cd86a17225:this-is-the-secret-2%2F7}
```

or

```
Authorization: Basic ZDQ1MDQ5YzMtMzQ0MS00MGVmLWFiNGQtYjljZDg2YTE3MjI1OnRoaXMtaXMtdGhlLXNlY3JldC0yJTJGNw==
```

Here's an example of what a valid HTTP POST might look like:

```
POST https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token HTTP/1.1
Authorization: Basic ZDQ1MDQ5YzMtMzQ0MS00MGVmLWFiNGQtYjljZDg2YTE3MjI1OnRoaXMtaXMtdGhlLXNlY3JldC0yJTJGNw==
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token&refresh_token=j12xcniournlsdf234bgsd
```

The authorization server responds to the HTTP POST request with a JSON object that includes the new access token. The response contains the following fields:

* access_token: This parameter contains the new access token issued.

* token_type: In Epic's OAuth 2.0 implementation, this parameter always includes the value bearer.

* expires_in: This parameter contains the number of seconds for which the access token is valid.

* scope: This parameter describes the access your application is authorized for.

An example response to the previous request may look like the following:

```
{
"access_token": "57CjhZEdiTcCh1nqIwQUw5rODOLP3bSTnMEGNYbBerSeNn8hIUm6Mlc5ruCTfQawjRAoR8sYr8S_7vNdnJfgRKfD6s8mOqPnvJ8vZOH
    jEvy7l3Ra9frDaEAUBbN-j86k",
"token_type": "bearer",
"expires_in": 3240,
"scope": "Patient.read Patient.search"
}
```

Using Refresh Tokens: If You Are Using JSON Web Token Authentication

Your application trades the refresh_token for a JSON object containing a new access token and contextual information by sending an HTTP POST to the token endpoint using a Content-Type HTTP header with value of "application/x-www-form-urlencoded". For more information, see RFC 6749 section 4.1.3.

You can use a one-time use JSON Web Token (JWT) to authenticate your app to the authorization server to exchange the refresh token for a new access token. There are several libraries for creating JWTs. See jwt.io for some examples. You'll pre-register a JSON Web Key Set or JWK Set URL for a given Epic community member environment on the Epic on FHIR website and then use the corresponding private key to create a signed JWT. Frontend apps using JWT authentication must use a JSON Web Key Set or JWK Set URL as static public keys are not supported.

See the Creating a Public Private Key Pair for JWT Signature and the Creating a JWT sections for details on how to create a signed JWT.

The following parameters are required in the POST body:

* grant_type: This should contain the value refresh_token.

* refresh_token: The refresh token received from a prior authorization request.

* client_assertion_type: This should be set to urn:ietf:params:oauth:client-assertion-type:jwt-bearer.

* client_assertion: This will be the one-time use JSON Web Token (JWT) your app generated using the steps mentioned above.

Here's an example of what a valid HTTP POST might look like:

```
POST https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token&client_assertion_type=urn%3Aietf%3Aparams%3Aoauth%3Aclient-assertion-type%3Ajwt-bearer&client_a
    ssertion=eyJhbGciOiJSUzM4NCIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJkNDUwNDljMy0zNDQxLTQwZWYtYWI0ZC1iOWNkODZhMTcyMjUiLCJzdWIiOi
    JkNDUwNDljMy0zNDQxLTQwZWYtYWI0ZC1iOWNkODZhMTcyMjUiLCJhdWQiOiJodHRwczovL2ZoaXIuZXBpYy5jb20vaW50ZXJjb25uZWN0LWZoaXItb2
    F1dGgvb2F1dGgyL3Rva2VuIiwianRpIjoiZjllYWFmYmEtMmU0OS0xMWVhLTg4ODAtNWNlMGM1YWVlNjc5IiwiZXhwIjoxNTgzNTI0NDAyLCJuYmYiOj
    E1ODM1MjQxMDIsImlhdCI6MTU4MzUyNDEwMn0.dztrzHo9RRwNRaB32QxYLaa9CcIMoOePRCbpqsRKgyJmBOGb9acnEZARaCzRDGQrXccAQ9-
    syuxz5QRMHda0S3VbqM2KX0VRh9GfqG3WJBizp11Lzvc2qiUPr9i9CqjtqiwAbkME40tioIJMC6DKvxxjuS-St5pZbSHR-
    kjn3ex2iwUJgPbCfv8cJmt19dHctixFR6OG-YB6lFXXpNP8XnL7g85yLOYoQcwofN0k8qK8h4uh8axTPC21fv21mCt50gx59XgKsszysZnMDt8OG_G4g
    jk_8JnGHwOVkJhqj5oeg_GdmBhQ4UPuxt3YvCOTW9S2vMikNUnxrhdVvn2GVg
```

The authorization server responds to the HTTP POST request with a JSON object that includes the new access token. The response contains the following fields:

* access_token: This parameter contains the new access token issued.

* token_type: In Epic's OAuth 2.0 implementation, this parameter always includes the value bearer.

* expires_in: This parameter contains the number of seconds for which the access token is valid.

* scope: This parameter describes the access your application is authorized for.

An example response to the previous request may look like the following:

```
{
  "access_token": "57CjhZEdiTcCh1nqIwQUw5rODOLP3bSTnMEGNYbBerSeNn8hIUm6Mlc5ruCTfQawjRAoR8sYr8S_7vNdnJfgRKfD6s8mOqPnvJ8vZ
    OHjEvy7l3Ra9frDaEAUBbN-j86k",
  "token_type": "bearer",
  "expires_in": 3240,
  "scope": "Patient.read Patient.search"
}
```

Offline Access for Native and Browser-Based Applications

For a native client app (for example, an iOS mobile app or a Windows desktop app) or a browser-based app (for example, a single-page application) to use the confidential app profile in the SMART App Launch framework, that app needs to use "additional technology (such as dynamic client registration and universal redirect_uris) to protect the secret." If you have a native client or browser-based app, you can register a dynamic client to integrate with Epic using the steps below.

Step 1: Get the initial access token you'll use to register a dynamic client

Your app uses a flow like the one described above for a Standalone Launch. Note your application will specifically not be using a client secret, and will use its initial client ID issued by Epic.

The end-result of this flow is your app obtaining an initial access token (defined in RFC 7591) used for registering a client instance (step 2). This token is intentionally one-time-use and should be used for registration immediately.

Step 2: Register the dynamic client

To register the dynamic client, your application needs to:

* Generate a public-private key pair on the user's phone or computer.

* Native apps can use the same strategies as backend apps to generate key sets

* Browser-based apps can use the WebCrypto API

* Securely store that device-specific key pair on the user's device.

* Browser-based apps using WebCrypto should follow key storage recommendations

* Use the access token from step 1 to register a dynamic client using the OAuth 2.0 Dynamic Client Registration Protocol.

The client ID and private key should be associated with the Epic environment they are communicating with. Identify the Epic environment with its OAuth 2.0 server URL. If this is a backend integration or a subspace integration, the environment health system identifier (HSI) can be used to identify the environment instead.

This request must have two elements:

* software_id: This parameter must contain the application's client ID issued by Epic.

* jwks: This parameter contains a JSON Web Key Set containing the public key from the key pair your application generated in step 2A.

Here's an example of what an HTTP POST request to register a dynamic client might look like:

```
POST https://fhir.epic.com/interconnect-fhir-oauth/oauth2/register HTTP/1.1
Content-Type: application/json
Authorization: Bearer Nxfve4q3H9TKs5F5vf6kRYAZqzK7j9LHvrg1Bw7fU_07_FdV9aRzLCI1GxOn20LuO2Ahl5RkRnz-
    p8u1MeYWqA85T8s4Ce3LcgQqIwsTkI7wezBsMduPw_xkVtLzLU2O

{
    "software_id": "d45049c3-3441-40ef-ab4d-b9cd86a17225",
    "jwks": {
        "keys": [{
                "e": "AQAB",
                "kty": "RSA",
                "n": "vGASMnWdI-ManPgJi5XeT15Uf1tgpaNBmxfa-
    _bKG6G1DDTsYBy2K1uubppWMcl8Ff_2oWe6wKDMx2-bvrQQkR1zcV96yOgNmfDXuSSR1y7xk1Kd-uUhvmIKk81UvKbKOnPetnO1IftpEBm5Llzy-
    1dN3kkJqFabFSd3ujqi2ZGuvxfouZ-S3lpTU3O6zxNR6oZEbP2BwECoBORL5cOWOu_pYJvALf0njmamRQ2FKKCC-
    pf0LBtACU9tbPgHorD3iDdis1_cvk16i9a3HE2h4Hei4-nDQRXfVgXLzgr7GdJf1ArR1y65LVWvtuwNf7BaxVkEae1qKVLa2RUeg8imuw",
            }
        ]
    }
}
```

The EHR stores your app's public key, issues it a dynamic client ID, and returns a response that might look like:

```
HTTP/1.1 201 Created
Content-Type: application/json

{
    "redirect_uris": [
        " https://fhir.epic.com/test/smart"
    ],
    "token_endpoint_auth_method": "none",
    "grant_types": [
        "urn:ietf:params:oauth:grant-type:jwt-bearer"
    ],
    "software_id": " d45049c3-3441-40ef-ab4d-b9cd86a17225",
    "client_id": "G65DA2AF4-1C91-11EC-9280-0050568B7514",
    "client_id_issued_at": 1632417134,
    "jwks": {
        "keys": [{
                "kty": "RSA",
                "n": "vGASMnWdI-ManPgJi5XeT15Uf1tgpaNBmxfa-
    _bKG6G1DDTsYBy2K1uubppWMcl8Ff_2oWe6wKDMx2-bvrQQkR1zcV96yOgNmfDXuSSR1y7xk1Kd-uUhvmIKk81UvKbKOnPetnO1IftpEBm5Llzy-
    1dN3kkJqFabFSd3ujqi2ZGuvxfouZ-S3lpTU3O6zxNR6oZEbP2BwECoBORL5cOWOu_pYJvALf0njmamRQ2FKKCC-
    pf0LBtACU9tbPgHorD3iDdis1_cvk16i9a3HE2h4Hei4-nDQRXfVgXLzgr7GdJf1ArR1y65LVWvtuwNf7BaxVkEae1qKVLa2RUeg8imuw",
                "e": "AQAB"
            }
        ]
    }
}
```

Step 3: Get an access token you can use to retrieve FHIR resources

Now that you have registered a dynamic client with its own credentials, there are two ways you can get an access token to make FHIR calls. We recommend that you use the JWT bearer flow. In the JWT bearer flow, the client issues a JWT with its client ID as the "sub" (subject) claim, indicating the access token should be issued to the users who registered the client. It is also possible to use the SMART standalone launch sequence again to get a refresh token and then use the refresh token flow. Note that both flows involve your app making a JSON Web Token and signing it with its private key.

Generate a JSON Web Token

Your app follows the guidelines below to create a JWT assertion using its new private key.

To get an access token, your confidential app needs to authenticate itself. Your dynamic client uses the private key it generated in step 2 to sign a JWT.

Here's an example of the JWT body:

```
{
    "sub": "G00000000-0129-A6FA-7EDC-55B3FB6E85F8",
    "aud": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token",
    "jti": "3fca7b08-e5a1-4476-9e4b-4e17f0ad7d1d",
    "nbf": 1639694983,
    "exp": 1639695283,
    "iat": 1639694983,
    "iss": "G00000000-0129-A6FA-7EDC-55B3FB6E85F8"
}
```

And here's the final encoded JWT:

```
eyJhbGciOiJSUzI1NiIsImtpZCI6ImJXbGphR0ZsYkNBOE15QnNZWFZ5WVE9PSIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJHMDAwMDAwMDAtMDEyOS1BNkZBLTd
    FREMtNTVCM0ZCNkU4NUY4IiwiYXVkIjpbImh0dHBzOi8vdnMtaWN4LmVwaWMuY29tL0ludGVyY29ubmVjdC1DdXJyZW50LUZpbmFsLVByaW1hcnkvIl0
    sImp0aSI6IjNmY2E3YjA4LWU1YTEtNDQ3Ni05ZTRiLTRlMTdmMGFkN2QxZCIsIm5iZiI6MTYzOTY5NDk4MywiZXhwIjoxNjM5Njk1MjgzLCJpYXQiOjE
    2Mzk2OTQ5ODMsImlzcyI6IkcwMDAwMDAwMC0wMTI5LUE2RkEtN0VEQy01NUIzRkI2RTg1RjgifQ.UyW_--
    xpDdfB3EbxQ1KwRRuNDGIb034Y9mpExaaYyoVVkfz3ophzueIFMRrcdqTWmH96Ivx7O-fnjvHkb9iv1ELGVS_UYZy-
    JNb7r5VHDhdEYoRJzomYQzAyZutK9CJuJQZSvJeQLOXFoFN-fuCHIUBmSoTY5FDGy8gmq_a5fD_L-PnrXCD6SyP663s8kP-
    cFWh1iuXP0pjg8EMuFFEwgSo-
    chvv6RO4LIBebqkkv5qkgO_nrcXVJpxu42FDNrP2618q2Rw7sdIaewDw_I5026T4jNSoJXT12dHurqzedvC20exOO2MA6Vh1o2iVyzIfPE1EnEf-
    hk4WRFtQTUlqCQ
```

Step 3a: Get an access token using the JWT bearer flow

For the duration the user selected when approving your app to get an authorization code in step 1, your app can get an access token whenever it needs using the JWT bearer authorization grant described in RFC 7523 Section 2.1.

Recall that in the JWT bearer flow, the client issues a JWT to itself with a "sub" (subject) claim that indicates the user on whose behalf the client acts. Because the dynamic client your app registered in step 2 is bound to the user who logged in during the SMART standalone launch sequence in step 1, the server only accepts a JWT where the subject is the user bound to that client.

Your app posts the JWT it issued itself in the Generate a JSON Web Token step to the server to get an access token. The following parameters are required in the POST body:

* grant_type: This parameter contains the static value urn:ietf:params:oauth:grant-type:jwt-bearer.

* assertion: This parameter contains the JWT your app generated above.

* client_id: This parameter contains the dynamic client ID assigned in step 2.

Here's an example of an HTTP POST request for an access token. You will replace the [assertion] and [client_id] placeholders with your own values.

```
POST https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded

grant_type=urn%3Aietf%3Aparams%3Aoauth%3Agrant-type%3Ajwt-bearer&assertion=[assertion]&client_id=[client_id]
```

The authorization server responds to the HTTP POST request with a JSON object that includes an access token. The response contains the following fields:

* access_token: This parameter contains the access token issued by Epic to your application and is used in future requests.

* See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling access tokens.

* token_type: In Epic's OAuth 2.0 implementation, this parameter always includes the value bearer.

* expires_in: This parameter contains the number of seconds for which the access token is valid.

* epic.dstu2.patient: This parameter identifies the DSTU2 FHIR ID for the patient, if a patient is in context at time of launch.

* scope: This parameter describes the access your application is authorized for.

* patient: This parameter identifies provides the FHIR ID for the patient.

Note that you can include additional fields in the response if needed based on the integration configuration. For more information, refer to Launching your apptopic. Here's an example of what a JSON object including an access token might look like:

```
{
    "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJhdWQiOiJ1cm46R1RNUUFDVVI6Y2UuZ3RtcWFjdXIiLCJjbGllbnRfaWQiOi
    JHMDAwMDAwMDAtMDEyOS1BNkZBLTdFREMtNTVCM0ZCNkU4NUY4IiwiZXBpYy5lY2kiOiJ1cm46ZXBpYzpzMmN1cmd0IiwiZXBpYy5tZXRhZGF0YSI6Im
    VTMUF1dng4NlhRanRmVExub2loeks4QVRjdU5OUDFnRHhCRklvR0EyVUhHdlpDRE5PYjVZQ3NFZVhlUjExa1UyTjBHaHIwZjIwcE5yQnJ0V1JUa0l4d3
    VoVnlYeEVYUjQwVGlkdXNKdTk5ZmZRLWsybjFJWEY4X2I0SS11a054IiwiZXBpYy50b2tlbnR5cGUiOiJhY2Nlc3MiLCJleHAiOjE2Mzk2OTg1ODgsIm
```

```
    lhdCI6MTYzOTY5NDk4OCwiaXNzIjoidXJuOkdUTVFBQ1VSOmNlLmd0bXFhY3VyIiwianRpIjoiYzQ1YzU4NjUtY2E3Ny00YWVlLTk3NDEtY2YzNzFiND
    lhY2FkIiwibmJmIjoxNjM5Njk0OTg4LCJzdWIiOiJlRmdDY3E4cW1PeXY1b3FEcUFyUS5MQTMifQ.G0Z3PBv4ldpTqkjBjNUnvRYeVnNzT8qvLsGAVN8
    S9YibJGGCH_Txd6Ph1c9yB2hlQW3dw9IkaAvxxlUuclGMzmtyPXeo8wcWC07t_0vVasS-
    Ya9VjeDtR1hO8rcqEgV1DhKZ1jsEbzlRvKuZvONew0gL25ug6dOolNXcPcluzK6sxEyf2UoosX-W3nsU0iYZPJI-mf7lMEbsMUOSY8CR-
    77uBap3suxxHy03BwtkAXP0GwW0KSjOVe7_bsxX9k4DEhWyZuEOgDjEhONQFe2TeuWgUcI2KQeK5HjzmxN3dp56rCZ8zlhlukgw-
    C0F2IDbkZ5on7g7rl8lm29I7_kq9g",
    "token_type": "Bearer",
    "expires_in": 3600,
    "scope": "patient/Immunization.Read patient/Immunization.read patient/Patient.read patient/Practitioner.read
    patient/PractitionerRole.read launch/patient offline_access",
    "state": "oYK9nyFdUgTb1n_hMHZESDea",
    "patient": "eUYSU3eH0lceii-4SYNvpPw3",
    "__epic.dstu2.patient": "TJR7HfYCw58VnihLrp.axehccg-4IcjmZd3lw7Spm1F8B"
}
```


Step 3b: Alternate flow with refresh tokens

If you completed Step 3A, you do not need to complete this step. This is an alternate workflow to the JWT bearer flow. We strongly recommend that you use the JWT bearer flow above so you can provide a smoother user experience in which the user only needs to authenticate once. To get a refresh token, your app would need to go through the Standalone Launch sequence a second time. The first time (see Step 1: Get the access token you'll use to register a dynamic client) you were using the public app profile and weren't issued a refresh token. Now that you have registered a dynamic client with its own credentials, you can use the confidential-asymmetric app profile.

Get an authorization code

This is the same as the Get an authorization code step from Step 1 except:

* The client_id should be that of your dynamic client

* Your token request will resemble a backend services grant, using the following parameters:

* client_assertion – set to a JWT signed with your dynamic client’s private key

* client_assertion_type – set to urn:ietf:params:oauth:client-assertion-type:jwt-bearer

* An additional parameter per grant_type (code or refresh_token) described below

Note that the user will once again need to log in.

Exchange the authorization code for an access token

When your app makes this request to the token, it needs to authenticate itself to be issued a refresh token. It uses the JWT it generated to authenticate itself per RFC 7523 section 2.2. This request will look like the JWT bearer request except the grant type is authorization_code instead of urn:ietf:params:oauth:grant-type:jwt-bearer and it includes the PKCE code verifier like the access token request you made before registering the dynamic client.

In response to this request, you’ll get an access token and a refresh token.

Use the refresh token to get subsequent access tokens

When the access token expires, your app can generate a new JWT and use the refresh token flow to get a new access token. This request will look like the JWT bearer request except the grant type is refresh_token instead of urn:ietf:params:oauth:grant-type:jwt-bearer.

SMART Backend Services (Backend OAuth 2.0)

Contents

* Building a Backend OAuth 2.0 App

* Complete Required Epic Community Member Setup to Audit Access from Your Backend Application

Using a JWT to Obtain an Access Token for a Backend Service

* Step 1: Creating the JWT

* Step 2: POSTing the JWT to Token Endpoint to Obtain Access Token

Overview

Backend apps (i.e. apps without direct end user or patient interaction) can also use OAuth 2.0 authentication through the client_credentials OAuth 2.0 grant type. Epic's OAuth 2.0 implementation for backend services follows the SMART Backend Services: Authorization Guide, though it currently differs from that profile in some respects. Application vendors pre-register a public key for a given Epic community member on the Epic on FHIR website and then use the corresponding private key to sign a JSON Web Token (JWT) which is presented to the authorization server to obtain an access token.

Building a Backend OAuth 2.0 App

To use the client_credentials OAuth 2.0 grant type to authorize your backend application's access to patient information, two pieces of information need to be shared between the authorization server and your application:

* Client ID: The client ID identifies your application to authentication servers within the Epic community and allows you to connect to any organization.

* Public key: The public key is used to validate your signed JSON Web Token to confirm your identity.

You can register your application for access to both the sandbox and Epic organizations here. You'll provide information, including a JSON Web Key set URL (JWK Set URL or JKU) that hosts a public key used to verify the JWT signature. Epic will generate a client ID for you. Your app will need to have both the Backend Systems radio button selected and the Use OAuth 2.0 box checked in order to register a JKU for backend OAuth 2.0.

Complete Required Epic Community Member Setup to Audit Access from Your Backend Application

Note: This build is not needed in the sandbox. The sandbox automatically maps your client to a user, removing the need for this setup.

When performing setup with an Epic community member, their Epic Client Systems Administrator (ECSA) will need to map your client ID to an Epic user account that will be used for auditing web service calls made by your backend application. Your app will not be able to obtain an access token until this build is completed.

Using a JWT to Obtain an Access Token for a Backend Service

You will generate a one-time use JSON Web Token (JWT) to authenticate your app to the authorization server and obtain an access token that can be used to authenticate your app's web service calls. There are several libraries for creating JWTs. See jwt.io for some examples.

Step 1: Creating the JWT

See the Creating a Public Private Key Pair for JWT Signature and the Creating a JWT sections for details on how to create a signed JWT.

Step 2: POSTing the JWT to Token Endpoint to Obtain Access Token

Your application makes a HTTP POST request to the authorization server's OAuth 2.0 token endpoint to obtain access token. The following form-urlencoded parameters are required in the POST body:

* grant_type: This should be set to client_credentials.

* client_assertion_type: This should be set to urn:ietf:params:oauth:client-assertion-type:jwt-bearer.

* client_assertion: This should be set to the JWT you created above.

Here is an example request:

```
POST https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token HTTP/1.1
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials&client_assertion_type=urn%3Aietf%3Aparams%3Aoauth%3Aclient-assertion-type%3Ajwt-bearer&cli
    ent_assertion=eyJhbGciOiJSUzM4NCIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJkNDUwNDljMy0zNDQxLTQwZWYtYWI0ZC1iOWNkODZhMTcyMjUiLCJzd
    WIiOiJkNDUwNDljMy0zNDQxLTQwZWYtYWI0ZC1iOWNkODZhMTcyMjUiLCJhdWQiOiJodHRwczovL2ZoaXIuZXBpYy5jb20vaW50ZXJjb25uZWN0LWZoa
    XItb2F1dGgvb2F1dGgyL3Rva2VuIiwianRpIjoiZjllYWFmYmEtMmU0OS0xMWVhLTg4ODAtNWNlMGM1YWVlNjc5IiwiZXhwIjoxNTgzNTI0NDAyLCJuY
    mYiOjE1ODM1MjQxMDIsImlhdCI6MTU4MzUyNDEwMn0.dztrzHo9RRwNRaB32QxYLaa9CcIMoOePRCbpqsRKgyJmBOGb9acnEZARaCzRDGQrXccAQ9-
    syuxz5QRMHda0S3VbqM2KX0VRh9GfqG3WJBizp11Lzvc2qiUPr9i9CqjtqiwAbkME40tioIJMC6DKvxxjuS-St5pZbSHR-
    kjn3ex2iwUJgPbCfv8cJmt19dHctixFR6OG-YB6lFXXpNP8XnL7g85yLOYoQcwofN0k8qK8h4uh8axTPC21fv21mCt50gx59XgKsszysZnMDt8OG_G4g
    jk_8JnGHwOVkJhqj5oeg_GdmBhQ4UPuxt3YvCOTW9S2vMikNUnxrhdVvn2GVg
```

And here is an example response body assuming the authorization server approves the request:

```
{
"access_token": "i82fGhXNxmidCt0OdjYttm2x0cOKU1ZbN6Y_-zBvt2kw3xn-MY3gY4lOXPee6iKPw3JncYBT1Y-kdPpBYl-
    lsmUlA4x5dUVC1qbjEi1OHfe_Oa-VRUAeabnMLjYgKI7b",
"token_type": "bearer",
"expires_in": 3600,
"scope": "Patient.read Patient.search"
}
```

* Note: See the Epic-Issued OAuth 2.0 Tokens appendix section for details on handling access tokens.

JSON Web Tokens (JWTs)

Contents

Creating a Public Private Key Pair for JWT Signature

* OpenSSL

* Windows PowerShell

* Finding the Public Key Certificate Fingerprint (Also Called Thumbprint)

* Creating a JWT

JSON Web Key Sets

* Key Identifier Requirements

Hosting a JWK Set URL

* App-Level vs Licensing Specific JWK Set URLs

* Registering a JWK Set URL

* Key Rotation

Providing Multiple Public Keys

* How

* Why Not?
* Use Cases

Background

JSON Web Tokens (JWTs)

jwt.io

Creating a Public Private Key Pair for JWT Signature

There are several tools you can use to create a key pair. As long as you can export your public key to a base64 encoded X.509 certificate (backend clients) or JSON Web Key Set (frontend confidential clients) for registration on the Epic on FHIR website, or you can host it on a properly formatted JWK Set URL, the tool you use to create the key pair and file format used to store the keys is not important. If your app is hosted by Epic community members ("on-prem" hosting) instead of cloud hosted, you should have unique key pairs for each Epic community member you integrate with. In addition, you should always use unique key pairs for non-production and production systems.

Here are examples of two tools commonly used to generate key pairs:

Creating a Public Private Key Pair: OpenSSL

You can create a new private key named privatekey.pem using OpenSSL with the following command:

openssl genrsa -out /path_to_key/privatekey.pem 2048

Make sure the key length is at least 2048 bits.

For backend apps, you can export the public key to a base64 encoded X.509 certificate named publickey509.pem using this command:

openssl req -new -x509 -key /path_to_key/privatekey.pem -out /path_to_key/publickey509.pem -subj '/CN=myapp'

Where '/CN=myapp' is the subject name (for example the app name) the key pair is for. The subject name does not have a functional impact in this case but it is required for creating an X.509 certificate.

For frontend confidential clients, refer to JSON Web Key Sets.

Creating a Public Private Key Pair: Windows PowerShell

You can create a key pair using Windows PowerShell with this command, making sure to run PowerShell as an administrator:

New-SelfSignedCertificate -Subject "MyApp"

Note that the Epic on FHIR website is extracting the public key from the certificate and discarding the rest of the certificate information, so it's not 'trusting' the self-signed certificate per se.

The subject name does not have a functional impact in this case, but it is required for creating an X.509 certificate. Note that this is only applicable to backend apps. For frontend confidential clients, refer to JSON Web Key Sets.

You can export the public key using either PowerShell or Microsoft Management Console.

If you want to export the public key using PowerShell, take note of the certificate thumbprint and storage location printed when you executed the New-SelfSignedCertificate command.

```
PS C:\dir> New-SelfSignedCertificate -Subject "MyApp"

PSParentPath: Microsoft.PowerShell.Security\Certificate::LocalMachine\MY

Thumbprint                                Subject
----------                                -------
3C4A558635D67F631E5E4BFDF28AE59B2E4421BA  CN=MyApp
```

Then export the public key to a X.509 certificate using the following commands (using the thumbprint and certificate location from the above example):

PS C:\dir> $cert = Get-ChildItem -Path Cert:\LocalMachine\My\3C4A558635D67F631E5E4BFDF28AE59B2E4421BA

PS C:\dir> Export-Certificate -Cert $cert -FilePath newpubkeypowerShell.cer

Export the binary certificate to a base64 encoded certificate:

PS C:\dir> certutil.exe -encode newpubkeypowerShell.cer newpubkeybase64.cer

If you want to export the public key using Microsoft Management Console, follow these steps:

* Run (Windows + R) > mmc (Microsoft Management Console).
* Go to File > Add/Remove SnapIn.
* Choose Certificates > Computer Account > Local Computer. The exact location here depends on the certificate store that was used to create and store the keys when you ran the New-SelfSignedCertificate PowerShell command above. The store used is displayed in the printout after the command completes.
* Then back in the main program screen, go to Personal > Certificates and find the key pair you just created in step 1. The "Issue To" column will equal the value passed to -Subject during key creation (e.g. "MyApp" above).
* Right click on the certificate > All Tasks > Export.
* Choose option to not export the private key.

* Choose to export in base-64 encoded X.509 (.CER) format.

* Choose a file location.

Finding the Public Key Certificate Fingerprint (Also Called Thumbprint)

The public key certificate fingerprint (also known as thumbprint in Windows software) is displayed for JWT signing public key certificates that are uploaded to the Epic on FHIR website. There are a few ways you can find the fingerprint for a public key certificate:

* If you created the public key certificate using Windows PowerShell using the steps above, the thumbprint was displayed after completing the command. You can also print public keys and their thumbprints for a given certificate storage location using the PowerShell command.

Get-ChildItem

For example, run Get-ChildItem -Path Cert:\LocalMachine\My to find all certificate thumbprints in the local machine storage.

* You can follow the steps here to find the thumbprint of a certificate in Microsoft Management Console.

* You can run this OpenSSL command to print the public key certificate fingerprint that would be displayed on the Epic on FHIR website, replacing openssl_publickey.cert with the name of your public key certificate:

$ openssl x509 -noout -fingerprint -sha1 -inform pem -in openssl_publickey.cert

Note that the output from this command includes colons between bytes which are not shown on the Epic on FHIR website.

Creating a JWT

The JWT should have these headers:

Header | Description

alg | The JWT authentication algorithm. Epic supports the following RSA signing algorithms for all confidential clients: RS256, RS384, and RS512. For clients that register a JSON Web Key Set URL, Epic also supports the following Elliptic Curve algorithms: ES256, ES384. Epic does not support manual upload of Elliptic Curve public keys, and we recommend new apps use a JWK Set URL and Elliptic Curve keys.

typ | If provided, this should always be set to JWT.

kid | For apps using JSON Web Key Sets (including dynamically registered clients), we recommend setting this value to the kid of the target public key from your key set.

jku | For apps using JSON Web Key Set URLs, optionally set this value to the URL you registered on your application

The JWT header should be formatted as follows:

```
{
"alg": "RS384",
"typ": "JWT"
}
```

The JWT should have these claims in the payload:

Claim | Description | Remarks

iss | Issuer of the JWT. This is the app's client_id, as determined during registration on the Epic on FHIR website, or the client_id returned during a dynamic registration | This is the same as the value for the sub claim.

sub | Issuer of the JWT. This is the app's client_id, as determined during registration on the Epic on FHIR website, or the client_id returned during a dynamic registration | This is the same as the value for the iss claim.

aud | The FHIR authorization server's token endpoint URL. This is the same URL to which this authentication JWT will be posted. See below for an example POST. | It's possible that Epic community member systems will route web service traffic through a proxy server, in which case the URL the JWT is posted to is not known to the authorization server, and the JWT will be rejected. For such cases, Epic community member administrators can add additional audience URLs to the allowlist, in addition to the FHIR server token URL if needed.

jti | A unique identifier for the JWT. | The jti must be no longer than 151 characters and cannot be reused during the JWT's validity period, i.e., before the exp time is reached. In the November 2024 version of Epic and later, this value must be a string.

exp | Expiration time integer for this authentication JWT, expressed in seconds since the "Epoch" (1970-01-01T00:00:00Z UTC). | The exp value must be in the future and can be no more than 5 minutes in the future at the time the access token request is received.

nbf | Time integer before which the JWT must not be accepted for processing, expressed in seconds since the "Epoch" (1970-01-01T00:00:00Z UTC). | The nbf value cannot be in the future, cannot be more recent than the exp value, and the exp - nbf difference cannot be greater than 5 minutes.

iat | Time integer for when the JWT was created, expressed in seconds since the "Epoch" (1970-01-01T00:00:00Z UTC). | The iat value cannot be in the future, and the exp - iat difference cannot be greater than 5 minutes.

Note: No JWT should contain more than 100 claims or be larger than 8 KB.

Here's an example JWT payload:

```
{
"iss": "d45049c3-3441-40ef-ab4d-b9cd86a17225",
"sub": "d45049c3-3441-40ef-ab4d-b9cd86a17225",
"aud": "https://fhir.epic.com/interconnect-fhir-oauth/oauth2/token",
"jti": "f9eaafba-2e49-11ea-8880-5ce0c5aee679",
"exp": 1583524402,
"nbf": 1583524102,
"iat": 1583524102
}
```

The header and payload are then base64 URL encoded, combined with a period separating them, and cryptographically signed using the private key to generate a signature. Conceptually:

```
signature = RSA-SHA384(base64urlEncoding(header) + '.' + base64urlEncoding(payload), privatekey)
```

The full JWT is constructed by combining the header, body and signature as follows:

```
base64urlEncoding(header) + '.' + base64urlEncoding(payload) + '.' + base64urlEncoding(signature)
```

A fully constructed JWT looks something like this:

```
eyJhbGciOiJSUzM4NCJ9.eyJpYXQiOjE3MDc0MjQzMzYsImlzcyI6IjAxZjBlMzllLTIyY2MtNGFiNS05NTE0LWMyN2Y5Mzg0NzFjOCIsInN1YiI6IjAxZjB
    lMzllLTIyY2MtNGFiNS05NTE0LWMyN2Y5Mzg0NzFjOCIsImF1ZCI6Imh0dHBzOi8vdmVuZG9yc2VydmljZXMuZXBpYy5jb20vaW50ZXJjb25uZWN0LWF
    tY3VycHJkLW9hdXRoL29hdXRoMi90b2tlbiIsImV4cCI6MTcwNzQyNDYzNiwianRpIjoiMGIyZjZmMTQtMThkZS01NTU1LWJkMjUtZTNkMzg0ZmEwZTV
    kIn0.hE8DAq72XjJWNci5JFEuN5TrLh9W1UZP4H904GTIhggZNnNxtJLvUPKPrpf9PYgMZmgHb6peVR5MMUnQRtFHLZsH_C2v9ridfIA8BgWBL5eNiXc
    riUM9rIEvHwImgbKyPQUY_a0D7BCXiLqdjMAdEZe_afxCQBW0I7EsGqbpA0agayhLIDrlhcfCtdBrGyBlwW2zlUjTUo8Rj2o59tL90w6mVykeZIsJfcL
    QF4jeSfNOko-908mha_Gga6szWCqxlCr5ed9i6mjRSUQByG-GAFnCIhPULeedCHEgb9WF2NC0cImjK92jGSM6i43Y19uHvCFYMEkBMDaDK7WQigm3yg
```

JSON Web Key Sets

Applications using JSON Web Token (JWT) authentication can provide their public keys to Epic as a JSON Web Key (JWK) Set. Epic accepts keys of type RSA or EC. EC is only support for JWKS returned from a JWKS URL and is not supported for static JWKS uploaded directly. When using a JWKS URL we recommend including the kid values in your keys and in the headers of your assertions. Note that when uploading a static JWKS directly, only the "kty", "e" and "n" properties will be saved from the uploaded keys. Other fields typically present in a JWKS like "kid" or "alg" can exist in the uploaded JWKS without an error but they will be discarded upon save.

In the example key set below, the "RSA" key can be used with the RS256, RS384 or RS512 signing algorithms. The "EC" key can only be used with the "ES384" algorithm because it specifies a curve of P-384. The algorithm is determined when you first generate the key, like in the example above.

```
        {
            "keys": [
                { "kty":"RSA", "e": "AQAB", "kid": "d1fc1715-3299-43ec-b5de-f943803314c2", "n": "uPkpNCkqbbismKNwKguhL0P
    4Q40sbyUkUFcmDAACqBntWerfjv9VzC3cAQjwh3NpJyRKf7JvwxrbELvPRMRsXefuEpafHfNAwj3acTE8xDRSXcwzQwd7YIHmyXzwHDfmOSYW7baAJt-
    g_FiqCV0809M9ePkTwNvjpb6tlJu6AvrNHq8rVn1cwvZLIG6KLCTY-EHxNzsBblJYrZ5YgR9sfBDo7R-
    YjE8c761PSrBmUM4CAQHtQu_w2qa7QVaowFwcOkeqlSxZcqqj8evsmRfqJWoCgAAYeRIsgKClZaY5KC1sYHIlLs2cp2QXgi7rb5yLUVBwpSWM4AWJ_J5
    ziGZBSQYB4sWWn8bjc5-k1JpUnf88-UVZv9vrrkMJjNam32Z6FNm4g49gCVu_TH5M83_pkrsNWwCu1JquY9Z-
    eVNCsU_AWPgHeVZyXT6giHXZv_ogMWSh-3opMt9dzPwYseG9gTPXqDeKRNWFEm46X1zpcjh-sD-8WcAlgaEES6ys_O8Z" },
                { "kty":"EC", "kid": "iTqXXI0zbAnJCKDaobfhkM1f-6rMSpTfyZMRp_2tKI8", "crv": "P-384", "x":
    "C1uWSXj2czCDwMTLWV5BFmwxdM6PX9p-Pk9Yf9rIf374m5XP1U8q79dBhLSIuaoj", "y":
    "svOT39UUcPJROSD1FqYLued0rXiooIii1D3jaW6pmGVJFhodzC31cy5sfOYotrzF" }
            ]
        }
```

Key Identifier Requirements

Epic recommends your application provide the kid field in JWK Sets and in your JWT Headers when using a JWKS URL, since Epic can use the identifier to find your intended key. Epic does not require your key identifiers to be thumbprints, and will accept any value that is unique within your key set.

Hosting a JWK Set URL

Starting in the February 2024 version of Epic, customers can require back-end apps using JWT authentication to use a JSON Web Key Set URL (JKU) to provide public keys to Epic for JWT verification. All customers will require either this or .pem files when they upgrade to the May 2026 version of Epic. JWK Set URLs streamline implementation and make key rotation feasible by providing a centralized and trusted place where Epic community members can retrieve your public keys.

Implementation Consideration: Starting in the May 2026 version of Epic, customers must manually configure your specified the JKU for back-end OAuth 2.0 clients in their system. This is referred to as the local JKU. In August 2026, they will have the option to alternatively upload a static key via .pem file.

App-Level vs Licensing Specific JWK Set URLs

You can provide an app-level JWK Set URL when building your app for use in the sandbox environments. For security reasons, we require you at least have different URLs for non-production and production. We also recommend that you use a different set of JWK Set URLs for each Epic community member, and that key sets are not shared between different JWK Set URLs. However, this is not required, and reusing JWK URLs across customers may be appropriate for cloud-based apps that can re-use a single private key (without making a copy).

However, if your application is hosted by Epic community members ("on-prem", applications), then you must provide different JWK Set URLs for every licensed community member. There are 2 reasons on-prem applications should use distinct URLs:

* These applications require physically separate servers, and cannot share a private key across installations without copying and transporting the key (a security risk)

* Attempting to aggregate multiple public keys behind one URL is a security risk if those keys are in physically different places. If any one of your server environments is compromised, it can be used to impersonate any other instance of your application

Non-Production and Production Keys

Vendor Services and Epic on FHIR will not allow non-production JWK Set URLs to be registered as production JWK Set URLs for new apps and new app versions.

Your application should use separate key pairs for non-production and production environments. If both JWK Set URLs host the same public key, the corresponding private key can authenticate your application to either environment. This means a compromise of your non-production key material, where access controls are typically less rigorous, could be used to obtain production access tokens with your app's authorized scopes.

Private keys should not be stored in plaintext configuration files, environment variables, or source code repositories in any environment. Use your hosting provider's key management service (such as AWS KMS, Azure Key Vault, GCP Cloud KMS, etc.) or a hardware security module where available.

Registering a JWK Set URL

When building or licensing an application that will authenticate against Epic, you are prompted to optionally upload non-production and production JWK Set URLs to your app. When registering these URLs, be sure of the following:

* Are secured with TLS

* Are publicly accessible

* Do not require authentication

* Will not change over time

* Are responsive

Epic will verify points 1-3 whenever you provide a JWK Set URL

There are separate non-production and production URLs because we recommend you use separate key sets for each.

Note that in addition to registering different URLs, you should ensure the public keys hosted at each URL are themselves different. Distinct URLs that serve the same underlying key set still allow the same private key to authenticate to both production and non-production environments, which leaves production vulnerable if the non-production private key is compromised.

Key Rotation

Before rotating your keys, remove any static public keys from your existing downloads

One benefit of JWK Set URLs is that your application can rotate its key as needed, and Epic can dynamically fetch the updated keys. It is a best practice to periodically rotate your private keys, and if you choose to do so, we recommend the following strategy:

* Create your new public-private key pair ahead of time

* Add your new public key to your JWK Set in addition to the existing one

* Start using your new private key for signing JWTs

* Once you are certain your system is no longer using the old private key, wait 5 minutes (the maximum JWT expiration time) remove the old public key from your JWK Set

In between steps 3 and 4, Epic will notice you are using a new private key and will automatically fetch the new key set. Every time you present a JWT to Epic we will check your JWK Set URL at most once, and only if the cached keys are expired or did not verify your JWT

Epic's default and maximum cache time is 29 hours, but your endpoint can override the default by using the Cache-Control HTTP response header. This is most useful during development and testing when you may want to update your non-production public keys ad-hoc, or to test the availability of your server from Epic. You could for instance respond with "Cache-Control": "no-store" to stop Epic from storing your keys at all. This is not recommended for production usage since you lose the performance benefits of caching.

Using separate key pairs per environment allows you to rotate non-production and production keys on independent schedules whereas shared key pairs make independent rotation not possible.

Providing Multiple Public Keys

How

Apps can provide multiple public keys to an Epic community member by providing a JWK Set URL that contains multiple keys.

Do not host more than 100 public keys on a single JKU. Epic does not support this.

Why Not?

In general, the only reason to provide multiple public keys on one JWK Set URL is if your system spans multiple separate servers or cloud environments, and so you cannot share a private key between them. If you can re-use a key without making copies of it, then you should re-use that key and focus on protecting it with the best policy available (For example, using your hosting provider's key store or using a Trusted Platform Module).

Use Cases

Example use cases:

* Having multiple isolated testing environments, where you may provide multiple non-production public keys (one for each environment)

* On-premises applications with multiple servers hosted by a given community member. In this case you would provide multiple production public keys (one for each server)

MyChart Central Community Authorization for TEFCA IAS Apps

To improve the IAS patient onboarding and app registration process and further simplify health information sharing with applications participating in TEFCA, MyChart Central offers patients the ability to share information from all their linked organizations at once.

Refer to TEFCA for the Individual Access Services for more information.

Appendix

Epic-Issued OAuth 2.0 Tokens

Starting in the May 2020 version of Epic, Epic community members can enable a feature that makes all OAuth 2.0 tokens and codes into JSON Web Tokens (JWTs) instead of opaque tokens. This feature increases the length of these tokens significantly and is enabled in the sandbox. Developers should ensure that app URL handling does not truncate OAuth 2.0 tokens and codes.

The format of these tokens is helpful for troubleshooting, but Epic strongly recommends against relying on these tokens being JWTs. This same feature can be turned off by Epic community members, and Epic may need to change the format of these tokens in the future.

In general, if the "aud" parameter of a JWT is not your app's client ID, you should not be decoding it programmatically. For the 5 types of JWTs mentioned in this tutorial, here's a breakdown of their audience (aud) and issuers (iss):

JWT Scenario | Audience (aud) | Issuer (iss)
Launch Token | Authorization server | Authorization server
Authorization Code | Authorization server | Authorization server
Access Token | Authorization server | Authorization server
Openid Connect ID Token | The app's client_id | Authorization server
Client Assertion | Authorization server | The app's client_id

Trusting an OAuth 2.0 Token

A common question that developers ask is how an app can "trust" an OAuth 2.0 token without cryptographically verifying it. Our answer is that your app should never trust a token by itself and should rely on the authorization server to verify a given token. When redeeming an authorization code for an access token, your app can trust that if the request succeeded that the authorization code was valid, and you can then create a session in your application. The only exception is an openid connect id_token, which is designed to be verified by the client application.

Epic-Issued OAuth 2.0 Tokens

Starting in the May 2020 Epic version, Epic community members can enable a feature that makes all OAuth 2.0 tokens and codes into JSON Web Tokens (JWTs) instead of opaque tokens. This feature increases the length of these tokens significantly and is enabled in the vendorservices.epic.com sandbox. Developers should ensure that app URL handling does not truncate OAuth 2.0 tokens and codes.

The format of these tokens is helpful for troubleshooting, but Epic strongly recommends against relying on these tokens being JWTs. This same feature can be turned off by Epic community members, and Epic may need to change the format of these tokens in the future.

In general, if the "aud" parameter of a JWT is not your app's client ID, you should not be decoding it programmatically. For the 5 types of JWTs mentioned in this tutorial, here's a breakdown of their audience (aud) and issuers (iss):

JWT Scenario | Audience (aud) | Issuer (iss)
Launch Token | Authorization server | Authorization server
Authorization Code | Authorization server | Authorization server
Access Token | Authorization server | Authorization server
Openid Connect ID Token | The app's client_id | Authorization server
Client Assertion | Authorization server | The app's client_id

Trusting an OAuth 2.0 Token

A common question that developers ask is how an app can "trust" an OAuth 2.0 token without cryptographically verifying it. Our answer is that your app should never trust a token by itself and should rely on the authorization server to verify a given token. When redeeming an authorization code for an access token, your app can trust that if the request succeeded that the authorization code was valid, and you can then create a session in your application. The only exception is an openid connect id_token, which is designed to be verified by the client application.

How you handle the authorization code is the most important step in an OAuth 2.0 flow. See our best practice for using this step to secure your application.

Refresh Tokens and Persistent Access Periods

The time during which a client can use refresh tokens to get new access is known as the persistent access period. In patient-facing contexts, it can be set by the patient through MyChart. In non-patient-facing contexts, the duration will be set by the refresh token itself. Specifically, the start of the period is set to the time the token is provisioned and the end of the period is set to the token's expiration date-time.

In many situations, only one access token will want to be requested with the original refresh token and the persistent access period just defines when the that refresh token can do so. However, a client will often need access over a longer period of time. In these situations, a single refresh token won't be an efficient solution as a new authorization request will have to be made after the token is used or the period ends. There are two workflows for getting an extended persistent access period by utilizing multiple refresh tokens: rolling refresh tokens and indefinite access.

Rolling Refresh Persistent Access Periods

In this workflow, multiple refresh tokens are used to have the persistent access period last as long as a single refresh token could. A first refresh token is provisioned and the persistent access period is set as normal (i.e., start of period = time of provisioning and end of period = token expiration date-time). This first refresh token can then be used in another access token request that specifies to return a refresh token. The refresh token granted from this access token request will be generated normally except for the fact that its expiration date-time will be set to the same as the first refresh token. All subsequent refresh tokens in the workflow will also have this same expiration date-time.

This means that the persistent access period of a rolling refresh workflow is defined by the first refresh token and all subsequent refresh tokens 'fall within' that period. The below diagram gives a visual representation of the persistent access period during a rolling refresh token workflow.

For patient-facing apps, the length of the persistent access period for a rolling refresh will be set by each patient during the OAuth 2.0 flow. They can choose from a variety of options (e.g., 1 hours, 1 day, 1 week, …) configured by the customer.

To enable rolling refresh access for non-patient-facing apps, check that your customers have properly configured their external client record.

Indefinite Persistent Access Periods

In this workflow, the persistent access period lasts indefinitely. It does so by using consecutive refresh tokens like in rolling refresh workflows. The notable difference is that, here, new refresh tokens have an expiration date-time greater than the one before it. In the rolling refresh workflow, this is always set to that of the first token. Since the persistent access period end date-time is set by the token's expiration, continually setting new refresh tokens to expire in the future will make the persistent access period last until no more refresh tokens are requested.

To enable indefinite access for patient-facing apps, instruct your customers to follow these configuration steps. Note that this only gives patients the option of indefinite access. Each patient still chooses their max access period.

* Navigate to Login and Access Configuration in MyChart.

* Navigate to OAuth Access Duration Configuration.

* Specify 'Indefinite' in the Global Max OAuth Access Duration field.

To enable indefinite access for non-patient-facing apps, check that your customers have properly configured their external client records.

Limit Use of Localhost URIs

Launched applications must register a list of URLs that Epic will use as an allowlist during OAuth 2.0. Epic will only deliver authorization codes to a URL that is listed on the corresponding app.

As a security best practice, apps should not include localhost/loopback URIs in this list. An exception is for Native applications which are designed to use localhost for the redirect, such as certain libraries available from the AppAuth group. An alternative available for native apps are platform specific link associations, discussed in our Considerations for Native Apps below.

If your developers wish to use localhost URIs for testing in our sandboxes, we recommend creating a second test application that will never be activated in customer environments and listing your localhost URLs there.

Considerations for Native Apps

Additional security considerations must be taken into account when launching a native app to sufficiently protect the authorization code from malicious apps running on the same device. There are a few options for how to implement this additional layer of security:

* Platform-specific link association: Android, iOS, Windows, and other platforms have introduced a method by which a native app can claim an HTTPS redirect URL, demonstrate ownership of that URL, and instruct the OS to invoke the app when that specific HTTPS URL is navigated to. Apple's iOS calls this feature "Universal Links"; Android, "App Links"; Windows, "App URI Handlers". This association of an HTTPS URL to native app is platform-specific and only available on some platforms.

* Proof Key for Code Exchange (PKCE): This is a standardized, cross-platform technique for public clients to mitigate the threat of authorization code interception. However, it requires additional support from the authorization server and app. PKCE is described in IETF RFC 7636 and supported starting in the August 2019 version of Epic. Note that the Epic implementation of this standard uses the S256 code_challenge_method. The "plain" method is not supported.

Starting in the August 2019 version of Epic, the Epic implementation of OAuth 2.0 supports PKCE and Epic recommends using PKCE for native mobile app integrations. For earlier versions or in cases where PKCE cannot be used, Epic recommends the use of Universal Links/App Links in lieu of custom redirect URL protocol schemes.

Subspace Security Considerations

This section applies to certain applications with a user type of "Backend Systems". If your application does not fit this criterion, you can skip this section. You can also skip this section if your Backend application does not use Subspace APIs; in which case you should remove the "Subspace" functionality and un-check the "Can Register Dynamic Clients" checkbox.

Beginning March 2, 2023, Epic no longer supports registering new client IDs with the following combination of settings:

User type of Backend Systems

Uses OAuth 2.0

Functionality: Subspace

Registers Dynamic Clients

Functionality: Incoming Web Services

The combination of settings 1-4 has historically been used for integration with Epic's Subspace Communication Framework. In this approach your application registers workstation-specific private keys to authorize Subspace API calls.

As a security best practice, for integrations using the above settings, instead it is recommended to authenticate the Incoming Web Services (Setting 5) using the OAuth 2.0 token obtained from a SMART on FHIR app launch or configuring your integration to use a multi-context application.

When a workstation-specific private key is allowed to authorize web service calls, a user may be able to bypass user-specific authentication and authorization checks within the web services themselves. This risk is mitigated in virtualized environments where the workstation (and the private key) is controlled by administrators. These security implications do not apply to Subspace APIs as Subspace APIs provide access to the Epic database only after the user has authenticated against Epic.

As a security best practice, rather than using the combination of settings above, an application that needs all these features (particularly web services) can do one of the following:

SMART App Launch

We recommend following the FHIRcast specification's guidance, which says to use the SMART App Launch to secure communication. One reason to use the SMART App Launch for Subspace integrations is that the launch provides user-specific security. The token provided by the launch is designed for end-users to use directly and provides the flexibility for you to call web services from any portion of your infrastructure.

Per the FHIRcast specification, the hub needs to launch your application before you can use Subspace APIs. Practically, this means your end users need to launch your application from Epic to begin their workflow. Epic's "User Toolbar" launch approach provides the ability for the user to launch your application before opening any patient and provides your application with authorization to view multiple patients' charts without re-launching. This requires coordination with the Epic Community Member to show the launch button to end users and requires instructing end users to start their workflow in this way.

By default, launched Subspace applications will receive 1 hour of API authorization per launch. If your application requires longer workflows, we offer the following options:

Epic offers a Dynamic Client Registration (DCR) workflow where you can register a temporary and in-memory private key to extend your application's access. Epic will revoke the key's access after a period of inactivity, so you should request a new access token before the most recent one expires. After the end user closes your application, you should delete the private key and Epic will revoke its access due to inactivity.

Note this workflow differs from the "Backend System" DCR workflow above, in that tokens issued to the client are tied to the user who launched the application. In this way, Epic can conduct user-specific security checks when your application calls web services.

(Not recommended) Epic Community Members can extend the default duration of your access tokens to any value, for example 4 hours.

Multi-Context Application

In this approach, your application handles the burden of authenticating the end-user and mediating their access to the Epic database. This can only be done securely by using server-side controls; safeguards put in place in a client-application (example: a native desktop app) are not sufficient, as they could potentially be bypassed by an end user.

An existing "Backend System" Subspace integration can achieve this with the following:

Created a new client ID with the following settings:

User type of Backend Systems

Uses OAuth 2.0

Feature: Incoming API

Select your required Web Services

Conduct Backend OAuth 2.0 as you did with your existing app, but skip the DCR step

Follow our security best practices for Backend applications, which solve the user-authentication issues above

SMART Scopes

SMART scopes are returned in the scope parameter of the OAuth 2.0 token endpoint response and determine which resources an application has permission to access and what actions the application is permitted to take. The scopes provided are based on the scopes that your app requested in your authorize request as well as the incoming APIs you have selected

on your app page.

As of the August 2024 version of Epic, both SMART v1 and SMART v2 scope formatting are supported. Prior to the August 2024 version, only SMART v1 scopes are supported.

The actions defined by SMART v1 scopes are .read and .write. SMART v2 scopes use CRUDS syntax and actions include: .c – create, .r – read, .u – update, .d – delete, and .s – search.

For example, the Observation.Create resource corresponds to either:

* SMART v1: {user-type}/Observation.write

* SMART v2: {user-type}/Observation.c

Which version should you select?

If your app uses SMART v1 scopes or if your app does not use SMART scopes at all, you should select SMART v1.

If your app uses SMART v2 scopes, you should select SMART v2.

Note: Prior to the August 2024 version of Epic, SMART v1 scope formatting will be returned regardless of the SMART Scope Version selected on your app page. If your app uses SMART scopes and your customer is on a version of Epic earlier than August 2024, you must support SMART v1 scopes.

Non-OAuth 2.0 Authentication

Only use these forms of authentication if OAuth 2.0 is not an option.

HTTP Basic Authentication

HTTP Basic Authentication requires that Epic community members provision a username and password that your application will provide to the web server to authenticate every request.

Epic supports HTTP Basic Authentication via a system-level user record created within Epic's database. You may hear this referred to as an EMP record. You will need to work with an organization's Epic user security team to have a username and password provided to you.

When calling Epic's web services with HTTP Basic Authentication the username must be provided as follows: emp$<username>. Replace <username> with the username provided by the organization during implementation of your application.

Base64 encode your application's credentials and pass them in the HTTP Authorization header. For example, if you've been given a system-level username/password combination that is username/Pa$$w0rd1, the Authorization header would have a value of ZW1wJHVzZXJuYW1lOlBhJCR3MHJkMQ== (the base64 encoded version of emp$username:Pa$$w0rd1):

```
GET http://localhost:8888/website HTTP/1.1
Host: localhost:8888
Proxy-Connection: keep-alive
Authorization: Basic ZW1wJHVzZXJuYW1lOlBhJCR3MHJkMQ==
```

Storing HTTP Basic Authentication Values

The username and password you use to authenticate your web service requests are extremely sensitive since they can be used to pull PHI out of an Epic organization's system. The credentials should always be encrypted and should not be stored directly within your client's code. You should make sure that access to decrypt the credentials should be limited to only the users that need access to it. For example, if a service account submits the web service requests, only that service account should be able to decrypt the credentials.

Client Certificates and SAML tokens

Epic also supports the use of Client Certificates and SAML tokens as an authentication mechanism for server-to-server web service requests. We do not prefer these methods because both require web server administrators to maintain a trusted certificate for authentication. As certificates expire or servers need to be moved around, this puts an additional burden on system administrators.

Additional Required Headers

When you use OAuth 2.0 authentication, Epic can automatically gather important information about the client making a web service request. When you use a non-OAuth 2.0 authentication mechanism, we require that this additional information be passed in an HTTP header.

Epic-Client-ID

This is the client ID you were given upon your app's creation on the Epic on FHIR site. This is always required when calling Epic's web services if your app doesn't use OAuth 2.0 authentication.

```
GET http://localhost:8888/website HTTP/1.1
Host: localhost:8888
Proxy-Connection: keep-alive
Authorization: Basic ZW1wJHVzZXJuYW1lOlBhJCR3MHJkMQ==
Epic-Client-ID: 0000-0000-0000-0000-0000
```

Epic-User-ID and Epic-User-IDType

This is required for auditing requests to FHIR resources. The Epic-User-ID corresponds to an EMP (Epic User) account that an organization creates for your application. This might be required depending on the web services you are calling. The

Epic-User-IDType is almost always EXTERNAL.

```
GET http://localhost:8888/website HTTP/1.1
Host: localhost:8888
Proxy-Connection: keep-alive
Authorization: Basic ZW1wJHVzZXJuYW1lOlBhJCR3MHJkMQ==
Epic-Client-ID: 0000-0000-0000-0000-0000
Epic-User-ID: username
Epic-User-IDType: EXTERNAL
```

1.23 OAuth 2.0 Tutorial

File: site/docs/01-guides/23-oauth2tutorial.html

Source: https://fhir.epic.com/Documentation?docId=oauth2tutorial

OAuth 2.0 Tutorial

Refer to the following table for Epic versions when this requirement will take effect.

Implementation Consideration: Starting in the May 2026 version of Epic, customers must manually configure the JWK Set URL (JKU) you specify for your backend OAuth 2.0 clients. This customer-specific configuration is referred to as the local JKU.

Update: Customers will be able to configure a local JWK using a static public key in .pem format that you provide, instead of using a JKU. If your app relies on static keys and your JWKs change, you will need to communicate the new keys directly to each customer to maintain functionality. For ease of key rotation and ongoing maintenance, Epic recommends supporting a JKU.

Epic Version | Epic Sandbox JKU Support | Epic Customer JKU Support

February 2025 and prior | Optional | Optional - Each customer can choose to enforce JKU support for backend OAuth apps. This enforcement is off by default.

August 2025 | Vendor Services and Epic on FHIR no longer allow new static key uploads for use in the sandbox.

February 2026 | Static keys no longer supported in the sandbox for backend OAuth. Vendor Services and Epic on FHIR websites no longer allow static key uploads for new apps or new customer download requests. Both sites still support rotating keys for existing live customers. | Vendor Services and Epic on FHIR websites no longer allow static key uploads for new apps or new customer download requests. Both sites still support rotating keys for existing live customers.

May 2026 | | Static keys are no longer supported in customer environments for backend OAuth.

August 2026 | | Customers may configure local JWKs if provided .pem files.

This tutorial complements and aims to replace the OAuth 2.0 Specification, which is mostly technical specs.

This tutorial focuses on the SMART App Launch framework, which is our recommended integration method for apps that meet all these criteria:

* Your application is UI-based, such as a web or native application

* Your end users have Epic credentials. Clinicians, staff, or administrative users use their Hyperspace credentials, while patients and their caretakers use their MyChart (Patient Portal) credentials

* You meet at least one of the following:

* You'd like to grant your users access to your application using their Epic credentials. In this case you're using OAuth 2.0 as a form of Single-Sign-On. This option may allow you to avoid having your own authentication system, where users would keep a separate login for your application.

* You'd like your application to launch from and optionally be embedded in Epic, and optionally receive contextual information at the time of launch such as the patient's ID.

If your application doesn't meet these criteria, but still wants to pull data from Epic, check out our Backend OAuth 2.0 Tutorial instead of this one. Otherwise, read on!

Your app may not fit cleanly into one workflow, so also consult our Choosing a User Context documentation to see if you need multiple components in your application.

Following this tutorial, you'll find information on:

* Choosing the right workflow for your use case

* Understanding how to secure your application using OAuth 2.0

* Building an application using a library

* Implementing a Standalone and/or EHR launch

* Reviewing common considerations for apps integrating with Epic

* Deploying your application at Epic Community Members

Choosing your Workflow

From criteria 3 above, there are 2 relevant workflows that the SMART App Launch is used for:

* Using OAuth 2.0 as a form of Single Sign-On (SSO)

* Launching from and optionally embedding in an Epic workflow

Depending on which workflows you'd like to use, consult the following table for next steps:

OAuth 2.0 for SSO | Launch from Epic | Next Steps
Yes | No | Secure your Application using a Standalone launch, meaning your app initiates an Epic login page.
Yes | Yes | Secure your Application using an EHR launch, meaning Epic initiates launching your app. This launch includes patient context and can optionally embed your app in Epic (for web apps, not native apps).
No | Yes | Review our launching without SSO appendix

Securing your Application

If you plan to use OAuth 2.0 for SSO, review this section with your security team. Using OAuth 2.0 incorrectly can introduce security vulnerabilities in your system.

If your application includes a database or protected business logic, you should already have authentication and session management systems in place. Your existing, protected app might function like this:

* A user submits a username and password to you.

* Your authentication system (on your web server) reviews the request.

* If the request is valid, your authentication system creates an authenticated user session, such as an HttpOnly Cookie or a token stored in your client-side.

* The user takes an action in your app, authenticated by their session cookie or token.

Your goal for integrating the SMART app launch into your app should be to replace your authentication system with the launch, where Epic authenticates the user. You should use the result of the launch to issue authenticated user sessions from your existing session system. This is illustrated below:

The labeled steps (a-d) are referenced in the session creation overview below.

Using a Library

Many OAuth 2.0 libraries are capable of or have documentation for integrating with your own session management system. Make sure you are using a server-side OAuth 2.0 library, and not a client-side one, so that you can securely issue sessions using the outcome of the launch. The library requirements below should guide you to a server-side library.

Session Creation from a Launch

Below is a high-level overview of how the SSO portion of the SMART app launch works (referencing labeled steps in the diagram above):

* (Optional) Epic initiates a launch to your app, including a launch token.

* You initiate an OAuth 2.0 authorization request.

* The user authenticates against Epic, and Epic redirects back to you with a one-time-use authorization code (step a). If your app is launched from Epic, your app will have a launch token to present in this request. In that case, the user doesn't re-authenticate.

* Your web server takes the code and sends it to Epic's token endpoint for review (step b).

* If the token request succeeds (), your system creates an authenticated user session and returns it to the user ().

step c

step d

* Epic can provide user identifiers to create or look up the user's account in your system.

* Epic can also provide contextual information like the patient or visit ID.

* The user takes an action in your app as usual, protected by their session. If the action requires data from Epic, your app can use the access token issued to it to retrieve the required data. We recommend storing this token on your server and making API calls from the server.

In step 4/b above, we recommend you redeem the authorization code directly from your web server, and use the result to immediately issue a user session. This contrasts with client-side libraries such as fhirclient, which redeem the authorization code from the user's browser. Use a server-side workflow instead, so you can trust the result to establish user sessions.

Building your Application

Choosing a Library

Using a SMART App Launch Library

A SMART app launch library needs to do all of the following:

* Run on your server, not the end-user's device.

* Support the EHR or standalone launch, depending on the launch type you chose above.

* Support private_key_jwt client authentication. We recommend this over client_secret_basic or client_secret_post.

* Support retrieving extra values from the token response. Use this option if you require a launch context beyond the standard patient and encounter identifiers.

Using an OAuth 2.0 Library

If you cannot find a SMART app launch library, here are OAuth 2.0 library requirements:

* Run on your server, not the end-user's device.

* Use an OAuth 2.0 Client Library or an OpenID Connect Relying Party library. Client refers to your role in the OAuth 2.0 flow, not whether you have a web server component.

* Support the authorization_code grant type.

* Support refresh_token grant type if you need >1 hour of API access.

* Support adding extra parameters to an authorize request ("aud" and "launch" parameters)

* Support private_key_jwt client authentication. We recommend this over client_secret_basic or client_secret_post.

* Support retrieving extra values from the token response so that you can retrieve any launch context including the standard patient and encounter identifiers.

Sample App Config

We've created a sample app to help you with initial development. The app has the following settings:

* User type: Clinicians, Staff, or Administrative Users

* Client ID: e3073934-68c1-4ca7-9b59-3d8b934187f1

* Redirect URIs:

* http://localhost:3000/callback

* https://localhost/callback

* http://localhost:3000/epic-sandbox/callback

* https://localhost/epic-sandbox/callback

* Incoming APIs:

* Practitioner.Read (R4)

* PractitionerRole.Search (R4)

* Patient.Read (R4)

* Encounter.Read (R4)

* Location.Read (R4)

* Location.Search (R4)

* Private Key:

* Download a sample private key here. When registering your own app, as a security best practice, you should create your own key pair. The sample key is provided exclusively for use with testing in the sandbox and should never be used with a mutual customer for testing or production purposes.

* The attached key is an RSA key (ex: RS384 alg). If your library supports Elliptic Curve algorithms (ex: ES384 alg) we recommend using that algorithm instead (requires hosting a JWK Set URL).

Define your Architecture

After you've identified a library, answer the following questions to define your architecture:

* How will my system/library differentiate between different Epic community members? See the Supporting Multiple FHIR Servers topic below.

* How will my system/library discover the needed OAuth 2.0 endpoints? See the Endpoint Discovery topic below.

* What client authentication does my library support?

* If it supports private_key_jwt (recommended), follow these steps to create private keys and public keys. On-premises apps will repeat this per installation.

* If it supports client_secret_basic (or _post), secret generation is covered in the Upload Your Credentials section below, and your implementation team will create separate secrets for every deployment of your app.

* Do not use "none" unless you use a launch without SSO.

* How will my app be deployed?

* If your app is Cloud-Based and if you support private_key_jwt authentication, consider sharing private keys across Epic Community Members

* If your app is installed On-Premises, or if you use client secret authentication, you must use separate authentication credentials per install.

For example, the deployment strategies below use the following architectures:

Cloud-Based Architecture

* Different URL paths (on a single domain) per Epic Community Member

* private_key_jwt authentication

* A single private key shared across Epic Community Members, hosted on a single JWKS URL

On-Premises Architecture

* Different origins (in this case, subdomains) per Epic Community Member.

* private_key_jwt authentication

* Unique private keys per Epic Community Members, hosted on separate JWKS URLs (one on each origin)

Build your Application

Now that you've identified the URLs and client authentication for your system, build an application with those items so that Epic can verify requests from your application.

Build an application with the following settings:

* Primary User Type. Choose one, depending on your use case.

* Patients - For patients and their caretakers authenticating with MyChart

* Clinicians, Staff, or Administrative Users - For users with Epic Hyperspace accounts

* Check "Incoming API"

* Check "Use OAuth 2.0"

* In Epic on FHIR, this box is always selected.

* Endpoint URI

* Enter the URIs for your development environments' OAuth 2.0 redirect_uris. Your library should have documentation on this value. Commonly, it ends in /callback.

* Only add URIs where your <hospital-system> is set to Epic's sandbox environment (example: "epic-sandbox"). When activating your app at an Epic community member, you'll be able to provide the community member's <hospital-system> value as a license-specific URI. This is covered in the Deploying your Application section.

* For EHR launched apps, do not include your launch URL unless it is the same value as your redirect_uri. Launch URLs are not checked against your registered app.

* Incoming APIs

* For all apps, we recommend reviewing our default "FHIR resource IDs" below, and requesting the associated "FHIR API" for the ones you need (example "encounter" and "Encounter.Read (R4)").

* For patient-facing apps, we recommend the Patient.Read (R4) and RelatedPerson.Read (R4) APIs for use with Openid Connect.

* For clinician, staff and administrative user-facing apps, we recommend the Practitioner.Read (R4) API for use with Openid Connect.

* App FHIR Version. We recommend setting this to R4 for new apps. See our documentation on this topic.

* Check "Is this app a confidential client?"

* We recommend all apps register as a confidential client, and then upload credentials in the next step.

* If your application launches without SSO, and specifically does not include a web server component at all, then you can keep this unchecked and skip the remaining setup. You are ready to test your app.

* Upload your credentials.

* For private_key_jwt apps, upload your public keys as an JWKS URL (recommended) or an X509 certificate.

* For On-Premises apps, provide the keys for your development environment at this time. See the example On-Premises deployment below for considerations on private key management in production.

* For client_secret apps, generate a "Sandbox Client Secret" and copy the secret into your development environment. When deploying your app, a similar screen appears where you will create separate client secrets per install. You should not share secrets between Epic community member installs.

* "Requires Persistent Access"

* Check this if your app needs more than one hour of API access. This is uncommon, especially if you only use APIs at the beginning of your workflow.

* If your app does need this access, check that your library supports the "refresh_token" grant type.

* This is only available if your app is a confidential client.

Now that you've configured your app, save it, and wait 30 minutes for your changes to sync. When testing your app against the sandbox, use the non-production client ID issued to you.

Implementing a Standalone Launch

Use the standalone launch workflow when you'd like to use OAuth 2.0 for SSO, but when you do not need to be launched from Epic. Your app initiates a login on its own, and Epic presents a login screen where the user can enter their credentials.

To start, make sure you've reviewed the securing and building your application sections above. Most OAuth 2.0 libraries can handle the standalone launch out of the box. Here's a diagram of what a standalone launch flow looks like.

Test a Standalone Launch

If you are using the sample application setup above, your application should create an authorize request URL that looks like this, and should navigate the user's browser to this URL (newlines included for clarity, values are URL encoded):

```
https://fhir.epic.com/interconnect-fhir-oauth/oauth2/authorize?
    client_id=e3073934-68c1-4ca7-9b59-3d8b934187f1&
    scope=openid%20fhirUser&
    response_type=code&
    redirect_uri=http%3A%2F%2Flocalhost%3A3000%2Fepic-sandbox%2Fcallback&
    state=example-state-value-should-fail&
    code_challenge_method=S256&
    code_challenge=PHHQCoInTVWPbe78fWbQSftF5f6nn7hgskfLfEm4L6s&
    aud=https%3A%2F%2Ffhir.epic.com%2Finterconnect-fhir-oauth%2Fapi%2FFHIR%2FR4
```

Note a few things about this request:

* aud

* The aud parameter is required for most workflows, and we recommend all apps include this parameter. Set this parameter to the FHIR base URL of the server you intend to call (the "iss" value). Starting in the August 2021 version of Epic, health care organizations can optionally configure their system to require the aud parameter for Standalone and EHR launch workflows if a launch context is included in the scope parameter. Starting in the May 2023 version of Epic, this parameter is required. The value to use is the base URL of the resource server the application intends to access, which is typically the FHIR server.

* Epic maintains a list of our Community Member's FHIR endpoints on epic.com

* This parameter is not required for standalone launches unless you request the "launch/patient" or "launch/encounter" scope. This means that traditional OAuth 2.0 and OpenID Connect clients do not need to include this parameter.

* code_challenge and _method

* We recommend that all apps use PKCE. If you have a native app, see the Considerations for Native Apps in the appendix. Most OAuth 2.0 libraries should support PKCE for "authorization_code" grants

* state

* The state parameter is set to a fake value in this example. Your library should auto-generate this for you, and the redirect back to your app should fail if the state does not match the generated one. This is used to mitigate login CSRF attacks. See RFC 6819 and the OWASP CSRF Cheat Sheet.

* scope

* The scope parameter is set to "openid fhirUser" (space delimited list). Epic issues additional scopes based on the incoming APIs you configured on your application.

* Remember to add scope "launch" if you later add EHR Launch support. Do not add it for standalone launches.

A login screen appears where you need to enter your sandbox user's credentials. Ask your organization's administrator to share your sandbox username and password with you.

After you're able to successfully redeem the code for an access token, do one of the following:

* If your target workflow is the EHR launch, review the next section.

* Otherwise, if your target workflow is the standalone launch, continue to the Identifying the User section below

Implementing an EHR Launch

The EHR launch flow allows Epic to launch your application and inform it of what user, patient and encounter was open at the time of launch. The user authenticates at the beginning of their Epic workflow and can launch your app without re-authenticating. Here's a diagram of what an EHR launch flow looks like.

To start, make sure you've reviewed the securing and building your application sections above. If you chose an OAuth 2.0 library that does not explicitly support the EHR launch, you need to customize it as described below. If you chose an EHR Launch library, skip to the Specific Redirect URIs section.

Customize an OAuth 2.0 Library

If you cannot find an EHR launch library for your framework, we recommend customizing an OAuth 2.0 library as a starting point. In this case, you should implement the standalone launch first, and then customize it for an EHR launch. Then, we recommend turning off your standalone launch if you do not plan to use it in production. Do the following customizations:

Launch Endpoint

Your OAuth 2.0 library should create a redirect_uri for you, but you'll need to create a launch URL on your own. This endpoint is used for step 1 in the EHR launch sequence, where the user's browser navigates to an endpoint like this:

```
https://app.com/launch?
  iss=https%3A%2F%2Fehr%2Ffhir&
  launch=xyz123
```

This request has three main pieces:

* The URL of https://app.com/launch

* This is defined by you, and you provide this to an Epic community member to configure in their system.

* Note this URL does not include a "<hospital-system>" value. We recommend you have a shared launch URL across hospital systems, and use the "iss" parameter defined below to differentiate systems.

* The iss parameter

* This is also known as the FHIR base URL and uniquely identifies the Epic community member that is initiating the request.

* Your application should create an allowlist of iss values (AKA resource servers) to protect against phishing attacks. Rather than accepting any iss value in your application, only accept those you know belong to the organizations you integrate with.

* Epic maintains a list of our Community Member's FHIR endpoints on open.epic.com

* The launch parameter

* You should treat this as an opaque token, and just include it in your later request (covered below).

Additional Authorize Request Parameters

After you've validated the iss and launch parameters, your server will initiate an authorize request. You'll need to add the following parameters as defined in the SMART app launch "App asks for Authorization" step:

* aud - The iss value from the launch request.

* scope - We recommend using "launch openid fhirUser" (space-delimited list), but "launch" at least is required. Epic issues additional scopes based on the "Incoming APIs" you configured on your application.

* launch - The launch parameter provided by the EHR at the launch endpoint above.

Specific Redirect URIs

We recommend your launch URL be the same for all hospital systems, and that you have multiple redirect URIs each specific to one system. Your launch should use the "iss" to differentiate between health systems and redirect to their specific URI.

For example, your app may have the following endpoints:

* https://app.com/launch

* https://app.com/health-system-a/callback

* https://app.com/health-system-b/callback

A launch from health-system-a would look like this:

* The health system will launch to:

* https://app.com/launch?iss=https%3A%2F%2Fsystem-a.fhir.com%2Fv1&launch=abc123

* And when your server constructs an authorization request, it would use this redirect_uri:

* https://app.com/health-system-a/callback

In this case, your server mapped on the "iss" value provided to choose the right <health-sytem> value.

Test an EHR Launch

Once you've built an application, you can test your integration with the Hyperdrive Client Testing Harness, which supports SMART on FHIR testing, and use that application's client ID in your test requests.

In either of these tools, you provide your launch URL and the tool will dynamically append the "iss" and "launch" parameters when you run a test.

A launch of the sample app above would look like this (newlines included for clarity, values are URL encoded):

```
http://localhost:3000/launch?
  launch=[omitted]&
  iss=https%3A%2F%2Ffhir.epic.com%2Finterconnect-fhir-oauth%2Fapi%2FFHIR%2FR4
```

In response, your app would navigate the user's browser to a new URL (newlines included for clarity, values are URL encoded). Note the redirect_uri includes "epic-sandbox", which is the <health-system> value this app chose for the sandbox's "iss". In addition, the request should include the "launch" scope, and also include the launch token presented above:

```
https://fhir.epic.com/interconnect-fhir-oauth/oauth2/authorize?
  client_id=e3073934-68c1-4ca7-9b59-3d8b934187f1&
  scope=launch%20openid%20fhirUser&
  response_type=code&
  redirect_uri=http%3A%2F%2Flocalhost%3A3000%2Fepic-sandbox%2Fcallback&
  state=example-state-value-should-fail&
  code_challenge_method=S256&
  code_challenge=PHHQCoInTVWPbe78fWbQSftF5f6nn7hgskfLfEm4L6s&
  aud=https%3A%2F%2Ffhir.epic.com%2Finterconnect-fhir-oauth%2Fapi%2FFHIR%2FR4
  launch=[omitted]
```

Note that Epic also supports POST-Based authorization for EHR Launches (but not standalone launches), allowing you to submit a traditional HTML form (second example here) for the authorize request. This moves the request parameters from the query string into the body of the form. If you use this method while embedded in Epic, you will need to allow the page to be embedded in an iframe.

Handling the State Parameter

We recommend following the SMART App Launch Framework's recommendation that the "state" parameter be an unpredictable value … with at least 122 bits of entropy (e.g., a properly configured random UUID is suitable), and not store any actual application state in the state parameter.

If you must store data in the "state" parameter, we recommend you use POST-Based authorization (discussed above). Large state values combined with Epic launch tokens may cause the request URL to exceed the authorization server's maximum URL length. By moving the data to a POST body, your request parameters do not contribute to the URL length.

Embedding Tips

If you are writing a web-based application that you intend to embed in Epic, you may find that your application may require minor modifications. We offer a simple testing harness to help you validate that your web application is compatible with the embeddable web application viewer in Epic. As embedding your application in Hyperspace may not behave the same as presenting it in a browser, ensure that your app works with the testing harness and that you can run multiple instances of the app at the same time within the same session (click the "Use Two Browsers" checkbox). Our launching & embedding document has more details, and we provide implementation options below.

Embedding in Multiple Tabs

Epic can embed multiple instances of your application by opening them in separate browser tabs. Given Epic can have multiple patients open at once, your app needs to handle having multiple browser tabs open at once, and remember what patient is open in each tab.

Rather than using HTML5 SessionStorage for tab segregation, you need to use a dedicated URL parameter representing the tab. You can then reference the launch session in that tab from the URL. There are several URL parameter options, such as:

* A path parameter (/app/{tabId})

* A query string parameter (/app?tabId={tabId})

* A URL fragment parameter (/app#tabId={tabId})

Most OAuth 2.0 libraries do not support tab-specific workflows out of the box, and you may need to customize the library to achieve this. This involves at least:

* Having the library redirect to a tab-specific URL after the launch.

* Securing the tab-specific data (patient context) behind your session management system.

Epic's Hyperspace browser does not support segregated session storage. If your app stores tab-specific data in session storage, you need to use the unique tabId from above to store off this data per tab. You can store/retrieve any data like this:

```
window.sessionStorage.setItem(tabId,"tab-specific-data")
window.sessionStorage.getItem(tabId) // Returns "tab-specific-data"
```

Other Browser Options

Epic community members can configure your app to launch in a floating window or the system default browser in case your app cannot support embedding. Note the system default browser is not recommended because switching between patients in Epic does not close your app. The user might not realize the open patient is different between Epic and your application, which might lead to them entering information or making care decisions based on a different patient.

Any apps that cannot support embedding in an iframe should attempt a floating window launch, and only use the system default browser as a fallback.

App Architecture Considerations

This section includes some common considerations for app developers, including:

* Supporting Multiple FHIR Servers - Since each Epic community member has their own copy of Epic.

* Endpoint Discovery - So you can limit your configuration to one or two URLs.

* Retrieving Launch Context - Outlines different ways Epic can share contextual data with your app.

* Identifying the User - Outlines Epic's support for sharing user identifiers and other user info.

Supporting Multiple FHIR Servers

Each Epic community member has a separate copy of Epic, and their own unique database of users and patients. This means that each Epic community member has their own authorization and servers, and that your app needs to differentiate between them. For example, patient records between different systems should not be mixed, because identifiers such as MRNs are not globally unique.

Defining your Endpoints

The two most common strategies we see for handling tenancy in apps are:

* Unique URL paths: https://app.com/<hospital-system>/app

* Unique subdomains: https://<hospital-system>.app.com/app

* Note that apps installed on-premises (hosted by Epic community members) likely have entirely different domains per community member.

Epic community members are most familiar with these strategies, and we recommend against using others such as query string parameters.

Multitenancy and ID Segregation

<hospital-system> is defined by you, and you can match this to your own tenant structure. You should always segregate data from different Epic community members, and keep in mind the following identifiers are not globally unique:

* FHIR IDs

* MRNs and other Epic internal identifiers

These IDs are only unique when stored off with an identifier-system, such as:

* FHIR ID + resource type + the FHIR Base URL

* ID + ID Concept + <hospital-system> - From our ID Types tutorial, OIDs are generally a combination of ID Concept and <hospital-system>

Endpoint Discovery

Epic maintains a list of our Community Member's FHIR endpoints on open.epic.com, where each <hospital-system> as defined above has one endpoint. The sandbox's R4 FHIR endpoint for example is:

`https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4`

Starting from this endpoint, you can manually edit or programmatically discover the other endpoints used by your library. Here are some examples:

* For the OAuth 2.0 authorize and token endpoints, you can use the programmatic methods mentioned here. If your app does programmatic discovery, cache these values on startup or pre-configure them, rather than making the request for every login.

* For the Openid Connect issuer endpoint, replace /api/FHIR/R4 with /oauth2. For example:

`https://fhir.epic.com/interconnect-fhir-oauth/oauth2`

Retrieving Launch Context

The SMART App Launch delivers patient context by adding parameters to the OAuth 2.0 token endpoint response. Your library needs to support retrieving these extra values from the token response, such as patient and encounter below:


```
{
  "access_token": "[omitted]",
  "token_type": "Bearer",
  "expires_in": 3599,
  "id_token": "eyJhbGciOiJSUzI1NiIsImtpZCI6ImxpQ3VsVElhaXRVempmVWgyQXFOaU1ybzQ3WDlIY1ZjZDlYUGk4TERKS0E9IiwidHlwIjoiSld
  UIn0.eyJhdWQiOiJkODZhNjBiNi01MDExLTRjOTEtYTg2Yi1iYmM0YWNjZjkyZTIiLCJleHAiOjE2ODA3MjUyOTEsImZoaXJVc2VyIjoiaHR0cHM6Ly9
  2ZW5kb3JzZXJ2aWNlcy5lcGljLmNvbS9pbnRlcmNvbm5lY3QtYW1jdXJwcmQtb2F1dGgvYXBpL0ZISVIvUjQvUHJhY3RpdGlvbmVyL2V0QnpNYVE0ZEh
  ONU9wVC1DTUZEMzhRMyIsImlhdCI6MTY4MDcyNDk5MSwiaXNzIjoiaHR0cHM6Ly92ZW5kb3JzZXJ2aWNlcy5lcGljLmNvbS9pbnRlcmNvbm5lY3QtYW1
  jdXJwcmQtb2F1dGgvb2F1dGgyIiwibm9uY2UiOiI4ejloXzAwbzNVeGo4ZTA2dVhXdVhQSjBWZEo3OGdXTW1nTFREa2Q4aFlVIiwic3ViIjoiZXRCek1
  hUTRkSE41T3BULUNNRkQzOFEzIn0.N8Ru9dlHN4ephBvYcv6qslcF3lMpAQEMspQAm7vLk-
  S9NekO5L1Sup4H1e7sDGsiUtlyvV012lxj45RAN9iW8NbqbvYM3yjFeonm19rL6T5AOS09Yf694Mg-
  gFOLamGeCnhITm7UNaEWOvso1GoXCsoSujfR3jEPjDG5mEDf9ArWNLdoy6_mHG2xCGELhuk3B4NIDvsU_FKitpx-_lNm1-
  FZNoot2taRW3sMEj9bziIz8wKhtNYPY1lDwoAfbIww-t1umMQsZ3KPnCIqTKOcmiWj2nozJlOW7ouuyeJouSK7Qk-90N5UK5zgQTk_8unAYWYOd-
  5WClGzjCkzKC2DdQ",
  "refresh_token": "[omitted]",
  "scope": "user/AdverseEvent.Read user/AdverseEvent.read user/AllergyIntolerance.read user/AllergyIntolerance.write
  user/Appointment.read user/Binary.Read user/Binary.read user/BodyStructure.read user/CarePlan.read
  user/CareTeam.read user/Communication.read user/Communication.write user/Condition.read user/Condition.write
  user/Consent.read user/Coverage.read user/CriteriaReview.read user/CriteriaReview.write user/Device.read
  user/DiagnosticReport.read user/DiagnosticReport.write user/DocumentReference.Read user/DocumentReference.read
  user/DocumentReference.write user/Encounter.read user/Endpoint.read user/EpisodeOfCare.read
  user/ExplanationOfBenefit.Read user/ExplanationOfBenefit.read user/Flag.read user/Goal.read user/ImagingStudy.Read
  user/ImagingStudy.read user/Immunization.read user/List.read user/Location.read user/Medication.read
  user/MedicationRequest.read user/Observation.read user/Observation.write user/Organization.read user/Patient.read
  user/Patient.write user/Practitioner.read user/PractitionerRole.read user/Procedure.read user/ProcedureRequest.Read
  user/Questionnaire.read user/QuestionnaireResponse.Read user/QuestionnaireResponse.write user/RelatedPerson.Read
  user/RequestGroup.read user/ResearchStudy.Read user/ResearchStudy.read user/ReviewCollection.read
  user/ReviewCollection.write user/ServiceRequest.read user/Specimen.Read user/Specimen.read user/Substance.Read
  user/Substance.read user/Task.read user/Task.write user/ValueSet.Read fhirUser launch launch/encounter
  launch/patient offline_access openid",
  "__epic.dstu2.patient": "TLqtVI-V.rX88jQHUnHxO8Npvd9Gcuz5fFeZuNPSeURIB",
  "encounter": "eUAW2-QkdwCE7KH7awS8sfw3",
  "location": "eweKBcq8ruM-1MzZAX-A-0uj2rDm4SMniPONiaM5VGQk3",
  "loginDepartment": "e4W4rmGe9QzuGm2Dy4NBqVc0KDe6yGld6HW95UuN-Qd03",
  "need_patient_banner": "false",
  "patient": "eRp.HuKDlKB8MhAkzHFthQQ3",
  "smart_style_url": "https://fhir.epic.com/interconnect-fhir-oauth/api/epic/2016/EDI/HTTP/style/33500004931/I0YwRjJGN
  HwjQzEyMTI3fCMxQUFCRkZ8I0QzRDhERXwjODZCNTQwfCMwMDAwMDB8MHB4fDEwcHh8fEFyaWFsLCBzYW5zLXNlcmlmfCdTZWdvZSBVSScsIEFyaWFsL
  CBzYW5zLXNlcmlmfHw%3D.json"
}
```


Epic supports a large library of context tokens, including these FHIR resource IDs (included by default):

Key	FHIR API	Explanation	Available For
patient	Patient.Read	Patient chart open during the launch	Most contexts, except for dashboard-type launches
encounter	Encounter.Read	Visit open during the launch	Provider-facing contexts, where a visit is open during the launch
location	Location.Read	Current location of the patient from the launched visit	Same as encounter, see above
loginDepartment	Location.Read	Department the provider logged into	Only in launches from Hyperspace and provider-facing mobile apps
appointment	Appointment.Read	Scheduled appointment that became the current visit	Available if a visit is open during the launch, and if that visit began as a scheduled appointment

Note that the tokens documented above are not always returned, and this is controlled by the specific workflow your app is used in. Limit the launch context your app requires to the minimum data needed and explain the context and expected

workflow to your installing community members. For example, does your app need visit context, or just patient context?

Identifying the User

If you are using an Openid Connect Relying Party library, it handles user authentication by relying on the "iss" URL and "sub" claim.

Note the "iss" for an OIDC flow is not the same as the "iss" from the first step of an EHR launch. For example:

* Fhir "iss": https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4

* Openid Connect "iss": https://fhir.epic.com/interconnect-fhir-oauth/oauth2

One simple user account strategy is to record the user's "sub" as the primary key in a database dedicated to one Openid Connect Issuer (See Multitenancy and ID Segregation above). Using this system, you can create an account on-the-fly for anyone that can launch your app from Epic, optionally retrieving their demographics using the strategies below.

On-the-Fly Account Creation

If your app requests the "openid" and "fhirUser" scopes, your id_token includes a claim where you can retrieve additional demographics about the user by making an API call.

For example, a launch from the sandbox could return these claims (among other claims omitted):

```
{

  "fhirUser": "https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4/Practitioner/eUXiD4IyBLJFoR2VHOjfTdg3",

  "sub": "eUXiD4IyBLJFoR2VHOjfTdg3"

}
```

So, an app scoped for the Practitioner.Read (R4) API could make a GET request to this URL (using your access token). This resource returns name, email, gender, and NPI, among other identifiers such as employee record ID. Note that not all users have an NPI.

Below is an example "identifier" object showing a provider's NPI. You can use the "system" value of "urn:oid:2.16.840.1.113883.4.6" across all Epic community members because it is the global OID for NPI:

```
{

  "use": "usual",

  "type": {

    "text": "NPI"

  },

  "system": "urn:oid:2.16.840.1.113883.4.6",

  "value": "1627363736"

}
```

Here is an example object showing a user's employee record ID. Note the system value is an OID that will vary per Epic community member, and between non-production and production:

```
{
  "use": "usual",
  "type": {
    "text": "EXTERNAL"
  },
  "system": "urn:oid:1.2.840.114350.1.13.0.1.7.2.697780",
  "value": "FAMMD"
}
```

See our documentation for more information on making FHIR requests using your access token.

Deploying your Application

Your app should have been issued two client IDs: a non-production one and a production one. These client IDs are used for all your Epic community member installs, not just Epic's sandbox. Make sure that your app's production servers are using your production client ID.

Your application should use separate key pairs for non-production and production environments.

After your app is deployed and made available to Epic community members, you should see requests for your application within the website. Depending on your app's architecture, your implementation team will provide specific settings per every install, using the instructions below. Note, only non-production URLs (-np, as a convention) are shown, and you will repeat this setup with your production URLs (-p) and production client ID as well.

Example App Deployments

Below are 2 example app deployments, one for a Cloud-Based App and one for an On-Premises App. Review these to see what fields you should provide on the website when licensing your app for a given Community Member.

Cloud-Based App

This example assumes your application supports private_key_jwt authentication. If not, and if you are using client secrets, you must use different secrets for each Epic Community Member.

An example cloud-based application with two installing health systems might have the following list of endpoints:

* https://my-app-np.com/config/jwks.json

* https://my-app-np.com/app/launch

* https://my-app-np.com/app/health-system-a/callback

* https://my-app-np.com/app/health-system-b/callback

Note there is only one jwks.json (and a production one not listed) because the app can in principle protect a shared private key in the cloud. If you do not want to re-use private keys across Epic community members, review the On-Premises App model below.

Steps:

* Make sure your app has an app-level JWK Set URL listed, because this is not a download specific value for cloud-based apps. Your organization's administrator should be able to edit this even after your application has been activated. Click "edit" at the bottom of your build apps page to unlock this field:

* When provisioning, un-check and remove the app-level endpoint URIs, and just supply the health system specific values:

* When provisioning, select the "JWK Set URL" option and explicitly use the app-level value. If the "JWK Set URL" option is not available, see step 4:

* (Conditional) Provide your public key manually in X509 format if the JWK Set URL option is disabled based on the community member's Epic version:

On-Premises App

An example on-premises (on prem) application with two installing health systems may have the following list of endpoints. Note there are multiple public key endpoints because on-premises apps should always have a JWK Set URL pointing at the community member's specific instance of your application, and its unique public key:

* https://health-system-a.my-app-np.com/config/jwks.json

* https://health-system-a.my-app-np.com/app/launch

* https://health-system-a.my-app-np.com/app/callback

* https://health-system-b.my-app-np.com/config/jwks.json

* https://health-system-b.my-app-np.com/app/launch

* https://health-system-b.my-app-np.com/app/callback

Steps:

* When provisioning, un-check and remove the app-level endpoint URIs, and just supply the health system specific values:

* When provisioning, select the "JWK Set URL" option, un-check "Use app-level JWK Set URL", and provide this health system's specific URL. If the "JWK Set URL" option is not available, see step 3:

* (Conditional) Provide your public key manually in X509 format if the JWK Set URL option is disabled based on the community member's Epic version:

Appendix

Considerations for Native Apps

There are additional security considerations when launching a native app to sufficiently protect the authorization code from malicious apps running on the same device. There are a few options for how to implement this additional layer of security:

* Platform-specific link association: Android, iOS, Windows, and other platforms have introduced a method by which a native app can claim an HTTPS redirect URL, demonstrate ownership of that URL, and instruct the OS to invoke the app when that specific HTTPS URL is navigated to. Apple's iOS calls this feature "Universal Links"; Android, "App Links"; Windows, "App URI Handlers". This association of an HTTPS URL to native app is platform-specific and only available on some platforms.

* Proof Key for Code Exchange (PKCE): This is a standardized, cross-platform technique for public clients to mitigate the threat of authorization code interception. However, it requires additional support from the authorization server and app. PKCE is described in IETF RFC 7636 and supported starting in the August 2019 version of Epic. Note that the Epic implementation of this standard uses the S256 code_challenge_method. The "plain" method is not supported.

Starting in the August 2019 version of Epic, the Epic implementation of OAuth 2.0 supports PKCE, and Epic recommends using PKCE for native mobile app integrations. For earlier versions or in cases where PKCE cannot be used, Epic recommends the use of Universal Links/App Links in lieu of custom redirect URL protocol schemes.

private_key_jwt authentication

During an OAuth 2.0 flow, your server may be required to authenticate itself to the token endpoint. We recommend private_key_jwt authentication over client_secret_basic (and client_secret_post) for a few reasons:

* It uses asymmetric cryptography, so you never have to distribute your credential (private key).

* It simplifies deploying your application because private keys can be reused across Epic community members (for cloud-based apps).

* When your public key is hosted on a JWK Set URL, you can easily rotate your public (and private) keys to help you meet NIST's recommendations on private key rotation.

Comparison to Mutual TLS

private_key_jwt authentication is similar to mutual TLS in that both use asymmetric cryptography. Both techniques give you strong assurance of who the app is and do not require you to manually distribute a secret like a password.

The two techniques differ in a few key ways:

* Mutual TLS can use a certificate chain to validate the expected public key, whereas private_key_jwt relies on the app developer pre-registering a public key with Epic.

* With private_key_jwt authentication, the app authenticates only once per hour per launch. With mutual TLS, the app authenticates on every web service call.

* Mutual TLS is conducted at the transport layer, whereas private_key_jwt is conducted in the application layer (see the OSI Model).

Practically, mutual TLS is more difficult to maintain because Epic community members need to register public keys manually through their Client Systems team. With private_key_jwt, Epic handles authentication directly in the Epic database.

Launching Without SSO

Use Cases

You might want to launch your application from Epic but not use the launch to authenticate your users. In general, we see one of two use cases for this:

Re-authenticating on Launch

While not our recommended workflow, your app can re-authenticate users during the launch and just use the launch to provide access to patient data.

No Authentication

Your application does not include a component that needs protecting. Examples of un-protected applications include:

* Public calculators, where anyone can submit data to your server without authentication and see the results.

* Front-end data visualization apps, where data loaded from the EHR is processed on the user's device.

If your un-protected application does not require user or patient context, then you do not need to use the SMART App Launch at all. You can simply provide a static URL for your website or Native app that Epic Community Members can configure in their system. This does not require registration with Epic.

If your un-protected application requires user or patient context, then you should use the launch to avoid putting PHI and PII into the page's URL.

Choosing a Library without SSO

Our recommendation for these apps is to use the same infrastructure as an application using OAuth 2.0 for SSO. However, for applications with no web server component this added infrastructure may be more complex than you need.

As an alternative, you can use a client-side OAuth 2.0 library such as the common, opensource fhirclient library (for single-page-apps) or one from the AppAuth SDK (for native apps). Use caution when using these libraries because they cannot be used to protect your app's web server (if you have one). They only secure the connection between your client-side and the EHR's authorization server.

If you decide to use a client-side OAuth 2.0 library, you will not register credentials (public keys or client secrets) when building your app and will configure your library to use client authentication type "none." This is called out in the "Upload your credentials" step of the Building your Application section.

Persistent Access

If you are using a client-side OAuth 2.0 library, you cannot use the "refresh_token" grant type for persistent access to APIs (>1 hour of access). If you need persistent access, we recommend using a server-side OAuth 2.0 library with refresh tokens, where the refresh token is stored on your server and protected by your session management system.

If you cannot add a server component to your application, you can add dynamic client registration to your client-side only application and follow the directions here to use your initial access token to get persistent access. This app is only available for Patient-facing applications at this time.

SMART Scopes

SMART scopes are returned in the scope parameter of the OAuth 2.0 token endpoint response and determine which resources an application has permission to access and what actions the application is permitted to take. The scopes provided are based on the scopes that your app requested in your authorize request as well as the incoming APIs you have selected on your app page.

As of the November 2024 version of Epic, both SMART v1 and SMART v2 scope formatting are supported. In the August 2024 and prior versions, only SMART v1 scopes are supported.

The actions defined by SMART v1 scopes are .read and .write. SMART v2 scopes use CRUDS syntax and actions include: .c – create, .r – read, .u – update, .d – delete, and .s – search.

For example, the Observation.Create resource corresponds to either:

* SMART v1: {user-type}/Observation.write

* SMART v2: {user-type}/Observation.c

Which version should you select?

If your app uses SMART v1 scopes or if your app does not use SMART scopes at all, you should select SMART v1.

If your app uses SMART v2 scopes, you should select SMART v2.

Note: Prior to the November 2024 version of Epic, SMART v1 scope formatting will be returned regardless of the SMART Scope Version selected on your app page. If your app uses SMART scopes and your customer is on a version of Epic earlier than November 2024, you must support SMART v1 scopes.

1.24 Patient Searching and Matching

File: site/docs/01-guides/24-eof-patientsearching.html

Source: https://fhir.epic.com/Documentation?docId=eof_patientsearching

If your app reads patient data from a customer system or writes data to patient records, it is essential that your app is accessing the correct patient record. When searching for or matching to a patient record, Epic strongly recommends that you require all of the following 3+1 demographics criteria:

* Patient First and Last Name.

* Legal Sex

* Date of Birth, and

* At least one additional identifier such as MRN, address, phone number, or email.

Keep the following considerations in mind depending on the type of app you are developing:

* For patient-facing apps using OAuth 2.0 and user-facing apps that launch from Epic with patient context:

* These apps obtain patient identifiers during the launch that can be used to call APIs and pull additional patient information for displaying to the patient or Epic user.

* If such a user-facing app writes data to a patient record in Epic, the user must be able to view enough patient information to confirm they are writing to the correct patient record.

* If the app has its own patient database, and information from that database should be written to a patient record in Epic, the app should perform matching between the two using the required demographic criteria.

* If there are multiple (patient) matches, patient-facing apps should neither display the data nor allow the data to be written to Epic.

* For user-facing apps that search for a patient record in Epic and display patient data to the user:

* These apps can leverage flexible search criteria, assuming the user has permission to view additional information for each matching patient record and can select the correct patient.

* It is important that an app only allow users to search for patient records in the app database, if the user has permission to view all patient records in the database (if an app database has patient information from multiple organizations, users may not have access to patient information outside of their organization).

* If such an app could write data to a patient record in Epic, the user must be able to view enough patient information to confirm they are going to write to the correct patient record.

* If the app has its own patient database, and information from that database could be written to a patient record in Epic, the app must match using the required demographic criteria; if multiple matches are returned, the app can provide additional patient information allowing the user to select which patient record to write to.

* For patient-facing apps searching for a patient record in Epic to display data to the patient or write back to the patient's record:

* These apps should search with the required demographics criteria and fail if multiple patient records are returned.

* These apps should not present more information from a patient record until the right patient has been identified and matched with data provided by the patient. Otherwise, you risk displaying another patient's information to the current patient and breaching patient privacy.

* It is best to use a patient lookup API such as Patient.$match that will not return more than one patient.

* It is recommended that you do multi-factor authentication as well in the absence of the ability to do OAuth MyChart authentication

* For apps that read or write patient data using backend calls:

* These apps should match patient records between the app and Epic databases using the required demographic criteria and fail if multiple patient records are returned before retrieving patient data.

* It is best to use a patient lookup API such as Patient.$match that will not return more than one patient.

* If such an app should write data to a patient record in Epic, it should pull demographics from Epic and validate the patient record is correct using the required demographics criteria before using a patient identifier to write to Epic. Apps generally must follow this best practice unless their app database is only populated programmatically by Epic, such as from flat file exports, SMART on FHIR launches or HL7/X12 interfaces. Validating with demographics before writing to a patient record is always a recommended guardrail.

* Apps writing data to a patient record in Epic using an HL7v2 clinical interface should send patient demographics in addition to a patient identifier in each message.

1.25 Search Parameters

File: site/docs/01-guides/25-searchparameters.html

Source: https://fhir.epic.com/Documentation?docId=searchparameters

Epic supports the _revInclude parameter to fetch provenance data for search interactions with resources that are part of the U.S Core Data for Interoperability starting in Epic version February 2020. Support for all other resources begins in August 2023.

FHIR Search Parameters

Contents

* Overview
* General Considerations
* Native Search Parameters
* Post-Filter Search Parameters
* Common FHIR Parameters
* _id Parameter
* _count Parameter
* _include Parameter
* _revInclude Parameter
* Provenance
* AuditEvent

Overview

This document describes general considerations for using FHIR search parameters, including the two categories of search parameters that Epic makes available across FHIR resources. It also explains which common (for example “underscore”) FHIR search parameters are currently supported for use across all Epic FHIR resources, what those common parameters do, and Epic’s recommendations for using them.

General Considerations

Epic supports many search parameters defined by the FHIR standard. These search parameters use two different types of logic to retrieve results and can be categorized as either native search parameters or post-filter search parameters. Refer to Epic’s documentation for individual FHIR resources to see which parameters are native search parameters and which are post-filter search parameters.

Native Search Parameters

Native search parameters are parameters that Epic supports with built-in database logic or indexing. To give one example, the patient parameter is a native search parameter available in all patient-centric Epic FHIR resources. All parameters released in the February 2024 version and earlier are native search parameters, and we plan to continue to provide support for and release new native search parameters in the future.

Post-Filter Search Parameters

Post-filter search parameters are available starting in the May 2024 version of Epic. Epic supports post-filter search parameters only for the R4 version of FHIR. Epic supports post-filtering so that your integration doesn’t have to. This gives you the flexibility to search and filter on the widest variety of parameters possible within the bounds of the HL7 FHIR specification.

When a FHIR query includes post-filter search parameters, Epic first performs a search using native search parameters and converts those results to the FHIR format. Then, the results are further reduced based on the post-filter search parameters that were supplied in the query. As a result, the performance cost of using post-filter search parameters is directly proportional to how many resources match to the native search parameters in a request. For example, if a theoretical patient has 5000 lab results, but only one of those lab results matches to a post-filter search parameter value in your query, the Epic database has to first retrieve all 5000 lab results before it can filter them down to the single result that is ultimately returned to your application.

"Unsupported" Post-Filter Search Parameters

Some post-filter parameters are marked as “Unsupported” in Epic documentation. This means that Epic does not currently support any of the corresponding response element(s) that the search parameter is meant to query for, so using the parameter will always cause all results to be filtered out of the response.

Common FHIR Parameters

Common FHIR search parameters, which include a variety of underscore-prefixed parameters, can modify how results are returned in a Search API’s response. See the table below for the common search parameters that are supported by Epic and the versions in which they are supported.

Parameter | Supported in DSTU2? | Supported in STU3? | Supported in R4?
_id | Yes | Yes | Yes
_count | Yes, for the Observation resource only | Yes | Yes
_include | No | Yes | Yes
_revInclude | No | No | Yes
_lastUpdated | No | Yes starting in Aug 26, for the DiagnosticReport (Results) resource only | Yes starting in Aug 26, for the DiagnosticReport (Results) and Observation (Labs) resources only

_id Parameter

The _id parameter is used to indicate a FHIR ID, allowing read-type functionality in a search FHIR API.

So why would you want to use _id in a search API instead of a read API? Well, what if you wanted to perform a read with a FHIR search parameter? The _id parameter allows you to do just that by using FHIR search parameters while still reading data about a single resource.

Examples:

```
GET https://hostname/instance/api/FHIR/STU3/Patient?_id=eYg3-1aJmCMq-
    umIIq2Njxw3&_include=Patient:generalPractitioner:Practitioner
```

_count Parameter

The _count parameter is used to limit the number of results returned by a search request. Once the number of results specified has been reached, the URL to the next page of results will be generated and included in the response. This URL contains a session ID which returns the next group of _count results. This is useful for resources that could return a large number of results (such as Observation), as using _count can minimize the demand on the server. _count can be set to any number from 1 to 999.

Important note on resources that return many results:

If your search returns more than 1000 results, the search will automatically be broken into pages with 1000 results each, regardless of whether or not you use the _count parameter. This means that your app needs to support this functionality to ensure that you receive all applicable data associated with the API request.

The API response when paging behaves differently depending on what FHIR version you are on, as described below.

STU3 and Later FHIR Versions

This section summarizes the steps to use _count functionality.

Iterate over the Bundle.link array and look for the link.relation parameter. If there is a link where link.relation is "next", use the request in link.url to obtain the next page of resources. Continue until there is no longer a link.url with a link.relation of "next", indicating you are on the final page.

There will be a second URL in link.url with a link.relation of "self" on every page. In STU3 paging, the self link will contain a session ID that points to the current page of results. In R4 paging, the self link will include the valid search parameters that were used in the original request.

STU3 Sample Request

```
GET https://apporchard.epic.com/interconnect-aocurprd-
    username/api/FHIR/STU3/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-signs&_count=10
```

STU3 Sample Response

```
{
   "resourceType": "Bundle",
    "type": "searchset",
    "total": 10,
    "link": [
        {
            "relation": "next",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/STU3/Observation?sessionID=16-BAF392808B2B11EA92E00050568B7BE6"
        },
        {
            "relation": "self",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/STU3/Observation?sessionID=16-854D71651G6711EA92E00050568B7BE6"
        }
...{response continues}
```

R4 Sample Request

```
GET https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/R4/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-signs&_count=10
```

R4 Sample Response

```
{
    "resourceType": "Bundle",
    "type": "searchset",
    "total": 10,
    "link": [
        {
            "relation": "next",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/R4/Observation?sessionID=16-BAF392808B2B11EA92E00050568B7BE6"
        },
        {
            "relation": "self",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/R4/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-signs&_count=10"
        }
...{response continues}
```

DSTU2 FHIR Versions

Similar to the STU3 version, there will be a link.relation parameter which tells you the relationship of the URL in the API response relative to the API URL in the current request. The link.relation parameters that can be returned are:

* self: a url to the same page of results as your current page

* previous: a url to the previous page of results relative to your current page

* next: a url to the next page of results relative to your current page

The first page of results will only return relations of “self” and “next” while the last page of results will only return relations of “self” and “previous”. If a page only returns the “self” parameter, this indicates that only one page of results exists. Omitting a page number and session ID defaults to displaying the first page of results. In the DSTU2 version, paging functionality only exists for the Observation resource.

* Perform a Search with the FHIR Observation resource, and include the parameter “_count=<# of resources to return>” in the URL. This will control the number of resources returned in a single API call.

```
GET https://apporchard.epic.com/interconnect-aocurprd-
    username/api/FHIR/DSTU2/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-signs&_count=10
```

* The beginning of the first response will look similar to that shown below. The “self” relation contains a URL that points to the current page of results. The “next” relation contains a URL that points to the page of results following the current page. If the first page contains all results, only the “self” relation will appear.

```
{
    "resourceType": "Bundle",
    "type": "searchset",
    "link": [
        {
            "relation": "self",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/DSTU2/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-signs&_count=10"
        },
        {
            "relation": "next",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/DSTU2/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-
    signs&_count=10&session=989750&start=2"
        }
...{response continues}
```

* As long as there are more than two pages, the response will contain three URLs now, “self”, “next”, and “previous”. As a counterpart to the “next” relation, the “previous” relation indicates a URL that points to the page of results preceding the current page of results in the response.

```
{
    "resourceType": "Bundle",
    "type": "searchset",
    "link": [
        {
            "relation": "self",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/DSTU2/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-
    signs&_count=10&session=989750&start=2"
        },
        {
            "relation": "next",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/DSTU2/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-
    signs&_count=10&session=989750&start=3"
        },
        {
            "relation": "previous",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/DSTU2/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-
    signs&_count=10&session=989750&start=1"
```

```
        }
...{response continues}
```

* When you reach the last page of results, the response will look something like that shown below. There will still be a “self” and “previous” relation, but no “next” relation as there are no additional pages to display.

```
{
    "resourceType": "Bundle",
    "type": "searchset",
    "link": [
        {
            "relation": "self",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/DSTU2/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-
    signs&_count=10&session=989750&start=3"
        },
        {
            "relation": "previous",
            "url": "https://apporchard.epic.com/interconnect-aocurprd-
    oauth/api/FHIR/DSTU2/Observation?patient=e63wRTbPfr1p8UW81d8Seiw3&category=vital-
    signs&_count=10&session=989750&start=2"
        }
...{response continues}
```

_include Parameter

The _include parameter is used to return resources related to the initial search result. Any other resource included as a reference can be returned in the response bundle. This reduces the number of requests needed to see information. For example, when searching for a patient's encounters, you can include the encounter providers as well.

In the February 2021 version of Epic and prior versions, included references are returned immediately following the associated search result. Starting in the May 2021 version of Epic for R4 FHIR resources, included resource references from all search results are at the end of the response bundle rather than after each individual search result, and each unique referenced resource instance is included only once in the bundle. Consumers of FHIR search results should be able to accept both bundle formats until a future version when we expect all resources to eventually transition to the latter format.

Epic supports the _include parameter primarily for FHIR search interactions. In addition, it is supported for particular operation types, like Appointment.$find.

If you want to return multiple referenced resources, this can be accomplished by repeating the _include criteria. A single _include parameter should not include multiple values. Instead, the parameter is repeated for each different include criteria.

For this parameter to work, the Search response must include a reference to another FHIR resource.

General format: _include = [resource you’re searching]:[name of reference element you want included]:[type of resource if element can refer to multiple (optional)]

* [resource you’re searching] – this is the initial API you are calling

* Ex: When calling Observation.Search, you are searching the Observation resource.

* [name of reference element you want included] – one of the response parameters for that API that has a type of “Reference”

* [type of resource if element can refer to multiple] – when the second parameter could refer to multiple resources, the third parameter specifies which resource to include

Examples:

* Immunization.Search

* Parameter in response: Practitioner.actor (Reference)

* From this, the second parameter would be “actor”

* _include=Immunization:actor:Practitioner

* Note: there is only one actor for Immunizations so the third parameter is optional (_include=Immunization:actor would also work here)

* Patient.Search

* Parameter in response: generalPractitioner (Reference (Practitioner))

* From this, the second parameter would be “generalPractitioner”

* _include=Patient:generalPractitioner:Practitioner

* Appointment $find (STU3)

* Parameters in response: Participant.actor (Reference(Patient|Practitioner|Location))

* From this, the second parameter would be "actor" just like in the first example

* Unlike the first example, you need to specify a third parameter as there are three potential actors referenced. Any of the following examples are valid here:

* _include=Appointment:actor:Patient

* _include=Appointment:actor:Practitioner

* _include=Appointment:actor:Location

* Observation.Search

* Parameters in response:

* performer (Reference (Practitioner))

* subject (Reference (Patient))

* To include both resources, the _include criteria is used multiple times

* _include=Observation:performer:Practitioner&_include=Observation:subject:Patient

_revInclude Parameter

The _revInclude parameter (reverse include) is used to return a particular resource, along with other resources that refer to that resource. For this parameter to return results, the FHIR resource specified in the search response must be referenced by the resource specified in the _revInclude parameter. Epic's implementation of the _revInclude parameter supports returning Provenance and AuditEvent data.

Provenance

Parameter format: _revInclude=Provenance:target

The response will have the following format:

Element | Type | Cardinality | Description
target | Reference (resource) | 1..* | The resource this provenance supports
recorded | Instant | 1..1 | Timestamp that the activity was recorded/updated
agent | Backbone Element | 1..* | Actor involved
agent.type | CodeableConcept | 0..1 | How the agent participated
agent.who | Reference (Practitioner | Patient | Organization) | 1..1 | Who participated
agent.onBehalfOf | Reference (Organization) | 0..1 | Who the agent is representing.

Examples: Sample Request

```
https://www.example.org/api/FHIR/R4/DocumentReference?_id=eiuHBlNpiGxJwXRUWVqHjN9QaKO0ODrkAJrvUByRLuMc3&_revInclude=Prov
    enance:target
```

Sample Response { "link": [{ "relation": "self", "url": "https://www.example.org/api/FHIR/R4/Provenance/eC7YEH-XJufbPDHTDFWShEk2b9Ki98xTRyLbhFXwQ6sfcjhyfmeem2nFqdxZYHcS0FhROyzJUToN.o1UE0DQeXQ3" }], "fullUrl": "https://www.example.org/api/FHIR/R4/Provenance/eC7YEH-XJufbPDHTDFWShEk2b9Ki98xTRyLbhFXwQ6sfcjhyfmeem2nFqdxZYHcS0FhROyzJUToN.o1UE0DQeXQ3", "resource": { "resourceType": "Provenance", "id": "eC7YEH-XJufbPDHTDFWShEk2b9Ki98xTRyLbhFXwQ6sfcjhyfmeem2nFqdxZYHcS0FhROyzJUToN.o1UE0DQeXQ3", "target": [{"reference": "DocumentReference/eiuHBlNpiGxJwXRUWVqHjN9QaKO0ODrkAJrvUByRLuMc3"}], "recorded": "2020-06-23T07:33:12Z", "agent": [{ "type": { "coding": [{ "system": "http://terminology.hl7.org/CodeSystem/provenance-participant-type", "code": "author", "display": "Author" }], "text": "Author" }, "who": { "reference": "Practitioner/elOnxJsdFcfgJ4iy716l3MA3", "display": "Starter Provider, MD" }, "onBehalfOf": {"display": "Example Organization"} }, { "type": { "coding": [{ "system": "http://terminology.hl7.org/CodeSystem/provenance-participant-type", "code": "transmitter", "display": "Transmitter" }], "text": "Transmitter" }, "who": { "display": "Example Organization" } }] }, "search": {"mode": "include"} } AuditEvent Epic currently supports using this parameter to fetch audit event data for search interactions with select FHIR resources starting in Epic version May 2021. AuditEvent returns audit event information for a specific resource instance. Examples of audited events include edits to an order or the signing of a note. AuditEvent is supported only for a subset of FHIR version R4 resources.

Parameter format: _revInclude=AuditEvent:entity

The response will have the following format:

Element | Type | Cardinality | Description
type | Coding | 1..1 | Type or identifier of the event
recorded | Instant | 1..1 | Time when the event was recorded
agent | BackboneElement | 1..* | Actors involved in the event
agent.role | CodeableConcept | 0..* | Agent role in the event
agent.who | Reference (PractitionerRole | Practitioner | Organization | Device | Patient | RelatedPerson) | 0..1 | Identifier of who participated
agent.requestor | Boolean | 1..1 | Whether the user is the initiator
source | BackboneElement | 1..1 | The audit event reporter
source.observer | Reference (PractitionerRole | Practitioner | Organization | Device | Patient | RelatedPerson) | 1..1 | The identity of the source detecting the event
entity | BackboneElement | 0..* | Data or objects used
entity.what | Reference (Any) | 0..1 | Specific instance of the resource
entity.detail | BackboneElement | 0..* | Additional information about the entity
entity.detail.type | String | 1..1 | Name of the property that changed
| String | 1..1 | Property value
entity.detail extension (audit-event-detail-state) | Extension | 0..1 | Indicates whether the value in entity.detail.valueString is the value before or after the audited event

Examples: Sample Request

```
https://www.example.org/api/FHIR/R4/MedicationRequest?_id=eiuHBlNpiGxJwXRUWVqHjN9QaKO0ODrkAJrvUByRLuMc3&_revInclude=Audi
    tEvent:entity
```

Sample Response { "link": [{ "relation": "self", "url": https://www.example.org/api/FHIR/R4/AuditEvent/erLtgh1AreI5Y0dE6YVZxVzXV3FaxnhsMWrsnBmt-JUcxF7D14JKfYlolngfGblRgloJNa8UzOyGJ38eJBF3aKn94wMPR9m6M.LWkVmDWuRxpwYKzNEloKILbZHuXeidD3 }], "fullUrl": https://www.example.org/api/FHIR/R4/AuditEvent/erLtgh1AreI5Y0dE6YVZxVzXV3FaxnhsMWrsnBmt-JUcxF7D14JKfYlolngfGblRgloJNa8UzOyGJ38eJBF3aKn94wMPR9m6M.LWkVmDWuRxpwYKzNEloKILbZHuXeidD3, "resource": { "resourceType": "AuditEvent", "id": "erLtgh1AreI5Y0dE6YVZxVzXV3FaxnhsMWrsnBmt-JUcxF7D14JKfYlolngfGblRgloJNa8UzOyGJ38eJBF3aKn94wMPR9m6M.LWkVmDWuRxpwYKzNEloKILbZHuXeidD3", "type": { "system": "http://open.epic.com/FHIR/StructureDefinition/audit-event-type", "code": "Modify Order", "display": "Modify Order" }, "recorded": "2020-12-09T21:00:03Z", "agent": [{ "role": [{ "coding": [{ "system": "http://open.epic.com/FHIR/StructureDefinition/audit-event-agent-role", "code": "Ordering Provider", "display": "Ordering Provider" }] }], "who": { "reference": "Practitioner/eEISPNvH1RKgu1t69j6Bx2g3", "display": "Starter Provider, MD" }, "requestor": true }], "source": { "observer": {"display": "Example Organization"} }, "entity": [{ "what": {"reference": "MedicationRequest/eiuHBlNpiGxJwXRUWVqHjN9QaKO0ODrkAJrvUByRLuMc3"}, "detail": [{ "type": "Comment", "valueString": "Order Modified" }, { "type": "Administration instructions", "valueString": "Administration Instructions edited" } }] } "search": {"mode": "include"} }

1.26 Sex, Gender, & Names

File: site/docs/01-guides/26-sgn.html

Source: https://fhir.epic.com/Documentation?docId=sgn

Sex, Gender, & Names

Introduction

For a given patient, Epic differentiates between several sex and gender properties, stores preferred and alternate names, and determines which to show to an end user based on the workflow. For example, patient interaction, clinical care, reimbursement, and patient identification and matching might use different sex, gender and names for the same person.

As an app developer, you should carefully determine the appropriate sex, gender, and name properties to retrieve and use based on your users, workflows, and healthcare setting. Some scenarios are more straightforward than others. Possible scenarios include:

* Patient interaction: Any time users interact with the patient (in person, through printed materials and handouts, letters, MyChart, or in other ways), the patient's gender identity should be respected when possible.

* Legal: Administrative systems need to identify a patient by their legal name and legal sex to interface with external systems such as state registries or systems that print legal forms for signature. For most standards-based data exchange, Epic communicates the patient’s legal sex as the administrative sex.

* Insurance: Insurance claims normally require the patient sex that is known to the payer (Medicare, Medicaid, other insurance). The best name and sex to use on an insurance claim might not be the best name to use when addressing the patient in person.

* Clinical: Clinicians need to know how to treat a patient based on the patient's current biology. When deciding whether to administer a prostate exam, the relevant question is whether the patient has a prostate, which is a distinct question from what sex is listed on the patient's driver's license. Epic has a Sex for Clinical Use concept that can be used as a broad categorization of the patient’s likely physiological properties in cases where the exchange of discrete organ inventory or other phenotypical properties is not practical.

Considerations for apps involving patient interaction

If the app needs a patient gender value only to communicate with the patient and not for clinical use, then you should use gender identity. However, not all patients have a gender identity documented, and not all organizations collect it, so you should plan to fall back to the administrative (legal) gender, which is almost always available.

Your app should support different values for gender identity (for patient communication) and administrative sex (for patient matching and as a fallback when gender identity isn’t documented).

Organizations might use different values on patient wristbands, room name cards, specimen labels, and other identifiers. Expect to coordinate with each organization to align the data your app uses for patient identification with the organization’s specific policies.

Considerations for administrative apps (legal, insurance)

Does your app need a patient’s sex for formal documents or communication with government agencies? If so, you should likely use the patient’s Legal Sex, which is communicated as the administrative sex in most data exchange standards. For

communicating with insurance, Coverage Sex tracks the sex the health plan has on file for the patient, and might be different than the patient's legal sex. For example, most plans keep either F or M on file for nonbinary patients with a legal sex of X or NB, and using the Coverage Sex is necessary for submitting successful claims.

Considerations for clinical apps

If a system needs a single sex value for the purpose of clinical decision making, be that by an end user of that system or by an automated process such as a reference range, then you should use the derived Sex for Clinical Use value instead of other related fields.

Epic calculates the Sex for Clinical Use value using a number of data points. If Sex for Clinical Use is unknown, individuals and systems should not assume sex by default.

Note that based upon jurisdictional law, health systems policies, and patient preferences, a patient proxy authorizing an app to access FHIR APIs might not have access to a patient's preferred name, gender identity, or sex for clinical use.

Available sex, gender, and name data elements

Element Name | Definition | Why it matters | API | HL7v2

Preferred Name | Name used by patient in day-to-day life | Note that using preferred name is something that can be important for more than just transgender patients. For example, a dementia patient who goes by Tessie might not remember or respond to her legal name of Theresa. It can be important to providing a better experience at the healthcare organization if everyone sees her name as Tessie in the chart and in ancillary systems so that everyone can use that name to address her. Preferred name might also be referred to as a chosen name, nickname, name in use, or lived name. | FHIR Patient resource (all versions): name.use = 'usual' | PID-5

Legal Sex | Legal sex of the patient | Legal sex might be required for patient identification, matching, or integration with government registries. | FHIR Patient resource (all versions): gender | PID-8

Coverage Sex | Sex documented on the patient's coverage | Health plans track a patient's demographics and compare against those submitted by a provider during administrative processes (for example, authorization or claims). Coverage Sex tracks the patient sex the health plan has on file, and can be different than the patient's legal sex. | Not supported | IN1-43 (subscriber sex)

Gender Identity | How the patient identifies their gender | Helps to assist users in the other system to address the patient respectfully. Might be required for reporting and registries. | FHIR Patient resource (STU3+): Patient.extension (gender-identity) | ZPD-20 or OBX value

Sex assigned at Birth | Sex assessed at birth | Helps to assist users in the other system with background information. | FHIR Patient resource (STU3+): Patient.extension (birth sex) | ZPD-22 or OBX value

Menarche Flag | Whether the patient can get pregnant | Useful for systems that deal with pregnancy and delivery, most commonly OB systems. This is intentionally not a gender/sex value and should not be used to determine if a patient is female. | Not supported | ZPD-23

Sex for Clinical Use | Patient proxy sex for clinical decisions | Sex for Clinical Use is useful for systems that make clinical decisions based on patient sex and can help prevent harmful assumptions about a patient's body and health. | FHIR Patient resource (STU3+): Patient.extension (sex-for-clinical-use) | ZPD-24

Individual Pronouns | Patient's pronouns | Referring to a patient using the wrong pronouns can lead to them feeling disrespected and excluded. | FHIR Patient resource (R4+): individual-pronouns | Patient Pronouns: ZPD-36 Pronoun Comment: ZPD-37

Sexual Orientation | Sexual Orientation | This information can be used for continuity of care during conversions or for reporting purposes. | FHIR Patient resource (STU3+) | ZPD-21

Appendix

Why related fields aren't reliable for clinical decision making

Here are some examples of why related sex and gender properties, such as sex assigned at birth, gender identity, and legal sex, aren't reliable for clinical decision making:

* Sex assigned at birth is often accurate for newborns without intersex conditions, but becomes less accurate as a person ages. A transgender patient might choose to undergo treatments to transition. In this case, the sex assigned at birth is not representative of the patient's current physiological status, but their gender identity might be.

* A patient might undergo hormone therapy, which could impact reference ranges for lab tests, but the patient might still have the organs they were born with.

* A trans patient might not choose to undergo treatments to transition. In this case, sex assigned at birth might be representative of the patient's current physiological status, and gender identity might not be.

* Many jurisdictions allow citizens to update the sex marker on their government documents, including the sex on their birth certificate, so the legal sex or sex assigned at birth recorded in the system might not represent current physiological status, and the sex assigned at birth documented in the system might not represent the physiological status of the patient at birth.

How Epic derives Sex for Clinical Use

To derive a Sex for Clinical Use value, the sex assigned at birth (SAAB), gender identity (GI), legal sex, OB history, and organ inventory are used as "hints" to determine if the patient is not fully represented by a physiological category of male or female. If any of these values (GI, SAAB, Legal Sex) differ, or if they are inconsistent with documented organ inventory or OB history, then the patient might have a physiological status that is not fully known or is not fully represented by physiological categories based only on the data.

What if Sex for Clinical Use is unknown?

Current diagnostics, analytics, and treatment best practices might be undefined or not aligned with existing sex-derived reference populations.

For these reasons, absent a Sex for Clinical Use, individuals and systems should not make assumptions about a patient's physiological status for the purpose of providing care, but should rather:

* Use default behavior that is safe for both male and female populations.

* Individually review treatment options with the patient.

* Carefully inspect relevant observations before proceeding with treatment.

1.27 Sandbox Test Data

File: site/docs/01-guides/27-testpatients.html

Source: https://fhir.epic.com/Documentation?docId=testpatients

This page is not comprehensive of all data that exists in the sandbox. Rather, it is meant to highlight patients that contain some common types of test data.

Sandbox URI - https://fhir.epic.com/interconnect-fhir-oauth/ Note that the Epic on FHIR sandbox is exclusively secured using OAuth 2.0 FHIR Endpoints - See 'FHIR API Endpoints' on open.epic

These test users can be used to authenticate when testing Provider-Facing Standalone OAuth 2.0 workflows with the FHIR sandbox.

Name | User Login | User Password | Description
FHIR, USER | FHIR | EpicFhir11! | User account without linked provider record. This user will not have a PractitionerRole resource.
FHIRTWO, USER | FHIRTWO | EpicFhir11! | User account with linked provider record. This user will have a PractitionerRole resource.

These are some of the test patients that exist in the Epic on FHIR sandbox. Any test patients with MyChart credentials listed can be used to authenticate when testing Patient-Facing Standalone OAuth 2.0 workflows with the FHIR sandbox.

Patient | Identifiers and Credentials | Applicable Resources
Camila Lopez | FHIR: erXuFYUfucBZaryVksYEcMg3 External: Z6129 MRN: 203713 MyChart Login Username: fhircamila MyChart Login Password: epicepic1 | DiagnosticReport Goal Medication MedicationOrder MedicationRequest MedicationStatement Observation (Labs) Patient Procedure
Derrick Lin | FHIR: eq081-VQEgP8drUUqCWzHfw3 External: Z6127 MRN: 203711 MyChart Login Username: fhirderrick MyChart Login Password: epicepic1 | CarePlan Condition Goal Medication MedicationOrder MedicationRequest MedicationStatement Observation (Smoking History) Patient
Desiree Powell | FHIR: eAB3mDIBBcyUKviyzrxsnAw3 External: Z6130 MRN: 203714 MyChart Login Username: fhirdesiree MyChart Login Password: epicepic1 | Immunization Observation (Vitals) Patient
Elijah Davis | FHIR: egqBHVfQlt4Bw3XGXoxVxHg3 External: Z6125 MRN: 203709 | AllergyIntolerance Binary Condition DocumentReference Medication MedicationOrder MedicationRequest MedicationStatement Observation (Smoking History) Patient
Linda Ross | FHIR: eIXesllypH3M9tAA5WdJftQ3 External: Z6128 MRN: 203712 | Condition Medication MedicationOrder MedicationRequest MedicationStatement Observation (Vitals) Patient
Olivia Roberts | FHIR: eh2xYHuzl9nkSFVvV3osUHg3 External: Z6131 MRN: 203715 | Binary Condition Device DocumentReference Patient
Warren McGinnis | FHIR: e0w0LEDCYtfckT6N.CkJKCw3 External: Z6126 MRN: 203710 | AllergyIntolerance Binary Condition DiagnosticReport DocumentReference Observation (Labs) Observation (Vitals) Patient Procedure

Below are Group FHIR IDs that can be used to test bulk FHIR calls in the sandbox.

To test bulk FHIR in the sandbox, you MUST select the Kick-off, Status Request, File Request, and Delete Request APIs along with the Search interaction of any APIs for which you want to export data.

For example, if you want to extract demographics and immunizations for patients in the relevant registry, you would add the following scopes to your application:

* Bulk Data Kick-off

* Bulk Data Status Request

* Bulk Data File Request

* Bulk Data Delete Request

* Patient.Search (R4)

* Immunization.Search (R4)

Your client record will automatically be granted access to the test FHIR Group below so long as you include all four bulk FHIR APIs in your registered application's API list.

Bulk FHIR Group | FHIR Group ID
Group consists of test patients in table above. | e3iabhmS8rsueyz7vaimuiaSmfGvi.QwjVXJANlPOgR83

Testing Community Authorization

The test patients below can authorize disclosure from two health systems, "Vendor Services MyChart Central Member" and "FHIR Sandbox", through MyChart Central instead of through their MyChart-site-specific credentials.

To prepare for testing:

* Your test environment should support community authorization.

* Your test environment should recognize the new responding environment, :

Vendor Services MyChart Central Member

* HCID: urn:oid:1.2.840.114350.1.13.529.1.7.3.688884.100.2200

* MyChart URL: https://open.epic.com/mychart-vsmyccmem/Authentication/Login

* Your test environment should be prepared to initiate for the test patients below.

* Workflow to test: all three test patients can authorize disclosure from the two health systems above via MyChart Central.

* For help, contact both QHINSupport@epic.com and open@epic.com.

Patient | Name (First / Middle / Last) | Sex | DOB | Email | Phone | Address
Camila Lopez | Camila / Maria / Lopez | F | 9/12/1987 | camilalopez@epic.com | 469-555-5555 | 3268 West Johnson St. Apt 117 Garland, Texas 75043
Derrick Lin | Derrick / (none) / Lin | M | 06/03/1973 | derricklin@epic.com | 785-555-5555 | 7324 Roosevelt Ave Indianapolis, Indiana 46201
Linda Ross | Linda / Jane / Ross | F | 4/27/1969 | lindaross@epic.com | 231-666-6666 | 912 Highway 14 Cadillac, MI 49601

1.28 Troubleshooting

File: site/docs/01-guides/28-troubleshooting-eof.html

Source: https://fhir.epic.com/Documentation?docId=troubleshooting_eof

Web App and Web Service Troubleshooting

Timing Changes to Your Application

The FHIR Developer Sandbox may take up to 30 minutes to sync changes you make to technical settings for your app, such as endpoint URIs and selected APIs.

Epic customers receive licenses of applications automatically on a rolling 12-hour cycle. Each customer maintains their own instance of Epic, so each environment they maintain is likely to have a unique client record sync cycle. Coordinate with customers and keep this sync cycle so that your app is licensed to their Non-Production and Production environments in time to accommodate testing and live use.

Contents

* OAuth 2.0 Errors

* API Call Errors

* Hyperspace/MyChart Embedding Errors

* Web Page Errors

OAuth 2.0 Errors

There are two types of OAuth 2.0 apps, launched apps and backend apps. Backend apps interact with only Epic's /token endpoint, while launched apps interact with the /token AND /authorize endpoints. We've broken out the error messages below by endpoint:

Authorize Endpoint

"Something went wrong trying to authorize the client. Please try logging in again."

* Check that that redirect URI in your request matches one of the redirect URIs registered on the app. This includes the following:

* If you've edited your app's URIs, wait for the client sync before testing

* URIs are case sensitive

* URIs must be exact matches. For instance "/" will not match "/index.html", even though browsers treat them the same

* Check that the client ID in the request matches the client ID of the app, and that the response_type is code.

* If you have a backend app encountering this error code, make sure the URL you're hitting ends in "/token" instead of "/authorize".

* Make sure you're including a proper "aud" parameter in the request.

* If using a POST request, make sure your body is of Content-Type: application/x-www-form-urlencoded instead of application/json, and make sure your request data is in the body instead of the query string (vice versa for GET requests).

"invalid_launch_token"

* The client ID in the FDI must match the client ID of the app. Confirm the client ID in the app on Epic on FHIR make sure the client record exists in the Epic Community Member's Epic instance and that they have used that client ID in the integration. Make sure your app record and the authorization code in your request match exactly.

* If you are redeeming a launch token at the wrong Epic instance (example: production vs non-production) this can occur.

404 or 500 error

* Request URL might be too long. Shorten your state parameter.

I'm being asked to log in, but I'm launching from Epic/the Hyperspace Simulator

* If you're launching from Epic (EHR launch) and still being asked to log in, have the app request the "launch" scope.

Token Endpoint

400 error:

* "invalid_request" - Formatting of the HTTP Request can cause this issue. Try out the following:

* Use the POST verb, not GET

* Content-Type should be "application/x-www-form-urlencoded".

* Do not send JSON, for example.

* Content should be in the body of the request, not the querystring

* There should be no "?" after "/token"

* "invalid_client" - Your client authentication may be invalid. Try the following:

* For apps using JWT authentication, refer to the linked JWT Auth Troubleshooting resource below.

* If you're still having issues, try re-uploading your client authentication (client secret or public key) and waiting for the sync.

400 error: "unauthorized_client"

* If using a Backend OAuth 2.0 integration, there is required community member setup that will cause this error if not completed. This step is auto-completed when testing in the FHIR sandbox.

API Call Errors

This section assumes your app has obtained an access token and is having issues making API calls using it. If you have not received an access token yet, refer to the OAuth 2.0 errors section above.

401 errors

* 400 usually means you had the right security, but the data in your request was incorrect somehow.

* Check that you're putting data in the right place (body vs querystring).

* Look for error messages in the response body to assist. Here is a common one:

* NO-PATIENT-FOUND - When searching using an ID such as MRN, the ID Type in Epic needs to have a descriptor. Make sure the descriptor (I IIT 600) is populated for the target ID Type, and distribute to the app for searching.

* It's helpful if your app is able to log the response body, since Epic returns detailed error messages for most APIs.

403 errors

* For these errors, Epic identified who the client and user were, but denied access.

* If the issue is with the , we generally don't return a response body, just 403 Forbidden

client

* In this case, check the API you are calling is attached to your client ID. Wait for the client to sync before testing any changes

* For scope errors, the WWW-Authenticate HTTP Response header should say this:

* Bearer error="insufficient_scope", error_description="The access token provided is valid, but is not authorized for this service"

* If the issue is with the , we generally return an error message describing why, such as:

user

* "The authenticated user is not authorized to view the requested data" is common for FHIR calls. In this case, try the following:

* When testing in the FHIR Developer Sandbox or a customer environment, changes made to the list of selected APIs on your app on the website may take up to 1 hour to sync with the Sandbox and 12 hours to sync with an Epic organization's Epic instance.

* Make sure you are handling access tokens correctly. If you mistakenly use one person's access token on another person's behalf, you'll see this error.

* Organizations set up user security checks that also apply to access tokens. These security checks, if failed, produce this error.

* If using a backend services app, these security checks are applied to the background user configured for your client ID. Otherwise, they are applied to the user who authenticated into your app.

Hyperspace/MyChart Embedding Errors

Many EHR Launched apps attempt to embed in Epic workflows, and rely on Epic build to do so. Below are common issues you might see when configuring one of these launches.

Attempted to embed in Hyperspace, but the window popped out to a new browser:

* Check the FDI record has the correct launch type.

* There are 2 types of FDIs and 2 types of launch activities, they need to match to respect launch settings:

* PACS FDIs - Need RIS_PACS_REDIRECTOR

* Web FDIs - Need MR_CLINKB_* (Active Guidelines)

Hyperspace launch displays a green/white spinning circle indefinitely (AGL spinner).

* Confirm that all domains in the login workflow are added to the Epic Browser Allowlist.

* Confirm that the workspace build points at the right FDI. Consider Active Guidelines debug mode to troubleshoot (shows URL navigated to).

App launches externally in MyChart mobile when launchtype specifies embedded launch.

* Confirm that all the app domains are allowlisted in the MyChart Mobile FDI record.

Web Page Errors

OAuth involves several web servers taking control of a browser in a sequence/handshake. The following errors could come from EITHER the Interconnect/Authorization server or the third-party app's server when they are trying to display HTML.

This content cannot be displayed in a frame.

* This could be hiding an OAuth 2.0 error, or an accidental login page.

* Interconnect cannot be iframed, and is trying to show an error page or login page.

* Will appear in trace logs, OR you can pop out the launch with FDI settings for "external window" workspace type.

* Refer to these sections above for scenarios where Interconnect returns HTML:

* "Something went wrong trying to authorize the client. Please try logging in again."

* "I'm being asked to login, but I'm launching from Epic"

* Otherwise, the third-party app is likely blocking the iframe.

* The app needs to adjust the X-Frame-Options response header. The website can omit the "X-Frame-Options" header or set it to "ALLOW-FROM https://domain". The domain should be that of the Hyperspace web server for the organization.

Content was blocked because it was not signed by a valid security certificate.

* Accompanied by pink background

* Implies a web site's certificate is invalid

* Check the third-party server and Interconnect server for expired or invalid certificates.

This page can't be displayed

* Error normally calls out the domain/server being requested

* One cause is the browser being unable to resolve the website's IP address. Make sure the website's domain is publicly available, or at least available from the Hyperspace server for provider-facing embedded launches

* If you can resolve the IP address, another cause is the server/website hanging up during the TLS handshake. Try using SSL Labs to diagnose why the server is hanging up. "Handshake Simulation" is especially useful since it calls out compatability with IE-11 and various windows versions

1.29 User Context

File: site/docs/01-guides/29-usercontext.html
Source: https://fhir.epic.com/Documentation?docId=usercontext

Application User Context

Epic defines three different user contexts for an application:

* Patients

* Clinicians, Staff, or Administrative Users*

* Backend Systems

*The "Clinicians, Staff, or Administrative Users" context is commonly referred to as "Provider" context, so the rest of this tutorial uses that term.

Choosing a User Context

Most applications have an end user who is interacting with them, and these users determine the user context of your application. To choose the appropriate user context, consult this table:

Primary User Type	User Has Epic Access	User Context
Patient	Yes	Patient
Patient	No	Backend System
Provider	Yes	Clinicians, Staff, or Administrative Users
Provider	No	Backend System
Background Process with No User Interaction	N/A	Backend System

Multi-Context Applications

Your application might not fit neatly into one user context, and in that case it could be a good fit for a multi-context application. With multi-context applications, you register separate apps or clients with Epic, one for each user context you support (typically no more than three).

Limit APIs to the Required Application

When building a multi-context application, an app that has multiple Client IDs, you might have overlap with which APIs are called by each client. For example, both a provider-facing and backend application might need to search for patients with the same API. While this search can be valid, you should in general only include the APIs necessary on a specific application. We recommend this for a few reasons:

* On the principle of least access, you should only request access to precisely what your workflow requires, and no more.

* You decrease the work needed to install your application. By precisely defining the APIs you need, you are also limiting the security that Epic community member's need to enable for your application to function.

For example, if your backend component searches for patients, but your provider-facing component does not, then you should only add the patient-searching API to your backend component.

Limit Use of Backend Apps

Whenever possible, your application should avoid using a backend component so that Epic can conduct end-user specific security checks during your API calls. By using a backend component, rather than a patient or provider-facing component, those security checks are run on a background user rather than the specific user interacting with your app.

This means your application should always use the OAuth 2.0 token of the user who authorized the application when one is available. The exception is for specific APIs that are not supported for the user context of your end users. It might be appropriate to include a backend component to call these APIs, but ensure you have robust security within your application to control how end users can interact with these APIs from your system. Your security policies should include:

* Never sharing access tokens or authentication credentials with your end user's device or browser. This way, a malicious end user cannot impersonate your application. Your backend server should be the only entity making API calls.

* Having your own authentication and session management systems. For most use cases, only authenticated end users should be able to trigger APIs.

* Validating all end-user input before forwarding the input to an API.

Use Cases for Backend Apps

Below is a list of common use cases for backend applications:

* Book Anywhere integrations

* Printing and faxing

* Remote patient monitoring (RPM, patient unauthenticated)

* Personnel management

* Patient Integrated Voice Response (IVR)

* Public dashboards

* Patient chatbot (patient unauthenticated)

Creating a Multi-Context App with Epic on FHIR

To support a multi-context app, you will need to create separate applications for each intended context. This provides you with a set of production and non-production client identifiers for you to use with each context of your app. Each application needs to include the APIs that will be used in that specific context. For example, if there is an API that will be used in the provider context, but not in the patient context, list that API on only the provider context application record.

Use consistent names across multi-context application records. Names followed by their context can be useful to show which client IDs are associated with which contexts. For example, if "OurNewHealthApp" is the name of the multi-context integration, "OurNewHealthApp - Provider" could be the name of the application record designed for the provider context app.

Epic Community Member Downloads

To allow an Epic community member to download your multi-context app, you will need to provide them with the Production or Non-Production Client IDs for each of the applications that together support the full multi-context integration.

API Supported User Types

Overview

Epic has determined that some APIs are not appropriate to be called directly by end users or patients. For example, an API used as part of an administrative user's workflow is likely not appropriate for a patient-facing application because a patient should not be interacting with that data.

To reflect these use cases, Epic has assigned a list of supported user types to each Epic API and introduced validation for user type on all API calls.

User Type Validation

Starting in the May 2022 version of Epic, Epic community members can enable user type validation for their APIs that checks the user context of the application against the supported user types of the API being called. If the context of the user calling the API does not match the API's supported user types, then the API call fails.

When Does an API Support a Given User Type?

An API can support a combination of three possible user types, and when an API supports a given user type, it means the following:

* Patients: This API is appropriate to call directly based on input from Patients, and its output can be shown directly to the authorized patient.

* Provider: This API is appropriate to call directly based on input from Provider users, and its output can be shown directly to the authorized provider.

* Backend Systems: This API can be called by a backend system, and when called from a backend application, it must have its input and output validated rather than letting end users interact with it directly.

API and User Type in Your Application

If your application uses OAuth 2.0, then the API user type needs to match the user context you chose when building the application.

If your application uses an alternative authentication method (such as basic authentication), Epic cannot authenticate against a specific user regardless of the app's selected context. These API calls are therefore treated as a Backend System user type even if the app's user context is Patient or Provider.

Should I Edit My App's User Context?

If an API does not support your app's user context, first validate that your chosen user context matches your use case using the table above. If you can appropriately switch to a supported context, then update your app.

If it is not appropriate to switch the context of your app, another option is to create a multi-context application. For most applications, this means provisioning an additional backend component that interacts with as few APIs as needed.

Additional User Security Validation

Epic's user context validation is just one step in our API security checks. This means that a request might pass context validation but fail at a later step. For example, Epic assesses the security of the individual user associated with a request. If a user does not have the correct security within Epic, the call fails, and the Epic community member needs to assess the use case and update the user's security as appropriate.

Refer to our troubleshooting tutorial for help assessing why a given API call has failed.

1.30 open.epic Terms of Use

File: site/docs/01-guides/30-terms.html

Source: https://fhir.epic.com/Documentation?docId=terms

open.epic Terms of Use

* Privacy Policy

* open.epic API Subscription Agreement

* Developer Guide

* Disclosures

Last updated: October 31, 2025

(Added Section 7 under terms “For Developers” to address data collection requirements of Insights Condition and Maintenance of Certification, 45 CFR 170.407)

Before using this website or any Materials, read these terms of use so you understand your responsibilities and Epic’s. You are accepting these terms by using this website or any Materials. Epic may change these terms from time to time - for example, to reflect changes in the Materials we offer or for legal, regulatory, or security reasons. Check back periodically for updates. By continuing to use this website or any Materials after an update, you are accepting the updated terms.

Introduction

Open.epic enables Epic Community Members and developers to use the open.epic Components and certain interfaces and other Epic-developed technologies to connect and interact with Epic Community Members’ Epic software. This website includes Materials to use as you develop, test, market, offer, deploy, and support products or services that interact with the technology available through open.epic.

The sections below describe the terms of use for the Materials and this website that apply specifically to Epic Community Members, those that apply specifically to developers, and those that apply to everyone. Capitalized words or phrases used in these terms are defined in the “Definitions” section below.

The interfaces, application programming interfaces (APIs), and other technologies available through open.epic represent a subset of the interoperability technology that Epic offers to support the access, exchange, and use of electronic health information and other data. Other APIs, technologies, and services may be available under different terms through different processes and programs.

For Epic Community Members

If you are an Epic Community Member, your use of Materials related to the open.epic Components is subject to the open.epic API Subscription Agreement.

You may use other Materials available through open.epic at your own discretion, subject to the terms “For Everyone” below and any applicable terms of your agreement(s) with Epic. You are solely responsible for how you use these Materials and this website to interoperate with products and services.

For Developers

If you are not an Epic Community Member, the following terms, and the terms “For Everyone” below, apply to your use of the Materials and this website.

* This website is the source of truth for current versions of the Materials. You can keep copies of the Materials for yourself. You may distribute them only by linking others directly to the Materials on the website.

* Epic provides the Materials for your use, and you decide how to use them to interoperate with Epic software and provide your products and services to Epic Community Members and end users. With this freedom comes the responsibility to make sure your products and services are behaving appropriately. Accordingly, as between you and Epic, you are solely responsible for your products and services, including how they interact with Epic Community Members’ systems, and for all liability and consequences (e.g., Claims by or on behalf of patients or related to patient harm; data corruption; mapping or saving data to patient records incorrectly; fraudulent or other unethical conduct, such as inappropriately prescribing narcotics or zeroing out a balance due; degraded system response time, performance, and availability; privacy breaches; and security vulnerabilities) that arise from or relate to the use of or inability to use your products or services, or that arise from or relate to data made available to or from your products or services or direct or indirect use of that data. You are also responsible for complying with all applicable laws, including not infringing on Epic’s or others’ intellectual property rights.

* You agree to indemnify, hold harmless and defend Epic, its subsidiaries, and all of its and their employees, officers, directors, contractors, and other personnel from and against any Claim that in any way arises out of or relates to the use of or inability to use any of your products or services. This obligation covers, for example, any Claim arising out of or related to:

* the accuracy, completeness, integrity, or compliance of your products and services; or

* any data made available to or from your products or services, or the direct or indirect use of that data in Epic Community Members’ Epic or downstream systems.

* Some interoperability technology made available through open.epic requires an Epic Community Member to have a license to other functionality or build additional workflows. Where possible, help our mutual customers avoid surprises by working closely with them and directing them to Epic with any questions related to Epic software.

* If you want to use open.epic or Epic's name or other trademarks, you may do so only in accordance with our Trademark Usage Guidelines. These terms of use do not authorize you to use an Epic Community Member’s logo or other intellectual property.

* Direct access to Epic’s software (including an Epic Community Member’s system) is not needed to develop, test, deploy, or support your product or service. You can test your product or service through the open.epic sandbox or by working with a particular Epic Community Member. Direct access to Epic’s software can be granted only with approval from both an Epic Community Member and Epic.

* Under the Insights Condition and Maintenance of Certification requirement at 45 CFR 170.407 (), Epic must measure and report information about applications that connect to our certified software to demonstrate how our software supports the application ecosystem. This includes details such as:

https://www.ecfr.gov/current/title-45/section-170.407

* the names of connected applications;

* their developers;

* their intended purposes and intended users; and

* whether the connected applications are actively used.

In connection with Epic’s obligations under the ONC Health IT Certification Program or other regulatory or legal requirements, Epic may collect data about you and your use of the Materials, including how your products or services interact with the Materials, and may disclose this data to third parties. If any of this data becomes public (e.g., through publication by the Assistant Secretary for Technology Policy/Office of the National Coordinator for Health Information Technology or by Epic as required under applicable regulatory or legal requirements), Epic reserves the right to further use the data at its discretion.

Refer to the Privacy Policy for additional information about how Epic collects and uses information you provide when creating an account with or otherwise accessing open.epic.com or fhir.epic.com.

For Everyone

The following additional terms apply to all users of the Materials and of this website, except that a Community Member’s use of any Materials related to the open.epic Components is instead governed by the open.epic API Subscription Agreement.

* THE MATERIALS AND THIS WEBSITE MAY INCLUDE INACCURACIES AND ERRORS, AND YOU USE THEM AT YOUR OWN RISK. THE MATERIALS AND THIS WEBSITE ARE PROVIDED AS IS WITHOUT ANY WARRANTY, EXPRESS OR IMPLIED. FOR EXAMPLE, WE DO NOT MAKE ANY PROMISES ABOUT THE CONTENT OR FEATURES OF THE MATERIALS OR THIS WEBSITE, INCLUDING THEIR ACCURACY, RELIABILITY, AVAILABILITY, OR ABILITY TO MEET YOUR NEEDS.

* You own what you develop using the Materials. Epic owns the Materials, as well as any Epic-provided improvements to the Materials or learnings based on the Materials, such as enhancements made by Epic to testing tools or documentation. We want to encourage a vibrant developer environment. If you suggest a way to improve the Materials, you are granting Epic the right to use your suggestion in any manner, including as part of the Materials for others to use, and without any obligation or notice to you.

* The testing sandbox made available through open.epic is meant to simulate an electronic health record environment and is for testing purposes only. The testing sandbox should be populated only with sample or synthetic data. You agree not to put individually identifiable information in any Epic-provided sandboxes. You acknowledge that Epic may wipe or otherwise remove data from those sandboxes in its sole discretion, including any individually identifiable data discovered in a sandbox.

* This website and the Materials are not intended and should not be used for performance, scalability, or security testing. Even so, if you identify an actual or potential security vulnerability with Epic software or the open.epic Components, you will inform Epic about such vulnerability as soon as possible so that Epic can address the issue.

* Wisconsin law will govern all disputes arising out of or relating to these terms of use, regardless of conflict of laws rules. These disputes will be resolved exclusively in the federal or state courts of Dane County, Wisconsin, and you and Epic consent to personal jurisdiction in those courts.

* These terms of use will be severable so that if a provision is not enforceable, it and related terms will be interpreted to best accomplish the essential purpose of that unenforceable provision. However, severability will not apply if it significantly changes the benefit of these terms to you or Epic.

* If these terms of use terminate (e.g., because Epic stops offering the Materials or this website), these terms “For Everyone” and clauses 2 and 3 under the terms ”For Developers” above will survive.

* In these terms, the word “include” and its variants are not words of limitation, and examples are for illustration and not limitation.

Definitions

* Claim means all claims, demands, investigations, inquiries, and actions, and all liabilities, damages, fines, and expenses arising out of or relating thereto, including settlement costs and attorneys’ fees.

* Epic Community Member means a healthcare organization that has a license and support agreement with Epic to use Epic software. For purposes of these terms of use, an Epic Community Member’s affiliated healthcare organizations that use Epic software under its license and support agreement with Epic are considered part of the Epic Community Member.

* Materials means resources made available on open.epic to support your use of the open.epic Components and other interoperability technology described on open.epic, including testing sandboxes, documentation, specifications, and other information related to Epic’s implementation of various interfaces and APIs.

* open.epic Components means the designated application programming interfaces listed here and other functionality licensed directly by Epic Community Members from Epic under the open.epic API Subscription Agreement to facilitate interoperability.

1.31 Frequently Asked Questions (FAQ)

File: site/docs/01-guides/31-faq.html
Source: https://fhir.epic.com/FAQ

General

What is FHIR?

the HL7 website

What is Epic on FHIR?

open.epic

What is USCDI?

Health IT website

Which FHIR resources, operations, and versions are available?

Epic on FHIR homepage

Who may use Epic on FHIR?

terms and conditions

Are there terms or costs associated with Epic on FHIR and use of the APIs listed there?

For app developers: Use of Epic on FHIR is subject to the terms and conditions of open.epic. Use of the Epic on FHIR website is free of charge. There are no licensing costs to the app developer for use of technologies made available through open.epic and Epic on FHIR. Epic Community Members should request an estimate for expected API licensing costs by contacting their Epic representatives - app developers should direct questions about Epic licensing costs back to Epic. We recommend that app developers provide the Epic Community Member with their app's client ID to facilitate an easy estimate process.

For app users (i.e., Epic Community Members): Epic Community Members' use of FHIR APIs is subject to the open.epic API Subscription Agreement, which may incur licensing costs. Epic Community Members with questions about licensing costs for specific APIs or app records should request an estimate for expected licensing costs by contacting their Epic representatives.

How does an app developer sign the open.epic agreement?

open.epic terms of use

Does Epic review, certify, or verify developers who use Epic on FHIR or their applications?

Client IDs and App Management

What are client records and client IDs?

A client record represents an individual client application that may potentially call an Epic Community Member's Epic system. Client records contain information such as the name of the app, the name of the company that registered the app, the type of user that will authorize the application to access the Epic system, and the scope of APIs that the app will use. Client records also contain production and non-production client IDs that uniquely identify the client record and application.

Client records and client IDs play an important roll in enabling Epic Community Members to control which applications access their Epic systems and monitor applications' use of APIs. An application must include its client ID when calling FHIR or other Epic APIs, and will only receive a successful response if the API called is contained in the API scope for the associated client record, and only if that client record has been authorized for the Epic system being called.

How do I get a client ID for my application?

How do I request my organization's Epic on FHIR app be listed in Connection Hub?

Requesting A Connection Hub Listing For Your App

Connection Hub on Showroom is an optional, paid marketing service that allows your app to be publicly discoverable. A listing is not required for customers to download your app.

Testing with the Sandbox

What is the sandbox?

How do I use the sandbox?

Where can I find test patient documentation for the sandbox?

Account Management

Can I sign up for both a Vendor Services and an Epic on FHIR account?

contact form

Authentication

Where can I find information about authentication?

How do I test authentication?

APIs

I don't see an API that I am looking for. How do I get non-FHIR APIs?

open.epic.

How can my app write data to Epic?

Epic supports a wide variety of standards-based and Epic-specific interfaces and APIs capable of writing information to Epic software. The Epic on FHIR site hosts FHIR Create APIs that allow data to be written into Epic. If these APIs do not meet your needs, you can find more information about the incoming, outgoing, and bidirectional standards-based interfaces that Epic supports at open.epic.

Writing data to the EHR requires great care on the part of the developer and the provider organization. All applications that write data to the EHR should be carefully tested before deployment to production environments.

What FHIR versions do Epic community members support?

App Licensing

How do I connect my FHIR application to Epic?

The Epic software does not consist of one centralized instance. Rather, each healthcare organization owns and manages their own instance of Epic and the data contained within it. This means that you must connect your application to each Epic healthcare organization individually.

Client records contain information about an application’s configuration and, when downloaded to an Epic environment, will allow that environment to recognize your application’s client ID and enable your application to make API calls against that environment’s servers.

The options you have for distributing your app’s client record depend on who will use your application and what data it will read from or write to Epic.

Patient-Facing Applications:

If you have developed a patient-facing application, there are two distribution models that can sync your application's client record to a healthcare organization's Epic instance, enabling your application to integrate with that organization's data.

* Auto-download – If your application meets the criteria specified on the Patient Facing Apps Using FHIR page, your application's client record will automatically download to most healthcare organizations' Epic instances. This type of distribution is meant to facilitate application use by individual patients and does not require any relationship with the healthcare organization.

* Manual download – For applications that do not meet this auto-download criteria, a representative from the healthcare organization must choose to download your client record to enable use of your application.

End-User and Backend Applications:

If you have developed an application for healthcare organization employees or backend system-to-system connections, your application's client record will always be distributed using the manual download process.

Getting Help

Can we schedule a meeting to talk about my integration?

Epic usually works through our customers to set up integrations. If we have a mutual healthcare organization that's asking to integrate, you should begin by working with that healthcare organization's IT team to design and set up the integration. The organization's IT team can reach out to their Epic support team if they need assistance.

If you want to engage with Epic directly, consider a subscription to Vendor Services where you would receive Epic support for designing, testing, and implementing your integration.

Otherwise, feel free to use all of the publicly available documentation and resources on open.epic.com and fhir.epic.com.

How do I get assistance implementing FHIR or authentication?

Epic on FHIR is a self-service resource. If you are interested in acquiring design, testing, troubleshooting, or install support services from Epic, consider Vendor Services, Epic's option for receiving support and other services directly from Epic representatives.

You may also find these industry resources helpful:

* HL7 FHIR

* HL7 FHIR Foundation

* FHIR Community Forum

* SMART Health IT

I am experiencing an error with the Epic on FHIR website. How do I get assistance?

We perform weekly maintenance on Thursdays. This results in a couple minutes of downtime, typically somewhere from 2 PM - 3 PM Central Time. We also post a banner when the Sandbox will be down for an upgrade.

If you experience a problem with the Epic on FHIR website, such as a broken link or an error message on page load, outside of these these times, contact us using the contact form at the bottom of the page. Please use the subject "FHIR – <Your Name or Organization>".

Showroom

What is Showroom?

Showroom is Epic's marketplace where anyone can discover services to help our customers achieve strategic goals, Health Grid Networks that bring information together, and products that can connect to Epic.

If you are interested in listing your product in Showroom, review this tutorial for more information.

What are the differences between the Showroom Product tiers?

* Cornerstone Partners: Technologies that are integral to Epic software.

* Toolbox: Products in specific categories that follow Epic's recommended practices for integration (aka Blueprint).

* Toolbox Under Construction: Products in Toolbox categories whose recommended practices are still being developed.

* Connection Hub: Products with a live connection to at least one Epic site.

Can the designation for a product in Showroom change?

What are the differences between open.epic, Epic on FHIR, Vendor Services, and Showroom?

* open.epic is a free online resource available to all developers to learn about and engage in data exchange with Epic. We provide technical specifications for our standards-based interfaces and APIs, self-service testing sandbox and client registration for FHIR clients, and a variety of self-services information to help developers locate the right Epic tools or services.

* Epic on FHIR is a free online resource and subset of open.epic that supports developers who create applications or services for use by provider organizations and wish to interoperate with the Epic comprehensive medical record. At fhir.epic.com, developers will find documentation on Epic's support for the HL7 FHIR APIs. Through this site, developers may also acquire a client ID for use with their application and test in our sandbox.

* Vendor Services is a collection of services offered by Epic to developers who are looking to integrate their product with Epic software. Any developer that meets the enrollment criteria can subscribe. Refer to our What is Vendor Services FAQ for more information.

* Showroom is Epic's marketplace where anyone can discover services to help our customers achieve strategic goals, Health Grid Networks that bring information together, and products that can connect to Epic.